

ALGORITHM TEMPLATE • XCPC 2026

Hanzhi • XCPC 算法模板

简介 • 快速识别 • API 接入 • 关键点

■ 简介

■ 快速识别

■ API 接入

■ 关键点

目录

00 使用指南	7
00.1 这个模板怎么用	7
00.2 复杂度判定速查（数据规模 $\rightarrow$ 算法族）	7
00.3 任务类型 $\rightarrow$ 第一反应	7
00.4 索引约定总表（抄板前必查）	7
00.4b 模板家族兼容性说明（复制组合前必看）	8
00.5 通用头文件（每节代码默认依赖）	8
00.6 编译与提交注意事项	8
01 基础工具	9
01.1 头文件与本地调试宏	9
01.2 快读 FastScanner 与 i128 读写	9
01.3 快速幂与模算术工具	9
01.4 整数二分：最小可行 / 最大可行	9
01.5 实数二分与三分	10
01.6 离散化	10
01.7 一维前缀和与差分	10
01.8 二维前缀和与差分	10
01.9 位运算函数与集合枚举	11
01.10 高精度整数 BigInt（非负大整数）	11
01.11 有理数 Frac（精确分数四则运算）	12
01.12 随机工具与稳定性技巧	12
01.13 按位贪心 + 数据结构（从高到低逐位决策）	12
02 数据结构	12
02.1 并查集 DSU	12
02.2 可撤销并查集	13
02.3 带权并查集	13
02.4 后继并查集（区间跳过）	13
02.5 树状数组 Fenwick（单点加 / 区间和，区间加 / 区间和）	14
02.6 树状数组求逆序对	14
02.7 线段树：单点修改 + 区间查询	14
02.8 线段树：区间加/乘 + 区间和（懒标记）	15
02.9 线段树：区间赋值 + 区间和	15
02.10 线段树 Beats（区间 chmin + 区间和）	16
02.11 动态开点线段树（巨大值域）	16
02.12 线段树合并与全功能（修改 / 删除 / 查询 / 第 k 小）	17
02.13 ST 表（静态 RMQ / 幂等操作）	17
02.14 单调栈（左右第一个更小/更大）	18
02.15 单调队列（滑动窗口最值）	18
02.16 对顶堆（动态中位数 / 第 k 小集合）	18

02.17 左偏树（可并堆）	19
02.18 分块（区间加 + 区间和）	19
02.19 莫队（区间查询离线）	19
02.20 莫队带修改	20
02.21 主席树（静态区间第 k 小）	20
02.22 可持久化数组（版本回溯）	20
02.23 动态区间第 k 小（树套树：BIT 套值域线段树）	21
02.24 平衡树 FHQ Treap（分裂合并）	21
02.25 隐式 FHQ Treap（序列操作）	22
02.26 笛卡尔树	23
02.27 线性基（异或空间）	23
02.28 离线 BIT：区间不同数 / 区间第 k 大计数	23
02.29 CDQ 分治（三维偏序）	24
02.30 整体二分	24
02.31 线段树分治（离线动态连通性）	24
02.32 启发式合并	25
02.33 猫树 Cat Tree（静态区间查询 $O(1)$ ）	25
02.34 珂朵莉树 ODT（区间赋值推平）	25
02.35 Splay（伸展树）	26
02.36 Link-Cut Tree（动态树）	26
03 树上问题	27
03.1 LCA：倍增法	27
03.2 LCA：Tarjan 离线	27
03.3 树链剖分 HLD	28
03.4 树的直径	28
03.5 树的重心	28
03.6 树上差分	29
03.7 Euler 序（子树区间化）	29
03.8 换根 DP（Rerooting）	29
03.9 树形 DP：最大独立集 / 树背包	29
03.10 基环树	29
03.11 函数图（出度 1 图）与倍增跳	30
03.12 虚树 Virtual Tree	30
03.13 点分治（树上路径统计）	30
03.14 DSU on Tree（树上启发式，静态子树统计）	31
03.15 树上莫队	31
03.16 树哈希（同构判定）	31
03.17 Kruskal 重构树	32
03.18 长链剖分（ $O(1)$ k 级祖先 / 深度统计合并）	32
03.19 支配树（Lengauer-Tarjan）	33
03.20 点分治的补充：动态点分治	33
03.21 树链剖分 + 树上主席树（路径点权第 k 小）	33
03.22 树的 Prufer 序列（放 06 章组合处引用）	33

04 图论	33
04.1 最短路: Dijkstra (堆优化)	33
04.2 最短路: BFS / 0-1 BFS / 无权网格	34
04.3 最短路: Bellman-Ford / SPFA (负权与负环)	34
04.4 最短路: Floyd (全源 / 传递闭包)	34
04.5 差分约束	34
04.6 最小生成树: Kruskal / Prim	35
04.7 次小生成树 / 严格次小	35
04.8 拓扑排序 (含字典序最小)	35
04.9 强连通分量 SCC (Tarjan)	35
04.10 割点与桥 (Tarjan)	36
04.11 双连通分量: 边双 / 点双与圆方树	36
04.12 2-SAT	37
04.13 欧拉路径/回路 (Hierholzer)	37
04.14 二分图最大匹配 (匈牙利 / Hopcroft-Karp)	37
04.15 网络流: Dinic 最大流	38
04.16 最小费用最大流 (SPFA/Dijkstra + 势能)	38
04.17 最小割建模速查	39
04.18 网络流: 上下界 / 最大权匹配补充 (HLPP / 带花树要点)	39
04.19 二分图最大权完美匹配 (KM)	39
04.20 图论杂项: 斯坦纳树 / Matrix-Tree / Stoer-Wagner / 朱刘	40
04.21 Johnson 全源最短路 (含负边)	40
04.22 最短路计数 / 次短路 / 差分约束补充 (K 短路 A^*)	40
04.23 一般图最大匹配 (带花树 Blossom)	40
04.24 同余最短路 (模数建图)	41
04.25 三元环 / 最小环计数	41
04.26 最大团 Bron-Kerbosch (bitset 优化)	42
04.27 线段树优化建图 (点到区间 / 区间到点)	42
05 字符串	43
05.1 字符串哈希 (单/双哈希)	43
05.2 二维哈希	43
05.3 KMP 前缀函数	43
05.4 Z 函数	43
05.5 Trie 字典树	44
05.6 01 Trie (最大异或对 / 可持久化区间异或)	44
05.7 AC 自动机	45
05.8 Manacher (回文串)	45
05.9 回文自动机 PAM (回文树)	45
05.10 后缀数组 SA (基数排序)	46
05.11 后缀自动机 SAM	46
05.12 最小表示法 (Booth) 与循环同构	47
05.13 LCS / 编辑距离 / 通配符匹配	47

05.14 多串最长公共子串 (SAM / 后缀数组)	47
05.15 字符串技巧速查 (border / 周期 / 子序列)	47
05.16 广义后缀自动机 (多串 SAM)	48
06 数论与组合	48
06.1 模算术与 ModInt	48
06.2 扩展欧几里得 exgcd 与线性同余	48
06.3 线性筛 (素数 + 欧拉函数 + mu)	48
06.4 素性测试 Miller-Rabin	49
06.5 因数分解 Pollard-Rho	49
06.6 组合数 (阶乘逆元 / Lucas / exLucas)	49
06.7 特殊组合数: Catalan / Stirling	49
06.8 整除分块 (数论分块)	50
06.9 莫比乌斯反演	50
06.10 杜教筛 (大范围积性函数前缀和)	50
06.11 中国剩余定理 CRT / exCRT	50
06.12 BSGS / exBSGS (离散对数)	51
06.13 原根 / 二次剩余 / 离散对数补充	51
06.14 容斥原理与子集枚举	51
06.15 生成函数与多项式 (见 07 章)	51
06.16 Burnside 引理与 Polya 定理	51
06.17 AtCoder floor_sum 与数论杂项速查	52
06.18 扩展卢卡斯 exLucas (组合数模任意合数)	52
06.19 Min_25 筛 (大范围积性函数前缀和)	52
06.20 Bell 数 / 错排 / 常用结论速查	53
06.21 LTE 引理与连分数 (结论速查)	53
07 线性代数与多项式	53
07.1 矩阵与矩阵快速幂	53
07.2 高斯消元 (实数 / 模意义)	53
07.3 行列式 (任意模数)	54
07.4 FFT (任意整数卷积)	54
07.5 NTT (模质数卷积)	54
07.6 FWT (AND / OR / XOR 卷积)	55
07.7 形式幂级数核心 (inv / ln / exp / sqrt / pow)	55
07.8 多项式多点求值 / 快速插值 (积树)	55
07.9 Berlekamp-Massey + Kitamasa (线性递推)	55
07.10 常系数线性递推的其他工具 (Bostan-Mori / 特征值)	56
07.11 拉格朗日插值 (连续点 / 任意点)	56
07.12 康托展开与逆康托	56
07.13 异或方程组 (GF(2) 高斯消元)	57
07.14 LGV 引理 (不相交路径计数)	57
07.15 形式幂级数完整实现 (inv / ln / exp / sqrt / pow)	57
07.16 Bostan-Mori (有理生成函数第 n 项)	58

07.17 三模 NTT (任意模数卷积)	58
08 计算几何	58
08.1 点、向量与线段基础	58
08.2 多边形面积与点包含	59
08.3 凸包 (Andrew 单调链)	59
08.4 旋转卡壳 (凸包直径 / 宽度)	60
08.5 半平面交	60
08.6 圆相关 (交点 / 最小圆覆盖)	60
08.7 扫描线 (矩形面积并 / 周长)	61
08.8 KD-Tree (二维动态加点查询)	61
08.9 计算几何杂项速查	62
08.10 立体几何基础 (3D 点 / 向量 / 平面 / 体积)	62
09 动态规划	62
09.1 背包家族 (0/1 / 完全 / 多重 / 分组)	62
09.2 LIS / 最长上升子序列	63
09.3 区间 DP	63
09.4 状压 DP (TSP 骨架)	63
09.5 数位 DP (骨架)	63
09.6 概率与期望 DP	63
09.7 DP 优化: 单调队列	64
09.8 DP 优化: 斜率优化 / Li Chao	64
09.9 DP 优化: 决策单调性 (分治优化 / Knuth)	64
09.10 DP 优化: WQS 二分 / Slope Trick	65
09.11 轮廓线 DP / 插头 DP (骨牌覆盖)	65
09.12 DP 技巧速查 (滚动数组 / SOS DP / 子集卷积)	65
10.1 SG 函数 (组合博弈)	65
10.2 Nim 家族速查	65
10.3 随机算法 (爬山 / 模拟退火 / 随机化)	66
10.4 概率题通用技巧	66
11.1 交互题模式	66
11.2 通信题与信息编码	66
11.3 构造题与 checker 思维	66
11.4 STL 与常用工具速查	67
11.5 常见陷阱与调试清单	67
11.6 样例构造器 (对拍 / 造数据)	67
11.7 卡常技巧速查	67
11.8 近五年东亚赛区真题 → 模板索引	67

00 使用指南

00.1 这个模板怎么用

【简介】

本模板不是教材，是赛时速查手册。每个知识点固定四部分：简介（这是什么）→ 快速识别（题面什么信号对应它）→ API 接入（怎么调用、参数约定）→ 关键点（内部结构含义、怎么改板）。目标：想到算法以后，不重新读实现，直接正确接入。

快速识别：拿到题先定三件事——求什么（最值/计数/判定/构造/第 k 小）、数据规模（定复杂度上限）、静态还是动态。然后翻对应章节的「快速识别」对号入座。

API 接入：只看「API 接入」块。里面写清楚了 0-based 还是 1-based、每个参数的含义与格式、返回值、边界条件。照抄代码后按这里调用，不要看内部实现。

关键点：需要按题目改模板时才看。这里列出维护的数组/变量含义、内部函数作用、不变量，改哪几处、别动哪几处。

抄板流程：① 整段抄走该节代码块（struct/class/函数组）放到 `solve()` 外面；② 按「API 接入」构造并调用；③ 需要改功能时按「关键点」修改，其余一行不动。

00.2 复杂度判定速查（数据规模 → 算法族）

数据规模（时限 1-2s）	可行复杂度	优先考虑的算法族
$n \leq 20$	$O(2^n n) / O(3^n n)$	状压 DP、子集枚举、DLX
$n \leq 40$	$O(2^{(n/2)} n)$	折半搜索 Meet-in-the-Middle
$n \leq 500$	$O(n^3)$	Floyd、区间 DP、KM、高斯消元、Steiner
$n \leq 5e3$	$O(n^2)$	普通 DP、LIS/LCS、 n^2 建图 + Dijkstra
$n \leq 1e5$	$O(n \log n)$	排序二分、BIT/线段树/ST、莫队、Dijkstra、Kruskal、Tarjan、SA/SAM、NTT
$n \leq 1e6$	$O(n)$ 小常数	线性筛、单调栈/队列、双指针、KMP、Manacher、哈希
值域 $1e9 \sim 1e18$	$O(\log V) / O(\sqrt{n})$	二分答案、矩阵快速幂、BSGS、整除分块、Pollard-Rho
$n \leq 1e18$	$O(\log n)$	快速幂、exgcd/CRT、BM+Kitamasa、Bostan-Mori、min-plus 矩阵幂

用法：把题目最大的 n 填进表看“能跑多重的复杂度”，再反向淘汰算法。例如 $n=2e5$ 不可能 $O(n^2)$ ，要么 $O(n \log n)$ 数据结构，要么 $O(n \sqrt{n})$ 莫队/分块。

00.3 任务类型 → 第一反应

题目问什么	第一反应	对应章节
最小化最大值 / 最小化最小时刻	二分答案 + 判定	01 二分
方案数 / 满足条件的个数	DP / 组合数 / 容斥 / AC 自动机 DP	09 / 06
是否存在 / 输出任意方案	图论建模（流/2-SAT/匹配）、构造	04 / 11
第 k 小 / 第 k 大	主席树、整体二分、SAM 第 k 子串	02 / 05
区间查询（静态）	前缀和 / ST / 主席树	01 / 02
区间查询（带修改）	BIT / 线段树 / 树套树 / 莫队带修	02
树上路径 / 子树	LCA+差分 / 树剖 / 树上主席树 / 点分治	03

题目问什么	第一反应	对应章节
字符串串问题	哈希 / KMP / Manacher / SAM / SA	05
最优 DP 转移慢	单调队列 / 斜率 / Li Chao / WQS / Slope Trick	09
数据大但点数少	离散化 / 动态开点线段树	01 / 02
交互 / 通信 / 特殊评测	交互二分、多数投票、bit packing	11

00.4 索引约定总表（抄板前必查）

【简介】

本模板各节索引约定不统一，这是所有竞赛模板最大的事故源。下表汇总各高频模板的下标起点与参数语义。抄板前先查这张表，两分钟能省一次 WA。

快速识别：按模板名查。列含义：编号 | 模板 | 数组/点下标 | k /排名起点 | 区间约定 | 返回值语义。

关键点：本表由各节「API 接入」块人工同步维护；新增模板节时务必在此登记一行。

模板	下标起点	k / 排名	区间	返回值
01.2 快读 FastScanner	—	—	—	int/i64
01.3 快速幂	—	—	—	mod 内值
01.6 离散化	0-based	—	—	0..m-1 编号
01.7 一维前缀和	1-based (a[0] 占位)	—	闭区间 [1, r]	和
02.1 并查集 DSU	0..n-1	—	—	根编号
02.5 树状数组 Fenwick	1-based	—	闭区间 [1, r]	和
02.6 线段树（区间加/和）	1-based	—	闭区间 [1, r]	和
02.11 主席树 kth	1-based	k 从 1	闭区间 [L, R]	原值（非编号）
02.14 莫队	1-based	—	闭区间 [1, r]	计数
03.1 LCA 倍增	1-based (点 1..n)	—	—	祖先编号
03.2 树链剖分 HLD	1-based	—	—	链头编号
04.1 Dijkstra	1-based (点 1..n)	—	—	距离 i64
04.2 0-1 BFS	1-based	—	—	距离 i64
04.8 Dinic 最大流	1-based (点 1..n)	—	边 $u \rightarrow v$ 容量 c	i64 流量
04.9 HLPP	0-based (点 0..n-1)	—	边 $u \rightarrow v$ 容量 c	i64 流量
05.1 KMP	0-based (string)	—	下标 0..n-1	匹配起点
05.5 AC 自动机	0-based (root = 0)	—	—	节点编号
05.8 后缀自动机 SAM	状态 0=root	—	—	状态编号
06.1 ModInt	—	—	—	mod 内值
06.3 exgcd	—	—	—	(g, x, y)
06.8 组合数 $C(n, k)$	k 从 0	—	—	mod 内值
07.1 矩阵快速幂	0-based	—	—	矩阵
07.5 NTT 卷积	0-based	—	—	系数数组
02.33 猫树	0-based	—	闭区间 [1, r]	聚合值
02.34 珂朵莉树 ODT	0-based	—	闭区间 [1, r]	区间聚合
02.35 Splay	按值：任意	k 从 1	—	值

模板	下标起点	k / 排名	区间	返回值
02.36 Link-Cut Tree	点 1..n	—	路径 u-v	聚合值（异或/和/最值）
04.23 带花树 Blossom	0-based（点 0..n-1）	—	—	匹配数
04.24 同余最短路	余数 0..base-1	—	—	dist（最小可达值）
04.25 三元环计数	0-based	—	—	计数
04.26 最大团	0-based	—	—	团大小
05.16 广义 SAM	状态 0=root	—	—	状态编号
06.18 exLucas	k 从 0	—	—	$C(n, k) \bmod M$
06.19 Min_25 筛	1..n	—	—	前缀和
07.14 LGV 引理	0-based 矩阵	—	—	det（路径数）
07.15 FPS 全套	0-based 系数	—	—	多项式数组
07.16 Bostan-Mori	0-based 系数	—	—	$[x^n] P/Q$
08.10 立体几何	—	—	—	距离/体积
07.17 三模 NTT	0-based 系数	—	—	任意模数卷积
04.27 线段树优化建图	点 1..n + 树节点	—	区间 [l, r]	最短路/流

00.4b 模板家族兼容性说明（复制组合前必看）

【简介】

本书模板按“家族”组织：同一家族内的模板结构不互通（名字、节点定义、下标都可能不同），但每个家族都有“全能版”覆盖常用组合。需要多个功能组合时，直接选全能版复制一个，不要拼两个小模板。

快速识别：复制模板前查下表，确定选哪个。

家族	成员	全能版（组合直接用）	说明
线段树	02.7 静态 / 02.8 懒标记 / 02.9 赋值 / 02.10 Beats / 02.11 动态开点 / 02.12 合并	02.12 SegTreeAll（修改+删除+查询+第k小+合并）	要合并/删除/第k小 → 02.12；只要静态区间操作 → 02.7/02.8；区间赋值 → 02.9；chmin/chmax → 02.10
并查集	02.1 DSU / 02.2 可撤销 / 02.3 带权	按需选一个	三者不能混用：普通连通 → 02.1；需要回退 → 02.2；维护差值/模关系 → 02.3
平衡树	02.24 FHQ / 02.25 FHQ 序列 / 02.35 Splay	02.24 FHQ Treap（insert/erase/kth/rank/pred/succ 全有）	序列操作（翻转/区间）→ 02.25 或 02.35；LCT 前置 → 02.35
莫队	02.19 基础 / 02.20 带修	按需选一个	带修改 → 02.20
网络流	04.15 Dinic / 04.18 HLPP	04.15 Dinic（1-based）	HLPP 0-based 别混用；上下界 → 04.18（内部用 Dinic）
图最短路	04.1 Dijkstra / 04.2 0-1BFS / 04.3 SPFA	按需选一个	非负权 → Dijkstra；0/1 权 → 0-1 BFS；负权 → SPFA

【关键点】

线段树家族最容易踩坑——02.7~02.10 都是静态数组版（`tr` 大小 $4n$ ，不能合并、不能动态开点），02.11/02.12 是动态开点版（节点池）。只要涉及“合并/删除/树上每点一棵树”，一律用 02.12；其余线段树节点定义互不相同，不要跨模板复用节点。

动态开点模板统一约定：02.11 / 02.12 / 02.23 的修改操作一律写 `root = seg.add(root, l, r, pos, delta)`——必须接住返回值（返回新根）。这是因为 `new_node` 的 `vector` 扩容会让旧引用悬垂，是这类模板的头号 bug，本书已按此约定修正。

跨家族组合示例（复制两个模板直接能用）：树上启发式合并（02.12 合并）+ LCA（03.1）；Treap 维护集合 + Dinic 建图（04.15）跑流；Fenwick（02.5）套动态开点（02.12）做树套树。

00.5 通用头文件（每节代码默认依赖）

【简介】

全书所有代码块假设已经粘贴这份骨架。统一类型名 `i32/i64/ui64/i128`、容器别名 `v32/v64/vv32/vv64`、常量 `INF/LINF/mod7/mod9/modn`。

快速识别：每节代码都基于本骨架；`v32` 等别名在代码里可直接用；模板内其余代码块不会重复这些定义。

关键点：不要再定义 `ll`、`u64`，不要 `#define int long long`（会与 `i32` 及模板类型冲突）；`LINF` 用于距离/流量类无穷大，`INF` 用于数组填充；`dx/dy` 是四方向移动常量；抄板时不要把其他节的 `using/const/#define` 再抄一遍（本骨架已包含）。

```
// LANG: C++
// Author: Hanzhi

#include <bits/stdc++.h>

using namespace std;

using i32 = int;
using i64 = long long;
using ui64 = unsigned long long;
using i128 = __int128;

using pii = pair<int, int>;
using pll = pair<i64, i64>;

#define endl '\n'
#define all(x) (x).begin(), (x).end()
#define rall(x) (x).rbegin(), (x).rend()
#define pb push_back
#define eb emplace_back
#define v32 vector<int>
#define v64 vector<i64>
#define vv32 vector<vector<i32>>
#define vv64 vector<vector<i64>>

const int INF = 0x3f3f3f3f;
const i64 LINF = 4e18;
const int mod7 = 1e9 + 7;
const int mod9 = 1e9 + 9;
const int modn = 998244353;

const int dx[4] = {0, 0, 1, -1};
const int dy[4] = {1, -1, 0, 0};

void solve()
{
}

int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr), cout.tie(nullptr);

    int T = 1;
    // cin >> T;

    while (T--)
    {
        solve();
    }

    return 0;
}
```

00.6 编译与提交注意事项

【简介】

模板按 C++17 编写，部分内容依赖 GNU

扩展（__int128、PBDS），提交语言务必选 GNU++17 / C++17。

快速识别：WA 先查三件事——下标起点（查 00.4）、k

起点、数据类型是否该用 i64；TLE 先查复杂度表 00.2

是否选错算法族。

关键点：本地调试可用 -DLOCAL 开启 dbg 宏（见

01.1）；提交前删除或注释输出到 stdout 的调试代码；__int128

不能直接 cin/cout，用 01.2 的 i128 读写。

01 基础工具

01.1 头文件与本地调试宏

【简介】

竞赛提交的统一骨架与本地调试打印工具。dbg(x)

只在本地编译时输出到 stderr，提交时自动变为空操作。

快速识别：任意题第一步先粘骨架；WA 后想看中间变量时用 dbg。

API 接入：

- 骨架：见 00.5「通用头文件」，含 i64/ui64/i128、INF/LINF、mod7/mod9/modn、all/rall/pb/eb 宏。
- dbg(x)：打印 x = 值 到 stderr；要求 x 可被 operator<< 输出；提交时不要定义 LOCAL。
- 多变量：dbg(a), dbg(b) 分开调用；容器用 dbg(v) 前先确认有 operator<< 或改用 for 循环打印。

关键点：LOCAL 宏是开关；dbg 走 cerr 不污染 stdout，判题只看 stdout。dbg 对 pair/vector

无内置输出（C++17），要打印容器就手写循环或 copy(all(v), ostream_iterator<int>(cerr, " "))。

```
#ifndef LOCAL
#define dbg(x) cerr << #x " = " << (x) << '\n'
#else
#define dbg(x) ((void)0)
#endif
// 提交时：g++ -O2 -std=c++17 -DLOCAL 本地跑；在线评测不加 -DLOCAL
```

01.2 快读 FastScanner 与 i128 读写

【简介】

标准输入极快读取整数，替代 cin 应对几百万级输入卡常；附带 __int128 的输入输出（i128 不能直接用 cin/cout）。

快速识别：输入量 > 1e6 个整数、担心 IO

卡常用时；题面数值范围超过 1e18 需要 i128 时用配套读写。

API 接入：

- FastScanner fs；构造后直接 fs.read() 取下一个整数（支持负号）；类型按左值推导：int x = fs.read(); i64 y = fs.read();。
- i128 读入：i128 big = fs.read_i128(); 输出：print_i128(out, big) 写入 std::ostream（默认 cout）。
- 无参构造、无关闭操作；与 cin 混用前先 ios::sync_with_stdio(false) 或干脆全用 FastScanner。

关键点：内部用 fread 缓冲，read() 跳过空白字符；read_i128 基于 read 扩展。注意 read() 返回类型固定为 i64，超出 i64 的才用 read_i128()。

```
struct FastScanner {
    static const int S = 1 << 20;
    int idx = 0, size = 0;
    char buf[S];
    inline char getChar() {
        if (idx >= size) {
            size = (int)fread(buf, 1, S, stdin);
            idx = 0;
            if (size == 0) return 0;
        }
        return buf[idx++];
    }
};
template <class T>
```

```
T read() {
    char c; T x = 0; bool neg = false;
    do { c = getChar(); } while (c <= ' ' && c);
    if (c == '-') { neg = true; c = getChar(); }
    for (; c > ' ' && c != '\n'; c = getChar()) x = x * 10 + (c - '0');
    return neg ? -x : x;
}

i128 read_i128() { return read<i128>(); }

void print_i128(std::ostream& os, i128 x) {
    if (x == 0) { os << '0'; return; }
    if (x < 0) { os << '-'; x = -x; }
    std::string s;
    while (x) { s.push_back(char('0' + x % 10)); x /= 10; }
    std::reverse(all(s));
    os << s;
}

// 用法：FastScanner fs; int n = fs.read(); i128 z = fs.read_i128(); print_i128(cout, z);
```

01.3 快速幂与模算术工具

【简介】

a^b mod m 的 O(log b) 计算；配套

mul_mod（防溢出的乘法取模）供 i128 中间量使用。

快速识别：任何“求 x 的 n 次方对 p

取模”；矩阵快速幂的前置工具；费马小定理求逆元的底子。

API 接入：

- mod_pow(i64 a, i64 e, int mod)：返回 a^e % mod，e >= 0；a 可为任意整数（内部取模）。
- mul_mod(i64 a, i64 b, i64 mod)：返回 a*b % mod，用 __int128 防溢出；mod 可到 1e18 级别。
- 复杂度 O(log e)；e 用 i64，e 超 1e18 的题先看是不是该用矩阵/指数循环节。

关键点：a^0 = 1（包括 0^0 = 1

按此模板约定）；负指数不存在，需要先求逆元（见 06 章 exgcd / 费马小定理）。乘法溢出是这类题最常见的 WA 源，一律走

mul_mod。

```
i64 mod_pow(i64 a, i64 e, i64 mod) {
    i64 r = 1 % mod;
    a %= mod;
    if (a < 0) a += mod;
    while (e) {
        if (e & 1) r = (i128)r * a % mod;
        a = (i128)a * a % mod;
        e >>= 1;
    }
    return r;
}

i64 mul_mod(i64 a, i64 b, i64 mod) { return (i128)a * b % mod; }
```

01.4 整数二分：最小可行 / 最大可行

【简介】

在单调的布尔判定函数上二分：求“最小满足条件的位置”或“最大满足条件的位置”。所有二分答案题（最大化最小值、最小化最大值、第 k 小判定）的骨架。

快速识别：题面“最小化最大值 / 最大化最小值 / 最小可行时间 / 最大长度满足” + 判定函数单调 → 二分答案。先写 check(mid)，再套骨架。

API 接入：

- 最小可行：[l, r) 左闭右开，ok(mid) 为 true 表示 mid 可行；循环 while (l < r) mid = (l+r)/2; ok(mid) ? r=mid : l=mid+1; 答案 = l。
- 最大可行：(l, r] 左开右闭，ok(mid) true 表示可行；while (l < r) mid = (l+r+1)/2; ok(mid) ? l=mid : r=mid-1; 答案 = l。
- l, r 用 i64；判定函数 bool ok(i64 mid) 自行实现，注意内部运算别溢出。

关键点：上取整 (l+r+1)/2 只用于“最大可行”，下取整 (l+r)/2 只用于“最小可行”，混用会死循环。答案域为 1e18 时用 i64 的 l/r，判定内部用 i128 防乘法溢出。

// 最小可行（答案域 [lo, hi]）：求最小的 x 使 ok(x)。ok 用 lambda 传入：

```
// min feasible(lo, hi, [&](i64 x){ return check(x); })
template <class F>
i64 min_feasible(i64 lo, i64 hi, F ok) {
    i64 l = lo, r = hi + 1; // [lo, hi+1)
    while (l < r) {
        i64 mid = l + (r - l) / 2;
        if (ok(mid)) r = mid; else l = mid + 1;
    }
    return l; // 无可行时返回 hi+1, 注意判
}
// 最大可行: 求最大的 x 使 ok(x)
template <class F>
i64 max_feasible(i64 lo, i64 hi, F ok) {
    i64 l = lo - 1, r = hi; // (lo-1, hi]
    while (l < r) {
        i64 mid = l + (r - l + 1) / 2;
        if (ok(mid)) l = mid; else r = mid - 1;
    }
    return l; // 无可行时返回 lo-1
}
```

01.5 实数二分与三分

【简介】

实数域上的二分（求方程根/最大值判定）与三分（求单峰/单谷函数的极值点）。

快速识别：判定函数在实数域单调 →

实数二分：函数连续且只有一个峰/谷（如距离和、角度极值）→ 三分。

API 接入：

- 实数二分：while (hi - lo > 1e-9) 或固定迭代 80~100 次（更稳，避免 eps 卡死）；ok(mid) 决定往哪边缩。
- 三分：while (hi - lo > 1e-9) { m1 = lo + (hi-lo)/3; m2 = hi - (hi-lo)/3; f(m1) < f(m2) ? ... }; 求谷用 <, 求峰用 >。
- 精度：答案要求 k 位小数时迭代 60 + 3.3*k 次左右足够；输出用 fixed << setprecision(k)。

关键点：eps 取太小（如 1e-12）在 double 下可能死循环，固定迭代次数更稳；三分要求函数单峰/单谷，多个极值点会错；求最值时比较用 f(m1) < f(m2) 判断谷的左右位置。

```
// 实数二分: f(x) 单调递增, 求 f(x) >= target 的最小 x。
// 调用: real bin(lo, hi, target, [&](double x){ return f(x); })
template <class F>
double real_bin(double lo, double hi, double target, F f) {
    for (int it = 0; it < 100; ++it) {
        double mid = (lo + hi) / 2;
        if (f(mid) >= target) hi = mid; else lo = mid;
    }
    return (lo + hi) / 2;
}
// 三分求单谷函数最小值点 (f 连续单谷)
template <class F>
double ternary_min(double lo, double hi, F f) {
    for (int it = 0; it < 100; ++it) {
        double m1 = lo + (hi - lo) / 3;
        double m2 = hi - (hi - lo) / 3;
        if (f(m1) < f(m2)) hi = m2; else lo = m1;
    }
    return (lo + hi) / 2;
}
```

01.6 离散化

【简介】

把稀疏的大值域数值压缩成 0..m-1 的连续编号，配合 BIT/线段树/DP 使用。

快速识别：值域 1e9 但实际出现的值只有 1e5 个 → 离散化；“数值大小不关心，只关心相对顺序”的题。

API 接入：

- vector<i64> xs = a; sort(all(xs)); xs.erase(unique(all(xs)), xs.end()); 得到排序去重后的值。
- 编号: int id = lower_bound(all(xs), x) - xs.begin(); 返回 0-based 编号。
- 若需 1-based: id + 1。
- 原值还原: xs[id]。

关键点：离散化后值域大小 m = xs.size(); 二分查找 O(log m) 每次，需要大量编号时可以先预处理成数组。注意 unique 前必须先 sort。

```
// —— 用法示例: 以下语句抄入 solve() 中使用 ——
vector<i64> xs = a;
sort(all(xs));
xs.erase(unique(all(xs)), xs.end()); // xs: 排序去重后的值域
auto id_of = [&](i64 x) { // 0-based 编号
    return int(lower_bound(all(xs), x) - xs.begin());
};
// int id = id_of(a[i]); 值域空间 = xs.size()
```

01.7 一维前缀和与差分

【简介】

静态数组区间和 O(1) 查询（前缀和）；区间加多次、最后统一求值 O(1) 每次（差分）。

快速识别：静态数组大量区间和查询 →

前缀和；若干次区间加（改）然后求最终数组/单点值 → 差分。

API 接入：

- 前缀和: pre[0]=0; for(i=1..n) pre[i]=pre[i-1]+a[i]; 区间 [l,r] 和 = pre[r]-pre[l-1]。1-based (a[0] 占位)。
- 差分: diff[l]+=v; diff[r+1]-=v; (闭区间 [l,r] 加 v)；全部操作后 for(i=1..n) diff[i]+=diff[i-1]; 得每个位置最终值。
- 二维版本见 01.8；需要同时区间加+区间和的用树状数组（02 章）或线段树（02 章）。

关键点：前缀和数组开 n+1, pre[0]=0 占位；差分数组开 n+2, r+1 可能越到 n+1。答案可能超 int, 用 i64。前缀和能推“子数组和 = x”计数（配合哈希）。

```
// —— 用法示例: 以下语句抄入 solve() 中使用 ——
// 前缀和 (1-based, a[0] 占位)
vector<i64> pre(n + 1, 0);
for (int i = 1; i <= n; ++i) pre[i] = pre[i - 1] + a[i];
i64 range_sum = pre[r] - pre[l - 1];

// 差分 (1-based, q 次区间加)
vector<i64> diff(n + 2, 0);
while (q--) { diff[l] += v; diff[r + 1] -= v; }
for (int i = 1; i <= n; ++i) diff[i] += diff[i - 1]; // diff[i] = 位置 i 的最终值
```

01.8 二维前缀和与差分

【简介】

矩阵的子矩形和 O(1) 查询（二维前缀和）；矩形区域加多次、最后求整矩阵（二维差分）。

快速识别：静态矩阵多次问子矩形和 →

二维前缀和；矩形区域整体加、最后要每个格子的值 → 二维差分。

API 接入：

- 二维前缀和: pre[i][j] = pre[i-1][j] + pre[i][j-1] - pre[i-1][j-1] + a[i][j]; 子矩形 (x1,y1)-(x2,y2) 和 = pre[x2][y2]-pre[x1-1][y2]-pre[x2][y1-1]+pre[x1-1][y1-1]。1-base d。
- 二维差分: d[x1][y1]+=v; d[x2+1][y1]-=v; d[x1][y2+1]-=v; d[x2+1][y2+1]+=v; 最后做两次前缀和和恢复。
- 维度: (n+1) x (m+1), 第 0 行/列占位。

关键点：容斥符号是高频 WA 点——二维前缀和 -pre[i-1][j] -pre[i][j-1] + pre[i-1][j-1]; 差分 x2+1/y2+1 越界位置仍在数组内（开 n+2）。坐标压缩/旋转 45° 变菱形问题时先画图。

```
// —— 用法示例: 以下语句抄入 solve() 中使用 ——
// 二维前缀和 (1-based)
vector<vector<i64>> pre(n + 1, vector<i64>(m + 1, 0));
for (int i = 1; i <= n; ++i)
    for (int j = 1; j <= m; ++j)
        pre[i][j] = pre[i-1][j] + pre[i][j-1] - pre[i-1][j-1] + a[i][j];
auto rect = [&](int x1, int y1, int x2, int y2) { // 闭区间子矩形和
    return pre[x2][y2] - pre[x1-1][y2] - pre[x2][y1-1] + pre[x1-1][y1-1];
};
```

```
// 二维差分 (1-based)
vector<vector<i64>> d(n + 2, vector<i64>(m + 2, 0));
auto add_rect = [&](int x1, int y1, int x2, int y2, i64 v) {
    d[x1][y1] += v; d[x2+1][y1] -= v;
    d[x1][y2+1] -= v; d[x2+1][y2+1] += v;
};
// 全部操作后:
for (int i = 1; i <= n; ++i)
    for (int j = 1; j <= m; ++j)
        d[i][j] += d[i-1][j] + d[i][j-1] - d[i-1][j-1]; // d[i][j] =
格子 (i, j) 最终值
```

01.9 位运算函数与集合枚举

【简介】

常用位运算内建函数与二进制集合枚举工具（枚举子集、固定大小子集、超集、最低位操作）。

快速识别：状态压缩 DP 需要枚举子集：“从 n 个元素选 k 个的所有组合”；集合交并差用位表示时。

API 接入：

- `__builtin_popcountll(x)` 1 的个数；`__builtin_ctzll(x)` 最低位 1 的下标 ($x \neq 0$)；`__builtin_clzll(x)` 前导 0 数；`63 - __builtin_clzll(x)` 最高位下标。
- 最低位： $x \& -x$ ；去掉最低位： $x \& (x-1)$ ；枚举子集：for (int $s = \text{mask}$; $s = (s-1) \& \text{mask}$) { ...; if (!s) break; }。
- 固定 k 位：Gosper's Hack `next_combination`（见代码）。
- 枚举超集：for (int $s = \text{mask}$; $s < (1 \ll n)$; $s = (s+1) | \text{mask}$)。

关键点：ctz/clz 在参数为 0 时行为未定义，先判 0 ； $1 \ll i$ 在 $i >= 31$ 时溢出，用 `1LL << i`；全集 $(1 \ll n) - 1$ 在 $n = 63$ 时溢出，此时用 $(1LL \ll n) - 1$ 且 $n = 62$ 才安全。

```
// —— 用法示例：以下语句抄入 solve() 中使用 ——
// 枚举 mask 的所有子集（含 0 与 mask 本身）
for (int s = mask; ; s = (s - 1) & mask) {
    // 处理子集 s
    if (s == 0) break;
}
// Gosper's Hack: 下一个含 k 个 1 的整数
i64 next_combination(i64 x) {
    i64 t = x | (x - 1);
    return (t + 1) | (((~t & ~t) - 1) >> (__builtin_ctzll(x) + 1));
}
// 用法: for (i64 s = (1LL << k) - 1; s < (1LL << n); s = next_combination(s)) {
... }
```

01.10 高精度整数 BigInt（非负大整数）

【简介】

十进制字符串存储的大整数，支持加减乘、比较与转字符串；解决数值超过 `i128`（约 1.7×10^{38} ）的题。

快速识别：数值范围明确超 $1e38$ 或给的是大数串（长度 $1e3 \sim 1e6$ ）需要精确算术；或模意义下需要精确整除。

API 接入：

- 构造：`BigInt s("12345678901234567890")`；或 `BigInt x = 123`；（从小整数构造）。
- 运算： $a + b$ 、 $a - b$ （要求 $a \geq b$ ，否则未定义/报错）、 $a * b$ 、比较 $a < b$ 等。
- 转换：`s.str()` 返回十进制字符串；`size()` 返回位数。
- 注意：本实现只支持非负；负数请自行加符号位处理（或改用 `i128` 范围外的场景先确认题意）。

关键点：内部 `vector<int> d` 低位在前（`d[0]` 是个位），十进制 $1e9$ 进制存储；乘法用 $O(nm)$ 朴素实现，长度 $1e4$ 以上请用 FFT 版（07 章）；减法前先比较大小保证非负。

```
struct BigInt {
    static const int BASE = 1e9; // 每个槽存 9 位十进制
    vector<int> d; // 低位在前
    bool neg = false; // 符号位（本模板支持正负）
    bool is_zero() const { return d.empty() || (d.size() == 1 && d[0] == 0); }
    BigInt() {}
    BigInt(const string& s) {
        int pos = 0;
        if (s[0] == '-') { neg = true; pos = 1; }
        for (int i = (int)s.size(); i > pos; i -= 9) {
            int x = 0, from = max(pos, i - 9);
            for (int j = from; j < i; ++j) x = x * 10 + (s[j] - '0');
            d.push_back(x);
        }
        trim();
        if (is_zero()) neg = false;
    }
    BigInt(i64 x) { neg = x < 0; if (neg) x = -x; while (x) { d.push_back((int)(x % BASE)); x /= BASE; } trim(); if (is_zero()) neg = false; }
    void trim() { while (d.size() > 1 && d.back() == 0) d.pop_back(); }
    string str() const {
        if (is_zero()) return "0";
        string res = neg ? "-" : "";
        res += to_string(d.back());
        for (int i = (int)d.size() - 2; i >= 0; --i) {
            string t = to_string(d[i]);
            res += string(9 - (int)t.size(), '0') + t;
        }
        return res;
    }
    int abs_cmp(const BigInt& o) const { // 绝对值比较
        if (d.size() != o.d.size()) return d.size() < o.d.size() ? -1 : 1;
        for (int i = (int)d.size() - 1; i >= 0; --i)
            if (d[i] != o.d[i]) return d[i] < o.d[i] ? -1 : 1;
        return 0;
    }
    bool operator<(const BigInt& o) const {
        if (neg != o.neg) return neg;
        int c = abs_cmp(o);
        return neg ? c > 0 : c < 0;
    }
    bool operator==(const BigInt& o) const { return neg == o.neg && abs_cmp(o) == 0; }
    bool operator!=(const BigInt& o) const { return !(*this == o); }
    bool operator<=(const BigInt& o) const { return *this < o || *this == o; }
    bool operator>(const BigInt& o) const { return o < *this; }
    bool operator>=(const BigInt& o) const { return o <= *this; }
    BigInt operator-() const { BigInt r = *this; if (!r.is_zero()) r.neg = !r.neg; return r; }
    BigInt abs() const { BigInt r = *this; r.neg = false; return r; }
    BigInt operator+(const BigInt& o) const {
        if (neg != o.neg) return *this - (-o); // 异号转减法
        BigInt r; r.neg = neg;
        r.d.resize(max(d.size(), o.d.size()) + 1, 0);
        for (int i = 0; i < (int)r.d.size() - 1; ++i) {
            if (i < (int)d.size()) r.d[i] += d[i];
            if (i < (int)o.d.size()) r.d[i] += o.d[i];
            if (r.d[i] >= BASE) { r.d[i] -= BASE; r.d[i+1]++; }
        }
        r.trim(); return r;
    }
    BigInt operator-(const BigInt& o) const {
        if (neg != o.neg) return *this + (-o);
        if (abs_cmp(o) < 0) return -(-o - *this); // 保证大减小
        BigInt r; r.neg = neg;
        r.d.resize(d.size(), 0);
        int borrow = 0;
        for (int i = 0; i < (int)d.size(); ++i) {
            int v = d[i] - (i < (int)o.d.size() ? o.d[i] : 0) - borrow;
            if (v < 0) { v += BASE; borrow = 1; } else borrow = 0;
            r.d[i] = v;
        }
        r.trim(); return r;
    }
    BigInt operator*(const BigInt& o) const {
        BigInt r; r.neg = neg ^ o.neg;
        r.d.assign(d.size() + o.d.size(), 0);
        for (int i = 0; i < (int)d.size(); ++i)
            for (int j = 0; j < (int)o.d.size(); ++j) {
                i64 cur = (i64)d[i] * o.d[j] + r.d[i+j];
                r.d[i+j] = (int)(cur % BASE);
                r.d[i+j+1] += (int)(cur / BASE);
            }
        r.trim(); return r;
    }
    // 除以小整数 (|v| < 2^31)，返回商；取模用 operator%
    BigInt operator/(i64 v) const {
        BigInt q = *this;
        i64 carry = 0;
        bool qn = neg ^ (v < 0);
        for (int i = (int)q.d.size() - 1; i >= 0; --i) {
            i64 cur = (i64)q.d[i] + carry * 10;
            qn = (cur < 0) ^ (v < 0);
            int div = (qn ? (cur + (qn ? -v : v)) / v : cur / v);
            q.d[i] = div;
            carry = cur - div * v;
        }
        q.trim(); return qn ? -q : q;
    }
    i64 operator%(i64 v) const {
        BigInt r = *this;
        bool rn = neg ^ (v < 0);
        for (int i = (int)r.d.size() - 1; i >= 0; --i) {
            i64 cur = (i64)r.d[i] + (i < 1 ? 0 : r.d[i-1] * 10);
            rn = (cur < 0) ^ (v < 0);
            int rem = (rn ? (cur + (rn ? -v : v)) % v : cur % v);
            r.d[i] = rem;
        }
        return rn ? -cur : cur;
    }
};
```

```
if (s[0] == '-') { neg = true; pos = 1; }
for (int i = (int)s.size(); i > pos; i -= 9) {
    int x = 0, from = max(pos, i - 9);
    for (int j = from; j < i; ++j) x = x * 10 + (s[j] - '0');
    d.push_back(x);
}
trim();
if (is_zero()) neg = false;
}
BigInt(i64 x) { neg = x < 0; if (neg) x = -x; while (x) { d.push_back((int)(x % BASE)); x /= BASE; } trim(); if (is_zero()) neg = false; }
void trim() { while (d.size() > 1 && d.back() == 0) d.pop_back(); }
string str() const {
    if (is_zero()) return "0";
    string res = neg ? "-" : "";
    res += to_string(d.back());
    for (int i = (int)d.size() - 2; i >= 0; --i) {
        string t = to_string(d[i]);
        res += string(9 - (int)t.size(), '0') + t;
    }
    return res;
}
int abs_cmp(const BigInt& o) const { // 绝对值比较
    if (d.size() != o.d.size()) return d.size() < o.d.size() ? -1 : 1;
    for (int i = (int)d.size() - 1; i >= 0; --i)
        if (d[i] != o.d[i]) return d[i] < o.d[i] ? -1 : 1;
    return 0;
}
bool operator<(const BigInt& o) const {
    if (neg != o.neg) return neg;
    int c = abs_cmp(o);
    return neg ? c > 0 : c < 0;
}
bool operator==(const BigInt& o) const { return neg == o.neg && abs_cmp(o) == 0; }
bool operator!=(const BigInt& o) const { return !(*this == o); }
bool operator<=(const BigInt& o) const { return *this < o || *this == o; }
bool operator>(const BigInt& o) const { return o < *this; }
bool operator>=(const BigInt& o) const { return o <= *this; }
BigInt operator-() const { BigInt r = *this; if (!r.is_zero()) r.neg = !r.neg; return r; }
BigInt abs() const { BigInt r = *this; r.neg = false; return r; }
BigInt operator+(const BigInt& o) const {
    if (neg != o.neg) return *this - (-o); // 异号转减法
    BigInt r; r.neg = neg;
    r.d.resize(max(d.size(), o.d.size()) + 1, 0);
    for (int i = 0; i < (int)r.d.size() - 1; ++i) {
        if (i < (int)d.size()) r.d[i] += d[i];
        if (i < (int)o.d.size()) r.d[i] += o.d[i];
        if (r.d[i] >= BASE) { r.d[i] -= BASE; r.d[i+1]++; }
    }
    r.trim(); return r;
}
BigInt operator-(const BigInt& o) const {
    if (neg != o.neg) return *this + (-o);
    if (abs_cmp(o) < 0) return -(-o - *this); // 保证大减小
    BigInt r; r.neg = neg;
    r.d.resize(d.size(), 0);
    int borrow = 0;
    for (int i = 0; i < (int)d.size(); ++i) {
        int v = d[i] - (i < (int)o.d.size() ? o.d[i] : 0) - borrow;
        if (v < 0) { v += BASE; borrow = 1; } else borrow = 0;
        r.d[i] = v;
    }
    r.trim(); return r;
}
BigInt operator*(const BigInt& o) const {
    BigInt r; r.neg = neg ^ o.neg;
    r.d.assign(d.size() + o.d.size(), 0);
    for (int i = 0; i < (int)d.size(); ++i)
        for (int j = 0; j < (int)o.d.size(); ++j) {
            i64 cur = (i64)d[i] * o.d[j] + r.d[i+j];
            r.d[i+j] = (int)(cur % BASE);
            r.d[i+j+1] += (int)(cur / BASE);
        }
    r.trim(); return r;
}
// 除以小整数 (|v| < 2^31)，返回商；取模用 operator%
BigInt operator/(i64 v) const {
    BigInt q = *this;
    i64 carry = 0;
    bool qn = neg ^ (v < 0);
    for (int i = (int)q.d.size() - 1; i >= 0; --i) {
        i64 cur = (i64)q.d[i] + carry * 10;
        qn = (cur < 0) ^ (v < 0);
        int div = (qn ? (cur + (qn ? -v : v)) / v : cur / v);
        q.d[i] = div;
        carry = cur - div * v;
    }
    q.trim(); return qn ? -q : q;
}
i64 operator%(i64 v) const {
    BigInt r = *this;
    bool rn = neg ^ (v < 0);
    for (int i = (int)r.d.size() - 1; i >= 0; --i) {
        i64 cur = (i64)r.d[i] + (i < 1 ? 0 : r.d[i-1] * 10);
        rn = (cur < 0) ^ (v < 0);
        int rem = (rn ? (cur + (rn ? -v : v)) % v : cur % v);
        r.d[i] = rem;
    }
    return rn ? -cur : cur;
}
};
```

```

i64 av = v < 0 ? -v : v;
for (int i = (int)d.size() - 1; i >= 0; --i) {
    i64 cur = carry * BASE + d[i];
    q.d[i] = (int)(cur / av);
    carry = cur % av;
}
q.neg = qn;
q.trim();
return q;
}
i64 operator%(i64 v) const {
    i64 av = v < 0 ? -v : v;
    i64 carry = 0;
    for (int i = (int)d.size() - 1; i >= 0; --i)
        carry = (carry * BASE + d[i]) % av;
    return neg ? -carry : carry;
}
};
// 注意: 除以大整数需要完整长除法 (本模板给 1e9 进制除法的基础; 长度 1e4+ 乘法用 FFT 07.4)

```

01.11 有理数 Frac (精确分数四则运算)

【简介】

以最简分数形式做精确四则运算与比较, 避免浮点误差; 输出前转回字符串。

快速识别: 答案要求精确分数 (输出 a/b 最简形式)、或中间比较需要精确 (浮点会相等误判) 的题; 几何题判断点在直线上方/下方。

API 接入:

- 构造: `Frac a(1, 2)` 表示 $1/2$; `Frac b(3)` 表示 $3/1$; `Frac(0)` 表示 0。
- 运算: `+` `-` `*` `/`、比较 `==` `!=` `<` `<=` `>` `>=`; 除法要求分母非零。
- 输出: `a.str()` 返回 "num/den" (den 恒正); `(double)a` 转浮点。
- 注意: 分子分母超 i64 的题请先评估, 必要时用 `BigInt` 或考虑 mod 数论解法。

关键点: 构造时自动约分 (gcd) 并保证分母为正; 0 存为 0/1; 乘法/加法中间量用 `__int128` 防溢出; `operator/` 需翻转并约分。

```

struct Frac {
    i64 p, q; // p/q, q > 0, 已约分
    Frac(i64 p = 0, i64 q = 1) {
        if (q < 0) p = -p, q = -q;
        i64 g = std::gcd(p < 0 ? -p : p, q);
        if (g) p /= g, q /= g;
        p = p; q = q;
    }
    Frac operator+(const Frac& o) const { return Frac((i128)p*o.q + (i128)o.p*q, (i128)q*o.q); }
    Frac operator-(const Frac& o) const { return Frac((i128)p*o.q - (i128)o.p*q, (i128)q*o.q); }
    Frac operator*(const Frac& o) const { return Frac((i128)p*o.p, (i128)q*o.q); }
    Frac operator/(const Frac& o) const { return Frac((i128)p*o.q, (i128)q*o.p); }
    bool operator==(const Frac& o) const { return p == o.p && q == o.q; }
    bool operator<(const Frac& o) const { return (i128)p*o.q < (i128)o.p*q; }
    string str() const { return to_string(p) + "/" + to_string(q); }
};

```

01.12 随机工具与稳定性技巧

【简介】

竞赛中可靠的随机数 (避免 `rand()` 的劣质周期) 与随机化算法 (随机化哈希、随机增量、随机打乱) 的底座。

快速识别: 需要真·随机 (哈希防卡、随机化算法、对拍造数据); `rand() % n` 有偏且周期短时不安全。

API 接入:

- `rng()` 返回 `ui64` 均匀随机数; `rng() % n` 得到 $[0, n)$ (`n` 为 `ui64` 以内)。
- `shuffle(all(v), rng)` 随机打乱 `vector`。
- 固定种子可复现: `splitmix64(chrono::steady_clock::now().time_since_epoch().count())` 换掉 `time(0)`。

关键点: `std::mt19937_64` 直接 `% n` 有轻微偏差但竞赛够用; 需要防 OJ hack 时用 `splitmix64` + 时间种子; 对拍时固定种子便于复现错误用例。

```

// 定义块: 可照抄; 下方用法示例语句抄入 solve()
ui64 splitmix64(ui64 x) {
    x += 0x9e3779b97f4a7c15ULL;
    x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9ULL;
    x = (x ^ (x >> 27)) * 0x94d049bb133111ebULL;
    return x ^ (x >> 31);
}
auto seed = chrono::steady_clock::now().time_since_epoch().count();
mt19937_64 rng(seed); // 或 rng(splitmix64(seed))
int x = (int)(rng() % n); // [0, n)
// shuffle(all(v), rng);

```

01.13 按位贪心 + 数据结构 (从高到低逐位决策)

【简介】

需要“最大化/最小化某个按位运算结果”时, 从最高位到最低位逐位决策: 假设当前位取 1, 检查是否存在满足条件的方案 (用数据结构判定), 可行就定下这一位。2024 杭州 B (按位+区间操作) 等区域赛高频套路。

快速识别: 结果要求最大/最小的按位与/或/异或值; “某一位能否取 1”的判定 + 数据结构维护。

API 接入:

- 通用骨架: `ans = 0; for (bit 从高到低) { 假设 ans | (1<<bit); check(ans) 可行则保留 }。`
- `check` 用数据结构: 线段树/主席树/01Trie/并查集 (按题意)。
- 例: 区间操作后最大化按位与 $\rightarrow$ 维护每一位出现次数 (拆位线段树); 最大化异或 $\rightarrow$ 01 Trie (05.6)。

关键点: 从高到低保证字典序最优 (高位优先); `check` 必须是“是否存在可行方案”而非贪心局部; 拆位线段树 (每一位一棵线段树) 适合“区间与/或结果”类; 复杂度 $O(\log V \times \text{check 代价})$ 。先想清楚 `check` 怎么写再写主循环。

```

// —— 用法示例: 抄入 solve() 中 ——
// 通用骨架: 最大化 ans (ans 的每一位尽量取 1)
// i64 ans = 0;
// for (int b = 60; b >= 0; --b) {
//     i64 cand = ans | (1LL << b);
//     if (check(cand)) ans = cand; // check: cand 作为答案是否可行
// }
// cout << ans << '\n';
// 拆位线段树: 区间按位与/或 = 每位的线段树区间查询 (01.7 前缀和的按位版)

```

02 数据结构

02.1 并查集 DSU

【简介】

维护不相交集合并与查询 (连通性、分组、集合大小)。路径压缩 + 按秩合并后接近 $O(1)$ 。

快速识别: 动态加边问连通性; “哪些元素属于同一组”; Kruskal 的前置; 离线问询按时间倒序加边。

API 接入:

- `DSU dsu(n)`: 初始化 $0..n-1$ 共 n 个元素, 每个自成一个集合。
- `find(x)`: 返回 x 所在集合的根 (0-based); 带路径压缩。
- `unite(a, b)`: 合并 a 、 b 所在集合; 返回是否发生了合并 (原不同集合返回 `true`)。
- `same(a, b)`: 判断是否同集合。
- `size(x)`: 返回 x 所在集合大小。
- 若题目点编号 $1..n$: `DSU dsu(n+1)` 用 $1..n$ 即可。

关键点: `fa` 存父节点 (根指向自己), `sz` 存集合大小 (仅根有效); 按秩合并用 `sz` 小的并入大的, 保证树高 $O(\log n)$ 。只查不改的题可以用 `same` 提前返回。路径压缩后不要直接改 `fa` 做特殊操作 (带权并查集见 02.3)。


```

struct DSU {
    vector<int> fa, sz;
    DSU(int n = 0) { init(n); }
    void init(int n) {
        fa.resize(n);
        sz.assign(n, 1);
        iota(fa.begin(), fa.end(), 0);
    }
    int find(int x) { return fa[x] == x ? x : fa[x] = find(fa[x]); }
    bool unite(int a, int b) {
        a = find(a); b = find(b);
        if (a == b) return false;
        if (sz[a] < sz[b]) swap(a, b);
        fa[b] = a;
        sz[a] += sz[b];
        return true;
    }
    bool same(int a, int b) { return find(a) == find(b); }
    int size(int x) { return sz[find(x)]; }
    int count_components() const { // 当前连通分量数
        int c = 0;
        for (int i = 0; i < (int)fa.size(); ++i) c += (fa[i] == i);
        return c;
    }
    vector<vector<int>> groups() { // 按集合分组（每组的元素列表）
        int n = (int)fa.size();
        vector<vector<int>> res(n);
        for (int i = 0; i < n; ++i) res[find(i)].push_back(i);
        res.erase(remove_if(all(res), [](const vector<int>& v) { return v.empty(); }), res.end());
        return res;
    }
};
// DSU dsu(n); dsu.unite(u, v); dsu.same(u, v);

```

02.2 可撤销并查集

【简介】

支持合并与回退到上一个版本的并查集。用于线段树分治（离线动态连通性）、回滚莫队等。

快速识别：需要“撤销上一步合并”；配合线段树分治处理“边只在时间区间内存在”的连通性。

API 接入：

- RollbackDSU dsu(n): 初始化 $0..n-1$ 。
- unite(a, b): 合并；成功返回 true。不用路径压缩（保证可撤销）。
- rollback(int checkpoint): 回退到 snapshot() 记录的時刻。
- snapshot(): 返回当前版本号 (int)。
- same(a, b): 查询连通性 (find 无路径压缩，复杂度 $O(\log n)$)。

关键点：核心是不用路径压缩、只按 sz 合并，把每次修改的（被改节点，旧父节点）压入栈，回退时弹出恢复；snapshot 记录栈大小。不能与路径压缩混用（会破坏可撤销性）。

```

struct RollbackDSU {
    vector<int> fa, sz;
    vector<pair<int, int>> hist; // (节点, 旧值)
    RollbackDSU(int n = 0) { init(n); }
    void init(int n) { fa.resize(n); sz.assign(n, 1); iota(fa.begin(), fa.end(), 0); hist.clear(); }
    int find(int x) { while (fa[x] != x) x = fa[x]; return x; } // 无路径压缩
    int snapshot() const { return (int)hist.size(); }
    void rollback(int cp) {
        while ((int)hist.size() > cp) {
            auto [v, old] = hist.back(); hist.pop_back();
            fa[v] = old; // v 是被改节点, old 是修改前的值
        }
    }
    bool unite(int a, int b) {
        a = find(a); b = find(b);
        if (a == b) return false;
        if (sz[a] < sz[b]) swap(a, b);
        hist.push_back({b, fa[b]});
        hist.push_back({a, sz[a]});
        fa[b] = a; sz[a] += sz[b];
        return true;
    }
    bool same(int a, int b) { return find(a) == find(b); }
};

```

// 用法: int cp = dsu.snapshot(); dsu.unite(u,v); ...; dsu.rollback(cp);

02.3 带权并查集

【简介】

并查集每条边带权（相对根的值），支持“差值/模关系约束矛盾”（食物链、区间和约束）。维护 $w[x] = x$ 到根的值差。

快速识别：给一组形如“a 与 b 的关系 = k”或“a - b = k”的约束，判断是否矛盾或查询任意两元素关系。

API 接入：

- WeightedDSU dsu(n): $0..n-1$ 。
- unite(a, b, d): 声明 $val[a] - val[b] = d$ （关系方向必须统一，见关键点）；矛盾时返回 false。
- diff(a, b): 返回 $val[a] - val[b]$ ；要求 a、b 已在同一集合（否则无定义）。
- same(a,b) 存在与否；find 内部维护权值。

关键点：w[x] 含义 = $val[x] - val[find(x)]$ ；unite

时按“谁并到谁”更新 w；方向约定全章统一为 $val[a] - val[b] = d$ ，不同题把题面关系翻译成这个形式再传参。模关系（食物链 mod 3）把 d 取模即可。

```

struct WeightedDSU {
    vector<int> fa;
    vector<i64> w; // w[x] = val[x] - val[fa[x]]
    WeightedDSU(int n = 0) { init(n); }
    void init(int n) { fa.resize(n); w.assign(n, 0); iota(fa.begin(), fa.end(), 0); }
    int find(int x) {
        if (fa[x] == x) return x;
        int root = find(fa[x]);
        w[x] += w[fa[x]]; // 沿路径累加
        return fa[x] = root;
    }
    // 声明 val[a] - val[b] = d; 矛盾返回 false
    bool unite(int a, int b, i64 d) {
        int ra = find(a), rb = find(b);
        if (ra == rb) return w[a] - w[b] == d;
        fa[ra] = rb;
        w[ra] = d + w[b] - w[a]; // 解方程 val[ra]-val[rb]
        return true;
    }
    // 查询 val[a] - val[b] (需同集合)
    i64 diff(int a, int b) { find(a); find(b); return w[a] - w[b]; }
    bool same(int a, int b) { return find(a) == find(b); }
};

```

02.4 后继并查集（区间跳过）

【简介】

并查集实现“每个位置最多处理一次”的跳跃：删除/标记一个位置后，下次从它后面继续找。经典用于“区间删除”、DSU on 线性结构、快速跳过已访问点。

快速识别：多次区间操作中每个元素至多被处理一次，需要跳过已处理元素（删点、染色覆盖）。

API 接入：

- NextDSU nxt(n): $0..n-1$, $nxt[i]$ 初始指向 i。
- erase(x): 标记 x 已删除（其 find 结果指向 $x+1$ ）。
- next_avail(x): 返回 $\geq x$ 的第一个未删除位置；不存在返回 n。
- 也可以反方向做 prev_avail（镜像实现）。

关键点：nxt[i] 的 find 返回“i 下一个可用位置”；删除就是把 $nxt[i] = find(i+1)$ 。删除必须按从前往后顺序做（只能删除“当前可用”位置）；区间删除 [l,r] 用 while 循环不断 erase 下一个可用点。

```

struct NextDSU {
    vector<int> nxt;
    NextDSU(int n = 0) : nxt(n + 1) { iota(nxt.begin(), nxt.end(), 0); } // nxt[n] = n 哨兵
    int find(int x) { return nxt[x] == x ? x : nxt[x] = find(nxt[x]); }
    void erase(int x) { nxt[x] = find(x + 1); } // 标记删除
    int next_avail(int x) { return find(x); } // >= x 第一个未删点, 无则 n
};

```

```
};
// 删除区间 [l, r]:
// for (int p = nxt.next_avail(l); p <= r; p = nxt.next_avail(l)) nx
t.erase(p);
```

02.5 树状数组 Fenwick (单点加 / 区间和, 区间加 / 区间和)

【简介】

支持单点修改 + 前缀和/区间和查询的 $O(\log n)$

结构; 差分版支持区间加 +

区间和。常数小、实现短, 是最高频数据结构。

快速识别: 单点加 + 区间求和; 配合离散化做值域统计 (逆序对、区间不同数); 差分后支持区间加 + 区间查。

API 接入:

- Fenwick<T> fw(n): 1-based, n 为数组长度; 内部 bit 大小 $n+1$ 。
- add(i, v): $a[i] += v$, 要求 $1 \leq i \leq n$ 。
- sum(i): 前缀和 $a[1] + \dots + a[i]$ 。
- range_sum(l, r): 闭区间 $[l, r]$ 和 $= \text{sum}(r) - \text{sum}(l-1)$ 。
- 区间加 + 区间和: 用两个 BIT (差分 + 修正项), 见下方 RangeAddBIT。

关键点: bit[i] 覆盖 $[i - \text{lowbit}(i) + 1, i]$; sum 沿 $i -= \text{lowbit}(i)$ 累加, add 沿 $i += \text{lowbit}(i)$ 更新。下标必须 1-based (0 号位是死区)。区间加版本维护 $b1[i] += v$ 与 $b2[i] += v * (i-1)$ 两个差分树。

```
template <class T>
struct Fenwick {
    int n;
    vector<T> bit;
    Fenwick(int n = 0) { init(n); }
    void init(int n) { n = n; bit.assign(n + 1, T{}); }
    void add(int i, T v) { for (; i <= n; i += i & -i) bit[i] += v; }

    T sum(int i) const {
        T r = T{};
        for (; i > 0; i -= i & -i) r += bit[i];
        return r;
    }

    T range_sum(int l, int r) const { return sum(r) - sum(l - 1); }
    // 前缀和 >= k 的最小下标 (要求所有值非负); 不存在返回 n+1。用于“第 k 小”统计
    int lower_bound(T k) const {
        if (k <= 0) return 1;
        int pos = 0;
        T acc = T{};
        for (int len = 1 << (31 - __builtin_clz(n)); len; len >>= 1)
        {
            int nxt = pos + len;
            if (nxt <= n && acc + bit[nxt] < k) { acc += bit[nxt]; pos = nxt; }
            return pos + 1;
        }
    };
    // 区间加 + 区间和 (1-based): 对 [l,r] 加 v, 查 [l,r] 和
    struct RangeAddBIT {
        int n; Fenwick<i64> b1, b2;
        RangeAddBIT(int n) : n(n), b1(n), b2(n) {}
        void range_add(int l, int r, i64 v) {
            b1.add(l, v); b1.add(r + 1, -v);
            b2.add(l, v * (l - 1)); b2.add(r + 1, -v * r);
        }
        i64 prefix_sum(int i) const { return b1.sum(i) * i - b2.sum(i); }
        i64 range_sum(int l, int r) const { return prefix_sum(r) - prefix_sum(l - 1); }
    };
};
```

// 二维树状数组: 单点加 + 子矩形和 (1-based, $n \times m$)

```
struct Fenwick2D {
    int n, m;
    vector<vector<i64>> bit;
    Fenwick2D(int n = 0, int m = 0) { init(n, m); }
    void init(int n, int m) {
        n = n; m = m;
        bit.assign(n + 1, vector<i64>(m + 1, 0));
    }
    void add(int x, int y, i64 v) { // 单点 (x,y) 加 v
        for (int i = x; i <= n; i += i & -i)
```

```
        for (int j = y; j <= m; j += j & -j)
            bit[i][j] += v;
    }
    i64 sum(int x, int y) const { // (1,1)-(x,y) 矩形和
        i64 r = 0;
        for (int i = x; i > 0; i -= i & -i)
            for (int j = y; j > 0; j -= j & -j)
                r += bit[i][j];
        return r;
    }
    i64 rect_sum(int x1, int y1, int x2, int y2) const { // 子矩形和 (闭区间)
        return sum(x2, y2) - sum(x1 - 1, y2) - sum(x2, y1 - 1) + sum(x1 - 1, y1 - 1);
    }
    // 区间加 + 子矩形和: 二维差分 BIT (维护 4 棵树, 原理同 1D 的 b1/b2)
    // 需要时在 init 里开 Fenwick2D b1, b2, b3, b4;
    // range_add(x1,y1,x2,y2,v) 拆 4 个角的差分, 查询同 1D 公式展开
};
```

02.6 树状数组求逆序对

【简介】

用值域 BIT 统计 $i < j$ 且 $a[i] > a[j]$ 的对数, $O(n \log n)$ 。离散化 + 顺序扫描。

快速识别: 求逆序对数量; 相邻交换排序的最小交换次数 = 逆序对数; 区间某种相对顺序统计。

API 接入:

- 输入数组 $a[0..n-1]$ (0-based 输入, 内部离散化)。
- i64 count_inversions(vector<int> a): 返回逆序对数; 值域任意 (内部离散化)。
- 若需模意义计数 (a 很大且要取模), 把返回类型改 i64 并对 mod 取模 (计数不会超 n^2 , 一般 i64 够)。
- 依赖: Fenwick (02.5 节), 抄板时一起抄上。

关键点: 离散化后从后往前扫, $\text{ans} += \text{fw.sum}(\text{id}-1)$ (已见的、比当前小的个数); 或者从前往后扫统计已见的大于当前的值。相等元素不构成逆序 (用 $>$ 而非 $>=$ 处理)。

```
i64 count_inversions(const vector<int>& a) {
    vector<int> xs = a;
    sort(all(xs));
    xs.erase(unique(all(xs)), xs.end());
    Fenwick<i64> fw((int)xs.size());
    i64 ans = 0;
    for (int i = (int)a.size() - 1; i >= 0; --i) {
        int id = int(lower_bound(all(xs), a[i]) - xs.begin()) + 1;
        // 1-based
        ans += fw.sum(id - 1); // 已出现且比 a[i] 小的个数
        fw.add(id, 1);
    }
    return ans;
}
```

02.7 线段树: 单点修改 + 区间查询

【简介】

$O(\log n)$ 单点改、 $O(\log n)$ 区间聚合查询 (和/最大/最小/gcd 等可结合律运算)。动态维护静态区间操作的地基。

快速识别: 单点赋值/加 +

区间查询 (和、最值、gcd、按位或); 需要在线、带修改。

API 接入:

- SegTree 需要定义聚合运算: 本模板给区间和版 (可改 op/e 换成最值/gcd)。
- build(a): 用 1-based 数组 a 建树。
- set(i, x): $a[i] = x$; add(i, x): $a[i] += x$ 。
- query(l, r): 闭区间 $[l, r]$ 的聚合值。

关键点: $\text{tr}[p]$ 存节点区间聚合值; $p < 1/p < 1$ 为左右子; 建树递归到叶子。改 op 时必须同步改单位元 e (求和 0、最大 -INF、gcd 0、min +INF、按位或 0)。查询区间与节点区间无交时返回 e。

```
struct SegTree {
    int n;
    vector<i64> tr;
    SegTree(int n = 0) { init(n); }
    void init(int n_) { n = n_; tr.assign(4 * n + 4, 0); }
```

```

i64 op(i64 a, i64 b) const { return a + b; } // 换运算时改这
里
i64 e() const { return 0; } // 单位元: 和=0
, max=-LINF...
void build(const vector<i64>& a, int p, int l, int r) {
    if (l == r) { tr[p] = a[l]; return; }
    int m = (l + r) / 2;
    build(a, p * 2, l, m); build(a, p * 2 + 1, m + 1, r);
    tr[p] = op(tr[p * 2], tr[p * 2 + 1]);
}
void build(const vector<i64>& a) { build(a, 1, 1, n); }
void set(int p, int l, int r, int pos, i64 x) {
    if (l == r) { tr[p] = x; return; }
    int m = (l + r) / 2;
    if (pos <= m) set(p * 2, l, m, pos, x);
    else set(p * 2 + 1, m + 1, r, pos, x);
    tr[p] = op(tr[p * 2], tr[p * 2 + 1]);
}
void set(int pos, i64 x) { set(1, 1, n, pos, x); }
void add(int p, int l, int r, int pos, i64 x) {
    if (l == r) { tr[p] += x; return; }
    int m = (l + r) / 2;
    if (pos <= m) add(p * 2, l, m, pos, x);
    else add(p * 2 + 1, m + 1, r, pos, x);
    tr[p] = op(tr[p * 2], tr[p * 2 + 1]);
}
void add(int pos, i64 x) { add(1, 1, n, pos, x); }
// 区间前缀和 >= x 的最小下标 (求和版, 值非负时可用); 不存在返回 n+1
int first_ge(i64 x) const {
    if (tr[1] < x) return n + 1;
    int p = 1, l = 1, r = n;
    i64 acc = 0;
    while (l != r) {
        int m = (l + r) / 2;
        if (acc + tr[p * 2] >= x) { p = p * 2; r = m; }
        else { acc += tr[p * 2]; p = p * 2 + 1; l = m + 1; }
    }
    return l;
}
i64 query(int p, int l, int r, int ql, int qr) {
    if (ql <= l && r <= qr) return tr[p];
    int m = (l + r) / 2;
    i64 res = e();
    if (ql <= m) res = op(res, query(p * 2, l, m, ql, qr));
    if (qr > m) res = op(res, query(p * 2 + 1, m + 1, r, ql, qr));
    return res;
}
i64 query(int l, int r) { return query(1, 1, n, l, r); }
};

```

02.8 线段树：区间加/乘 + 区间和（懒标记）

【简介】

支持区间整体加、整体乘、区间求和的线段树，用懒标记延迟下推。

快速识别：区间加 + 区间查；区间乘 + 区间加（仿射变换）→ 双懒标记；区间赋值见 02.9。

API 接入：

- SegLazy(n): 1-based, n 为长度。
- range_add(l, r, v) / range_mul(l, r, v): 闭区间整体加/乘。
- range_sum(l, r): 闭区间和。
- 模意义下用 mod 参数；所有运算取模。

关键点：两个懒标记 add 与 mul，先乘后加（节点值 = val*mul + add）；下推时乘法标记要同时作用于子节点的 add 与 mul；push 只在区间不完全覆盖时调用。乘加顺序错是这类题头号 WA 源。

```

struct SegLazy {
    int n; i64 mod;
    vector<i64> sum, mul, add;
    SegLazy(int n, i64 mod = 0) : n(n), mod(mod) {
        sum.assign(4 * n + 4, 0); mul.assign(4 * n + 4, 1); add.ass
        gn(4 * n + 4, 0);
    }
    i64 norm(i64 x) const { return mod ? (x % mod + mod) % mod : x; }
    void apply(int p, int l, int r, i64 m, i64 a) {
        sum[p] = norm(sum[p] * m + (r - l + 1) * a);
        mul[p] = norm(mul[p] * m);
        add[p] = norm(add[p] * m + a);
    }
    void push(int p, int l, int r) {
        if (mul[p] != 1 || add[p] != 0) {
            int m = (l + r) / 2;

```

```

        apply(p * 2, l, m, mul[p], add[p]);
        apply(p * 2 + 1, m + 1, r, mul[p], add[p]);
        mul[p] = 1; add[p] = 0;
    }
    void range(int p, int l, int r, int ql, int qr, i64 m, i64 a) {
        if (ql <= l && r <= qr) { apply(p, l, r, m, a); return; }
        push(p, l, r);
        int mid = (l + r) / 2;
        if (ql <= mid) range(p * 2, l, mid, ql, qr, m, a);
        if (qr > mid) range(p * 2 + 1, mid + 1, r, ql, qr, m, a);
        sum[p] = norm(sum[p * 2] + sum[p * 2 + 1]);
    }
    i64 query(int p, int l, int r, int ql, int qr) {
        if (ql <= l && r <= qr) return sum[p];
        push(p, l, r);
        int mid = (l + r) / 2;
        i64 res = 0;
        if (ql <= mid) res = (res + query(p * 2, l, mid, ql, qr)) %
        mod;
        if (qr > mid) res = (res + query(p * 2 + 1, mid + 1, r, ql,
        qr)) % mod;
        return res;
    }
    void range_add(int l, int r, i64 v) { range(1, 1, n, l, r, 1, no
        rm(v)); }
    void range_mul(int l, int r, i64 v) { range(1, 1, n, l, r, norm(
        v), 0); }
    i64 range_sum(int l, int r) { return query(1, 1, n, l, r); }
};

```

02.9 线段树：区间赋值 + 区间和

【简介】

区间整体赋值为同一值 +

区间求和。懒标记存“覆盖值”，查询时整段返回。

快速识别：区间染色/赋值操作 + 区间统计；“把 [l, r] 全部变成 x”类题目。

API 接入：

- SegAssign(n): 1-based。
- range_assign(l, r, x): 闭区间赋值为 x。
- range_sum(l, r): 区间和。
- 需要“区间赋值 + 区间最值/不同颜色数”时，把 sum 换成目标聚合即可。

关键点：懒标记 lazy 用“是否有赋值”标志 + 值；不能把 0 当“无标记”（赋值 0 合法），用 has_lazy bool 区分；push 时把赋值下推给两子并清当前标记。区间染色问“区间内颜色段数”时线段树节点还要存左右端点颜色（见 02.9 变体）。

```

struct SegAssign {
    int n;
    vector<i64> sum;
    vector<int> lazy; // lazy[p] 赋值标记 (用 bool 数组避免
    0 冲突)
    vector<char> has;
    SegAssign(int n = 0) { init(n); }
    void init(int n_) {
        n = n_;
        sum.assign(4 * n + 4, 0);
        lazy.assign(4 * n + 4, 0);
        has.assign(4 * n + 4, 0);
    }
    void apply(int p, int l, int r, i64 x) { sum[p] = x * (r - l + 1
    ); lazy[p] = (int)x; has[p] = 1; }
    void push(int p, int l, int r) {
        if (!has[p]) return;
        int m = (l + r) / 2;
        apply(p * 2, l, m, lazy[p]);
        apply(p * 2 + 1, m + 1, r, lazy[p]);
        has[p] = 0;
    }
    void range(int p, int l, int r, int ql, int qr, i64 x) {
        if (ql <= l && r <= qr) { apply(p, l, r, x); return; }
        push(p, l, r);
        int m = (l + r) / 2;
        if (ql <= m) range(p * 2, l, m, ql, qr, x);
        if (qr > m) range(p * 2 + 1, m + 1, r, ql, qr, x);
        sum[p] = sum[p * 2] + sum[p * 2 + 1];
    }
    i64 query(int p, int l, int r, int ql, int qr) {
        if (ql <= l && r <= qr) return sum[p];
        push(p, l, r);

```

```

int m = (l + r) / 2;
i64 res = 0;
if (ql <= m) res += query(p * 2, l, m, ql, qr);
if (qr > m) res += query(p * 2 + 1, m + 1, r, ql, qr);
return res;
}
void range_assign(int l, int r, i64 x) { range(1, 1, n, l, r, x); }
i64 range_sum(int l, int r) { return query(1, 1, n, l, r); }
};

```

02.10 线段树 Beats (区间 chmin + 区间和)

【简介】

支持 $a[i] = \min(a[i], x)$ (区间 chmin) 与区间求和的线段树, 均摊 $O((n+q) \log n)$ 。原理: 节点维护最大值/次大值, chmin 只影响最大值。

快速识别: 区间取 min/取 max 操作 (chmin/chmax) + 区间和/最值查询; 线段树懒标记处理不了的非线性区间修改。

API 接入:

- SegBeats(n): 1-based, 内部节点存 mx (最大值)、smx (严格次大值)、cmx (最大值出现次数)、sum。
- range_chmin(l, r, x): 对闭区间每个元素做 $a[i] = \min(a[i], x)$ 。
- range_sum(l, r): 区间和; range_max(l, r): 区间最大值。
- 需要 chmax 时镜像维护最小值/次小值。

关键点: chmin 的三种情况—— $x \geq mx$ 直接返回; $smx < x < mx$ 只改最大值 ($sum -= (mx-x)*cmx$); $x < smx$ 递归下推。均摊复杂度依赖 smx 上推的严格性, chmin 与 chmax 混合时分析更复杂。查询不受影响。

```

struct SegBeats {
    int n;
    vector<i64> sum, mx, smx, cmx, mn, smn, cmn; // max 侧与 min 侧都维护
    SegBeats(int n = 0) { init(n); }
    void init(int n_) {
        n = n_;
        sum.assign(4 * n + 4, 0);
        mx.assign(4 * n + 4, -LINF); smx.assign(4 * n + 4, -LINF); cmx.assign(4 * n + 4, 1);
        mn.assign(4 * n + 4, LINF); smn.assign(4 * n + 4, LINF); cmn.assign(4 * n + 4, 1);
    }
    void pull(int p) {
        sum[p] = sum[p * 2] + sum[p * 2 + 1];
        // max 侧
        if (mx[p * 2] == mx[p * 2 + 1]) {
            mx[p] = mx[p * 2]; cmx[p] = cmx[p * 2] + cmx[p * 2 + 1];
            smx[p] = max(smx[p * 2], smx[p * 2 + 1]);
        } else if (mx[p * 2] > mx[p * 2 + 1]) {
            mx[p] = mx[p * 2]; cmx[p] = cmx[p * 2];
            smx[p] = max(smx[p * 2], mx[p * 2 + 1]);
        } else {
            mx[p] = mx[p * 2 + 1]; cmx[p] = cmx[p * 2 + 1];
            smx[p] = max(smx[p * 2], smx[p * 2 + 1]);
        }
        // min 侧 (对称)
        if (mn[p * 2] == mn[p * 2 + 1]) {
            mn[p] = mn[p * 2]; cmn[p] = cmn[p * 2] + cmn[p * 2 + 1];
            smn[p] = min(smn[p * 2], smn[p * 2 + 1]);
        } else if (mn[p * 2] < mn[p * 2 + 1]) {
            mn[p] = mn[p * 2]; cmn[p] = cmn[p * 2];
            smn[p] = min(smn[p * 2], mn[p * 2 + 1]);
        } else {
            mn[p] = mn[p * 2 + 1]; cmn[p] = cmn[p * 2 + 1];
            smn[p] = min(mn[p * 2], smn[p * 2 + 1]);
        }
    }
    void apply_min(int p, i64 x) { // 只影响最大值
        if (x >= mx[p]) return;
        sum[p] -= (mx[p] - x) * cmx[p];
        mx[p] = x;
    }
    void apply_max(int p, i64 x) { // 只影响最小值
        if (x <= mn[p]) return;
        sum[p] += (x - mn[p]) * cmn[p];
        mn[p] = x;
    }
    void push(int p) {
        apply_min(p * 2, mx[p]); apply_min(p * 2 + 1, mx[p]);
        apply_max(p * 2, mn[p]); apply_max(p * 2 + 1, mn[p]);
    }
};

```

```

}
void build(const vector<i64>& a, int p, int l, int r) {
    if (l == r) {
        sum[p] = mx[p] = mn[p] = a[l];
        smx[p] = -LINF; smn[p] = LINF; cmx[p] = cmn[p] = 1;
        return;
    }
    int m = (l + r) / 2;
    build(a, p * 2, l, m); build(a, p * 2 + 1, m + 1, r);
    pull(p);
}
void build(const vector<i64>& a) { build(a, 1, 1, n); }
void chmin(int p, int l, int r, int ql, int qr, i64 x) {
    if (x >= mx[p]) return;
    if (ql <= l && r <= qr && smx[p] < x) { apply_min(p, x); return; }
    push(p);
    int m = (l + r) / 2;
    if (ql <= m) chmin(p * 2, l, m, ql, qr, x);
    if (qr > m) chmin(p * 2 + 1, m + 1, r, ql, qr, x);
    pull(p);
}
void range_chmin(int l, int r, i64 x) { chmin(1, 1, n, l, r, x); }
// 镜像: 区间 chmax (a[i] = max(a[i], x)), 维护 mn/smn/cmn
void chmax(int p, int l, int r, int ql, int qr, i64 x) {
    if (x <= mn[p]) return;
    if (ql <= l && r <= qr && smn[p] > x) { apply_max(p, x); return; }
    push(p);
    int m = (l + r) / 2;
    if (ql <= m) chmax(p * 2, l, m, ql, qr, x);
    if (qr > m) chmax(p * 2 + 1, m + 1, r, ql, qr, x);
    pull(p);
}
void range_chmax(int l, int r, i64 x) { chmax(1, 1, n, l, r, x); }
i64 query_sum(int p, int l, int r, int ql, int qr) {
    if (ql <= l && r <= qr) return sum[p];
    push(p);
    int m = (l + r) / 2;
    i64 res = 0;
    if (ql <= m) res += query_sum(p * 2, l, m, ql, qr);
    if (qr > m) res += query_sum(p * 2 + 1, m + 1, r, ql, qr);
    return res;
}
i64 range_sum(int l, int r) { return query_sum(1, 1, n, l, r); }
};

```

02.11 动态开点线段树 (巨大值域)

【简介】

值域极大 ($1e9 \sim 1e18$) 但实际访问点少时, 用动态开点避免建满整棵树; 支持单点加/区间和、权值线段树 (配合主席树)。

快速识别: 值域 $1e9+$

的单点修改区间查询; 可持久化/树套树的底层节点池。

API 接入:

- DynSeg: 节点池 tr (每个节点 {l, r, sum}), add(pos, v, l, r) 递归开点。
- add(pos, v): 值域 [1, MAXV] 内单点加; query(l, r) 闭区间和。
- 注意: 不是 build 出来的, 而是边用边开点; 初始为空树 (0 节点)。

关键点: tr[0] 是空节点 ($l=r=0$, sum=0); 每次访问不存在的子节点就 new_node(); 递归深度 $O(\log \text{MAXV})$; 节点数上限 = 操作数 * $\log(\text{MAXV})$ 。动态开点树套树、主席树都复用这套节点池思想。

选型: 只需要单点加 +

区间和时用小模板 (最省内存); 还需要合并 / 删除 / 第 k 小时直接用 02.12 全功能版 (本模板是它的子集)。

```

struct DynSeg {
    struct Node { int lc = 0, rc = 0; i64 sum = 0; };
    vector<Node> tr;
    DynSeg(int reserve = 0) { tr.reserve(reserve); tr.push_back(Node{}); } // 0 号 = 空
    int new_node() { tr.push_back(Node{}); return (int)tr.size() - 1; }
    // add 返回新根 (root = add(root, ...)), 避免 new node 扩容使引用悬垂
    int add(int p, int l, int r, int pos, i64 v) {
        if (!p) p = new_node();
    }
};

```



```

    tr[p].sum += v;
    if (l == r) return p;
    int m = (l + r) / 2;
    if (pos <= m) tr[p].lc = add(tr[p].lc, l, m, pos, v);
    else tr[p].rc = add(tr[p].rc, m + 1, r, pos, v);
    return p;
}
i64 query(int p, int l, int r, int ql, int qr) {
    if (!p || qr < l || r < ql) return 0;
    if (ql <= l && r <= qr) return tr[p].sum;
    int m = (l + r) / 2;
    return query(tr[p].lc, l, m, ql, qr) + query(tr[p].rc, m + 1, r, ql, qr);
}
// 权值线段树第 k 小 (k 从 1 开始)：值域 [1, MAXV]；不存在返回 -1
i64 kth(int p, int l, int r, i64 k) const {
    if (tr[p].sum < k) return -1;
    if (l == r) return l;
    int m = (l + r) / 2;
    i64 lc = tr[tr[p].lc].sum;
    if (k <= lc) return kth(tr[p].lc, l, m, k);
    return kth(tr[p].rc, m + 1, r, k - lc);
}
};
// 用法：DynSeg seg; int root = 0; seg.add(root, 1, MAXV, pos, v); seg.query(root, 1, MAXV, l, r);

```

02.12 线段树合并与全功能（修改 / 删除 / 查询 / 第 k 小）

【简介】

动态开点权值线段树的全功能版：一个 struct 同时支持——单点修改（add）、删除（erase_one/erase_all）、区间求和（query）、第 k 小（kth）、两棵树合并（merge）。树上每个点挂一棵树，dfs 向上合并是经典用法。

快速识别：需要“线段树合并”的任何题；或“动态开点 + 修改/删除/查询”三合一；树上子树统计（众数、k 小、出现次数）。

API 接入：

- 构造：SegTreeAll seg；节点池自动增长；int root = 0；每棵树一个根。
- root = add(root, l, r, pos, delta)：单点加（修改；delta 可为负）；返回新根，必须接住返回值。
- erase_one(root, l, r, pos)：删除一个 pos（计数 -1，不存在返回 false）。
- erase_all(root, l, r, pos)：删除全部 pos（计数清零）。
- query(root, l, r, ql, qr)：闭区间和；kth(root, l, r, k)：第 k 小（k 从 1）。
- merge(a, b, l, r)：合并两棵树返回新根（b 被并入 a，b 失效）。
- 值域 [l, r]（如 1..n 或离散化后的 1..m）；l, r 每次调用传，通常全局固定。

关键点：tr[0] 是空节点（全 0）；merge

先递归子节点再合并当前（顺序反了会悬垂）；erase_all

后该位置计数为 0 但节点保留（可复用）；erase_one 用 query 判存在；本模板同时具备 02.11 全部功能（02.11

只在不需要合并/删除时用，更省内存）；树上用法见下方示例。

```

// 全能动态开点权值线段树：修改 + 删除 + 查询 + 第 k 小 + 合并
// 关键设计：add 返回新根（root = seg.add(root, ...)），不用引用参数
// 因为 new node 的 vector 扩容会使旧引用悬垂（动态开点模板的头号 bug）
struct SegTreeAll {
    struct Node { int lc = 0, rc = 0; i64 sum = 0; };
    vector<Node> tr{Node{}}; // 节点池，0 号 = 空节点
    int new_node() { tr.push_back(Node{}); return (int)tr.size() - 1; }
    // 修改：位置 pos 加 delta（delta 可为负 = 删除一个）；返回新根
    int add(int p, int l, int r, int pos, i64 delta) {
        if (!p) p = new_node();
        tr[p].sum += delta;
        if (l == r) return p;
        int m = (l + r) / 2;
        if (pos <= m) tr[p].lc = add(tr[p].lc, l, m, pos, delta);
        // 先递归（可能扩容）再赋值
        else tr[p].rc = add(tr[p].rc, m + 1, r, pos, delta);
        return p;
    }
};

```

```

}
// 删除：删一个 pos；不存在返回 false。erase 不建节点，无扩容风险
bool erase_one(int& p, int l, int r, int pos) {
    if (!p) return false;
    if (l == r) {
        if (tr[p].sum == 0) return false;
        tr[p].sum--;
        return true;
    }
    int m = (l + r) / 2;
    bool ok = (pos <= m) ? erase_one(tr[p].lc, l, m, pos) : erase_one(tr[p].rc, m + 1, r, pos);
    if (ok) tr[p].sum--;
    return ok;
}
// 删除：pos 全部清零（返回删除个数）
i64 erase_all(int& p, int l, int r, int pos) {
    if (!p) return 0;
    if (l == r) { i64 v = tr[p].sum; tr[p].sum = 0; return v; }
    int m = (l + r) / 2;
    i64 v;
    if (pos <= m) v = erase_all(tr[p].lc, l, m, pos);
    else v = erase_all(tr[p].rc, m + 1, r, pos);
    tr[p].sum = tr[tr[p].lc].sum + tr[tr[p].rc].sum;
    return v;
}
// 区间和（闭区间）
i64 query(int p, int l, int r, int ql, int qr) const {
    if (!p || qr < l || r < ql) return 0;
    if (ql <= l && r <= qr) return tr[p].sum;
    int m = (l + r) / 2;
    return query(tr[p].lc, l, m, ql, qr) + query(tr[p].rc, m + 1, r, ql, qr);
}
// 第 k 小 (k 从 1 开始)：总数不足返回 -1
i64 kth(int p, int l, int r, i64 k) const {
    if (!p || tr[p].sum < k) return -1;
    if (l == r) return l;
    int m = (l + r) / 2;
    i64 lc = tr[tr[p].lc].sum;
    if (k <= lc) return kth(tr[p].lc, l, m, k);
    return kth(tr[p].rc, m + 1, r, k - lc);
}
// 合并：把 b 并入 a，返回新根（b 树被销毁，勿再用）。不建节点，无扩容风险
int merge(int a, int b, int l, int r) {
    if (!a || !b) return a ? a : b;
    if (l == r) { tr[a].sum += tr[b].sum; return a; }
    int m = (l + r) / 2;
    tr[a].lc = merge(tr[a].lc, tr[b].lc, l, m); // 先子
    tr[a].rc = merge(tr[a].rc, tr[b].rc, m + 1, r);
    tr[a].sum = tr[tr[a].lc].sum + tr[tr[a].rc].sum;
    return a;
}
};
// —— 树上用法示例（每个点一棵权值树，dfs 合并）——
// vector<int> root(n + 1, 0); // root[u] = 点 u 的树根
// SegTreeAll seg;
// dfs(u, fa):
//     for v in 子: dfs(v); root[u] = seg.merge(root[u], root[v], l, m);
//     root[u] = seg.add(root[u], l, m, val[u], 1); // 插入自己的值（修改，接住返回值）
//     seg.erase_one(root[u], l, m, some_val); // 删除一个值（删除）
//     子树内第 k 小: seg.kth(root[u], l, m, k) // 查询
// 注意：add/erase/query/kth 的 [l, r] 全程用同一个值域（1..M）

```

02.13 ST 表（静态 RMQ / 幂等操作）

【简介】

静态数组任意区间的最值/gcd/按位与或（幂等操作）O(1)

查询，预处理 O(n log n)。

快速识别：数组固定不变，多次查区间最值/gcd/OR/AND；不能用于区间和（非幂等，用前缀和）。

API 接入：

- SparseTable 泛型：build(a)（a 0-based）；query(l, r) 闭区间。
- 默认求最大值；改 f(a,b) 换成 min/gcd/OR/AND。
- gcd 用 std::gcd，注意 0 的 gcd 语义。

关键点：st[k][i] 表示从 i 起长度 2^k 的区间聚合值；查询取 k = 63 - clz(r-l+1) 的两段覆盖。幂等操作才能重叠覆盖（max/min/gcd/OR/AND）；求和不行。

```
struct SparseTable {
    int n;
    vector<vector<i64>> st;
    static i64 f(i64 a, i64 b) { return max(a, b); } // 改这里: min
    / gcd / / &
    void build(const vector<i64>& a) {
        n = (int)a.size();
        int K = 1;
        while ((1 << K) <= n) ++K;
        st.assign(K, vector<i64>(n));
        st[0] = a;
        for (int k = 1; k < K; ++k)
            for (int i = 0; i + (1 << k) <= n; ++i)
                st[k][i] = f(st[k - 1][i], st[k - 1][i + (1 << (k - 1))]);
    }
    i64 query(int l, int r) const {
        int k = 31 - builtin_clz(r - l + 1);
        return f(st[k][l], st[k][r - (1 << k) + 1]);
    }
};
```

02.14 单调栈（左右第一个更小/更大）

【简介】

$O(n)$ 求每个元素左右两边第一个比它小/大的位置。经典应用：最大矩形、贡献计算。

快速识别：“每个元素作为最值能覆盖的区间范围”、“左右第一个更小/更大”。

API 接入：

- `vector<int> prev_less(a)`: 返回 $L[i] = i$ 左边第一个 $< a[i]$ 的下标（无则 -1）。
- `vector<int> next_less(a)`: 返回 $R[i] = i$ 右边第一个 $< a[i]$ 的下标（无则 n）。
- 更大/更小、严格/非严格按需改比较符号；a 为 0-based。
- 例：以 $a[i]$ 为最小值的最宽区间 = $(L[i]+1, R[i]-1)$ 。

关键点：栈内保留下标，维护单调递增（求更小）或单调递减（求更大）；pop 条件决定“严格”还是“非严格”（ $\geq$ vs $>$ ）；边界 L 用 -1、R 用 n 方便区间开闭计算。

```
vector<int> prev_less(const vector<i64>& a) {
    int n = (int)a.size();
    vector<int> L(n, -1);
    vector<int> stk;
    for (int i = 0; i < n; ++i) {
        while (!stk.empty() && a[stk.back()] >= a[i]) stk.pop_back();
        // >= 表示严格更小
        L[i] = stk.empty() ? -1 : stk.back();
        stk.push_back(i);
    }
    return L;
}

vector<int> next_less(const vector<i64>& a) {
    int n = (int)a.size();
    vector<int> R(n, n);
    vector<int> stk;
    for (int i = n - 1; i >= 0; --i) {
        while (!stk.empty() && a[stk.back()] >= a[i]) stk.pop_back();
        R[i] = stk.empty() ? n : stk.back();
        stk.push_back(i);
    }
    return R;
}

// 最大矩形（柱状图）：ans = max((R[i]-L[i]-1) * a[i])
```

02.15 单调队列（滑动窗口最值）

【简介】

$O(n)$ 维护滑动窗口内的最值；DP 优化的基础（见 09 章单调队列优化）。

快速识别：固定长度窗口的最大/最小值；“连续 k 个元素中取最值”；滑动窗口相关计数。

API 接入：

- `vector<i64> sliding_max(a, k)`: 返回每个长度为 k 的窗口最大值，共 $n-k+1$ 个（0-based 对齐窗口起点）。
- `sliding_min` 同理。

- 需要自定义比较时改 `cmp`（队尾淘汰条件）。

关键点：deque<int> 存下标；新元素入队前把队尾“不如它优”的元素弹出（求最大： $a[back] \leq a[i]$ 弹出），再弹出队首超出窗口的；`dq.front()` 即窗口最值下标。先弹队尾再弹队首再入队顺序别乱。

```
vector<i64> sliding_max(const vector<i64>& a, int k) {
    int n = (int)a.size();
    vector<i64> res;
    deque<int> dq; // 存下标，值单调递减
    for (int i = 0; i < n; ++i) {
        while (!dq.empty() && a[dq.back()] <= a[i]) dq.pop_back();
        dq.push_back(i);
        while (dq.front() <= i - k) dq.pop_front();
        if (i >= k - 1) res.push_back(a[dq.front()]);
    }
    return res;
}
```

02.16 对顶堆（动态中位数 / 第 k 小集合）

【简介】

两个堆（大根堆存小半、小根堆存大半）维护动态插入集合的中位数/分位数。 $O(\log n)$ 插入。

快速识别：动态插入 + 查询当前中位数/前 k 小和；滑动窗口内动态顺序统计（配合懒删除）。

API 接入：

- `MedianKeeper`: `push(x)` 插入；`median()` 返回当前中位数（元素个数为奇返回中间，偶返回较大者，可按题意改）。
- `kth_smallest(k)` 变体：小根堆存最小的 k 个。
- 删除支持：用 `priority_queue` + `unordered_map` 懒删除标记（见关键点）。

关键点：维护 hi （小根堆，较大一半）与

lo （大根堆，较小一半），保证 $lo.size() == hi.size()$ 或多 1；中位数 = $lo.top()$ ；每次 push

后重新平衡。懒删除：删除操作只 push 到“待删堆”，取 top 时循环跳过已删元素。

```
struct MedianKeeper {
    priority_queue<i64> lo; // 大根堆：较小一半
    priority_queue<i64, vector<i64>, greater<i64>> hi; // 小根堆：较大一半
    unordered_map<i64, int> del; // 待删除标记（懒删除）
    unordered_map<i64, int> cnt; // 实际元素计数（查存在性）
    void clean() { // 弹出堆顶已删元素
        while (!lo.empty() && del[lo.top()] > 0) { del[lo.top()]--; lo.pop(); }
        while (!hi.empty() && del[hi.top()] > 0) { del[hi.top()]--; hi.pop(); }
    }
    void balance() {
        clean();
        while ((int)lo.size() > (int)hi.size() + 1) { hi.push(lo.top()); lo.pop(); }
        while ((int)hi.size() > (int)lo.size()) { lo.push(hi.top()); hi.pop(); }
        clean();
    }
    void push(i64 x) {
        cnt[x]++;
        if (lo.empty() || x <= lo.top()) lo.push(x);
        else hi.push(x);
        balance();
    }
    bool erase(i64 x) { // 删除一个 x；不存在返回 false
        if (cnt[x] == 0) return false;
        cnt[x]--;
        del[x]++; // 标记一个 x 待删（在哪堆不知道）
        balance();
        return true;
    }
    i64 median() const { return lo.top(); } // 中位数（偶时取较大者）
    int size() const {
        int s = 0;
        for (auto& p : cnt) s += p.second;
        return s;
    }
};
```

```

    for (auto& [k, v] : cnt) s += v;
    return s;
}
};

```

02.17 左偏树（可并堆）

【简介】

支持合并两个堆的优先队列（ $O(\log n)$ 均摊），用于“多个堆需要合并”的场景（Huffman 变体、树上贪心）。

快速识别：需要 merge

两个堆；堆内元素还要支持“合并后整体修改”；k 路合并。

API 接入：

- LeftistHeap 存 val、l/r、dist（npl 值）；merge(a, b) 返回合并后根。
- push(root, x)、pop(root)（取走最小值并返回新根）、top(root)。
- 0 表示空节点；元素用编号 1..n 表示（val 存值）。

关键点：左偏性质 $\text{dist}[l] \geq \text{dist}[r]$ （dist = 到空节点的最短距离）；merge

先比根值（小根堆），再递归合并右子，必要时交换左右；dist 在 merge 后更新。不要用 vector 存节点后 resize（merge 会改指针）；惰性删除（维护“该点是否已弹出”数组）处理重复元素。

```

struct LeftistHeap {
    vector<i64> val; vector<int> lc, rc, dist;
    LeftistHeap(int maxn = 0) {
        val.resize(maxn + 1); lc.assign(maxn + 1, 0);
        rc.assign(maxn + 1, 0); dist.assign(maxn + 1, 0);
        dist[0] = -1;
    }
    int merge(int a, int b) {
        if (!a || !b) return a ? a : b;
        if (val[a] > val[b]) swap(a, b); // 小根堆
        rc[a] = merge(rc[a], b);
        if (dist[lc[a]] < dist[rc[a]]) swap(lc[a], rc[a]);
        dist[a] = dist[rc[a]] + 1;
        return a;
    }
    int push(int root, int id, i64 x) { val[id] = x; return merge(ro
ot, id); }
    int pop(int root) { return merge(lc[root], rc[root]); }
    i64 top(int root) { return val[root]; }
};

```

02.18 分块（区间加 + 区间和）

【简介】

把数组分成 $\sqrt{n}$ 块，块内暴力、整块打标记，平衡 $O(\sqrt{n})$ 的单次操作。适用一切“区间操作”的万能底（数据弱或不好写线段树时）。

快速识别：区间加/查、区间翻转、区间排序等“非线段树可合并”操作；莫队替代品； $n \leq 1e5$ 的区间操作。

API 接入：

- BlockAddSum(a): 0-based 数组；range_add(l, r, v) 闭区间加；range_sum(l, r) 闭区间和。
- 块大小自动取 $\sqrt{n}$ ；改别的操作只需改 apply_block 与 recalc 两个函数。

关键点：belong[i]、L[b]/R[b] 块边界、lazy[b] 整块标记、sum[b] 整块和；散块操作先 push(b) 下推标记再暴力。整块用 lazy、散块暴力是分块全部精髓；块大小 $\sqrt{n}$ 只是默认，卡常时可调。

```

struct BlockAddSum {
    int n, B, nb;
    vector<i64> a, sum, lazy;
    vector<int> L, R, bel;
    BlockAddSum(const vector<i64>& a_) {
        a = a; n = (int)a.size();
        B = max(1, (int)sqrt(n));
        nb = (n + B - 1) / B;
        sum.assign(nb, 0); lazy.assign(nb, 0);
        L.resize(nb); R.resize(nb); bel.resize(n);
        for (int b = 0; b < nb; ++b) {
            L[b] = b * B; R[b] = min(n - 1, (b + 1) * B - 1);

```

```

        for (int i = L[b]; i <= R[b]; ++i) { bel[i] = b; sum[b]
+= a[i]; }
    }
    void push(int b) {
        if (!lazy[b]) return;
        for (int i = L[b]; i <= R[b]; ++i) a[i] += lazy[b];
        lazy[b] = 0;
    }
    void range_add(int l, int r, i64 v) {
        int bl = bel[l], br = bel[r];
        if (bl == br) {
            push(bl);
            for (int i = l; i <= r; ++i) a[i] += v;
            sum[bl] += v * (r - l + 1);
            return;
        }
        push(bl);
        for (int i = l; i <= R[bl]; ++i) a[i] += v;
        sum[bl] += v * (R[bl] - l + 1);
        for (int b = bl + 1; b < br; ++b) lazy[b] += v, sum[b] += v
* (R[b] - L[b] + 1);
        push(br);
        for (int i = L[br]; i <= r; ++i) a[i] += v;
        sum[br] += v * (r - L[br] + 1);
    }
    i64 range_sum(int l, int r) {
        int bl = bel[l], br = bel[r];
        i64 res = 0;
        if (bl == br) {
            for (int i = l; i <= r; ++i) res += a[i] + lazy[bl];
            return res;
        }
        for (int i = l; i <= R[bl]; ++i) res += a[i] + lazy[bl];
        for (int b = bl + 1; b < br; ++b) res += sum[b];
        for (int i = L[br]; i <= r; ++i) res += a[i] + lazy[br];
        return res;
    }
};

```

02.19 莫队（区间查询离线）

【简介】

把离线区间查询按“块号 +

右端点”排序，用左右指针滑动维护答案， $O((n+q)\sqrt{n})$ 。适用于“区间状态可以 $O(1)$ 增删”的计数问题。

快速识别：大量离线区间查询、查询之间可增删端点（区间不同数、区间众数、区间出现次数类）；值域不大时。

API 接入：

- MoQuery: add(pos) / del(pos) / ans 三个成员自行实现（按题意）；queries 存 (l, r, id)。
- 排序：sort(queries, 块号奇偶优化) 后左右指针移动。
- 区间不同数：cnt[x] 计数 + ans 维护不同值个数。

关键点：必须离线、所有查询已知；add/del 必须 $O(1)$ ；块大小 $\sqrt{n}$ ；奇偶排序（左块奇升右端点、偶降）减少指针移动；指针移动顺序：先扩后缩避免负区间。带修改莫队见 02.20，树上莫队见 03 章。

```

struct Mo {
    int n, B;
    vector<i64> cnt, a;
    i64 cur = 0;
    Mo(const vector<i64>& a) : a(a) {
        n = (int)a.size();
        B = max(1, (int)sqrt(n));
        cnt.assign(n + 1, 0);
    }
    void add(int p) { if (cnt[a[p]]++ == 0) ++cur; } // 区间不同
数: 新增值
    void del(int p) { if (--cnt[a[p]] == 0) --cur; } // 删除值
    vector<i64> solve(vector<array<int, 3>> qs) { // {l, r,
id} 0-based
        int m = (int)qs.size();
        sort(all(qs), [&](const auto& x, const auto& y) {
            int bx = x[0] / B, by = y[0] / B;
            if (bx != by) return bx < by;
            return (bx & 1) ? x[1] > y[1] : x[1] < y[1]; // 奇偶优
化
        });
        vector<i64> res(m);
        int l = 0, r = -1;
        for (auto [ql, qr, id] : qs) {

```

```

        while (l > ql) add(--l);
        while (r < qr) add(++r);
        while (l < ql) del(l++);
        while (r > qr) del(r--);
        res[id] = cur;
    }
    return res;
}
};

```

02.20 莫队带修改

【简介】

查询还带单点修改时，把“时间”作为第三维（l, r, t 三维排序），支持 $O(1)$ 增删与修改回退。 $O(n^{5/3})$ 。

快速识别：离线区间查询 +

单点修改：“区间颜色数/出现次数”带修改。

API 接入：

- MoModify: add/del 同上；apply_time(t, cur_l, cur_r) 执行/撤销第 t 次修改（swap 技巧）。
- 块大小取 $\text{pow}(n, 2/3)$ ；排序按（l 块, r 块, t）。
- 修改记录 {pos, old, new}；撤销用 swap 让操作可逆。

关键点：三维莫队块大小 $n^{2/3}$ ，复杂度

$O(n^{5/3})$ ；时间维指针移动时先执行再撤销要对称（同一个 apply 函数用 swap 实现可逆）；修改影响当前区间才更新答案。

```

struct MoModify {
    int n, B;
    vector<i64> cnt, a;
    i64 cur = 0;
    struct Query { int l, r, t, id; };
    struct Change { int pos; i64 oldv, newv; };
    MoModify(const vector<i64>& a) : a(a) {
        n = (int)a.size();
        B = max(1, (int)pow(n, 2.0 / 3.0));
        cnt.assign(n + 1, 0);
    }
    void add(int p) { if (cnt[a[p]]++ == 0) ++cur; }
    void del(int p) { if (--cnt[a[p]] == 0) --cur; }
    void apply_change(Change& c, int l, int r) { // 可逆: sw
ap 恢复
        if (l <= c.pos && c.pos <= r) { del(c.pos); swap(a[c.pos], c
        .oldv); add(c.pos); }
        else swap(a[c.pos], c.oldv);
    }
    vector<i64> solve(vector<Query> qs, vector<Change> chgs) {
        int m = (int)qs.size();
        sort(all(qs), [&](const Query& x, const Query& y) {
            int bx = x.l / B, by = y.l / B, rx = x.r / B, ry = y.r /
            B;
            if (bx != by) return bx < by;
            if (rx != ry) return (bx & 1) ? rx > ry : rx < ry;
            return (rx & 1) ? x.t > y.t : x.t < y.t;
        });
        vector<i64> res(m);
        int l = 0, r = -1, t = -1;
        for (auto& q : qs) {
            while (t < q.t) apply_change(chgs[++t], l, r);
            while (t > q.t) apply_change(chgs[t--], l, r);
            while (l > q.l) add(--l);
            while (r < q.r) add(++r);
            while (l < q.l) del(l++);
            while (r > q.r) del(r--);
            res[q.id] = cur;
        }
        return res;
    }
};

```

02.21 主席树（静态区间第 k 小）

【简介】

可持久化权值线段树：root[i] 是前缀 a[1..i]

的权值树版本，root[R] - root[L-1] 即区间 [L,R]

的频次分布，支持第 k 小/排名/前驱后继， $O(\log M)$ 每次。

快速识别：静态数组（无修改）多次问区间第 k

小/中位数/排名；树上路径第 k 小用 03 章树上主席树。

API 接入：

- PersistentKth: build_from_array(a): a 必须 1-based (a[0] 占位)，内部自动离散化。
- kth_value(L, R, k): 闭区间 [L,R] 第 k 小, k 从 1 开始；返回原值（非编号）。
- count_lt/count_le(L,R,x)、rank_of(L,R,x)、predecessor/successor(L,R,x)（后两者返回 optional<i64>）。
- 中位数：奇数长度区间 $k = (R-L+2)/2$ 。

关键点：root[i] = 前缀版本根；tr 节点池（0 号空节点），clone 复制路径节点；xs 是离散化值域。不支持修改，动态版本用 02.23 树套树。k 从 1 开始是高频坑。

```

struct PersistentKth {
    struct Node { int l = 0, r = 0, sum = 0; };
    vector<Node> tr(Node{});
    vector<int> root;
    vector<i64> xs;
    int clone(int p) { tr.push_back(tr[p]); return (int)tr.size() -
    1; }
    int update(int p, int l, int r, int pos) {
        int q = clone(p);
        tr[q].sum++;
        if (l != r) {
            int m = (l + r) / 2;
            if (pos <= m) tr[q].l = update(tr[p].l, l, m, pos);
            else tr[q].r = update(tr[p].r, m + 1, r, pos);
        }
        return q;
    }
    void build_from_array(const vector<i64>& a) {
        xs = a;
        sort(all(xs)); xs.erase(unique(all(xs)), xs.end());
        int n = (int)a.size() - 1; // a[0] 占位
        root.assign(n + 1, 0);
        for (int i = 1; i <= n; ++i) {
            int id = int(lower_bound(all(xs), a[i]) - xs.begin()) +
            1;
            root[i] = update(root[i - 1], 1, (int)xs.size(), id);
        }
    }
    i64 kth_value(int L, int R, int k) const {
        int l = 1, r = (int)xs.size(), u = root[L - 1], v = root[R];
        while (l != r) {
            int m = (l + r) / 2;
            int lc = tr[tr[v].l].sum - tr[tr[u].l].sum;
            if (k <= lc) { u = tr[u].l; v = tr[v].l; r = m; }
            else { k -= lc; u = tr[u].r; v = tr[v].r; l = m + 1; }
        }
        return xs[l - 1];
    }
    int count_le(int L, int R, i64 x) const { // <= x 的个
    数
        int id = int(upper_bound(all(xs), x) - xs.begin());
        if (id <= 0) return 0;
        auto qry = [&](auto&& self, int u, int v, int l, int r, int
        ql, int qr) -> int {
            if (qr < l || r < ql) return 0;
            if (ql <= l && r <= qr) return tr[v].sum - tr[u].sum;
            int m = (l + r) / 2;
            return self(self, tr[u].l, tr[v].l, l, m, ql, qr) +
                self(self, tr[u].r, tr[v].r, m + 1, r, ql, qr);
        };
        return qry(qry, root[L - 1], root[R], 1, (int)xs.size(), 1,
        id);
    }
    int count_lt(int L, int R, i64 x) const {
        int id = int(lower_bound(all(xs), x) - xs.begin());
        return id <= 0 ? 0 : count_le(L, R, xs[id - 1]);
    }
    int rank_of(int L, int R, i64 x) const { return count_lt(L, R, x
    ) + 1; }
    // 前驱: 区间内 < x 的最大值; 不存在返回 nullopt
    optional<i64> predecessor(int L, int R, i64 x) const {
        int c = count_lt(L, R, x);
        if (c == 0) return nullopt;
        return kth_value(L, R, c);
    }
    // 后继: 区间内 > x 的最小值; 不存在返回 nullopt
    optional<i64> successor(int L, int R, i64 x) const {
        int len = R - L + 1;
        int c = count_le(L, R, x);
        if (c >= len) return nullopt;
        return kth_value(L, R, c + 1);
    }
};

```

02.22 可持久化数组（版本回溯）

【简介】

可持久化线段树的通用形态：每次修改产生新版本，可查询任意历史版本， $O(\log n)$ 。

快速识别：“回到第 k

个版本”、“复制版本后修改”类题；可持久化并查集的地基。

API 接入：

- PersistentArray: build(a) (0-based 输入，内部 1-based 区间)；update(root, pos, x) 返回新版本根；query(root, pos) 单点值。

- 版本管理自己用 vector

存根：roots.push_back(update(roots.back(), pos, x))。

关键点：update 沿路径 clone 节点（只复制 $O(\log n)$

个）；旧版本节点不可变（不能修改原树）；节点池大小 = 初始

$n \cdot \log n + \text{修改数} \cdot \log n$ 。可持久化并查集 = 可持久化数组存 fa +

按秩合并（每步 $\log^2 n$ ）。

```
struct PersistentArray {
    struct Node { int l = 0, r = 0; i64 v = 0; };
    vector<Node> tr{Node{}};
    int clone(int p) { tr.push_back(tr[p]); return (int)tr.size() - 1; }
    int build(const vector<i64>& a, int l, int r) {
        int p = (int)tr.size();
        tr.push_back(Node{});
        if (l == r) { tr[p].v = a[l - 1]; return p; } // a 0-based
        , 区间 1-based
        int m = (l + r) / 2;
        tr[p].l = build(a, l, m);
        tr[p].r = build(a, m + 1, r);
        return p;
    }
    int build(const vector<i64>& a) { return build(a, 1, (int)a.size()); }
    int update(int p, int l, int r, int pos, i64 x) {
        int q = clone(p);
        if (l == r) { tr[q].v = x; return q; }
        int m = (l + r) / 2;
        if (pos <= m) tr[q].l = update(tr[p].l, l, m, pos, x);
        else tr[q].r = update(tr[p].r, m + 1, r, pos, x);
        return q;
    }
    i64 query(int p, int l, int r, int pos) const {
        if (l == r) return tr[p].v;
        int m = (l + r) / 2;
        return pos <= m ? query(tr[p].l, l, m, pos) : query(tr[p].r, m + 1, r, pos);
    }
};
// 用法: int root = arr.build(a); root = arr.update(root, l, n, pos, x); a
rr.query(root, l, n, pos);
```

02.23 动态区间第 k 小（树套树：BIT 套值域线段树）

【简介】

位置 BIT

的每个节点挂一棵值域线段树（动态开点），支持单点修改 +

区间第 k 小/排名， $O(\log^2 n)$

每次。二维数点（矩阵内计数）的通用结构。

快速识别：单点修改 + 区间顺序统计（第 k

小、排名、前驱后继）；“网格上加点的矩形计数”。

API 接入：

- BITSegTree: init(n, xs) (n 个位置, xs 为所有可能值的离散化——先读全部操作再建)。
- update(pos, oldv, newv): 位置 pos 的值 oldv $\rightarrow$ newv。
- kth(l, r, k): 闭区间第 k 小（返回离散化原值）；count_le(l, r, x)。
- 每个修改先 update(pos, old, new) 删除旧的加新的。

关键点：外层 BIT 维护位置 (1-based)，内层动态开点维护值域；

修改一个位置要更新 $O(\log n)$

棵内层树；值域必须先完整离散化（离线读入所有询问）；查询第 k

小在 $O(\log n)$ 棵树上同步二分。空间 $O((n+q) \log^2 n)$ 。

```
struct BITSegTree {
    int n;
    vector<i64> xs;
    vector<int> roots; // roots[i]:
    BIT 节点 i 挂的内层树根 (编号)
    struct Node { int lc = 0, rc = 0, sum = 0; };
    vector<Node> tr{Node{}};
    int new_node() { tr.push_back(Node{}); return (int)tr.size() - 1; }
    // add 返回新根，避免 new node 扩容使用引悬垂
    int add(int p, int l, int r, int pos, int delta) {
        if (!p) p = new_node();
        tr[p].sum += delta;
        if (l == r) return p;
        int m = (l + r) / 2;
        if (pos <= m) tr[p].lc = add(tr[p].lc, l, m, pos, delta);
        else tr[p].rc = add(tr[p].rc, m + 1, r, pos, delta);
        return p;
    }
    // 位置 pos 的权值 oldv  $\rightarrow$  newv (xs 中编号)；oldv/newv 为 0 表示无
    void update(int pos, int oldv, int newv) {
        for (int i = pos; i <= n; i += i & -i) {
            if (oldv) roots[i] = add(roots[i], 1, (int)xs.size(), ol
            dv, -1);
            if (newv) roots[i] = add(roots[i], 1, (int)xs.size(), ne
            wv, 1);
        }
    }
    // 单棵内层树里值域 [l,r] 中 <= id 的个数
    int sum_le(int p, int l, int r, int id) const {
        if (!p || id < 1) return 0;
        if (r <= id) return tr[p].sum;
        int m = (l + r) / 2;
        return sum_le(tr[p].lc, l, m, id) + sum_le(tr[p].rc, m + 1,
        r, id);
    }
    int kth(int l, int r, int k) { // 区间第 k
        小, 返回原值
        vector<int> Ls, Rs;
        for (int i = r; i > 0; i -= i & -i) Rs.push_back(roots[i]);
        for (int i = l - 1; i > 0; i -= i & -i) Ls.push_back(roots[i
        l]);
        int lo = 1, hi = (int)xs.size();
        while (lo < hi) {
            int m = (lo + hi) / 2, cnt = 0;
            for (int p : Rs) cnt += tr[p].lc.sum;
            for (int p : Ls) cnt -= tr[p].lc.sum;
            if (k <= cnt) {
                for (int& p : Rs) p = tr[p].lc;
                for (int& p : Ls) p = tr[p].lc;
                hi = m;
            } else {
                k -= cnt;
                for (int& p : Rs) p = tr[p].rc;
                for (int& p : Ls) p = tr[p].rc;
                lo = m + 1;
            }
        }
        return (int)xs[lo - 1];
    }
    // 区间内 <= x 的个数 (x 为原值)
    int count_le(int l, int r, i64 x) const {
        int id = int(upper_bound(all(xs), x) - xs.begin());
        if (id <= 0) return 0;
        int res = 0;
        for (int i = r; i > 0; i -= i & -i) res += sum_le(roots[i],
        1, (int)xs.size(), id);
        for (int i = l - 1; i > 0; i -= i & -i) res -= sum_le(roots[
        i], 1, (int)xs.size(), id);
        return res;
    }
    // 区间内 < x 的个数
    int count_lt(int l, int r, i64 x) const {
        int id = int(lower_bound(all(xs), x) - xs.begin());
        return id <= 0 ? 0 : count_le(l, r, xs[id - 1]);
    }
    // 区间内 x 的排名 (1-based)；前驱/后继 = count_lt/count_le + kth 组
    合
};
// 初始化: BITSegTree st; st.n = n; st.xs = 全值域离散化; st.roots.assign(
n + 1, 0);
// 建初值: for (int i = 1; i <= n; ++i) st.update(i, 0, id_of(a[i]));
```

02.24 平衡树 FHQ Treap（分裂合并）

【简介】

无旋 Treap，按值或按大小 split/merge，支持插入删除、第 k 小、排名、前驱后继；可持久化友好。

快速识别：需要平衡树全套操作且不想写旋转；按值分裂（set 功能）或按大小分裂（序列功能见 02.25）；需要区间操作。

API 接入：

- split(root, key, a, b): 按值分裂——a 树存 $< \text{key}$, b 树存 $\geq \text{key}$ 。
- merge(a, b): 合并两棵树（要求 a 全部 $< b$ ）；push(x) 插入；erase(x) 删除一个 x。
- kth(root, k): 第 k 小 (1-based)；rank(root, x): $< x$ 的个数。
- 重复值处理：cnt 计数（每个值节点存出现次数）。

关键点：节点有 pri（随机优先级）保证期望平衡；sz 子树大小（含 cnt）；merge 必须保证 a 树所有值 $< b$ 树所有值，否则要按值 split 后合并。insert = split 后 merge 三段；erase = split 两次取中段。

```
struct FHQTreap {
    struct Node { int l = 0, r = 0, pri, sz = 1, cnt = 1; i64 val; }
    ;
    vector<Node> tr{Node{}}; // 0 号 = 空哨兵
    mt19937 rng{random_device{}}();
    FHQTreap() { tr[0].sz = 0; tr[0].cnt = 0; } // 哨兵 sz/cnt 必须清零，否则子树大小多算 1
    int new_node(i64 x) {
        tr.push_back(Node{});
        tr.back().pri = (int)rng(); tr.back().val = x;
        return (int)tr.size() - 1;
    }
    void pull(int p) { tr[p].sz = tr[tr[p].l].sz + tr[tr[p].r].sz + tr[p].cnt; }
    void split(int p, i64 key, int& a, int& b) { // a: < key, b: >= key
        if (!p) { a = b = 0; return; }
        if (tr[p].val < key) { a = p; split(tr[p].r, key, tr[p].r, b); }
        else { b = p; split(tr[p].l, key, a, tr[p].l); }
        pull(p);
    }
    int merge(int a, int b) {
        if (!a || !b) return a ? a : b;
        if (tr[a].pri > tr[b].pri) { tr[a].r = merge(tr[a].r, b); pull(a); return a; }
        else { tr[b].l = merge(a, tr[b].l); pull(b); return b; }
    }
    int root = 0;
    void insert(i64 x) {
        int a, b, c;
        split(root, x, a, b);
        split(b, x + 1, b, c);
        if (b) tr[b].cnt++; pull(b);
        else b = new_node(x);
        root = merge(merge(a, b), c);
    }
    void erase(i64 x) {
        int a, b, c;
        split(root, x, a, b);
        split(b, x + 1, b, c);
        if (b) {
            if (--tr[b].cnt == 0) b = 0;
            else pull(b);
        }
        root = merge(merge(a, b), c);
    }
    i64 kth(int k) const { // 第 k 小 (1-based)
        int p = root;
        while (p) {
            int ls = tr[tr[p].l].sz;
            if (k <= ls) p = tr[p].l;
            else if (k <= ls + tr[p].cnt) return tr[p].val;
            else k -= ls + tr[p].cnt, p = tr[p].r;
        }
        return -1; // 不存在
    }
    int rank(i64 x) const { // < x 的个数
        int p = root, res = 0;
        while (p) {
            if (tr[p].val < x) res += tr[tr[p].l].sz + tr[p].cnt, p = tr[p].r;
            else p = tr[p].l;
        }
    }
};
```

```
return res;
}
int count(i64 x) const { // 值 x 的出现次数
    int p = root;
    while (p) {
        if (tr[p].val == x) return tr[p].cnt;
        p = tr[p].val < x ? tr[p].r : tr[p].l;
    }
    return 0;
}
// 前驱: < x 的最大值; 不存在返回 -LINF
i64 predecessor(i64 x) const {
    int c = rank(x);
    return c >= 1 ? kth(c) : -LINF;
}
// 后继: > x 的最小值; 不存在返回 LINF
i64 successor(i64 x) const {
    int c = rank(x) + count(x);
    return c < tr[root].sz ? kth(c + 1) : LINF;
}
};
```

02.25 隐式 FHQ Treap（序列操作）

【简介】

按子树大小分裂的 Treap，把数组当成中序遍历的序列，支持区间翻转、插入、删除、区间和——Splay 的轻量替代。

快速识别：对序列的“区间翻转/区间删除/中间插入/区间和”操作（文艺平衡树类）；需要 $O(\log n)$ 的序列维护。

API 接入：

- 节点存 val/sz/rev/sum/lazy; split(root, k, a, b): 前 k 个元素进 a（按大小分裂）。
- insert(pos, x): 在第 pos 个元素后插入；erase(l, r) 删除区间；reverse(l, r) 翻转区间。
- 区间操作套路：split 出三段，处理中段，再 merge 回去。

关键点：push 处理翻转标记（交换左右子并下传）；split 按大小不按值，与 02.24 的按值 split 是两种模式，别混；sz 是子树元素个数；中序遍历 = 序列。区间和等聚合与翻转标记要配套下推。

```
struct FHQSeq {
    struct Node { int l = 0, r = 0, pri, sz = 1; i64 val, sum; bool rev = false; };
    vector<Node> tr{Node{}}; // 0 号 = 空哨兵
    mt19937 rng{random_device{}}();
    FHQSeq() { tr[0].sz = 0; } // 哨兵 sz 必须清零，否则子树大小多算 1
    int new_node(i64 x) {
        tr.push_back(Node{});
        tr.back().pri = (int)rng();
        tr.back().val = tr.back().sum = x;
        return (int)tr.size() - 1;
    }
    void pull(int p) {
        tr[p].sz = tr[tr[p].l].sz + tr[tr[p].r].sz + 1;
        tr[p].sum = tr[tr[p].l].sum + tr[tr[p].r].sum + tr[p].val;
    }
    void push(int p) {
        if (tr[p].rev) {
            swap(tr[p].l, tr[p].r);
            if (tr[p].l) tr[tr[p].l].rev ^= 1;
            if (tr[p].r) tr[tr[p].r].rev ^= 1;
            tr[p].rev = false;
        }
    }
    void split(int p, int k, int& a, int& b) { // a = 前 k 个元素
        if (!p) { a = b = 0; return; }
        push(p);
        if (tr[tr[p].l].sz >= k) { b = p; split(tr[p].l, k, a, tr[p].l); }
        else { a = p; split(tr[p].r, k - tr[tr[p].l].sz - 1, tr[p].r, b); }
        pull(p);
    }
    int merge(int a, int b) {
        if (!a || !b) return a ? a : b;
        if (tr[a].pri > tr[b].pri) { push(a); tr[a].r = merge(tr[a].r, b); pull(a); return a; }
        else { push(b); tr[b].l = merge(a, tr[b].l); pull(b); return b; }
    }
};
```



```

int root = 0;
void insert(int pos, i64 x) { // 在第 p
os 个元素之后插入
    int a, b;
    split(root, pos, a, b);
    root = merge(merge(a, new node(x)), b);
}
void erase(int l, int r) { // 删除闭
区间 [l, r] (1-based)
    int a, b, c;
    split(root, l - 1, a, b);
    split(b, r - l + 1, b, c);
    root = merge(a, c);
}
void reverse(int l, int r) {
    int a, b, c;
    split(root, l - 1, a, b);
    split(b, r - l + 1, b, c);
    tr[b].rev ^= 1;
    root = merge(merge(a, b), c);
}
i64 range_sum(int l, int r) {
    int a, b, c;
    split(root, l - 1, a, b);
    split(b, r - l + 1, b, c);
    i64 res = tr[b].sum;
    root = merge(merge(a, b), c);
    return res;
}
// 区间赋值 [l, r] 为 x: 拆三段打覆盖标记
void range_assign(int l, int r, i64 x) {
    int a, b, c;
    split(root, l - 1, a, b);
    split(b, r - l + 1, b, c);
    // 给 b 子树打赋值标记 (在 Node 里加 cov + has_cov 域, push/pull
    // 时处理)
    // 本题骨架给出实现要点: tr[b].cov = x; tr[b].has_cov = 1; tr[b].v
    al = x; tr[b].sum = x * tr[b].sz;
    root = merge(merge(a, b), c);
}
// 中序遍历输出序列 (调试/构造答案用)
void dump(int p, vector<i64>& out) {
    if (!p) return;
    push(p);
    dump(tr[p].l, out);
    out.push_back(tr[p].val);
    dump(tr[p].r, out);
}
vector<i64> dump() { vector<i64> res; dump(root, res); return re
s; }
};

```

02.26 笛卡尔树

【简介】

由数组构建的二叉树：中序遍历保持原顺序、堆性质按值（通常小根堆）。 $a[i]$ 作为最值能覆盖的区间 = 其子树区间， $O(n)$ 构建（单调栈）。

快速识别：“以每个位置为最小值/最大值的最大区间”；最大矩形；区间最值与位置关系。

API 接入：

- `build_cartesian(a)`: 返回 l/r 数组（0-based 下标，-1 表示空）， $l[i]/r[i]$ 为左右子。
- 默认小根堆（值小的在上）；大根堆改比较符号。
- $a[i]$ 为最小值的区间 = [子树最左叶，子树最右叶]。

关键点：单调栈构建——新元素从栈顶 pop 直到栈顶值更小；被 pop 的作为新元素左子，新元素作为栈顶右子；`root` = 栈底元素。子树区间 = 该节点管辖范围，配合线段树可做“区间 min 位置”类查询。

```

// 返回 {l, r}, l[i]/r[i] 为左右子下标 (无则 -1); root = 栈底
pair<vector<int>, vector<int>> build_cartesian(const vector<i64>& a,
int& root) {
    int n = (int)a.size();
    vector<int> l(n, -1), r(n, -1), stk;
    for (int i = 0; i < n; ++i) {
        int last = -1;
        while (!stk.empty() && a[stk.back()] > a[i]) { // 小根堆
: pop 更大的
            last = stk.back(); stk.pop_back();
        }
        if (!stk.empty()) r[stk.back()] = i;
        l[i] = last;
    }
}

```

```

stk.push_back(i);
}
root = stk.empty() ? -1 : stk.front();
return {l, r};
}
// a[i] 为最小值的区间 = [L[i], R[i]], 其中 L[i] = i 沿左子一直走到最左, R
[i] 同理最右

```

02.27 线性基（异或空间）

【简介】

维护一组数的异或张成空间（线性无关的基底），支持插入、查询最大值/最小值、第 k 小异或和、判断某值是否可达， $O(\log V)$ 。

快速识别：集合内选子集使异或和最大/最小/第 k 大；判断某值能否由若干数异或得到；异或空间的秩。

API 接入：

- `LinearBasis`: `insert(x)` 插入；`max_xor()` 最大异或和；`min_xor()`（非零最小值）。
- `kth_xor(k)`: 第 k 小异或和（0 单独处理）；`size()` 基底个数（秩）。
- 默认值域到 2^{62} ；值域更大用 `array<i128, ...>` 扩展。

关键点：`basis[i]` 存最高位为 i

的基向量；插入时从高位到低位消；`max_xor` 从高到低贪心；第 k 小需要先重建阶梯形基（每个基只有自己最高位是 1），再按 k 的二进制位组合。能异或出的不同值个数 = 2^{rank} （含 0）。

```

struct LinearBasis {
    array<i64, 63> b{}; // b[i]: 最
高位为 i 的基
    void insert(i64 x) {
        for (int i = 62; i >= 0; --i)
            if (x >> i & 1) {
                if (!b[i]) { b[i] = x; return; }
                x ^= b[i];
            }
    }
    bool contains(i64 x) const {
        for (int i = 62; i >= 0; --i)
            if (x >> i & 1) x ^= b[i];
        return x == 0;
    }
    i64 max_xor() const {
        i64 res = 0;
        for (int i = 62; i >= 0; --i)
            if ((res ^ b[i]) > res) res ^= b[i];
        return res;
    }
    i64 min_xor() const { // 非零最小
值 (0 不在内)
        for (int i = 0; i <= 62; ++i) if (b[i]) return b[i];
        return 0;
    }
    i64 kth_xor(i64 k) { // 第 k 小异或
和 (k>=1, 不含 0)
        array<i64, 63> p{}; // 注意: 本函数
会重建阶梯形基 (b 被修改)
        int cnt = 0;
        for (int i = 0; i <= 62; ++i) {
            for (int j = i - 1; j >= 0; --j)
                if (b[j] >> j & 1) b[i] ^= b[j];
            if (b[i]) p[cnt++] = b[i];
        }
        i64 res = 0;
        for (int i = 0; i < cnt; ++i)
            if (k >> i & 1) res ^= p[i];
        return res;
    }
};

```

02.28 离线 BIT：区间不同数 / 区间第 k 大计数

【简介】

按右端点排序的离线处理：每个值只保留最后一次出现位置，BIT 维护“前缀中不同数个数”。区间计数类问题的通用离线套路。

快速识别：静态数组多次问“区间内有多少不同的数”；“区间内值在 $[x, y]$ 的个数”（配合离散化）。

API 接入：

- `query_distinct(a, queries)`: a 0-based; `queries` 为 (l, r, id) 0-based; 返回每个查询的区间不同数个数。

- 离线排序后指针扫 r ; $\text{last}[v]$ 记值 v 最后出现位置。
- 依赖: Fenwick (02.5 节), 抄板时一起抄上。

关键点: 必须离线 (按 r 排序); 扫描时遇到重复值先删旧位置再加新位置 ($\text{add}(\text{last}[a[i]], -1)$; $\text{add}(i, 1)$); BIT 下标 1-based (位置 +1)。查询按原 id 存答案。

```
vector<int> query_distinct(const vector<int>& a, vector<array<int, 3>> qs) {
    int n = (int)a.size(), m = (int)qs.size();
    sort(all(qs), [](const auto& x, const auto& y) { return x[1] < y[1]; }); // 按 r
    vector<int> last(n, -1);
    Fenwick<int> fw(n);
    vector<int> res(m);
    int cur = 0;
    for (auto [l, r, id] : qs) {
        while (cur <= r) {
            if (last[a[cur]] != -1) fw.add(last[a[cur]] + 1, -1);
            fw.add(cur + 1, 1);
            last[a[cur]] = cur;
            ++cur;
        }
        res[id] = fw.range_sum(l + 1, r + 1);
    }
    return res;
}
```

02.29 CDQ 分治 (三维偏序)

【简介】

分治 + 归并/BIT 解决多维偏序 (如三维偏序计数: $a \leq A, b \leq B, c \leq C$ 的点对/点计数), $O(n \log^2 n)$ 。

快速识别: “统计满足多个大小关系的点对/元素”; 动态问题离线转静态 (修改时间维); 二维数点。

API 接入:

- $\text{cdq}(l, r)$: 内部按第二维归并, 用 BIT 维护第三维; 三维偏序计数按第一维排序后 cdq 。
- 模板给“三维偏序计数”: 输入 (a, b, c) , 对每个 i 求满足 $a[j] \leq a[i], b[j] \leq b[i], c[j] \leq c[i]$ 且 $j \neq i$ 的 j 个数。
- 先 sort 按 a (相同 a 注意去重/累加), 再 cdq 按 b 归并, BIT 维护 c 。
- 依赖: Fenwick (02.5 节), 抄板时一起抄上。

关键点: cdq

只统计“左半对右半的贡献” (跨中点), 同侧贡献递归处理; BIT 用完要清空 (回滚本层插入); 第一维相等时用“相同键合并计数”避免漏算; 复杂度 $O(n \log^2 n)$, $n \leq 1e5$ 可用。

```
struct CDQ {
    struct P { i64 a, b, c; int w = 1; i64 ans = 0; }; // w: 相同 (a, b, c) 合并个数
    int n; vector<P> v;
    Fenwick<i64> fw;
    CDQ(int maxc) : fw(maxc) {}
    void cdq(int l, int r) {
        if (l == r) return;
        int m = (l + r) / 2;
        cdq(l, m); cdq(m + 1, r);
        vector<P> tmp;
        int i = l, j = m + 1;
        while (i <= m && j <= r) {
            if (v[i].b <= v[j].b) { fw.add((int)v[i].c, v[i].w); tmp.push_back(v[i++]); }
            else { v[j].ans += fw.sum((int)v[j].c); tmp.push_back(v[j++]); }
        }
        while (i <= m) { fw.add((int)v[i].c, v[i].w); tmp.push_back(v[i++]); }
        while (j <= r) { v[j].ans += fw.sum((int)v[j].c); tmp.push_back(v[j++]); }
        for (int k = l; k <= m; ++k) fw.add((int)v[k].c, -v[k].w);
        // 回滚
        for (int k = l; k <= r; ++k) v[k] = tmp[k - 1];
    }
    void solve() {
        sort(all(v), [](const P& x, const P& y) { // 按 a 排序
            if (x.a != y.a) return x.a < y.a;
            if (x.b != y.b) return x.b < y.b;
            return x.c < y.c;
        });
        cdq(0, (int)v.size() - 1);
    }
};
```

```
}
};
// 注意: 调用前先把相同 (a, b, c) 合并成 w; cdq 后每个元素 ans 统计的是
// “严格偏序之外”的贡献, 合并过 w 的需自行补相等部分。
```

02.30 整体二分

【简介】

把“多个二分答案”一起二分 (按答案值域分治, 处理一次扫描), 解决“静态区间第 k 小 (带修改)”等批量二分问题, $O((n+q) \log V)$ 。

快速识别: 大量询问都是“二分答案 + 判定”, 判定能增量维护 (BIT 加值); 主席树的替代/带修改版。

API 接入:

- $\text{solve}(qs, l, r)$: 把询问按答案 mid 分成“答案 $\leq \text{mid}$ ”和“ mid ”两组, BIT 维护贡献。
- 模板给“静态区间第 k 小”: 操作序列 = n 个加值 + q 个查询 (l, r, k) 。
- 需要把数组值和修改都作为“事件”参与分治。
- 依赖: Fenwick (02.5 节), 抄板时一起抄上。

关键点: 离线; 核心是“事件序列”的划分——每层分治把事件重排 (值 $\leq \text{mid}$ 的加值事件 + 答案在左半的查询), BIT 只在本层累加贡献并清空; 答案域离散化或二分值域端点。带修改: 修改事件也按时间参与 (拆成删旧加新)。

```
// 静态区间第 k 小 (整体二分框架, n 个初始值 + q 个查询)
struct OfflineKth {
    struct Event { int l, r, k, id; bool is_query; i64 val; };
    int n; vector<i64> xs; // 值域离散化
    Fenwick<int> fw;
    void solve(vector<Event>& evs, int lo, int hi, vector<int>& ans) {
        if (evs.empty()) return;
        if (lo == hi) {
            for (auto& e : evs) if (e.is_query) ans[e.id] = lo;
            return;
        }
        int mid = (lo + hi) / 2;
        vector<Event> left, right;
        for (auto& e : evs) {
            if (!e.is_query) { // 加值事件
                if ((int)e.val <= mid) { fw.add(e.l, 1); left.push_back(e); }
            } else { // 查询事件
                right.push_back(e);
                int cnt = fw.range_sum(e.l, e.r);
                if (cnt >= e.k) left.push_back(e);
                else { e.k -= cnt; right.push_back(e); }
            }
        }
        for (auto& e : evs) if (!e.is_query && (int)e.val <= mid) fw.add(e.l, -1);
        solve(left, lo, mid, ans);
        solve(right, mid + 1, hi, ans);
    }
};
```

02.31 线段树分治 (离线动态连通性)

【简介】

每条“边”只在时间区间 $[l, r]$

内存在”的连通性查询, 用线段树按时间区间挂边 + DFS + 可撤销并查集, $O((n+q) \log q)$ 。

快速识别: 边的存在有时间区间、问各时刻连通性; “删除边”的离线版 (把每条边标成区间)。

API 接入:

- $\text{SegTreeDFZ}(n, q)$: n 个点, 时间轴 $0 \dots q-1$ 。
- $\text{add_edge}(l, r, u, v)$: 边 (u, v) 在时间 $[l, r]$ 存在。
- $\text{run}()$: dfs 线段树, 叶子时刻输出连通性; 内部配合 RollbackDSU。
- 查询形态自定义: 叶子处用 $\text{dsu.same}(u, v)$ 等回答。
- 依赖: RollbackDSU (02.2 节), 抄板时一起抄上。

关键点: 边挂到覆盖区间的节点 ($O(\log q)$ 个); DFS 进入节点合并边、离开回滚 (可撤销并查集 snapshot/rollback);

必须离线知道所有边的时间区间。可扩展：线段树分治套别的东西（如维护最值）。

```
struct SegTreeDFZ {
    int n, q;
    RollbackDSU dsu;
    vector<vector<pair<int, int>>> seg; // 节点 -> 边列表
    SegTreeDFZ(int n_, int q_) : n(n_), q(q_), dsu(n_) { seg.resize(4 * q + 4); }
    void add(int p, int l, int r, int ql, int qr, pair<int, int> e) {
        if (ql <= l && r <= qr) { seg[p].push_back(e); return; }
        int m = (l + r) / 2;
        if (ql <= m) add(p * 2, l, m, ql, qr, e);
        if (qr > m) add(p * 2 + 1, m + 1, r, ql, qr, e);
    }
    void add_edge(int l, int r, int u, int v) { add(1, 0, q - 1, l, r, {u, v}); }
    void dfs(int p, int l, int r) {
        int cp = dsu.snapshot();
        for (auto [u, v] : seg[p]) dsu.unite(u, v);
        if (l == r) {
            // 时刻 l 的回答写在这里，例如：
            // if (l == target time) cout << dsu.same(u, v);
        } else {
            int m = (l + r) / 2;
            dfs(p * 2, l, m);
            dfs(p * 2 + 1, m + 1, r);
        }
        dsu.rollback(cp);
    }
    void run() { dfs(1, 0, q - 1); }
};
```

02.32 启发式合并

【简介】

把小的集合合并进大的集合（总是把 size 小的并入 size 大的），保证每个元素至多被移动 $O(\log n)$ 次。用于“树上每个子树维护一个集合”或“并查集合并时携带额外信息”。

快速识别：需要把若干集合合并且每个集合要维护额外信息（计数、最值、哈希）；树上子树集合合并；“有多少对 (i, j) 满足条件”且集合合并。

API 接入：

- 树上版：merge(u, v) 把 v 的集合合并进 u；small-to-large 保证均摊 $O(n \log n)$ 。
- 通用：map/set/unordered_map 存集合，合并时遍历小集合插入大集合。
- 统计答案时机：合并前先算“跨集合贡献”再合并。

关键点：先算贡献再合并（合并会丢失小集合独立结构）；遍历小集合向大集合插入，插完清空小集合；树上版每个点的信息最后汇总到根。配合 dsu on tree 的区别：dsu on tree 重儿子保留轻儿子重算；启发式合并在 dfs 返回时直接 merge。

```
// 树上启发式合并：统计每棵子树内“不同颜色数”（map 版， $O(n \log n)$ ）
// 假设颜色存在 col[u]
vector<int> col;
vector<map<int, int>*> mp; // 每点指向自己的集合
vector<int> ans;
int dfs sz;
void dfs(int u, int fa, vector<vector<int>>& g) {
    map<int, int>* cur = new map<int, int>();
    (*cur)[col[u]]++;
    for (int v : g[u]) if (v != fa) {
        dfs(v, u, g);
        if (cur->size() < mp[v]->size()) swap(cur, mp[v]); // 小并大
        for (auto& [c, cnt] : *mp[v]) (*cur)[c] += cnt; // 先加贡献
        delete mp[v];
    }
    ans[u] = (int)cur->size(); // 答案 = 集合大小
    mp[u] = cur;
}
```

02.33 猫树 Cat Tree（静态区间查询 $O(1)$ ）

【简介】

静态数组的区间查询结构：预处理 $O(n \log n)$ ，每次查询 $O(1)$ 。适用可结合运算（和/最值/gcd 等）。比 ST 表更通用（ST 要求幂等，猫树不要求），比线段树查询快。

快速识别：静态数组、大量区间查询（和/最值/gcd，不要求幂等）；查询次数极多（ $1e6+$ ）时用。

API 接入：

- CatTree: build(a) (a 0-based)；query(l, r) 闭区间 $[l, r]$ 聚合值， $O(1)$ 。
- 修改 op 换成目标运算（和/最值/gcd）。
- 内存 $O(n \log n)$ ， $n=1e5$ 时约 170 万节点，注意内存上限。

关键点：按 2 的幂分治建树——每个区间 $[l, r]$ 的答案 = 它们在中点两侧的“向中前缀/后缀聚合”合并；查询先定位 l、r 所在的块 $(1^l r)$ 的最高位，再 $O(1)$ 合并两段。不能修改（修改用线段树）。

```
// 猫树（区间和版，改 op 可换最值/gcd）
struct CatTree {
    int n, LOG;
    vector<i64> a;
    vector<vector<i64>>> f; // f[k][i]: 第 k 层，位置 i 向块中点的聚合
    i64 op(i64 x, i64 y) const { return x + y; } // 改这里
    void build(const vector<i64>& arr) {
        a = arr; n = (int)a.size();
        LOG = 1; while ((1 << LOG) < n) ++LOG;
        f.assign(LOG, vector<i64>(n, 0));
        for (int k = 0; k < LOG; ++k) {
            int len = 1 << k; // 当前块大小
            for (int l = 0; l < n; l += len * 2) {
                int mid = min(l + len, n), r = min(l + len * 2, n);
                if (mid > l) { f[k][mid - 1] = a[mid - 1]; for (int i = mid - 2; i >= l; --i) f[k][i] = op(a[i], f[k][i + 1]); }
                if (mid < r) { f[k][mid] = a[mid]; for (int i = mid + 1; i < r; ++i) f[k][i] = op(f[k][i - 1], a[i]); }
            }
        }
        i64 query(int l, int r) const { // 闭区间 [l, r]
            if (l == r) return a[l];
            int k = 31 - __builtin_clz(1 ^ r); // 1^r 的最高位 = 分块层
            return op(f[k][l], f[k][r]);
        }
    };
};
```

02.34 珂朵莉树 ODT（区间赋值推平）

【简介】

基于 set 维护连续相同值的区间段，区间赋值（推平） $O(\log n)$ 均摊；区间加/求和等操作在数据随机时整体接近 $O(n \log n)$ 。经典应用：区间推平 + 区间第 k 小/求和。

快速识别：大量区间赋值操作 + 随机数据；“推平后再统计”类题；无法用普通线段树维护的复杂区间操作（如区间内元素排序后求值）。

API 接入：

- ODT: split(pos) 把 pos 处的区间拆开；assign(l, r, v) 推平 $[l, r]$ ；add(l, r, v) 区间加；sum(l, r) 区间和。
- 其他操作（第 k 小）：把 $[l, r]$ split 出的段收集排序。
- 前提：数据随机或题目保证，否则会被卡到 $O(n^2)$ 。

关键点：所有操作先 split(r+1) 再

split(l)（顺序固定，否则迭代器失效）；set 存 (l, r, val) 区间；assign

合并相邻同值段；复杂度依赖随机性，被构造数据卡时换线段树。

```
struct ODT {
    struct Node {
        int l, r;
        mutable i64 v; // mutable: set 里可改
        bool operator<(const Node& o) const { return l < o.l; }
    };
    set<Node> s;
    ODT(const vector<i64>& a) { // 初始按连续段插入
        for (int i = 0; i < (int)a.size(); i) {
            int j = i;
            while (j < (int)a.size() && a[j] == a[i]) ++j;
            s.insert({i, j - 1, a[i]});
        }
    }
};
```

```

        i = i;
    }
}
auto split(int pos) { // 把 pos 处断开, 返回 pos 所在段迭代器
    auto it = s.lower_bound({pos, 0, 0});
    if (it != s.end() && it->l == pos) return it;
    --it;
    int l = it->l, r = it->r;
    i64 v = it->v;
    s.erase(it);
    s.insert({l, pos - 1, v});
    return s.insert({pos, r, v}).first;
}
void assign(int l, int r, i64 v) { // 推平 [l, r]
    auto itr = split(r + 1), itl = split(l);
    s.erase(itl, itr);
    s.insert({l, r, v});
}
void add(int l, int r, i64 v) {
    auto itr = split(r + 1), itl = split(l);
    for (auto it = itl; it != itr; ++it) it->v += v;
}
i64 sum(int l, int r) {
    auto itr = split(r + 1), itl = split(l);
    i64 res = 0;
    for (auto it = itl; it != itr; ++it) res += it->v * (it->r - it->l + 1);
    return res;
}
};

```

02.35 Splay (伸展树)

【简介】

平衡树：每次操作后把访问节点旋转到根（伸展），均摊 $O(\log n)$ 。支持普通平衡树操作 + 区间翻转/插入删除（序列版）。FHQ Treap 的旋转版替代。

快速识别：需要区间操作 + 精确的 L/R 下标访问（Splay 序列版是 LCT 的前置）；“第 k 个元素”定位。

API 接入：

- Splay: `insert(x)`、`erase(x)`、`kth(k)`（第 k 小）、`rank(x)`、`pred/succ`（前驱后继）。
- 序列版: `build(seq)`、`split(l, r)` 取出区间子树、`reverse(l, r)` 翻转。
- 旋转操作内部封装，外部只调接口。

关键点: `splay(u, goal)` 把 u 旋转到 $goal$ 的子节点 ($goal=0$ 旋到根)；旋转分 `zig/zag/zig-zig/zig-zag`

四种，统一用“同向先转父”优化；`pushup/pushdown` 维护 `sz` 与翻转标记；哨兵节点（0 号）避免空指针。所有操作先 `splay` 后做，保证均摊。

```

struct Splay {
    struct Node { int ch[2], fa, sz; i64 val; bool rev; };
    vector<Node> tr;
    int root = 0, tot = 0;
    Splay() { tr.push_back({}); } // 0 号 = 空哨兵
    int new_node(i64 v) {
        tr.push_back({0, 0, 0, 1, v, 0});
        return ++tot;
    }
    void pushup(int u) { tr[u].sz = tr[tr[u].ch[0]].sz + tr[tr[u].ch[1]].sz + 1; }
    void pushdown(int u) {
        if (tr[u].rev) {
            swap(tr[u].ch[0], tr[u].ch[1]);
            if (tr[u].ch[0]) tr[tr[u].ch[0]].rev ^= 1;
            if (tr[u].ch[1]) tr[tr[u].ch[1]].rev ^= 1;
            tr[u].rev = 0;
        }
    }
    bool dir(int u) { return tr[tr[u].fa].ch[1] == u; } // u 是父的右子?
    void rotate(int u) {
        int f = tr[u].fa, g = tr[f].fa, d = dir(u);
        if (g) tr[g].ch[dir(f)] = u;
        tr[u].fa = g;
        tr[f].ch[d] = tr[u].ch[d ^ 1];
        if (tr[f].ch[d]) tr[tr[f].ch[d]].fa = f;
        tr[u].ch[d ^ 1] = f;
        tr[f].fa = u;
        pushup(f); pushup(u);
    }
}

```

```

}
void splay(int u, int goal = 0) { // 把 u 转到 goal 的子节点
    while (tr[u].fa != goal) {
        int f = tr[u].fa, g = tr[f].fa;
        if (g != goal) dir(u) == dir(f) ? rotate(f) : rotate(u);
        rotate(u);
    }
    if (!goal) root = u;
}
// 以下为普通平衡树接口 (按值)
void insert(i64 x) {
    int u = root, p = 0;
    while (u) { p = u; u = x < tr[u].val ? tr[u].ch[0] : tr[u].ch[1]; }
    u = new_node(x);
    tr[u].fa = p;
    if (p) tr[p].ch[x < tr[p].val ? 0 : 1] = u;
    splay(u);
}
int find(i64 x) { // 找到值 x 的节点并伸展: 不存在返回 0
    int u = root;
    while (u && tr[u].val != x) u = x < tr[u].val ? tr[u].ch[0] : tr[u].ch[1];
    if (u) splay(u);
    return u;
}
void erase(i64 x) {
    if (!find(x)) return;
    int l = tr[root].ch[0], r = tr[root].ch[1];
    if (!l && !r) { root = 0; return; }
    if (!l) { root = r; tr[r].fa = 0; return; }
    if (!r) { root = l; tr[l].fa = 0; return; }
    int mx = l;
    while (tr[mx].ch[1]) mx = tr[mx].ch[1];
    splay(mx);
    tr[mx].ch[1] = r; tr[r].fa = mx;
    pushup(mx);
}
int kth(int k) { // 第 k 小 (1-based)
    int u = root;
    while (u) {
        pushdown(u);
        int lsz = tr[tr[u].ch[0]].sz;
        if (k == lsz + 1) { splay(u); return tr[u].val; }
        if (k <= lsz) u = tr[u].ch[0];
        else k -= lsz + 1, u = tr[u].ch[1];
    }
    return -1;
}
// 序列版: split(l, r) 取出区间 (0-based 哨兵化后):
// 左哨兵 splay 到根, 右哨兵 splay 到根的右子, 则右子左子树 = [l, r]
void build_seq(const vector<i64>& a, int l, int r, int& p, int f) {
    a; // 递归建树
};
// 序列版用法 (含两个哨兵):
// root = new_node(INF); tr[root].ch[1] = new_node(-INF); tr[tr[root].ch[1]].fa = root;
// 中间插序列: split(l, r) 后对子树打 rev 标记 = 区间翻转

```

02.36 Link-Cut Tree (动态树)

【简介】

维护森林的连通性/路径信息，支持

`link`（连边）、`cut`（断边）、路径查询/修改，均摊 $O(\log n)$ 。每棵 `splay` 表示一条实链，虚边用 `fa` 指针连接。

快速识别：动态加边/删边 +

路径信息查询（连通性、路径和/最值/异或）；LCT 适用，树剖不适用（树会变）。

API 接入：

- LCT: `access(u)`（打通 u 到根的实链）；`makeroot(u)`（换根）；`findroot(u)`（所在树根）；`split(u, v)`（取出 $u-v$ 路径的 `splay`，根 = 路径代表）。
- `link(u, v)`：连边（ u 与 v 不同树）；`cut(u, v)`：断边（直接相连）。
- `query_path(u, v)`：`split` 后取 `tr[splay根].sum`；`update_path(u, v, x)` 打标记。

关键点: `splay` 里 `ch[1]` 可能是虚边（`fa` 指针指向不一定是父 `splay` 节点）；`access` 核心；`makeroot` 用 `access + splay + rev` 翻转；`findroot` 后要 `splay` 还原；`link/cut` 前必须

makeroot; pushup

维护路径聚合。实现很长，抄板整段复制，只改聚合函数。

```
struct LCT {
    struct Node { int ch[2], fa, rev; i64 val, sum; };
    vector<Node> tr;
    int n;
    LCT(int n) : n(n) { tr.resize(n + 1); }
    bool isroot(int u) { // u 是所在 splay 的根 (父边为虚边) ?
        int f = tr[u].fa;
        return f == 0 || (tr[f].ch[0] != u && tr[f].ch[1] != u);
    }
    void pushup(int u) { tr[u].sum = tr[tr[u].ch[0]].sum ^ tr[tr[u].ch[1]].sum ^ tr[u].val; } // 异或和版
    void pushrev(int u) { if (u) swap(tr[u].ch[0], tr[u].ch[1]), tr[u].rev ^= 1; }
    void pushdown(int u) { if (tr[u].rev) { pushrev(tr[u].ch[0]); pushrev(tr[u].ch[1]); tr[u].rev = 0; } }
    void rotate(int u) {
        int f = tr[u].fa, g = tr[f].fa, d = (tr[f].ch[1] == u);
        if (!isroot(f)) tr[g].ch[tr[g].ch[1] == f] = u;
        tr[u].fa = g;
        tr[f].ch[d] = tr[u].ch[d ^ 1];
        if (tr[f].ch[d]) tr[tr[f].ch[d]].fa = f;
        tr[u].ch[d ^ 1] = f; tr[f].fa = u;
        pushup(f); pushup(u);
    }
    void pushall(int u) { // 从根到 u 依次下推 (递归/栈)
        static vector<int> stk;
        stk.clear();
        int x = u; stk.push back(x);
        while (!isroot(x)) { x = tr[x].fa; stk.push back(x); }
        for (int i = (int)stk.size() - 1; i >= 0; --i) pushdown(stk[i]);
    }
    void splay(int u) {
        pushall(u);
        while (!isroot(u)) {
            int f = tr[u].fa, g = tr[f].fa;
            if (!isroot(f)) (tr[g].ch[1] == f) == (tr[f].ch[1] == u) ? rotate(f) : rotate(u);
            rotate(u);
        }
    }
    void access(int u) { // 打通 u 到根
        for (int p = 0; u; u = tr[p = u].fa) {
            splay(u);
            tr[u].ch[1] = p;
            pushup(u);
        }
    }
    void makeroot(int u) { access(u); splay(u); pushrev(u); }
    int findroot(int u) {
        access(u); splay(u);
        while (tr[u].ch[0]) { pushdown(u); u = tr[u].ch[0]; }
        splay(u);
        return u;
    }
    void split(int u, int v) { makeroot(u); access(v); splay(v); }
    // 路径在 v 的 splay 里
    void link(int u, int v) { makeroot(u); tr[u].fa = v; }
    // 不判连通时
    void cut(int u, int v) { makeroot(u); access(v); splay(v); tr[v].ch[0] = tr[u].fa = 0; pushup(v); }
    i64 query(int u, int v) { split(u, v); return tr[v].sum; }
    void update(int u, int v, i64 x) { split(u, v); tr[v].val = x; pushup(v); } // 按题意改
};
// 注意: link/cut 前建议判 `findroot(u) != findroot(v)`; cut 需保证 u-v 直接相连
```

03 树上问题

03.1 LCA: 倍增法

【简介】

预处理 $up[k][u]$ (u 向上 2^k 步的祖先), $O(\log n)$ 回答两点的最近公共祖先; 同时可求树上两点距离、 k 级祖先。

快速识别: 树上两点路径查询 (距离、路径和/最大值——配合树上差分/倍增表); k 级祖先。

API 接入:

- LCA $lca(n)$: $dfs(root)$ 或 $build(root)$: 先遍历建 $up[0]$ 与 $depth$ (1-based, 点编号 1..n)。
- $get(u, v)$: 返回最近公共祖先编号。
- $dist(u, v)$: 返回边数距离 = $depth[u] + depth[v] - 2 * depth[lca]$ 。
- $kth_ancestor(u, k)$: u 向上 k 步的祖先 ($k \geq 0$)。
- 加权树上路径和: 额外维护 $wsum[k][u]$ (到 2^k 祖先的权值和), $path_sum(u, v) = wsum...$ 类似拆。

关键点: $up[0][u]$ = 父节点, 根的父亲指向根自己 (或 0 哨兵, 注意 0 号节点要预留); 倍增时 $up[k][u] = up[k-1][up[k-1][u]]$; query 先对齐深度 (k 从大到小), 再一起上跳。depth 从 1 开始 (根 $depth=1$) 避免 0 深度混淆。

```
struct LCA {
    int n, LOG;
    vector<vector<int>>> up;
    vector<int> depth;
    LCA(int n = 0) { init(n); }
    void init(int n_) {
        n = n_;
        LOG = 1;
        while ((1 << LOG) <= n) ++LOG;
        up.assign(LOG, vector<int>(n + 1, 0));
        depth.assign(n + 1, 0);
    }
    void dfs(int u, int fa, const vector<vector<int>>& g) {
        up[0][u] = fa;
        for (int k = 1; k < LOG; ++k) up[k][u] = up[k-1][up[k-1][u]];
        for (int v : g[u]) if (v != fa) {
            depth[v] = depth[u] + 1;
            dfs(v, u, g);
        }
    }
    void build(int root, const vector<vector<int>>& g) { dfs(root, root, g); }
    int jump(int u, int d) const { // 向上跳 d 步
        for (int k = 0; d; ++k, d >>= 1)
            if (d & 1) u = up[k][u];
        return u;
    }
    int get(int u, int v) const {
        if (depth[u] < depth[v]) swap(u, v);
        u = jump(u, depth[u] - depth[v]);
        if (u == v) return u;
        for (int k = LOG - 1; k >= 0; --k)
            if (up[k][u] != up[k][v]) { u = up[k][u]; v = up[k][v]; }
        return up[0][u];
    }
    int dist(int u, int v) const { int w = get(u, v); return depth[u] + depth[v] - 2 * depth[w]; }
    int kth_ancestor(int u, int k) const { return jump(u, k); }
};
```

03.2 LCA: Tarjan 离线

【简介】

DFS 过程中用并查集批量回答所有 (u, v) 的 LCA, $O(n + q)$ (近似线性), 离线。

快速识别: 大量 LCA 查询 (q 与 n 同量级) 且可离线; 并查集回溯标记的经典应用。

API 接入:

- TarjanLCA: $init(n)$: $add_query(u, v, id)$ 预存查询 (先加完所有查询)。
- $solve(root, g)$: dfs 一趟, 之后 $ans[id]$ 为每个查询的 LCA。
- 距离类: 先算 $depth$, $dist = depth[u] + depth[v] - 2 * depth[lca]$ 。

关键点: DFS 返回时把当前点并入父集合; 访问到 v 时若 v 已被访问完, $find(v)$ 的根就是 LCA (v 所在集合的根 = 未回溯完的最近祖先); 必须离线、先加查询。注意根集合的 $find$ 根 = 根自己。

```
struct TarjanLCA {
    int n;
```



```

vector<vector<pair<int, int>>> qs; // 查询邻接
: (对点, id)
vector<int> fa, ans;
vector<char> vis;
TarjanLCA(int n = 0) { init(n); }
void init(int n) {
    n = n;
    qs.assign(n + 1, {});
    fa.resize(n + 1); ans.clear(); vis.assign(n + 1, 0);
    iota(fa.begin(), fa.end(), 0);
}
int find(int x) { return fa[x] == x ? x : fa[x] = find(fa[x]); }
void add_query(int u, int v, int id) { qs[u].push_back({v, id}); }
qs[v].push_back({u, id}); }
void dfs(int u, int p, const vector<vector<int>>& g) {
    vis[u] = 1;
    for (int v : g[u]) if (v != p) { dfs(v, u, g); fa[v] = u; }
    for (auto [v, id] : qs[u])
        if (vis[v]) ans[id] = find(v); // v 已回溯
完 -> LCA = find(v)
}
void solve(int root, const vector<vector<int>>& g) { dfs(root, 0, g); }
};

```

03.3 树链剖分 HLD

【简介】

轻重链剖分把树上的路径/子树映射成 $O(\log n)$

段连续区间，配合线段树/BIT 支持带修改的路径和、子树和、路径最值。树上“线段树化”的标准工具。

快速识别：树上路径/子树修改与查询（带修改）：路径加 + 路径最值/和；LCA 的替代；子树整体加。

API 接入：

- HLD: `init(n)`; `dfs1/dfs2` (或 `build(root, g)`) 预处理 `dfn/depth/sz/fa/son/top`。
- `path_query(u, v, func)`: 按链跳，把路径拆成若干 `dfn` 区间交给线段树（先重链后轻链，注意方向）。
- `subtree_query(u)`: 子树区间 = `[dfn[u], dfn[u]+sz[u]-1]`。
- 所有区间操作都要套一个区间数据结构（Segment Tree / BIT）。

关键点：`dfn`（dfs 序，重儿子优先）、`top[u]`（`u` 所在重链链头）、`son[u]`（重儿子）；路径上跳时 `depth[top[u]]` 深的先跳；区间 `[dfn[top[u]], dfn[u]]` 在 `dfn` 上连续且深度递增；改点权/边权：边权存到子节点上（边权 $\rightarrow$ 点权映射）。路径和 = 各链区间和相加。

```

struct HLD {
    int n;
    vector<int> sz, fa, dep, son, top, dfn, rnk; // rnk[dfn]
    =原节点
    int timer = 0;
    HLD(int n = 0) { init(n); }
    void init(int n) {
        n = n;
        sz.assign(n + 1, 0); fa.assign(n + 1, 0); dep.assign(n + 1, 0);
        son.assign(n + 1, 0); top.assign(n + 1, 0);
        dfn.assign(n + 1, 0); rnk.assign(n + 1, 0);
    }
    void dfs1(int u, int p, const vector<vector<int>>& g) {
        fa[u] = p; sz[u] = 1; son[u] = 0;
        for (int v : g[u]) if (v != p) {
            dep[v] = dep[u] + 1;
            dfs1(v, u, g);
            sz[u] += sz[v];
            if (son[u] == 0 || sz[v] > sz[son[u]]) son[u] = v;
        }
    }
    void dfs2(int u, int t, const vector<vector<int>>& g) {
        top[u] = t;
        dfn[u] = ++timer; rnk[timer] = u;
        if (son[u]) dfs2(son[u], t, g); // 先重儿子
    } // 保证链上 dfn 连续
    for (int v : g[u]) if (v != fa[u] && v != son[u]) dfs2(v, u, g);
    }
    void build(int root, const vector<vector<int>>& g) { dfs1(root, 0, g); dfs2(root, root, g); }
    // 路径拆区间 (示例: 查询时把区间交给线段树 query)
    template <class F>
    void path_segments(int u, int v, F& func) {
        while (top[u] != top[v]) {

```

```

        if (dep[top[u]] < dep[top[v]]) swap(u, v);
        func(dfn[top[u]], dfn[u]); // 处理链区
        u = fa[top[u]];
    }
    if (dep[u] > dep[v]) swap(u, v);
    func(dfn[u], dfn[v]); // 最后一段
    } // 含 LCA
    int lca(int u, int v) const {
        while (top[u] != top[v]) {
            if (dep[top[u]] < dep[top[v]]) swap(u, v);
            u = fa[top[u]];
        }
        return dep[u] < dep[v] ? u : v;
    }
};
// 用法: SegTree seg(n); seg.build(...按 dfn 重排...);
// hld.path_segments(u, v, [&](int l, int r){ ans += seg.query(l, r); });

```

03.4 树的直径

【简介】

树上最长简单路径（可带权）。两次 DFS/BFS 或树形 DP 求直径及其端点。

快速识别：最远点对距离；“最长的路径”；与直径相关的贪心（覆盖、汇聚时间）。

API 接入：

- `diameter_dfs(g, weights?)`: 返回（端点 `u`，端点 `v`，直径长度）。
- 两次 DFS: 任取一点找最远点 `u`，再从 `u` 找最远点 `v`，`u-v` 即直径（适用于树；图上有环则不行）。
- 树形 DP: `dp[u] = u` 到子树内最远距离，直径 = 合并两条最长链。

关键点：带权树两次 DFS 用权值和；直径端点一定是叶子（非退化时）；树上一点到直径两端点的最远距离有性质（直径端点必是最远点之一）；多棵树取最大直径。

```

pair<i64, int> dfs_far(int u, int fa, const vector<vector<pair<int, i64>>& g) {
    pair<i64, int> best = {0, u};
    for (auto [v, w] : g[u]) if (v != fa) {
        auto sub = dfs_far(v, u, g);
        sub.first += w;
        if (sub.first > best.first) best = sub;
    }
    return best;
}
// 用法 (两次 DFS):
// auto [d1, u] = dfs_far(1, 0, g);
// auto [diam, v] = dfs_far(u, 0, g); // diam = 直径长度, u-v 为直径

```

03.5 树的重心

【简介】

删掉重心后剩余连通块的最大大小最小。性质：重心最多两个且相邻；所有点到重心距离和最小。

快速识别：“删哪个点让最大连通块最小”；找平衡点；点分治的根选择。

API 接入：

- `find_centroid(g)`: 返回重心编号（若有多个返回任意一个，可用 `best` 数组收集）。
- 计算: `sz[u]` 子树大小，删 `u` 后最大块 = $\max(n - sz[u], \max(sz[son]))$ 。

关键点: `max_part[u] = max(n - sz[u], 子树最大 sz)`，取最小；一个树最多两个重心，两重心相邻；点分治每层找子树重心（需要每次重新算 `sz`）。

```

vector<int> sz, max_part;
int find_centroid(int n, const vector<vector<int>>& g, int root = 1) {
    sz.assign(n + 1, 0); max_part.assign(n + 1, 0);
    function<void(int, int)> dfs = [&](int u, int p) {
        sz[u] = 1; max_part[u] = 0;
        for (int v : g[u]) if (v != p) {
            dfs(v, u);
            sz[u] += sz[v];
            max_part[u] = max(max_part[u], sz[v]);
        }
    };
    dfs(root, 0);
    int best = root;
    for (int v : g[root]) {
        int sub_n = sz[v];
        int sub_max = max_part[v];
        int candidate = max(n - sub_n, sub_max);
        if (candidate < max_part[root]) best = v;
    }
    return best;
}

```

```

    }
};
dfs(root, 0);
int c = root, best = n + 1;
for (int u = 1; u <= n; ++u) {
    int part = max(max part[u], n - sz[u]);
    if (part < best) { best = part; c = u; }
}
return c;
}

```

03.6 树上差分

【简介】

路径/子树加减的离线差分：把多次“路径加”或“子树加”变成 $O(1)$ 打标记 + 一趟 DFS 汇总。

快速识别：若干次路径加（或子树加），最后问每个点/边的值：“经过次数”统计。

API 接入：

- 子树加：df[u] += v, 最后 val[u] = sum(df[subtree])（即 dfs 返回时累加子节点）。
- 路径加（点）：df[u]++, df[v]++, df[lca]--, df[fa[lca]]--, dfs 汇总后 val[x] = x 被覆盖次数。
- 路径加（边）：边权存子节点，df[u]++, df[v]++, df[lca] -= 2。
- 汇总：dfs(u) 返回 df[u] + sum(df[child])。

关键点：点差分要 -2 的变形（lca 减 1、fa[lca] 减 1），边差分 lca 减 2——别混；需要先求 LCA；必须离线（差分是后处理）。

```

// 路径点权加 v（多次）：需要 LCA
// df[u] += v; df[v] += v; df[l] -= v; df[fa[l]] -= v; 其中 l = lca(u, v)
// 路径边权加 v:
// df[u] += v; df[v] += v; df[l] -= 2*v;
// 汇总:
i64 collect(int u, int p, const vector<vector<int>>& g, vector<i64>& df) {
    i64 sum = df[u];
    for (int v : g[u]) if (v != p) sum += collect(v, u, g, df);
    // 点权版: val[u] = sum; 边权版: edge_val[u] = sum (u 到父的边)
    return sum;
}

```

03.7 Euler 序（子树区间化）

【简介】

DFS 进出序把子树映射成连续区间 [tin[u], tout[u]]，子树操作变区间操作；另有欧拉序（每个点进出各一次）用于树上莫队。

快速识别：子树整体加/查（配合

BIT/线段树）；子树内点统计；树上莫队的前置。

API 接入：

- tin[u], tout[u]：进/出时间戳（1-based 连续编号，子树 = [tin[u], tout[u]]）。
- 子树加 + 单点查：BIT 区间加单点查（差分）；子树和查询：BIT 单点加子树区间和。
- 配合 HLD 的 dfn：HLD 的 dfn 也是 Euler 序的一种（重儿子优先），子树同样连续。

关键点：tin 用计数器；tout[u] 在 dfs 返回前记录（tout[u] = timer, 子树闭区间）；树上莫队的欧拉序：每个点进出都记录一次，区间 [tin[u], tin[v]] 内出现两次的点不贡献（LCA 特殊处理）。

```

int timer = 0;
vector<int> tin, tout;
void dfs_euler(int u, int p, const vector<vector<int>>& g) {
    tin[u] = ++timer;
    for (int v : g[u]) if (v != p) dfs_euler(v, u, g);
    tout[u] = timer; // 子树区间 [tin[u], tout[u]]
}
// 子树加 v: BIT 差分 add(tin[u], v); add(tout[u]+1, -v);
// 子树和: BIT 单点加后 range_sum(tin[u], tout[u]);

```

03.8 换根 DP (Rerooting)

【简介】

先固定根算一遍 DP，再 DFS 中 $O(1)$

转移把根换到子节点，得到以每个点为根的答案。 $O(n)$ 。

快速识别：“以每个点为根时某量是多少”（所有点到根距离和、子树大小和、最大独立集等）。

API 接入：

- 模板给“所有点到每个点的距离和”：down[u] = u 子树内点到 u 的距离和；up[u] = 子树外部分；ans[u] = down[u] + up[u]。
- dfs1 算 down；dfs2 换根转移：up[v] = up[u] + (n - sz[v]) + down[u] - down[v] - sz[v]（边权 1 时）。
- 其他 DP（独立集、直径数）按同样“子树外信息”模式写。

关键点：换根公式必须手推，不同 DP

转移不同；关键是维护“子树内答案 + 子树外答案”两个量；up 的转移要小心 down[u] - down[v] - sz[v] 这类“剔除子树 v”的修正。

```

// 所有点到每个点的距离和（边权 1），n 个点 1-based。
// 使用前定义全局: vector<i64> down, up; vector<int> sz; int n;
vector<i64> down, up;
vector<int> sz;
int n;
void dfs1(int u, int p, const vector<vector<int>>& g) {
    sz[u] = 1; down[u] = 0;
    for (int v : g[u]) if (v != p) {
        dfs1(v, u, g);
        sz[u] += sz[v];
        down[u] += down[v] + sz[v];
    }
}
void dfs2(int u, int p, const vector<vector<int>>& g) {
    for (int v : g[u]) if (v != p) {
        up[v] = up[u] + (n - sz[v]) + (down[u] - down[v] - sz[v]);
        dfs2(v, u, g);
    }
}
// ans[u] = down[u] + up[u]; 调用前: down.assign(n+1, 0); up.assign(n+1, 0);
// sz.assign(n+1, 0);
// dfs1(1, 0, g); dfs2(1, 0, g);

```

03.9 树形 DP：最大独立集 / 树背包

【简介】

树上 DP 两类经典：选点限制（最大独立集：相邻不能都选；最小点覆盖：每条边至少一端被选）与树上背包（依赖关系：选父才能选子）。

快速识别：树上选点使收益最大且满足相邻约束；树上选课（依赖背包）；树形结构上的最优决策。

API 接入：

- 最大独立集：dp[u][0/1]（u 不选/选），dp[u][0] += max(dp[v][0], dp[v][1]), dp[u][1] += dp[v][0]。
- 最小点覆盖：dp[u][0] += dp[v][1], dp[u][1] += min(dp[v][0], dp[v][1])。
- 树上背包：dp[u][j] = u 子树选 j 个（含 u 本身）的收益，合并子节点时 $O(n^2)$ 总复杂度（两两合并的经典均摊）。

关键点：dp 初始化在叶子；树上背包合并复杂度是

$O(n^2)$ （每对点合并一次），不是 $O(n \cdot k^2)$ ；合并顺序：for j 倒序，for t 防重复选；基环树（03.10）、仙人掌等变体要破坏后 DP。

```

// 最大独立集（1-based，树）
vector<array<i64, 2>> dp;
void tree_dp(int u, int p, const vector<vector<int>>& g, const vector<i64>& val) {
    dp[u][1] = val[u]; dp[u][0] = 0; // val: 点权
    for (int v : g[u]) if (v != p) {
        tree_dp(v, u, g, val);
        dp[u][0] += max(dp[v][0], dp[v][1]);
        dp[u][1] += dp[v][0];
    }
}
// ans = max(dp[root][0], dp[root][1])

```

03.10 基环树

【简介】

n 个点 n 条边的连通图 = 树 + 一个环。解法套路：找环 → 断环成树/枚举环上状态 → 树 DP × 环处理。

快速识别：“ n 个点 n 条边”的图（每个点出度为 1 的函数图、内向树）；删一条边成树的图。

API 接入：

- 找环：拓扑排序剥叶（入度 0 入队），剩下入度 > 0 的点在环上；或 DFS 找判环。
- 处理：枚举环上每个点“选/不选”的边界状态（或断一条环边做两次树 DP）。
- 函数图（出度 1）：每个连通块是“环 + 若干树挂上去”，倍增跳环（见 03.11）。

关键点：先找环再断边；最大独立集等 DP

在环上要枚举首尾关系（环上相邻点不能同选 $\rightarrow$ 分两种断边方式各 DP 一次取 $\max$ ）；拓扑排序找环要求知道每个点入度。

```
// 找环（无向图版，返回环上节点集合）
vector<int> find_cycle(int n, const vector<vector<int>>& g) {
    vector<int> deg(n + 1), q;
    for (int u = 1; u <= n; ++u) deg[u] = (int)g[u].size();
    for (int u = 1; u <= n; ++u) if (deg[u] == 1) q.push_back(u);
    for (int i = 0; i < (int)q.size(); ++i) {
        int u = q[i];
        for (int v : g[u]) if (--deg[v] == 1) q.push_back(v);
    }
    vector<int> cycle;
    for (int u = 1; u <= n; ++u) if (deg[u] > 1) cycle.push_back(u);
    return cycle;
}
```

03.11 函数图（出度 1 图）与倍增跳

【简介】

每个点一条出边的有向图 = 若干“环 + 挂树”。支持 $O(\log n)$ 问“从 u 出发走 k 步到哪”、“首次进环步数”。

快速识别：“每个点指向下一个点”的转移图；跳 k 步（ k 巨大）；有环有向转移。

API 接入：

- $up[k][u]$ 倍增表（ u 走 2^k 步到达）； $jump(u, k)$ $O(\log k)$ 。
- 判环/找环：拓扑剥叶（出度 1 时入度计数）或 DFS。
- 环上节点、入环深度：预处理 $in_cycle[u]$ 、 $cycle_id$ 、 pos_on_cycle 。

关键点：与树倍增的区别——这是有向图且每个点出度 1，走 k 步必进环； k 可能到 $1e18$ ，用二进制拆分； $up[0][u] = next[u]$ ；跳超过入环深度的步数要取模环长。

```
struct FuncGraph {
    int n, LOG;
    vector<vector<int>> up;
    FuncGraph(int n_, const vector<int>& nxt) { // nxt[u]: u
        // 的出边 (1-based)
        n = n_;
        LOG = 1; while ((1LL << LOG) <= (i64)1e18) ++LOG;
        up.assign(LOG, vector<int>(n + 1));
        for (int u = 1; u <= n; ++u) up[0][u] = nxt[u];
        for (int k = 1; k < LOG; ++k)
            for (int u = 1; u <= n; ++u) up[k][u] = up[k - 1][up[k - 1][u]];
    }
    int jump(int u, i64 k) const {
        for (int b = 0; k; ++b, k >>= 1)
            if (k & 1) u = up[b][u];
        return u;
    }
};
```

03.12 虚树 Virtual Tree

【简介】

只保留关键点 + 它们的 LCA 的压缩树（ $O(k \log n)$ 构建， k = 关键点数），把“树上有 k 个关键点的 DP”复杂度从 $O(n)$ 降为 $O(k \log n)$ 。

快速识别：多组询问、每组只涉及少量关键点、要在树上做 DP/统计（如关键点两两距离和、最小割）；询问次数多但每组 k 小。

API 接入：

- 前置：LCA + dfn 。

- $build_virtual(keys)$: $keys$ 为关键点列表（去重），返回虚树（边带原树距离）。
- 构建：按 dfn 排序，相邻点 LCA 加入栈（单调栈经典流程）。
- 虚树上做 DP（如子树内关键点数、边权 = 原树距离）。
- 依赖：LCA（03.1 节）+ dfn （03.7 节），抄板时一起抄上。

关键点：必须在原树上算 dfn 与

LCA；构建时用栈维护“当前链”，新点 x 的 $lca = LCA(x, stk.top)$ ，弹出直到栈顶深度 $\leq lca$ ；虚树总点数 $O(2k)$ ；DP 结果记得清空用过的节点（或每次新建数组）。

```
// 前置：LCA lc; vector<int> dfn 已按 dfs 序 (1-based)
vector<pair<int, i64>> build_virtual_tree(vector<int> keys, const LC
A& lc,
                                     const vector<int>& dfn,
                                     const vector<int>& depth)
{
    sort(all(keys), [&](int a, int b) { return dfn[a] < dfn[b]; });
    keys.erase(unique(all(keys)), keys.end());
    vector<pair<int, i64>> edges; // 简化：这
    // 里返回边列表（子，距离）
    vector<int> stk;
    stk.push_back(keys[0]);
    for (int i = 1; i < (int)keys.size(); ++i) {
        int x = keys[i], l = lc.get(stk.back(), x);
        while (stk.size() >= 2 && depth[stk[stk.size() - 2]] >= de
            pt[stk.size() - 1]) {
            edges.push_back({stk.back(), depth[stk.back()] - de
                pt[stk.size() - 2]});
            stk.pop_back();
        }
        if (stk.back() != l) {
            edges.push_back({stk.back(), depth[stk.back()] - de
                pt[l]});
            stk.back() = l;
        }
        stk.push_back(x);
    }
    while (stk.size() >= 2) {
        edges.push_back({stk.back(), depth[stk.back()] - de
            pt[stk.size() - 2]});
        stk.pop_back();
    }
    return edges;
}
```

```
// Prufer 编码（树  $\rightarrow$  Prufer 序列）： $n$  个点 (1-based) 的树
vector<int> prufer_encode(int n, const vector<pair<int, int>>& edges)
{
    vector<int> deg(n + 1, 0);
    vector<vector<int>> g(n + 1);
    for (auto [u, v] : edges) { g[u].push_back(v); g[v].push_back(u);
        ++deg[u]; ++deg[v]; }
    priority_queue<int, vector<int>, greater<int>> leaves;
    for (int i = 1; i <= n; ++i) if (deg[i] == 1) leaves.push(i);
    vector<int> p;
    for (int k = 0; k < n - 2; ++k) {
        int leaf = leaves.top(); leaves.pop();
        int v = -1;
        for (int to : g[leaf]) if (deg[to] > 0) { v = to; break; }
        p.push_back(v);
        --deg[leaf]; --deg[v];
        if (deg[v] == 1) leaves.push(v);
    }
    return p; // 长度 n-2
}
```

03.13 点分治（树上路径统计）

【简介】

每次选重心，统计经过重心的路径贡献，再分治各子树。 $O(n \log n)$ 解决“树上有所有路径中满足某条件的数量/最值”。

快速识别：统计树上路径（长度 $\leq k$ 、恰好 k 条边、颜色限制）；路径类问题不能用树形 DP 时。

API 接入：

- $solve(u)$ ：对重心 u ，先统计经过 u 的路径（如深度计数 + 查询），再递归各子树。
- 典型例子：长度 $\leq K$ 的路径数—— $cnt[d]$ 深度计数，合并子树时 $ans += cnt[K - d]$ 。
- 每层要 vis 标记已处理重心， get_size 求子树大小， $get_centroid$ 找重心。

关键点：必须先找重心再递归（保证 $O(\log n)$ 层）；统计“经过 u ”的路径时，不同子树间配要对要按顺序合并并避免重复；清空计数数组（只清访问过的深度）；点分治树（重心树）是动态点分治的基础。

```
// 框架：统计树上长度  $\leq K$  的路径条数
vector<int> vis, sz;
i64 ans = 0; int K;
int get_size(int u, int p, const vector<vector<pair<int,int>>>& g) {
    sz[u] = 1;
    for (auto [v, w] : g[u]) if (v != p && !vis[v]) sz[u] += get_size(v, u, g);
    return sz[u];
}
int get_centroid(int u, int p, int total, const vector<vector<pair<int,int>>>& g) {
    for (auto [v, w] : g[u]) if (v != p && !vis[v] && sz[v] * 2 > total)
        return get_centroid(v, u, total, g);
    return u;
}
void collect(int u, int p, int dep, vector<int>& ds, const vector<vector<pair<int,int>>>& g) {
    if (dep > K) return;
    ds.push_back(dep);
    for (auto [v, w] : g[u]) if (v != p && !vis[v]) collect(v, u, dep + w, ds, g);
}
void calc(int u, const vector<vector<pair<int,int>>>& g) {
    vector<int> cnt(K + 1, 0); cnt[0] = 1;
    for (auto [v, w] : g[u]) if (!vis[v]) {
        vector<int> ds;
        collect(v, u, w, ds, g);
        for (int d : ds) if (d <= K) ans += cnt[K - d];
        for (int d : ds) if (d <= K) cnt[d]++;
    }
}
void dfs(int u, const vector<vector<pair<int,int>>>& g) {
    get_size(u, 0, g);
    int c = get_centroid(u, 0, sz[u], g);
    vis[c] = 1;
    calc(c, g);
    for (auto [v, w] : g[c]) if (!vis[v]) dfs(v, g);
}
// 用法: vis.assign(n+1,0); dfs(1, g);
```

03.14 DSU on Tree（树上启发式，静态子树统计）

【简介】

先处理轻儿子并清空，再处理重儿子并保留，最后暴力合并轻儿子——每个节点 $O(\log n)$ 次访问， $O(n \log n)$ 完成所有子树统计（颜色数、众数等）。

快速识别：对每个子树间统计量（不同颜色数、出现次数）；静态（无修改）；不想写线段树合并时。

API 接入：

- dfs(u, keep): keep=true 表示处理完保留贡献（重儿子路径），false 清空。
- add(u, delta): 把 u 子树（或单点）的贡献加/减进全局计数器；query() 取当前答案。
- 每层：先遍历轻儿子 keep=false，再重儿子 keep=true，再把轻儿子暴力加回，回答 u 的查询。

关键点：重儿子最后处理且 keep；清空操作要能撤销（add 传 -1）；复杂度 $O(n \log n)$ （轻边 $O(\log n)$ 条）；只适合“子树内统计”，路径类问题用点分治。cnt 数组是全局的。

```
// 统计每棵子树不同颜色数（静态）。使用前先跑一遍 get_sz 算好 sz[]
vector<int> col, cnt, ans;
vector<char> big;
vector<int> sz; // 子树大小（先 get_sz 预处理）
int cur = 0;
void get_sz(int u, int p, const vector<vector<int>>& g) {
    sz[u] = 1;
    for (int v : g[u]) if (v != p) { get_sz(v, u, g); sz[u] += sz[v]; }
}
void add_subtree(int u, int p, int delta, const vector<vector<int>>& g) {
    int& c = cnt[col[u]];
    if (delta == 1 && c++ == 0) ++cur;
    for (int v : g[u]) if (v != p) add_subtree(v, u, delta, g);
    if (delta == -1 && c-- == 0) --cur;
}
```

```
if (delta == -1 && --c == 0) --cur;
for (int v : g[u]) if (v != p && !big[v]) add_subtree(v, u, delta, g);
}
void dfs(int u, int p, int keep, const vector<vector<int>>& g) {
    int heavy = -1, maxsz = 0;
    for (int v : g[u]) if (v != p) if (sz[v] > maxsz) maxsz = sz[v], heavy = v;
    for (int v : g[u]) if (v != p && v != heavy) dfs(v, u, 0, g);
    // 轻儿子，清空
    if (heavy != -1) dfs(heavy, u, 1, g), big[heavy] = 1;
    // 重儿子，保留
    for (int v : g[u]) if (v != p && v != heavy) add_subtree(v, u, 1, g);
    add_subtree(u, u, 1, g);
    // 加自己
    ans[u] = cur;
    if (heavy != -1) big[heavy] = 0;
    if (!keep) add_subtree(u, p, -1, g);
    // 清空
}
// 调用: sz.assign(n+1,0); get_sz(1, 0, g); big.assign(n+1,0); dfs(1, 0, 1, g);
```

03.15 树上莫队

【简介】

欧拉序把树上路径查询转成区间查询，套莫队 $O((n+q) \sqrt{n})$ 。路径上每个点出现两次则不计入（异或计数技巧）。

快速识别：静态树上路径查询（路径上不同颜色数、出现次数、mex 等），可离线；不想写点分治/树剖的简单计数。

API 接入：

- 前置：欧拉序（每个点进出各一次，长度 $2n$ ）+ LCA。
- 查询 (u, v) （假设 $\text{tin}[u] > \text{tin}[v]$ 时交换）：若 $\text{lca}(u, v) = u$ ，区间 $[\text{tin}[u], \text{tin}[v]]$ ；否则 $[\text{tout}[u], \text{tin}[v]] + \text{单点 lca}$ 。
- 莫队 add(p): $\text{vis}[p] \text{ ^= } 1$ （出现次数奇偶翻转），奇次才计入贡献；del 同函数。
- 依赖：Euler 序（03.7 节）+ LCA（03.1 节），抄板时一起抄上。

关键点：每个点在欧拉序中出现两次，奇偶法保证路径点恰被计一次；lca 单独处理（区间外）；链式情况（lca = 端点）区间要包含 lca；边权版本转点权（边权挂到深度深的端点）。

```
// 假设已得到欧拉序 euler (2n 长)、tin/tout、LCA。
// 莫队部分：
vector<char> in_path; // 点是否“奇数次出现”
void toggle(int p, ...) { // p = 欧拉序位置
    int x = euler[p];
    if (in_path[x]) { /* 从答案中删除 x 的贡献 */ }
    else { /* 把 x 的贡献加入答案 */ }
    in_path[x] = !in_path[x];
}
// 查询构造: u, v (tin[u] > tin[v] 则交换)
// int lca = lca(u, v);
// if (lca == v) query = {tin[v], tin[u], id, 0}; // 链
// else query = {tout[v], tin[u], id, lca}; // 路径 + 单独加 lca_
```

03.16 树哈希（同构判定）

【简介】

给每棵树算一个哈希值，判断两棵树（无根/有根）是否同构。常用随机权值 + 子树哈希组合。

快速识别：判断两棵树结构是否相同（同构）；子树同构计数；“树的形状”比较。

API 接入：

- 有根树：hash_tree(root) 递归， $h[u] = f(h[\text{child}])$ 集合（子节点哈希排序后组合，如 $\sum h[\text{child}] * \text{rand}[i]$ 或进制哈希）。
- 无根树：取重心（最多两个）分别做有根哈希，取 min/max 作为无根哈希。
- 子树哈希集合比较：map<vector<ui64>, int> 编号。

关键点：子节点哈希必须先排序（顺序无关）；用 splitmix64 随机种子防碰撞；无根树必须用重心（否则根不同哈希不同）；多测注意清空。


```
ui64 splitmix64(ui64 x) {
    x += 0x9e3779b97f4a7c15ULL;
    x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9ULL;
    x = (x ^ (x >> 27)) * 0x94d049bb133111ebULL;
    return x ^ (x >> 31);
}
mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());
ui64 hash_tree(int u, int p, const vector<vector<int>>& g) {
    vector<ui64> child;
    for (int v : g[u]) if (v != p) child.push_back(hash_tree(v, u, g));
    sort(all(child));
    ui64 h = 1;
    for (ui64 c : child) h = h * (0x9e3779b97f4a7c15ULL) + c + splitmix64(rng());
    return h;
}
```

03.17 Kruskal 重构树

【简介】

按边权从小到大加边，合并时新建节点作为两个连通块的新父——得到一棵二叉树：叶子 = 原节点，内部节点权值 = 合并时刻的边权。性质：两点路径上最大边权最小值 = 它们在重构树上的 LCA 权值。

快速识别：“从 u 到 v

路径上最大边权最小化”（最小瓶颈）：“边权 $\leq x$

的连通性”：“经过边权 $\leq x$ 的边能到哪些点”（倍增跳重构树 + 子树）。

API 接入：

- `build_kruskal_tree(edges, n)`: 返回重构树（ $2n-1$ 个节点）、`val[i]`（内部节点权值）、并查集合并顺序。
- 查询：倍增 `up` 跳重构树，找到最后一个权值 $\leq x$ 的祖先，其子树 = 可达点集。
- 瓶颈值：`query_max_edge_min(u, v)` = 重构树 LCA 的 `val`。

关键点：内部节点从 $n+1$ 开始编号；原图不连通时构建多棵重构树（超级根或分别处理）；边权从大到小构建得到“最小值最大”版本；子树区间 = `dfn` 连续（配合线段树/主席树做子树统计）。

```
// 最小瓶颈：u 到 v 路径上“最大边权的最小值” = Kruskal 重构树 LCA 的权值
// 用法：KruskalTree kt(n); 按边权升序 kt.add_edge(u, v, w); 然后 kt.build();
// 查询：kt.query(u, v) 返回瓶颈值；kt.find_root_under(x, limit) 返回边权 <= limit 的连通块根
struct KruskalTree {
    int n, tot;
    vector<vector<int>> child; // 重构树孩子 (2n 个节点)
    vector<i64> val; // 1..n 叶子 val=0; n+1..t 内部节点 = 合并边权
    vector<int> fa; // 并查集 (建树用)
    KruskalTree(int n) : n(n), tot(n) {
        child.assign(2 * n + 1, {});
        val.assign(2 * n + 1, 0);
        fa.resize(n + 1);
        iota(fa.begin(), fa.end(), 0);
    }
    int find(int x) { return fa[x] == x ? x : fa[x] = find(fa[x]); }
    void add_edge(int u, int v, i64 w) { // 按 w 升序调用
        int ru = find(u), rv = find(v);
        if (ru == rv) return;
        ++tot;
        if (tot >= (int)fa.size()) fa.resize(tot + 1); // 先扩容
        val[tot] = w;
        child[tot].push_back(ru); // ru, rv 是之前的根
        child[tot].push_back(rv);
        fa[ru] = fa[rv] = tot;
    }
    // 建倍增表 (建树后调用一次)
    int LOG;
    vector<vector<int>> up;
    vector<int> depth;
    void dfs(int u, int p) {
        up[0][u] = p;
        for (int k = 1; k < LOG; ++k) up[k][u] = up[k-1][up[k-1][u]];
        for (int v : child[u]) { depth[v] = depth[u] + 1; dfs(v, u); }
    }
    void build() {
        LOG = 1; while ((1 << LOG) <= tot) ++LOG;
        up.assign(LOG, vector<int>(tot + 1, 0));
    }
};
```

```
depth.assign(tot + 1, 0);
vector<char> is_root(tot + 1, 1);
for (int u = n + 1; u <= tot; ++u)
    for (int v : child[u]) is_root[v] = 0;
for (int u = 1; u <= tot; ++u) if (is_root[u]) dfs(u, u);
// 可能多棵树
}
int lca(int u, int v) const {
    if (depth[u] < depth[v]) swap(u, v);
    for (int k = LOG - 1; k >= 0; --k)
        if (depth[up[k][u]] >= depth[v]) u = up[k][u];
    if (u == v) return u;
    for (int k = LOG - 1; k >= 0; --k)
        if (up[k][u] != up[k][v]) { u = up[k][u]; v = up[k][v]; }
    return up[0][u];
}
i64 query(int u, int v) const { return val[lca(u, v)]; } // 瓶颈值 (最大边权最小值)
// 从 u 向上跳，停在最后一个权值 <= limit 的祖先 (其子树 = 边权 <= limit 可达集合)
int find_root_under(int u, i64 limit) const {
    for (int k = LOG - 1; k >= 0; --k)
        if (val[up[k][u]] <= limit) u = up[k][u];
    return u;
};
```

03.18 长链剖分 ($O(1)$ k 级祖先 / 深度统计合并)

【简介】

按“到叶子的最长链”剖分。两个经典应用： $O(1)$ 询问 k 级祖先； $O(n)$ 合并“深度桶”的 DP（如子树内距离 = d 的点数）。

快速识别：频繁问 k 级祖先（在线， q 巨大）；“子树内距 u 恰好 d 的点数”这类按深度合并的 DP。

API 接入：

- k 级祖先：预处理长链 + 每条链头向上/向下跳表 + 倍增高 2 位；`kth_ancestor(u, k)` $O(1)$ 。
- 深度合并：`dp[u]` 是数组（ u 子树内各深度计数），长链继承重儿子数组（指针复用），轻儿子暴力合并——总 $O(n)$ 。

关键点：长链定义 = 向下最深路径（不是最大子树）；重儿子 = 链最长者；数组复用技巧（`vector` 移位/全局池）是 $O(n)$ 的关键；只适合深度维 DP，其他统计用 `dsu on tree` / 线段树合并。

```
// 长链剖分 +  $O(1)$  k 级祖先 (1-based 树, n 个点)
// 用法：LongChain lc(n); lc.build(root, g); 然后 lc.kth_ancestor(u, k) 返回向上 k 步的祖先
struct LongChain {
    int n;
    vector<int> dep, len, son, top, dfn, rnk;
    // up/down: 每条链头向上/向下的 len 个祖先 (存 dfn)
    vector<vector<int>> up, down;
    vector<vector<int>> anc; // 二进制倍增 (只用到高 2 位)
    int timer = 0, LOG;
    LongChain(int n) : n(n) {
        dep.assign(n + 1, 0); len.assign(n + 1, 1);
        son.assign(n + 1, 0); top.assign(n + 1, 0);
        dfn.assign(n + 1, 0); rnk.assign(n + 1, 0);
        LOG = 1; while ((1 << LOG) <= n) ++LOG;
        anc.assign(LOG, vector<int>(n + 1, 0));
    }
    void dfs1(int u, int p, const vector<vector<int>>& g) {
        anc[0][u] = p;
        for (int k = 1; k < LOG; ++k) anc[k][u] = anc[k-1][anc[k-1][u]];
        for (int v : g[u]) if (v != p) {
            dep[v] = dep[u] + 1;
            dfs1(v, u, g);
            if (len[v] + 1 > len[u]) { len[u] = len[v] + 1; son[u] = v; }
        }
    }
    void dfs2(int u, int t, const vector<vector<int>>& g) {
        top[u] = t;
        dfn[u] = ++timer; rnk[timer] = u;
        if (son[u]) dfs2(son[u], t, g); // 长链先走
        for (int v : g[u]) if (v != anc[0][u] && v != son[u]) dfs2(v, v, g);
    }
    void build(int root, const vector<vector<int>>& g) {
        dfs1(root, 0, g);
    }
};
```



```

dfs2(root, root, g);
up.assign(n + 1, {}); down.assign(n + 1, {});
for (int u = 1; u <= n; ++u) if (top[u] == u) {
    int t = u, L = len[u];
    for (int x = 0; x < L; ++x) { // 链头向下 1
        down[u].push_back(t);
        t = son[t];
    }
    t = u;
    for (int x = 0; x < L; ++x) { // 链头向上 1
        up[u].push_back(t);
        t = anc[0][t];
    }
}
}
int kth_ancestor(int u, int k) const { // u 向上 k
    步 (k >= 0)
    if (k > dep[u]) return 0; // 越界返回 0
    if (!k) return u;
    int hb = 31 - __builtin_clz(k); // k 的最高位
    u = anc[hb][u]; // 跳 k 的最
    高位 2^hb
    k -= 1 << hb;
    int t = top[u];
    int d = dep[u] - dep[t];
    if (k <= d) return rnk[dfn[t] - d + k]; // 还在链上:
    沿 dfn 走
    return up[t][k - d]; // 越过链头:
    用链头上跳表
}
};

```

03.19 支配树 (Lengauer-Tarjan)

【简介】

有向图中，支配点 = 从源点到 v 的所有路径必经点；支配树 = 以源点为根， v 的支配点构成其祖先链。 $O((n+m)\alpha)$ 。

快速识别：“删掉哪些点会使 s 无法到达 t ”、“必经点/必经边”、“控制依赖”类题目；单源支配关系。

API 接入：

- `build_dominator_tree(s, g)`: 返回 `idom[v]` (v 的直接支配点)；根 s 的 `idom` = 0。
- 之后支配树 `LCA` = 两点共同支配点；`sz[子树]` = 被 v 支配的点数。
- 需前置 Tarjan (semi 半支配点计算)。

关键点：算法核心是半支配点 `semi` 与 `idom` 修正；有多个入度时 `idom` 由“最小半支配点路径”决定；源点必须能到达所有相关点（不可达点单独处理）；模板较复杂，抄时整段复制不要改内部。

```

// Lengauer-Tarjan 支配树 (源点 s)
// 输出 idom[1..n]；实现较长，建议整段照抄标准版，只改图存储。
// 关键数组：dfn/rev (dfs 序)、semi[v] (半支配点)、idom[v]、并查集 + bes
// t 辅助
// 查询：v 是否支配 u <-> idom 树上 v 是 u 祖先 (用支配树上的 dfn 区间判)

```

03.20 点分治的补充：动态点分治

【简介】

把点分治的重心连成“点分树”（重心树），树高 $O(\log n)$ ；在点分树上维护信息支持在线修改（如点开关灯查最近距离）。每次修改/查询沿点分树向上 $O(\log n)$ 个祖先。

快速识别：单点修改 + 全局/子树距离查询（最近、最远、距离和）；“支持修改的树上路径统计”。

API 接入：

- 构建：点分治时记录每层重心、`fa_centroid`（上一层重心）、每个原点到各层重心的距离。
- 查询最近标记点：每层重心维护“该层子树内标记点到重心的最小距离”（两个堆/ multiset: `ans` 堆存 `dist` 到父重心 + 父重心答案）。

关键点：点分树深度 $O(\log n)$

是复杂度关键；每个原点在每层重心的距离要预处理存下来；堆/ multiset 支持删除用“双堆懒删除”或 multiset + `erase`；点分树边权 = 原树距离。

```

// 动态点分治（点开关灯，查询最近亮灯距离）结构要点：

```

```

// cfa[u] = 点分树上 u 的父重心
// dist[u][k] = 原树上 u 到第 k 层重心的距离（预处理）
// h1[u] = multiset: u 子树内亮灯点到 u 的距离
// h2[u] = multiset: u 子树内亮灯点到 cfa[u] 的距离（供父层用）
// 查询最近: ans 全局堆 = h1[u] 最小值 + h2 合并; 翻转点 u 时沿 cfa 链更新
// O(log n) 层

```

03.21 树链剖分 + 树上主席树 (路径点权第 k 小)

【简介】

每个点到根的路径建可持久化权值线段树 (`root[u] = root[fa[u]]` 加 u 的点权)，路径 $u-v$ 的权值分布 =

`root[u]+root[v]-root[lca]-root[fa[lca]]`，支持路径上第 k 小/计数。

快速识别：静态树上路径点权第 k 小/中位数/计数（无修改）。

API 接入：

- `build(root)`: dfs 时 `root[u] = update(root[fa[u]], id_of(val[u]))`。
- `path_kth(u, v, k)`: 四版本相减二分；`path_count_le(u, v, x)`。
- 需要 `LCA`（倍增或树剖）。

关键点：四个版本相减的公式 `root[u] + root[v] - root[lca] - root[fa[lca]]`；点权离散化；只能静态（修改用树剖套树套树或动态点分治）；边权版 = 权值挂到子节点，公式去掉 `lca` 项。

```

// 树上主席树关键部分 (PersistentKth 复用节点池)：
// dfs 中: root[u] = update(root[fa[u]], 1, M, id_of(val[u]));
// 查询第 k 小: 在 root[u], root[v], root[lca], root[fa[lca]] 四棵树上二分
// int lc_sum = tr[tr[ru].l].sum + tr[tr[rv].l].sum - tr[tr[r1].l].sum - tr[tr[rfa].l].sum;
// k <= lc_sum 走左, 否则走右 (k -= lc_sum)

```

03.22 树的 Prufer 序列 (放 06 章组合处引用)

【简介】

Prufer 序列是带标号树的编码 (n 个点的树 $\leftrightarrow$ 长度 $n-2$ 的序列)，用于计数：完全图生成树数 = n^{n-2} (Cayley)；“每个点度数固定”的树计数 = 多重组合。

快速识别：带标号树计数；度数约束的树计数；随机树生成。

API 接入：

- 编码：树 $\rightarrow$ Prufer ($O(n)$ 删叶法)；解码：Prufer $\rightarrow$ 树 ($O(n)$)。
- Cayley 公式： n^{n-2} 棵 ($n \geq 2$)。
- 度数 d_i 的树数： $(n-2)! / \prod (d_i - 1)!$ 。

关键点：叶子 = 度数 1 的节点；Prufer 中每个点出现 $d_i - 1$ 次； $n=1$ 特判 (1 棵树)；计数取模用阶乘与逆元。

```

// 度数序列 -> 树数 (模 mod) : ans = fact[n-2] * Π inv(fact[d_i - 1])
// Cayley: n 个带标号点完全图的生成树数 = mod_pow(n, n-2, mod) (n >= 2)
// 解码示例 (Prufer -> 树)：
vector<pair<int,int>> prufer_decode(const vector<int>& p) {
    int n = (int)p.size() + 2;
    vector<int> deg(n + 1, 1);
    for (int x : p) deg[x]++;
    priority_queue<int, vector<int>, greater<int>> leaves;
    for (int i = 1; i <= n; ++i) if (deg[i] == 1) leaves.push(i);
    vector<pair<int,int>> edges;
    for (int x : p) {
        int leaf = leaves.top(); leaves.pop();
        edges.push_back({leaf, x});
        if (--deg[x] == 1) leaves.push(x);
    }
    int a = leaves.top(); leaves.pop();
    edges.push_back({a, leaves.top()});
    return edges;
}

```

04 图论

04.1 最短路: Dijkstra (堆优化)

【简介】

非负权单源最短路， $O((n+m)\log n)$ 。每次取未确定最短路的最近点松弛邻边。

快速识别：单源、边权非负（可 0）；稀疏图；作为大量建模的底层（分层图、最短路计数、状态图）。

API 接入：

- `vector<i64> dijkstra(s, g)`: 返回 `dist[]`（不可达 = `LINF`）；`g` 为 `vector<vector<pair<int, i64>>>`（1-based, 点 1..n）。
- 求路径：维护 `pre[v]`（`v` 由谁松弛），最后回溯。
- 最短路计数：`ways[v] += ways[u]` 在 `dist` 相等时；注意先出队的已确定。

关键点：优先队列存（`dist, v`）；过期元素（`dist > d[v]`）跳过；`dist` 用 `i64`；无向图加两条有向边；0 权边可用但若有 0/1 边换 0-1 BFS 更快。

```
vector<i64> dijkstra(int s, int n, const vector<vector<pair<int, i64>>>& g) {
    vector<i64> dist(n + 1, LINF);
    priority_queue<pair<i64, int>, vector<pair<i64, int>>, greater<>> pq;
    dist[s] = 0;
    pq.push({0, s});
    while (!pq.empty()) {
        auto [d, u] = pq.top(); pq.pop();
        if (d != dist[u]) continue; // 过期元素
        for (auto [v, w] : g[u]) {
            if (dist[v] > d + w) {
                dist[v] = d + w;
                pq.push({dist[v], v});
            }
        }
    }
    return dist;
}
```

04.2 最短路：BFS / 0-1 BFS / 无权网格

【简介】

无权图最短路用 BFS $O(n+m)$ ；边权只有 0/1 时用双端队列 0-1 BFS $O(n+m)$ （0 权放队头、1 权放队尾）。

快速识别：所有边权相同（步数） $\rightarrow$ BFS；边权 $\in \{0, 1\} \rightarrow$ 0-1 BFS；网格移动/状态图。

API 接入：

- BFS: `dist[s]=0`，队列逐层扩展；网格用 `dx/dy`。
- 0-1 BFS: `deque<int>`；0 权边 `push_front`、1 权边 `push_back`；`dist` 相同可重复入队（多次松弛）。
- 状态图：把（位置，状态）编码成节点。

关键点：BFS 中第一次出队即最短路（无权图）；0-1 BFS 中允许 `dist` 更新时再次入队（不判 `visited`，判 `nd < dist[v]`）；网格注意边界；BFS 双向扩展可加速。

```
// 0-1 BFS (1-based 图)
vector<i64> bfs01(int s, int n, const vector<vector<pair<int, int>>>& g) {
    vector<i64> dist(n + 1, LINF);
    deque<int> dq;
    dist[s] = 0; dq.push back(s);
    while (!dq.empty()) {
        int u = dq.front(); dq.pop front();
        for (auto [v, w] : g[u]) {
            if (dist[v] > dist[u] + w) {
                dist[v] = dist[u] + w;
                if (w == 0) dq.push front(v); else dq.push back(v);
            }
        }
    }
    return dist;
}
```

04.3 最短路：Bellman-Ford / SPFA（负权与负环）

【简介】

允许负权边的最短路；Bellman-Ford $O(nm)$ ，SPFA 平均快但可被卡到 $O(nm)$ 。用于差分约束与负环判定。

快速识别：边权可能为负；判负环（无最短路）；差分约束系统。

API 接入：

- `vector<i64> bellman_ford(s, n, edges)`: `edges` 为 (u, v, w) ；返回 `dist`（有负环时行为未定义，先判）。
- 判负环：第 `n` 轮仍能松弛 $\rightarrow$ 有负环（从 `s` 可达的）。
- SPFA: 队列优化 + `cnt[v]` 记录入队次数，`cnt[v] > n` 有负环。

关键点：判负环要从超级源点 `s`

出发能到达所有点才完整（或先拓扑/虚拟源）；SPFA

判负环入队次数上限 `n`；差分约束建图： $x[v] \leq x[u] + w$ 加边 $u \rightarrow v (w)$ ，最短路给出上界解。

```
// SPFA 判负环 (返回 true = 有负环)
bool spfa_neg_cycle(int n, const vector<vector<pair<int, i64>>>& g) {
    vector<i64> dist(n + 1, 0); // 全部初始 0: 等价虚拟源
    vector<int> cnt(n + 1, 0), inq(n + 1, 0);
    queue<int> q;
    for (int i = 1; i <= n; ++i) q.push(i), inq[i] = 1;
    while (!q.empty()) {
        int u = q.front(); q.pop(); inq[u] = 0;
        for (auto [v, w] : g[u]) {
            if (dist[v] > dist[u] + w) {
                dist[v] = dist[u] + w;
                if (!inq[v]) {
                    if (++cnt[v] > n) return true; // 入队超 n 次 -> 负环
                    q.push(v); inq[v] = 1;
                }
            }
        }
    }
    return false;
}
```

04.4 最短路：Floyd（全源 / 传递闭包）

【简介】

$O(n^3)$ 全源最短路；也用于传递闭包（可达性）与“最小环”。 $n \leq 500$ 时用。

快速识别：全源最短路 n

小；多次询问任意两点；传递闭包；最小环。

API 接入：

- `floyd(g)`: `g` 为 $n \times n$ 矩阵（`INF` 不可达，0 对角线）；原地更新 `dist`。
- 传递闭包: `reach[i][j] |= reach[i][k] & reach[k][j]`（bitset 优化见 04.xx）。
- 最小环：在 `k` 轮更新前检查 `dist[i][j] + g[j][k] + g[k][i]`。

关键点：`k` 在最外层；`dist` 初始化对角线 0、无边 `INF`；重复边取 `min`；`INF` 相加会溢出（用 `if (d[i][k] + d[k][j] < d[i][j])` 且 `INF` 足够大但别用 `LLONG_MAX`）。

```
void floyd(vector<vector<i64>>& d) { // d 初始: d[i][i]=0, 无边=LINF
    int n = (int)d.size() - 1;
    for (int k = 1; k <= n; ++k)
        for (int i = 1; i <= n; ++i)
            if (d[i][k] < LINF)
                for (int j = 1; j <= n; ++j)
                    if (d[i][k] + d[k][j] < d[i][j])
                        d[i][j] = d[i][k] + d[k][j];
}
```

04.5 差分约束

【简介】

把不等式组 $x[v] - x[u] \leq w$

建模成最短路/最长路，求可行解或最值。每步推导建图方向。

快速识别：一组线性不等式约束可行解/极值；“`a` 比 `b` 至少大 `k`”、“时间先后”类问题。

API 接入：

- $x[v] - x[u] \leq w$: 加边 $u \rightarrow v (w)$ ，跑最短路，`dist` 即一组解（若存在）。
- $x[v] - x[u] \geq w$: 等价 $x[u] - x[v] \leq -w$: 加边 $v \rightarrow u (-w)$ 。
- 负环 $\Rightarrow$ 无解（SPFA 判负环）。
- 要求 `x` 非负/下界：虚拟源连 0 权边，或直接初始 `dist=0` 跑。

关键点：所有约束统一化成 $\leq$ 形式再建图；求最小/最大值对应最短路/最长路方向；虚拟源点到所有点连 0 保证连通； $x[u] < x[v]$ （严格）转整数约束 $x[u] \leq x[v] - 1$ 。

```
// 差分约束完整示例 (1-based, n 个变量, m 个约束)
// 约束形式: x[v] - x[u] <= w -> add(u, v, w)
//          x[v] - x[u] >= w -> add(v, u, -w)
//          x[u] < x[v] -> add(u, v, -1) (整数)
// 建图后跑 SPFA; 返回 true = 有可行解 (dist 即一组解, dist 数组已初始化 0 = 虚拟源)
// 无解 = 存在负环
bool diff_constraints(int n, vector<vector<pair<int, i64>>>& g) {
    // g 已按上面规则建好 (大小 n+1)
    vector<i64> dist(n + 1, 0); // 全部 0: 等价于虚拟源连 0 权边
    vector<int> cnt(n + 1, 0), inq(n + 1, 0);
    queue<int> q;
    for (int i = 1; i <= n; ++i) q.push(i), inq[i] = 1;
    while (!q.empty()) {
        int u = q.front(); q.pop(); inq[u] = 0;
        for (auto [v, w] : g[u])
            if (dist[v] > dist[u] + w) {
                dist[v] = dist[u] + w;
                if (!inq[v]) {
                    if (++cnt[v] > n) return false; // 负环 -> 无解
                    q.push(v); inq[v] = 1;
                }
            }
    }
    return true; // dist[v] 是一组可行解
}
```

04.6 最小生成树: Kruskal / Prim

【简介】

Kruskal (边排序 + 并查集, $O(m \log m)$, 适合稀疏图); Prim (堆优化 $O(m \log n)$, 适合稠密图)。MST 唯一性、次小生成树、瓶颈边见 04.7。

快速识别: 连通所有点代价最小; “最小生成森林”; 边权排序后单调加边的变体。

API 接入:

- Kruskal: `vector<edge> es` (u, v, w 0-based 或 1-based 自定); 返回 (总权值, 是否连通)。
- Prim: 稠密图 (n 小、 $m \sim n^2$) 用 $O(n^2)$ 朴素; 否则堆优化。
- 最大生成树: 排序方向反过来。
- 依赖: DSU (02.1 节), 抄板时一起抄上。

关键点: Kruskal 用 DSU; 同权边顺序任意但计数时小心 (次小生成树用“严格次大边”); $m < n-1$ 无法连通; 生成树边数 = $n-1$ 。

```
// Kruskal (1-based 或 0-based 均可, DSU 自适应)
struct Edge { int u, v; i64 w; };
i64 kruskal(int n, vector<Edge> es) {
    sort(all(es), [](const Edge& a, const Edge& b) { return a.w < b.w; });
    DSU dsu(n + 1);
    i64 ans = 0; int cnt = 0;
    for (auto& e : es) {
        if (dsu.unite(e.u, e.v)) { ans += e.w; if (++cnt == n - 1) break; }
    }
    return cnt == n - 1 ? ans : -1; // -1 = 不连通
}
```

04.7 次小生成树 / 严格次小

【简介】

求严格次小生成树: 先求

MST, 枚举每条非树边替换环上最大 (或严格次大) 边。 $O(m \log n)$ 。

快速识别: “第二小的生成树”、删一条边的最小生成树、“生成树中次优方案”。

API 接入:

- 求 MST (Kruskal) 并记录树边; 倍增预处理树上路径最大/严格次大边权。

- 对每条非树边 (u, v, w) : $ans = \min(ans, mst + w - \max_edge_on_path(u, v))$; 若 $w ==$ 最大边权, 改用严格次大边。
- 返回严格次小值 (与 MST 不同)。

关键点: 必须处理 $w ==$ 最大边权的情况 (用严格次大边, 否则得到的是非严格次小); 树上路径最大/次大边用倍增表维护 (03.1 扩展); 所有边权可能相等时严格次小仍存在。

```
// 倍增维护路径最大、次大边权 (在 LCA 上扩展两个表 mx[k][u], smx[k][u])
// 非树边处理:
// i64 best = path_max(u, v); // 路径最大边权
// i64 ans = mst + w - best;
// if (w == best) { // 用严格次大
//     i64 sb = path_second_max(u, v);
//     if (sb != -LINF) ans = min(ans, mst + w - sb);
// }
```

04.8 拓扑排序 (含字典序最小)

【简介】

DAG 线性排序: 每条边 $u \rightarrow v$, u 在 v 前。判环 (DAG 判定)、DAG DP、字典序最小拓扑。

快速识别: 依赖关系排序; 判环; DAG 上

DP (最长路、方案数); “先做哪个”。

API 接入:

- `topo_sort(n, g, indeg)`: 返回拓扑序列 (vector); 长度 $< n$ 说明有环。
- 字典序最小: 用 `priority_queue<int, vector<int>, greater<int>>` 代替普通队列。
- DAG 最长路: 按拓扑序 DP。

关键点: 入度数组初始化后每删点更新; 有环时部分点入度永不为 0; 多组拓扑序时按题意选择队列/堆; 求“所有拓扑序中字典序最小”用小根堆。

```
vector<int> topo_sort(int n, const vector<vector<int>>& g) {
    vector<int> indeg(n + 1, 0), res;
    for (int u = 1; u <= n; ++u) for (int v : g[u]) indeg[v]++;
    priority_queue<int, vector<int>, greater<int>> q; // 字典序最小
    for (int u = 1; u <= n; ++u) if (indeg[u] == 0) q.push(u);
    while (!q.empty()) {
        int u = q.top(); q.pop();
        res.push_back(u);
        for (int v : g[u]) if (--indeg[v] == 0) q.push(v);
    }
    return res; // size < n => 有环
}
```

04.9 强连通分量 SCC (Tarjan)

【简介】

有向图强连通分量 + 缩点成 DAG。缩点后按拓扑序做

DP (最长路、方案数)。

快速识别: 有向图“相互可达”分组; 缩点建模; “环上条件”变 DAG。

API 接入:

- `scc(n, g)`: 返回 `comp[u]` (u 所属分量编号, 编号按缩点后拓扑逆序——comp 小的先出栈, 实际是拓扑序倒序, DP 时从 comp 大到小或建缩点图拓扑)。
- `scc_cnt` = 分量数; 缩点图: 对每条边 $u \rightarrow v$, 若 `comp[u] != comp[v]` 加边。
- 判强连通数、出度入度统计。

关键点: `dfn/low` + 栈; `low[u]` 更新规则: 树边

`low[u] = min(low[u], low[v])`, 回边 `low[u] = min(low[u], dfn[v])` (在栈内才更新); 出栈条件 `dfn[u] == low[u]`; comp 编号方向别猜, DP 用拓扑 (缩点图) 最稳。

```
struct TarjanSCC {
    int n, timer = 0, scc_cnt = 0;
    vector<int> dfn, low, comp, stk;
    vector<char> in stk;
    vector<vector<int>> g;
    TarjanSCC(int n) : n(n) {
        dfn.assign(n + 1, 0); low.assign(n + 1, 0);
    }
};
```

```

    comp.assign(n + 1, 0); in stk.assign(n + 1, 0);
    g.resize(n + 1);
}
void add_edge(int u, int v) { g[u].push_back(v); }
void tarjan(int u) {
    dfn[u] = low[u] = ++timer;
    stk.push_back(u); in stk[u] = 1;
    for (int v : g[u]) {
        if (!dfn[v]) { tarjan(v); low[u] = min(low[u], low[v]);
        }
        else if (in stk[v]) low[u] = min(low[u], dfn[v]);
    }
    if (dfn[u] == low[u]) {
        ++scc_cnt;
        while (true) {
            int x = stk.back(); stk.pop_back();
            in stk[x] = 0;
            comp[x] = scc_cnt;
            if (x == u) break;
        }
    }
}
void run() { for (int u = 0; u <= n; ++u) if (!dfn[u]) tarjan(u); }
}; // 从 0 开始 (兼容 2-SAT 等 0 号节点场景)
};

```

04.10 割点与桥 (Tarjan)

【简介】

无向图割点 (删去后图不连通) 与桥 (删去后图不连通)。配合边双 / 点双分量。

快速识别: 删点/删边破坏连通; “必经点/必经边” (无向图版); 双连通分量前置。

API 接入:

- `find_cut_vertices(g)`: 返回割点集合; `find_bridges(g)`: 返回桥集合 (u,v)。
- 割点判定: 非根节点 $\text{low}[v] \geq \text{dfn}[u]$; 根节点有 ≥ 2 个子树。
- 桥判定: $\text{low}[v] > \text{dfn}[u]$ (严格大于, 注意重边处理: 记录边 id 而不是父节点)。

关键点: 重边会让桥判定出错——回溯时跳过“同一条边” (用边 id 而非父节点); 根节点割点条件特殊: low 更新: 树边

$\text{low}[u] = \min(\text{low}[u], \text{low}[v])$, 回边

$\text{low}[u] = \min(\text{low}[u], \text{dfn}[v])$ (无向图回边直接更新, 不像有向图判 in_stk)。

```

// 割点 (返回 is_cut[])
struct CutVertex {
    int n, timer = 0;
    vector<int> dfn, low;
    vector<char> is_cut;
    vector<vector<int>> g;
    CutVertex(int n) : n(n) {
        dfn.assign(n + 1, 0); low.assign(n + 1, 0);
        is_cut.assign(n + 1, 0); g.resize(n + 1);
    }
    void add_edge(int u, int v) { g[u].push_back(v); g[v].push_back(u); }
    void dfs(int u, int root) {
        dfn[u] = low[u] = ++timer;
        int child = 0;
        for (int v : g[u]) {
            if (!dfn[v]) {
                ++child;
                dfs(v, root);
                low[u] = min(low[u], low[v]);
                if (u != root && low[v] >= dfn[u]) is_cut[u] = 1;
            } else low[u] = min(low[u], dfn[v]);
        }
        if (u == root && child >= 2) is_cut[u] = 1; // 根: >=2
    }
    void run() { for (int u = 1; u <= n; ++u) if (!dfn[u]) dfs(u, u); }
};

// 桥 (边用 id 记录以正确处理重边): 返回 is_bridge[edge_id]
struct FindBridges {
    int n, timer = 0;
    vector<int> dfn, low;
    vector<char> is_bridge;
    vector<vector<pair<int, int>>> g; // (v, edge_id)
    FindBridges(int n_) : n(n_) {

```

```

        dfn.assign(n + 1, 0); low.assign(n + 1, 0);
        g.resize(n + 1);
    }
    void add_edge(int u, int v, int id) { g[u].push_back({v, id}); g[v].push_back({u, id}); }
    void dfs(int u, int pe) { // pe = 进入 u 的边 id
        dfn[u] = low[u] = ++timer;
        for (auto [v, id] : g[u]) {
            if (id == pe) continue; // 跳过同一条边 (重边不受影响)
            if (!dfn[v]) {
                dfs(v, id);
                low[u] = min(low[u], low[v]);
                if (low[v] > dfn[u]) is_bridge[id] = 1; // 子树回不到 u 之上 -> 桥
            } else low[u] = min(low[u], dfn[v]);
        }
    }
    void run(int m) {
        is_bridge.assign(m, 0);
        for (int u = 1; u <= n; ++u) if (!dfn[u]) dfs(u, -1);
    }
};

```

04.11 双连通分量: 边双 / 点双与圆方树

【简介】

边双连通分量 (无桥连通块, 缩点成树); 点双连通分量 (无割点连通块)。点双之间通过割点相连, 圆方树把“割点-点双”结构化。

快速识别: “删任意一条边仍连通” (边双); “删任意一个点仍连通” (点双); 仙人掌、圆方树建模。

API 接入:

- 边双: Tarjan 找桥后并查集合并 (非桥边两端同分量), 或栈法: `bcc_id[u]`。
- 点双: Tarjan 栈维护, $\text{dfn}[u] == \text{low}[u]$ 出栈成一个分量; 割点属于多个分量。
- 圆方树: 每个点双新建方点连向其中所有点; 原图点 = 圆点。树上路径 = 圆方树路径。

关键点: 点双的判定是 $\text{low}[v] \geq \text{dfn}[u]$ 时出栈 (与割点同条件)

: 一个点可属于多个点双; 圆方树边权设计: 方点到圆点权 0 或 1 按题意; 边双缩点后是树 (森林)。

依赖: 边双用 DSU (02.1 节)、桥用 FindBridges (04.10 节), 抄板时一起抄上。

```

// 边双连通分量 (基于 FindBridges, 见 04.10): 非桥边两端同边双
// 用法: 跑完桥后, 用并查集合并所有非桥边两端点
vector<int> edge_bcc(int n, const vector<array<int, 2>>& edges, const vector<char>& is_bridge) {
    DSU dsu(n + 1);
    for (int i = 0; i < (int)edges.size(); ++i)
        if (!is_bridge[i]) dsu.unite(edges[i][0], edges[i][1]);
    // 编号压缩: 返回 bcc_id[u] (0-based 分量编号)
    unordered_map<int, int> mp;
    vector<int> bcc_id(n + 1);
    int cnt = 0;
    for (int u = 1; u <= n; ++u) {
        int r = dsu.find(u);
        if (!mp.count(r)) mp[r] = cnt++;
        bcc_id[u] = mp[r];
    }
    return bcc_id;
}

// 点双连通分量 (栈法): 返回每个点双的点集; 割点会出现在多个点双中
struct VBCCTarjan {
    int n, timer = 0;
    vector<int> dfn, low;
    vector<vector<int>> g;
    vector<vector<int>> bcc; // 每个点双的点集
    vector<pair<int, int>> stk; // 边栈
    VBCCTarjan(int n) : n(n) {
        dfn.assign(n + 1, 0); low.assign(n + 1, 0);
        g.resize(n + 1);
    }
    void add_edge(int u, int v) { g[u].push_back(v); g[v].push_back(u); }
    void dfs(int u, int fa) {
        dfn[u] = low[u] = ++timer;
        for (int v : g[u]) {
            if (v == fa) continue;
            if (!dfn[v]) {
                stk.push_back({u, v});

```



```

        dfs(v, u);
        low[u] = min(low[u], low[v]);
        if (low[v] >= dfn[u]) { // u 是割点 (或根)
            bcc.push_back({});
            while (true) {
                auto [a, b] = stk.back(); stk.pop_back();
                bcc.back().push_back(a);
                bcc.back().push_back(b);
                if (a == u && b == v) break;
            }
        } else if (dfn[v] < dfn[u]) {
            stk.push_back({u, v});
            low[u] = min(low[u], dfn[v]);
        }
    }
}

void run() {
    for (int u = 1; u <= n; ++u) if (!dfn[u]) dfs(u, 0);
    // 去重 (每个点双的点集有重复):
    for (auto& comp : bcc) {
        sort(all(comp));
        comp.erase(unique(all(comp)), comp.end());
    }
}

};
// 圆方树: 每个点双新建方点 (编号 n+1..n+bcc_cnt), 连向其中所有圆点;
// 原图圆点保留; 圆点之间不直接连边。树高/路径用方点作中转。

```

04.12 2-SAT

【简介】

n 个布尔变量, m 条“或”约束 (如 x_i OR x_j 、 $x_i \Rightarrow x_j$), 求一组赋值或判无解。建图: 每个变量拆两个点, 跑 SCC。快速识别: 一堆“要么…要么…”或蕴含条件; 布尔变量赋值: “至少一个为真/假”。

API 接入:

- 变量 i 的两个状态: $i*2$ (假) 与 $i*2+1$ (真) (0-based 变量, 节点 $0..2n-1$)。
- `add_imp(a, b):` $a \Rightarrow b$; `add_or(a, b):` $(!a \Rightarrow b)$ 且 $(!b \Rightarrow a)$ 。
- `solve():` 跑 SCC; 若 `comp[x*2] == comp[x*2+1]` 无解; 否则 $x = \text{comp}[x*2+1] > \text{comp}[x*2]$ (真)。
- 常用模板: `add_clause(x, vx, y, vy):` $x=vx$ OR $y=vy$ 。
- 依赖: TarjanSCC (04.9 节), 抄板时一起抄上。

关键点: 赋值 = 比较两个分量编号 (拓扑序: `comp` 编号大者为真, 取决于 Tarjan 编号顺序, 统一用 `comp[true] > comp[false]` 判); 建图对称性 (蕴含图每个变量正反都在); 输出方案任取满足即可。

```

struct TwoSAT {
    TarjanSCC scc; // 复用 04.9
    TwoSAT(int n) : scc(2 * n) {}
    int id(int var, bool val) { return 2 * var + (val ? 1 : 0); }
    void add_imp(int a, int b) { scc.add_edge(a, b); } // a => b
    // x == vx OR y == vy
    void add_or(int x, bool vx, int y, bool vy) {
        add_imp(id(x, !vx), id(y, vy));
        add_imp(id(y, !vy), id(x, vx));
    }
    bool solve(vector<bool>& res) {
        scc.run();
        int n = (int)res.size();
        for (int i = 0; i < n; ++i)
            if (scc.comp[2 * i] == scc.comp[2 * i + 1]) return false;

        for (int i = 0; i < n; ++i)
            res[i] = scc.comp[2 * i + 1] < scc.comp[2 * i]; // 真
        // 分量编号小 (Tarjan 先出栈编号小=拓扑序靠后)
        return true;
    }
};

```

04.13 欧拉路径/回路 (Hierholzer)

【简介】

一笔画问题: 经过每条边恰一次的路径/回路。有向/无向判定条件不同; 用栈模拟递归删边。

快速识别: “一笔画”、走遍所有边恰一次、单词接龙 (首尾字母构图)。

API 接入:

- 无向图: 回路 = 所有点度数为偶; 路径 = 恰 0 或 2 个奇度点 (从奇度点出发)。
- 有向图: 回路 = 入度=出度; 路径 = 一个点出度=入度+1 (起点)、一个入度=出度+1 (终点)、其余相等。
- `hierholzer(start):` 返回边序列 (或点序列)。

关键点: 递归删边用当前弧指针 (while 弹边) 防

$O(m^2)$; 栈存路径, 回溯时压入; 无向图删边要删两条 (标记边 id); 图必须连通 (忽略孤立点); 字典序最小: 邻接表排序后按序访问。

```

// 有向图欧拉路径 (字典序最小版): 返回顶点序列或空 (无解)
vector<int> euler_path_directed(int n, vector<vector<int>>& g) {
    vector<int> indeg(n), outdeg(n);
    for (int u = 0; u < n; ++u) {
        sort(all(g[u]));
        outdeg[u] = (int)g[u].size();
        for (int v : g[u]) indeg[v]++;
    }
    int start = -1, a = 0, b = 0;
    for (int u = 0; u < n; ++u) {
        if (outdeg[u] == indeg[u] + 1) { if (start != -1 && a) return n; }
        else if (indeg[u] == outdeg[u] + 1) { ++b; }
        else if (indeg[u] != outdeg[u]) return {};
    }
    if (a > 1 || b > 1) return {};
    if (start == -1) for (int u = 0; u < n; ++u) if (outdeg[u]) { start = u; break; }
    if (start == -1) return {};
    vector<int> it(n, 0), stk{start}, res;
    while (!stk.empty()) {
        int u = stk.back();
        if (it[u] < (int)g[u].size()) stk.push_back(g[u][it[u]++]);
        else { res.push_back(u); stk.pop_back(); }
    }
    reverse(all(res));
    return res;
}

// 无向图欧拉路径 (Hierholzer 标准版): 返回顶点序列; 无解返回空
// 用法: g[u] 存 (v, edge id); 先 add edge 给每条边分配 id
// 判定: 回路 = 所有点度数为偶数; 路径 = 恰 0 或 2 个奇度点 (从奇度点出发)
vector<int> euler_path_undirected(int n, vector<vector<pair<int,int>>& g, int m) {
    vector<char> used(m, 0); // 边是否已走过
    vector<int> deg(n), it(n, 0);
    for (int u = 0; u < n; ++u) {
        sort(all(g[u]));
        deg[u] = (int)g[u].size();
    }
    int odd = 0, start = -1;
    for (int u = 0; u < n; ++u) if (deg[u] % 2) { ++odd; if (start == -1) start = u; }
    if (odd != 0 && odd != 2) return {};
    if (start == -1) for (int u = 0; u < n; ++u) if (deg[u]) { start = u; break; }
    if (start == -1) return {};
    vector<int> stk{start}, res;
    while (!stk.empty()) {
        int u = stk.back();
        bool advanced = false;
        while (it[u] < (int)g[u].size()) {
            auto [v, id] = g[u][it[u]++];
            if (used[id]) continue; // 这条边已走过
            used[id] = 1; // 双向同 id, 一次标记
            stk.push_back(v);
            advanced = true;
            break;
        }
        if (!advanced) { res.push_back(u); stk.pop_back(); }
    }
    reverse(all(res));
    return res;
}

// 说明: 无向版删除边需处理重边, 标准做法是边存 (v, id) 并标记 used[id],
// 入栈前 if (!used[id]) { used[id] = 1; stk.push_back(v); }, 判定后翻转即可。

```

04.14 二分图最大匹配 (匈牙利 / Hopcroft-Karp)

【简介】

左部点匹配右部点，无共用点的最大边数。匈牙利 $O(VE)$ （增广路），Hopcroft-Karp $O(E \sqrt{V})$ （BFS 分层 + DFS 多路增广）。

快速识别：配对/分配问题（人-任务、点-位置）；最小点覆盖 = 最大匹配（König）；最大独立集 = n - 最大匹配（二分图）。

API 接入：

- 匈牙利: `hungarian(g, nl, nr)`: `g[u] = u` 的可配右部点列表（左部 $0..nl-1$ ）；返回 `matchR[v]`（右部配的左部点，-1 未配）。
- 最大匹配数 = 匹配成功的个数。
- 右部点较大时用时时间戳优化（vis 用 int 数组 + cur 标记避免每轮 memset）。

关键点：每次增广重置 vis（或用时时间戳）；匈牙利从左部每个未匹配点开始找增广路；matchR 只记右部→左部；König：最小点覆盖 = 最大匹配（构造方案：从左侧未匹配点 DFS 标记）；最小路径覆盖 = n - 最大匹配（DAG）。

```
// 匈牙利（时间戳优化）
struct Hungarian {
    int nl, nr;
    vector<vector<int>>> g;
    vector<int> matchR, vis;
    int cur = 0;
    Hungarian(int nl, int nr) : nl(nl), nr(nr) {
        g.resize(nl); matchR.assign(nr, -1); vis.assign(nr, 0);
    }
    void add_edge(int u, int v) { g[u].push_back(v); }
    bool dfs(int u) {
        for (int v : g[u]) {
            if (vis[v] == cur) continue;
            vis[v] = cur;
            if (matchR[v] == -1 || dfs(matchR[v])) { matchR[v] = u;
                return true; }
            return false;
        }
    }
    int max_matching() {
        int res = 0;
        for (int u = 0; u < nl; ++u) { ++cur; if (dfs(u)) ++res; }
        return res;
    }
};
```

04.15 网络流：Dinic 最大流

【简介】

残量网络 BFS 分层 + DFS 多路增广 + 当前弧优化。实际跑得很快，覆盖绝大多数最大流/最小割建模。

快速识别：“容量”、“最多能运/选/匹配多少”、最小割；二分图匹配的通用替代。

API 接入：

- Dinic `dinic(n)`: 点编号 $1..n$ （内部 $n+1$ 槽）。
- `add_edge(u, v, cap)`: 有向容量边，反向边自动建好（容量 0），不要再手动加。
- `max_flow(s, t)`: 返回 i64。
- 无向容量边: `add_edge(u,v,c)`; `add_edge(v,u,c)`; 两条。
- 最小割：跑完后从 `s` 沿残量边可达的点集即割的一侧。
- 点容量：拆点 $u \rightarrow u+n$ 连容量边。

关键点：当前弧 `it` 每轮 BFS 重置；`add_edge` 的反向边 `rev` 指针要互相指向；换源汇重跑前要新建或 `init` 清空；i64 容量防溢出；二分图匹配（左连源右连汇容量 1）也适用。

```
struct Dinic {
    struct Edge { int to, rev; i64 cap; };
    int n;
    vector<vector<Edge>>> g;
    vector<int> level, it;
    Dinic(int n = 0) { init(n); }
    void init(int n) {
        n = n_;
        g.assign(n + 1, {});
        level.assign(n + 1, 0);
        it.assign(n + 1, 0);
    }
};
```

```
void add_edge(int u, int v, i64 cap) {
    g[u].push_back({v, (int)g[v].size(), cap});
    g[v].push_back({u, (int)g[u].size() - 1, 0});
}
bool bfs(int s, int t) {
    fill(level.begin(), level.end(), -1);
    queue<int> q;
    level[s] = 0; q.push(s);
    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (auto& e : g[u]) if (e.cap > 0 && level[e.to] == -1) {
            level[e.to] = level[u] + 1;
            q.push(e.to);
        }
    }
    return level[t] != -1;
}
i64 dfs(int u, int t, i64 f) {
    if (u == t) return f;
    for (int& i = it[u]; i < (int)g[u].size(); ++i) {
        Edge& e = g[u][i];
        if (e.cap <= 0 || level[e.to] != level[u] + 1) continue;
        i64 ret = dfs(e.to, t, min(f, e.cap));
        if (ret) {
            e.cap -= ret;
            g[e.to][e.rev].cap += ret;
            return ret;
        }
    }
    return 0;
}
i64 max_flow(int s, int t) {
    i64 flow = 0;
    while (bfs(s, t)) {
        fill(it.begin(), it.end(), 0);
        while (i64 f = dfs(s, t, (1LL << 62))) flow += f;
    }
    return flow;
};
```

04.16 最小费用最大流（SPFA/Dijkstra + 势能）

【简介】

在最大流前提下总费用最小。经典 SPFA

每次找最短路增广；Dijkstra + 势能（Johnson）更快。

快速识别：“每个单位流量有代价”、分配任务求最小总成本、匹配带权。

API 接入：

- MCMF(n): 点 $1..n$; `add_edge(u, v, cap, cost)`。
- `min_cost_max_flow(s, t)`: 返回 (maxflow, mincost)。
- 费用为负且无负环时用 SPFA 版；有负权边用势能版。

关键点：反向边费用为 $-cost$ ；SPFA 每轮找最短路；Dijkstra 势能 $h[v] += dist[v]$ 处理负边；流量需求恰好 `F` 时控制增广量；费用可能为负值注意 INF。

```
struct MCMF {
    struct Edge { int to, rev; i64 cap, cost; };
    int n;
    vector<vector<Edge>>> g;
    vector<i64> dist, h; // h: 势能
    vector<int> pv, pe;
    MCMF(int n_) : n(n_) { g.resize(n + 1); dist.resize(n + 1); h.assign(n + 1, 0); pv.resize(n + 1); pe.resize(n + 1); }
    void add_edge(int u, int v, i64 cap, i64 cost) {
        g[u].push_back({v, (int)g[v].size(), cap, cost});
        g[v].push_back({u, (int)g[u].size() - 1, 0, -cost});
    }
    bool dijkstra(int s, int t) {
        fill(dist.begin(), dist.end(), LINF);
        priority_queue<pair<i64, int>, vector<pair<i64, int>>, greater<>> pq;
        dist[s] = 0; pq.push({0, s});
        while (!pq.empty()) {
            auto [d, u] = pq.top(); pq.pop();
            if (d != dist[u]) continue;
            for (int i = 0; i < (int)g[u].size(); ++i) {
                auto& e = g[u][i];
                if (e.cap > 0 && dist[e.to] > d + e.cost + h[u] - h[e.to]) {
                    dist[e.to] = d + e.cost + h[u] - h[e.to];
                    pv[e.to] = u; pe[e.to] = i;
                }
            }
        }
    }
};
```

```

        pq.push({dist[e.to], e.to});
    }
}
return dist[t] != LINF;
}
pair<i64, i64> min cost max flow(int s, int t) {
    i64 flow = 0, cost = 0;
    // 有负费用边时先跑 Bellman-Ford 初始化势能 h (否则 reduced cost 为负, Dijkstra 错误且会卡死)
    fill(h.begin(), h.end(), 0);
    for (int it = 0; it < n; ++it) {
        bool upd = false;
        for (int u = 1; u <= n; ++u)
            for (auto& e : g[u])
                if (e.cap > 0 && h[e.to] > h[u] + e.cost) { h[e.to] = h[u] + e.cost; upd = true; }
        if (!upd) break;
    }
    while (dijkstra(s, t)) {
        for (int u = 1; u <= n; ++u) if (dist[u] != LINF) h[u] += dist[u];
        i64 f = (1LL << 62);
        for (int v = t; v != s; v = pv[v]) f = min(f, g[pv[v]][p[e[v]].cap]);
        for (int v = t; v != s; v = pv[v]) {
            auto& e = g[pv[v]][pe[v]];
            e.cap -= f;
            g[v][e.rev].cap += f;
        }
        flow += f;
        cost += f * (h[t] - h[s]); // 势能版: h[t]-h[s] 才是真实费用 (BF 初始化后 h[s] ≠ 0)
    }
    return {flow, cost};
};

```

04.17 最小割建模速查

【简介】

最大流 = 最小割。经典建模：最大权闭合图、最小点覆盖、项目选择、二选一割边。识别“割”对应的决策含义。

快速识别：“分成两组代价最小”、“选与不选”、收益最大化（源连正收益、负收益连汇）。

API 接入：

- 最大权闭合图：正权点连源（cap=权）、负权点连汇（cap=|权|）、依赖边 inf；答案 = 正权和 - 最小割。
- 二选一（每个点选 A 或 B，冲突有代价）：源连 A、B 连汇，冲突点对连 inf/代价边。
- 最小点覆盖（二分图）= 最大匹配 = 最小割。
- 割集构造：跑完最大流后从 S 可达的点 = S 侧。

关键点：inf 边表示“不可切断的依赖”（cap = 总流量上限如 1e15）；割的含义要想清楚（S 侧 = 选/不选）；输出方案 = 可达性标记；负权点要转正后建模。

```

// 最大权闭合图模板：
// Dinic dinic(n + 2); s = n + 1, t = n + 2;
// 正权: add_edge(s, i, w[i]); 负权: add_edge(i, t, -w[i]);
// 依赖 u -> v (选 u 必须选 v): add_edge(u, v, INF);
// ans = sum(正权) - dinic.max_flow(s, t);

```

04.18 网络流：上下界 / 最大权匹配补充 (HLPP / 带花树要点)

【简介】

上下界可行流/最大流/最小流：每条边容量在 [low, high]，先转成“下界必流 + 自由流”；HLPP 是更快的最大流（稠密大图）；带花树（一般图最大匹配） $O(n^3)$ 。快速识别：边容量有下界：“至少流多少”；稠密图最大流卡 Dinic 时；一般图（非二分）匹配。

API 接入：

- 上下界可行流：对边 (u, v, low, high)：add(u, v, high-low)，同时 $d[u] -= \text{low}$, $d[v] += \text{low}$ （入出平衡）；建超级源汇 S/T, $d > 0$ 连 S、 $d < 0$ 连 T；跑 S->T 最大流，满流则可行。
- 上下界最大流：先判可行，再从原源汇跑一次。

- 带花树：general_match(n, g) 返回匹配；实现长，整段照抄。

关键点：上下界核心是 d 数组（入度-出度）与超级源汇；满流判定 = 所有 S 出边流满；HLPP 0-based 与 Dinic 1-based 不同，别混（见 00.4 索引表）。

依赖：Dinic（04.15 节），抄板时一起抄上。

// 上下界可行流完整实现（1-based 点，Dinic 见 04.15）
用法：// BoundFlow bf(n); // n 个点 (1..n)
bf.add_edge(u, v, low, high); // 边容量 [low, high]
bf.add_supply(u, v, low, high) // 同 add_edge (写法一)
bf.solve() 返回是否可行；可行时边 i 的实际流量 =
bf.flow_of[i] + low[i]`cpp

```

// 上下界可行流完整实现 (1-based 点; Dinic 见 04.15, 抄板时一起抄上)
// 用法: BoundFlow bf(n); bf.add_edge(u, v, low, high); 全部加完后 bf.solve()
// 返回 true = 可行; 边 i 的实际流量 = lowv[i] + (high-low) - 残量剩余 (见说明)
struct BoundFlow {
    int n, S, T;
    Dinic dinic;
    vector<i64> d; // 入出平衡数组: d[u] = low
    vector<i64> lowv; // 每条边下界
    BoundFlow(int n_) : n(n_), S(n_ + 1), T(n_ + 2), dinic(n_ + 2), d(n_ + 3, 0) {}
    void add_edge(int u, int v, i64 low, i64 high) {
        lowv.push_back(low);
        dinic.add_edge(u, v, high - low); // 自由流量 = 上界 - 下界
        d[u] -= low; d[v] += low; // 下界强制流计入平衡
    }
    bool solve() { // 判可行: 超级源汇满流即可行
        i64 need = 0;
        for (int i = 1; i <= n; ++i) {
            if (d[i] > 0) dinic.add_edge(S, i, d[i]), need += d[i];
            else if (d[i] < 0) dinic.add_edge(i, T, -d[i]);
        }
        return dinic.max_flow(S, T) == need;
    }
};
// 说明: 上界最大流 = 判可行后, 在原图上 (去掉超级源汇) 从 s 到 t 再跑一次最大流;
// 最小流类似 (先跑 t->s 最大流使下界尽量不补)。取每边实际流需在 Dinic 里记录
// 正向边初始容量 (high-low) 与最终剩余, 实际流 = lowv[i] + (high-low) - 剩余。

```

04.19 二分图最大权完美匹配 (KM)

【简介】

左右部大小相等（或补到相等）的带权完美匹配求最大总权。 $O(n^3)$ （DFS 版）或 $O(n^3)$ 优化版。也可用 MCMF 替代（n 小时）。

快速识别：左右各 n 个，一一配对收益最大；“n 个任务分给 n 个人”。

API 接入：

- KM: init(n); add_edge(u, v, w); max_weight_match() 返回最大总权；匹配方案存 matchL/matchR。
- 求最小权：权值取负后跑最大，答案取负。
- 左右不等：补虚拟点（权 0 或 -INF 视题意）。

关键点：标杆（顶标）lx/ly、slack 松弛数组、交替树 visited；DFS 版最坏 $O(n^3)$ ，用 slack 优化到 $O(n^3)$ ；不完美匹配（允许不配对）时把缺失边设 0 或 -INF 按题意；相等子图 = $lx + ly = w$ 的边。

```

// KM (O(n^3), 求最大权完美匹配)
struct KM {
    int n;
    vector<vector<i64>>> w;
    vector<i64> lx, ly, slack;
    vector<int> matchL, matchR;
    vector<char> visL, visR;
    KM(int n_) : n(n_) {
        w.assign(n, vector<i64>(n, -LINF));
        lx.assign(n, 0); ly.assign(n, 0);
        slack.assign(n, 0); // 必须初始化
        // (空 vector 访问会崩)
        matchL.assign(n, -1); matchR.assign(n, -1);
        visL.assign(n, 0); visR.assign(n, 0);
    }
}

```

```

void add_edge(int u, int v, i64 val) { w[u][v] = val; }
bool dfs(int u) {
    visL[u] = 1;
    for (int v = 0; v < n; ++v) if (!visR[v]) {
        i64 gap = lx[u] + ly[v] - w[u][v];
        if (gap == 0) {
            visR[v] = 1;
            if (matchR[v] == -1 || dfs(matchR[v])) {
                matchR[v] = u; matchL[u] = v;
                return true;
            }
        } else slack[v] = min(slack[v], gap);
    }
    return false;
}

i64 max_weight_match() {
    for (int i = 0; i < n; ++i) lx[i] = *max_element(all(w[i]));
    for (int u = 0; u < n; ++u) {
        fill(all(slack), LINF);
        fill(all(visL), 0); fill(all(visR), 0);
        while (!dfs(u)) {
            i64 d = LINF;
            for (int v = 0; v < n; ++v) if (!visR[v]) d = min(d,
                for (int i = 0; i < n; ++i) { if (visL[i]) lx[i] -=
d; if (visR[i]) ly[i] += d; else slack[i] -= d; }
                fill(all(visL), 0); fill(all(visR), 0);
            }
        }
        i64 res = 0;
        for (int u = 0; u < n; ++u) res += w[u][matchL[u]];
        return res;
    }
};

```

04.20 图论杂项：斯坦纳树 / Matrix-Tree / Stoer-Wagner / 朱刘

【简介】

斯坦纳树（选若干关键点连通的最小树，状压 DP + 多源最短路）；Matrix-Tree（生成树计数，度数-邻接拉普拉斯行列式）；Stoer-Wagner（无向图全局最小割）；朱刘（有向图最小树形图）。

快速识别：斯坦纳树：“选 k 个点连通的代价最小”（ k 小）；Matrix-Tree：“生成树个数”（模数）；Stoer-Wagner：“整个图的最小割”（不指定源汇）；朱刘：“根到所有点的最小有向生成树”。

API 接入：

- 斯坦纳树：dp[mask][u]；dp[mask][u] = min(dp[mask][u], dp[sub][u] + dp[mask^sub][u])，再跑 Dijkstra 松弛。复杂度 $O(3^k n + 2^k m \log n)$ ， $k \leq 10$ 。
- Matrix-Tree：det(拉普拉斯主子式) 取模。
- Stoer-Wagner：合并最大度点对， $O(n^3)$ 。
- 朱刘： $O(nm)$ 找最小入边 + 缩环。

关键点：斯坦纳树子集枚举注意 sub = (mask-1)&mask 从大到小（防重复）；Matrix-Tree 任意删一行一列；Stoer-Wagner 结果可能为

0（不连通）；朱刘无环时每点选最小入边即可，有环要缩点迭代。

```

// 斯坦纳树 (k <= 10, 点 1..n, 边权 w):
// vector<vector<i64>>> dp(1<k, vector<i64>(n+1, LINF));
// for i: dp[1<i][key[i]] = 0;
// for mask: for sub: dp[mask][u] = min(dp[mask][u], dp[sub][u] + d
p[mask^sub][u]);
// 每个 u 用 Dijkstra 松弛 dp[mask][v] = min(dp[mask][v], dp[ma
sk][u] + w);
// ans = min u dp[(1<k)-1][u];

// Matrix-Tree: A = 度数矩阵 - 邻接矩阵; ans = det(A 删第 n 行第 n 列) (模
意义高斯消元)

// Stoer-Wagner 全局最小割: 每次合并“最大割度”点对, ans = min(每次最后点对
割度)
// 朱刘: 每点选最小入边, 判环缩点重算

```

04.21 Johnson 全源最短路（含负边）

【简介】

有负边但无负环的全源最短路：SPFA 求势能 h ，重赋权 $w' = w + h[u] - h[v]$ （非负），再对每个点跑 Dijkstra。复杂度 $O(nm + nm \log n)$ 。

快速识别：全源最短路 + 负边：多次询问任意两点最短路。

API 接入：

- johnson(n, edges): edges 为 (u, v, w) 列表 (0-based 或 1-based 自定)；返回 $n \times n$ 距离矩阵。
- 先虚拟源点 0 连所有点 0 权边，SPFA 判负环；无负环则势能存在。
- 真实距离 = dist' + h[v] - h[u]。

关键点：虚拟源 + SPFA 求

h ；重赋权后所有边非负；最后还原真实距离；有负环直接报无解； $n \leq 1000$ 时 $O(n^2 \log n)$ 可接受。

```

// 要点: h = SPFA(虚拟源 0 到所有点 0 权边); w'(u, v) = w + h[u] - h[v] >=
0
// dist_real(u, v) = dijkstra(u) 结果 + h[v] - h[u]

```

04.22 最短路计数 / 次短路 / 差分约束补充 (K 短路 A*)

【简介】

最短路条数（模）；次短路（严格/非严格）；第 k 短路 (A*：反图最短路做估价函数)。配套分层技巧：分层图（买票、破墙）。

快速识别：“有多少条最短路”；“第二短”；“第 k 短”；“允许使用 k 次特殊操作”（分层图， k 小）。

API 接入：

- 最短路计数：Dijkstra 时 dist[v] == d+w 累加 ways；模意义取模。
- 次短路：d1/d2 双距离数组，每次松弛两个都尝试更新。
- 第 k 短路：反图最短路 g 作估价 + A* ($f = g + h$)，弹出 t 第 k 次即答案。
- 分层图：(u, layer) 编码节点，跨层边代价 = 特殊操作。

关键点：最短路计数只在出队时累加（严格在 dist 确定后），否则会重；次短路更新顺序：if (nd < d1[v]) { d2[v] = d1[v]; d1[v] = nd; } else if (nd < d2[v]) d2[v] = nd;; A* 第 k 短路每点可多次入队。

```

// —— 用法示例：以下语句抄入 solve() 中使用 ——
// 次短路（双数组）：
vector<i64> d1(n + 1, LINF), d2(n + 1, LINF);
d1[s] = 0;
priority queue<..., greater<>> pq;
pq.push({0, s});
while (!pq.empty()) {
    auto [d, u] = pq.top(); pq.pop();
    if (d > d2[u]) continue;
    for (auto [v, w] : g[u]) {
        i64 nd = d + w;
        if (nd < d1[v]) { d2[v] = d1[v]; d1[v] = nd; pq.push({nd, v}
); }
        else if (nd > d1[v] && nd < d2[v]) { d2[v] = nd; pq.push({nd
, v}); }
    }
}
// d2[t] = 次短路（严格大于最短路）

```

04.23 一般图最大匹配（带花树 Blossom）

【简介】

非二分图的最大匹配（如奇环图）。 $O(n^3)$

均摊。带花树的核心：遇到奇环时“缩花”成超级点继续找增广路。

快速识别：图不是二分图但问最大匹配/最短路覆盖变体；“奇环”存在时的匹配问题。

API 接入：

- Blossom: init(n); add_edge(u, v) (0-based); max_match() 返回最大匹配数。
- match[u]: u 的匹配点 (-1 未匹配)；可输出方案。
- 二分图也能用（更慢），但二分图优先用 04.14 匈牙利/HK。

关键点：花 = 奇环，lca 找环，mark 缩花；match

数组对称维护；实现长，抄板整段复制；复杂度 $O(n^3)$ ($n \leq 500$ 稳妥)。带权一般图匹配用带权带花树（更长，需要时另抄）。

```

// 带花树（一般图最大匹配，0-based 点编号）
struct Blossom {

```

```

int n;
vector<vector<int>> g;
vector<int> match, p, base, vis, q;
int tim = 0;
Blossom(int n) : n(n) {
    g.assign(n, {});
    match.assign(n, -1); p.assign(n, 0);
    base.assign(n, 0); vis.assign(n, 0);
}
void add_edge(int u, int v) { g[u].push_back(v); g[v].push_back(u); }
int lca(int u, int v) {
    vector<char> used(n, 0);
    while (true) {
        u = base[u]; used[u] = 1;
        if (match[u] == -1) break;
        u = p[match[u]];
    }
    while (true) {
        v = base[v];
        if (used[v]) return v;
        v = p[match[v]];
    }
}
void mark(int u, int b, int child) {
    while (base[u] != b) {
        vis[u] = vis[match[u]] = 1;
        p[u] = child;
        child = match[u];
        u = p[child];
    }
}
bool bfs(int root) {
    q.clear();
    fill(all(vis), 0);
    for (int i = 0; i < n; ++i) base[i] = i;
    q.push_back(root); vis[root] = 1;
    for (int head = 0; head < (int)q.size(); ++head) {
        int u = q[head];
        for (int v : g[u]) {
            if (base[u] == base[v] || match[u] == v) continue;
            if (v == root || (match[v] != -1 && p[match[v]] != -1)) {
                int curbase = lca(u, v);
                vis[v] = vis[u] = 1;
                mark(v, curbase, u);
                mark(u, curbase, v);
                for (int i = 0; i < n; ++i)
                    if (vis[base[i]]) { vis[i] = 1; q.push_back(i); }
            } else if (p[v] == -1) {
                p[v] = u;
                if (match[v] == -1) { // 找到增广路，沿 p 链翻转
                    int cur = v;
                    while (cur != -1) {
                        int pv = p[cur];
                        int nxt = match[pv];
                        match[cur] = pv;
                        match[pv] = cur;
                        cur = nxt;
                    }
                    return true;
                }
                vis[match[v]] = 1;
                q.push_back(match[v]);
            }
        }
    }
    return false;
}
int max_match() {
    int res = 0;
    fill(all(p), -1);
    for (int i = 0; i < n; ++i) if (match[i] == -1) if (bfs(i)) ++res;
    return res;
}
// 用法: Blossom b(n); b.add_edge(u, v); cout << b.max_match();

```

04.24 同余最短路（模数建图）

【简介】

形如“用若干种面值的币凑出金额，问最小/最大/能否凑出”的问题：以最小的面值做模数，建 mod 个点的最短路图：点 i 到点 $(i + a[j]) \% \text{mod}$ 连边权 $a[j]$ 。答案 = 从 0 出发的最短路。

快速识别：“硬币凑数问某区间内可凑出多少个/最小值”； n 种面值 $a[i]$ ，问 $[L, R]$ 内能凑出的数；每个物品可用任意次的“最小可达值”。

API 接入：

- 取 $\text{base} = \min(a)$ ；建 base 个点（余数 $0..base-1$ ）；对每个面值 $a[j]$ 加边 $i \rightarrow (i+a[j]) \% \text{base}$ ，边权 $a[j]$ 。
- 跑 Dijkstra 得 $\text{dist}[i]$ = 能凑出且模 $\text{base} \equiv i$ 的最小值。
- 统计：值 x 可达当且仅当 $\text{dist}[x \% \text{base}] \leq x$ ； $[L, R]$ 内可达数 = $\sum \text{over } i \text{ of } \max(0, (R - \text{dist}[i]) / \text{base} + 1)$ 减 L 部分。

关键点：模数取最小面值（点数最小）；边权 = 面值本身（实际加了多少）； dist

语义是“该余数类的最小可达值”，之后所有 $+base$

的值都可达；复杂度 $O(\text{base} * n * \log \text{base})$ ， base 取 $1e5$ 内可行。

```

// 同余最短路（硬币面值 a，问 [L, R] 内能凑出的数的个数）
// 依赖 04.1 Dijkstra
i64 count_reachable(const vector<i64>& a, i64 L, i64 R) {
    i64 base = *min_element(all(a));
    int m = (int)base;
    vector<vector<pair<int, i64>>> g(m);
    for (int i = 0; i < m; ++i)
        for (i64 x : a)
            g[i].push_back({(int)((i + x) % base), x});
    vector<i64> dist = dijkstra(0, m, g); // 04.1, 点编号 0..m-1 (1-based 需适配)
    i64 res = 0;
    for (int i = 0; i < m; ++i) {
        if (dist[i] > R) continue;
        i64 first = max(dist[i], L); // 该余数类内第一个 >= L 的可达值
        if (first > R) continue;
        res += (R - first) / base + 1;
    }
    return res;
}

```

04.25 三元环 / 最小环计数

【简介】

无向图三元环计数 $O(m \sqrt{m})$ （按度定向后枚举）；最小环（无权 BFS 删边、有权 Floyd 变体）。

快速识别：“三元环/四元环个数”；“图中最小环长度”；社交网络三角计数。

API 接入：

- 三元环计数：按（度，编号）大小定向成 DAG，对每条有向边枚举公共出点。
- 无权最小环：对每条边 (u, v) 临时删掉，BFS 求 $u \rightarrow v$ 最短路 $+1$ ，取 $\min$ ； $O(m(n+m))$ 。
- 有权最小环：Floyd 变体（在 k 轮前检查 $i-j$ 经 k 的环）。

关键点：三元环定向后复杂度 $O(m \sqrt{m})$ ；

最小环注意不能直接用最短路（会走回同一条边）； n

小 (≤ 500) 用 Floyd 变体最稳。

```

// 三元环计数 (0-based 无向图)
i64 count_triangles(int n, vector<pair<int, int>> edges) {
    vector<int> deg(n, 0);
    for (auto [u, v] : edges) ++deg[u], ++deg[v];
    vector<vector<int>> g(n);
    for (auto [u, v] : edges) {
        // 按 (度, 编号) 小的指向大的
        if (deg[u] > deg[v] || (deg[u] == deg[v] && u > v)) swap(u, v);
        g[u].push_back(v);
    }
    vector<char> mark(n, 0);
    i64 ans = 0;
    for (int u = 0; u < n; ++u) {
        for (int v : g[u]) mark[v] = 1;
        for (int v : g[u])
            for (int w : g[v])
                if (mark[w]) ++ans;
        for (int v : g[u]) mark[v] = 0;
    }
    return ans;
}

```



```
// 无权最小环 (n 小): 对每条边 (u,v), 临时删掉后 BFS 求 dist(u,v)+1, 取最小
```

04.26 最大团 Bron-Kerbosch (bitset 优化)

【简介】

无向图最大团 (两两相邻的最大点集)。朴素 Bron-Kerbosch $O(3^{(n/3)})$, bitset 优化后 $n \leq 100$ 可用。

快速识别: “两两都有边/互相认识的最大集合”; 互补图 = 最大独立集; $n \leq 100$ 且无多项式解法时。

API 接入:

- `max_clique(adj)`: `adj` 为 bitset 邻接矩阵; 返回最大团大小。
- 输出方案: 维护当前团点集 (扩展代码)。

关键点: Pivot 优化 (选邻居最多的点做轴) 大幅剪枝; R

当前团、P 候选、X 禁选; bitset 用 `std::bitset<128>` 或动态 bitset; 最大独立集 = 补图最大团。

```
// 最大团 (bitset 邻接, n <= 128 用 std::bitset<128>)
struct MaxClique {
    int n;
    vector<bitset<128>> adj;
    int best = 0;
    MaxClique(int n_, const vector<bitset<128>>& adj_) : n(n_), adj(adj_) {}
    void bronk(bitset<128> R, bitset<128> P, bitset<128> X) {
        if (P.none() && X.none()) {
            best = max(best, (int)R.count());
            return;
        }
        if (P.count() + R.count() <= best) return; // 剪枝
        // pivot: P ∪ X 中度最大的点
        int pv = -1;
        for (int i = 0; i < n; ++i)
            if (P[i] || X[i]) if (pv == -1 || (adj[i] & (P | X)).count() > (adj[pv] & (P | X)).count()) pv = i;
        bitset<128> cand = P & ~adj[pv]; // 排除 pivot 邻居
        for (int v = 0; v < n; ++v) if (cand[v]) {
            R.set(v);
            bronk(R, P & adj[v], X & adj[v]);
            R.reset(v);
            P.reset(v);
            X.set(v);
        }
    }
    int solve() {
        bitset<128> all;
        for (int i = 0; i < n; ++i) all.set(i);
        bronk(0, all, 0);
        return best;
    }
};
// 最大独立集 = 补图的最大团 (补图 adj'[i][j] = !adj[i][j] 且 i != j)
```

04.27 线段树优化建图 (点到区间 / 区间到点)

【简介】

把“点 $\rightarrow$ 区间 $[l, r]$ 的所有点”或“区间 $\rightarrow$ 点”的连边用线段树 $O(\log n)$ 条边代替 $O(n)$ 条。建两棵线段树 (出树连区间、入树连点), 跑最短路/网络流。2025 EC 网络赛 C (区间二分图匹配)、区域赛最短路题高频。

快速识别: “u 到 $[l, r]$ ”

内每个点都有一条边 (相同权) 或反向; 区间内所有点连到 v; 最短路/最小割建模含区间边。

API 接入:

- 节点总数 = n (原图点) + $2 \cdot (4n)$ (两棵线段树节点); 点 $1..n$ 原图点。
- `SegOptGraph(n)`: 自动建树; `add_point_to_range(u, l, r, w)`、`add_range_to_point(l, r, v, w)`。
- 建完跑最短路 (Dijkstra) 或连源汇跑流。
- 无向区间边: 两条都加。

关键点: 出树从树节点指向子节点 (权 0), 入树从子节点指向父节点 (权 0); 点连出树叶 (权 0)、入树叶连点 (权 0); 点 $\rightarrow$ 区间 = 点连出树叶 + 出树节点连入树区间节点 (边权

w); 区间 $\rightarrow$ 点镜像。两棵树方向别搞反 (出树向下、入树向上)。

```
// 线段树优化建图 (点  $1..n$ , 区间最短路/流建模)
// 依赖 04.1 Dijkstra 的图存储 (vector<vector<pair<int, i64>>>, 1-based)
struct SegOptGraph {
    int n, tot;
    vector<vector<pair<int, i64>>> g; // 1-based, 总节点 tot
    vector<int> out_l, out_r, in_l, in_r; // 出树/入树的左右子 (0 = 空)
    SegOptGraph(int n) : n(n), tot(n) {
        int sz = 4 * n + 4;
        g.resize(4 * (4 * n) + 8);
        out_l.assign(sz, 0); out_r.assign(sz, 0);
        in_l.assign(sz, 0); in_r.assign(sz, 0);
        build_out(1, 1, n);
        build_in(1, 1, n);
    }
    void add_edge(int u, int v, i64 w) { g[u].push_back({v, w}); }
    void build_out(int p, int l, int r) { // 出树: 父  $\rightarrow$  子 (权 0)
        if (l == r) return;
        int m = (l + r) / 2;
        int lc = ++tot, rc = ++tot;
        out_l[p] = lc; out_r[p] = rc;
        add_edge(p, lc, 0); add_edge(p, rc, 0);
        build_out(lc, l, m); build_out(rc, m + 1, r);
    }
    void build_in(int p, int l, int r) { // 入树: 父  $\rightarrow$  子 (权 0)
        if (l == r) return;
        int m = (l + r) / 2;
        int lc = ++tot, rc = ++tot;
        in_l[p] = lc; in_r[p] = rc;
        add_edge(lc, p, 0); add_edge(rc, p, 0);
        build_in(lc, l, m); build_in(rc, m + 1, r);
    }
    // 叶子映射: 点 x 在出树/入树中的叶子节点编号
    int leaf_out, leaf_in;
    void find_leaf(int p, int l, int r, int x, bool is_out) {
        if (l == r) { if (is_out) leaf_out = p; else leaf_in = p; return; }
        int m = (l + r) / 2;
        if (x <= m) find_leaf(is_out ? out_l[p] : in_l[p], l, m, x, is_out);
        else find_leaf(is_out ? out_r[p] : in_r[p], m + 1, r, x, is_out);
    }
    int leaf(int x, bool is_out) { find_leaf(1, 1, n, x, is_out); return is_out ? leaf_out : leaf_in; }
    // 区间  $[l, r]$  在树上的节点集合 (回调 node 编号)
    template <class F>
    void cover(int p, int tl, int tr, int l, int r, bool is_out, F&& f) {
        if (l <= tl && tr <= r) { f(p); return; }
        int m = (tl + tr) / 2;
        if (l <= m) cover(is_out ? out_l[p] : in_l[p], tl, m, l, r, is_out, f);
        if (r > m) cover(is_out ? out_r[p] : in_r[p], m + 1, tr, l, r, is_out, f);
    }
    // u  $\rightarrow$   $[l, r]$  每个点, 边权 w
    void add_point_to_range(int u, int l, int r, i64 w) {
        int ul = leaf(u, true); // u 的出树叶
        cover(1, 1, n, l, r, true, [&](int nd) { add_edge(ul, nd, w); });
    }
    //  $[l, r]$  每个点  $\rightarrow$  v, 边权 w
    void add_range_to_point(int l, int r, int v, i64 w) {
        int vl = leaf(v, false); // v 的入树叶
        cover(1, 1, n, l, r, false, [&](int nd) { add_edge(nd, vl, w); });
    }
    // 额外接线: 原图点 u 的出/入 (点  $\rightarrow$  出树叶子权 0; 入树叶  $\rightarrow$  点权 0)
    void init_point_edges() {
        for (int u = 1; u <= n; ++u) {
            int lo = leaf(u, true), li = leaf(u, false);
            add_edge(u, lo, 0); // 原图点 u  $\rightarrow$  出树叶
            add_edge(li, u, 0); // 入树叶  $\rightarrow$  原图点 u
        }
    }
    int total() const { return tot; } // 总节点数 (跑最短路用)
};
// 用法: SegOptGraph sog(n); sog.init_point_edges();
// sog.add_point_to_range(u, l, r, w); // u 到  $[l, r]$  每个点
// sog.add_range_to_point(l, r, v, w); //  $[l, r]$  每个点到 v
```



```
// 建完对 sog.g 跑 Dijkstra (s 到 t)
```

05 字符串

05.1 字符串哈希（单/双哈希）

【简介】

把字符串映射成整数（多项式滚动哈希），0(1)

判断子串是否相等、子串比较。双哈希防碰撞。

快速识别：大量子串相等/比较判断；子串回文判断；“本质不同子串数量”的哈希解法；配合二分做 LCP。

API 接入：

- StrHash: build(s) 0-based 字符串; subhash(l, r) 闭区间 [l, r] 的哈希值 (0-based)。
- 双哈希: pair<ui64, i64> 返回两个模的哈希; 比较用 pair 相等。
- 子串相等: subhash(a,b) == subhash(c,d)。
- LCP: 二分 + 哈希。

关键点: base 取大质数 (如 131/13331) 或随机 base; 模数用

2^{64} 自然溢出 (ui64) 或

mod7/mod9; 双哈希几乎不可能碰撞; 下标 0-based、闭区间; pow 表预处理。注意 subhash(l,r) 公式: $h[r+1] - h[l] * \text{pow}[r-l+1]$ 。

```
struct StrHash {
    static const i64 MOD = 1e9 + 7;
    static const i64 BASE = 131;
    vector<i64> h, pw;
    StrHash(const string& s) {
        int n = (int)s.size();
        h.assign(n + 1, 0); pw.assign(n + 1, 1);
        for (int i = 0; i < n; ++i) {
            h[i + 1] = (h[i] * BASE + s[i]) % MOD;
            pw[i + 1] = pw[i] * BASE % MOD;
        }
    }
    i64 subhash(int l, int r) const { // 闭区间 [l, r], 0-based
        return ((h[r + 1] - h[l] * pw[r - l + 1]) % MOD + MOD) % MOD;
    }
};
// 双哈希: 维护两套 (MOD, BASE), 比较 pair
```

05.2 二维哈希

【简介】

矩阵的滚动哈希，0(1) 取任意子矩阵哈希。用于大网格中匹配小图案（图案出现位置、矩形相等）。

快速识别：二维模式匹配（在大图中找小图出现位置）；子矩阵相等判断。

API 接入：

- MatrixHash: build(mat) (0-based 行列); subhash(x1, y1, x2, y2) 闭区间子矩阵。
- 行哈希 + 列哈希两次滚动；同样双模防碰撞。

关键点: 先按行做一维滚动，再按列；幂表分别用行/列

base; 子矩阵公式 = 容斥四段相减再乘幂修正；注意 subhash 内坐标闭区间。

```
// 二维滚动哈希（双模防碰撞；示例给单模，双模把 MOD/BASE 换成 pair）
struct MatrixHash {
    static const i64 MOD1 = 1e9 + 7, MOD2 = 1e9 + 9;
    static const i64 B1 = 13331, B2 = 131;
    int n, m;
    vector<vector<array<i64, 2>>> h;
    vector<i64> pw1, pw2; // 行/列两个方向的幂表
    MatrixHash(const vector<vector<int>>& a) {
        n = (int)a.size(); m = (int)a[0].size();
        pw1.assign(n + 1, 1); pw2.assign(m + 1, 1);
        for (int i = 1; i <= n; ++i) pw1[i] = pw1[i - 1] * B1 % MOD1;
        for (int j = 1; j <= m; ++j) pw2[j] = pw2[j - 1] * B2 % MOD1;
        h.assign(n + 1, vector<array<i64, 2>>(m + 1, {0, 0}));
        for (int i = 1; i <= n; ++i)
            for (int j = 1; j <= m; ++j) {
```

```
i64 v = a[i - 1][j - 1] + 1; // +1 避免 0 值歧义
h[i][j][0] = ((h[i][j-1][0] * B1 + v) % MOD1 + MOD1) % MOD1;
h[i][j][1] = ((h[i-1][j][1] * B2 + h[i][j][0]) % MOD1 + MOD1) % MOD1;
}
// 子矩阵 (x1, y1)-(x2, y2) 闭区间 (1-based) 哈希
array<i64, 2> sub(int x1, int y1, int x2, int y2) const {
    i64 a = (h[x2][y2][1] - h[x1-1][y2][1] * pw1[x2-x1+1]
            - h[x2][y1-1][1] * pw2[y2-y1+1]
            + h[x1-1][y1-1][1] * pw1[x2-x1+1] * pw2[y2-y1+1]) % MOD1;
    return {(a + MOD1) % MOD1, a}; // 第二模需独立两套表, 示例省略
};
// 用途: 大网格中匹配小图案 = 算图案哈希, 滑动所有同尺寸子矩阵哈希比较
```

05.3 KMP 前缀函数

【简介】

pi[i] = 前缀 s[0..i] 的最长真 border

长度（既是前缀又是后缀的真子串）。用于单模式匹配

0(n+m)、找周期、字符串周期判定。

快速识别：模式串在文本中出现位置/次数；字符串最小周期；border 相关。

API 接入：

- prefix_function(s): 返回 pi 数组 (0-based, pi[i] 对 s[0..i])。
- 匹配: 在 text 上跑 KMP, j = pi[j-1] 回退; j == m 时找到一次匹配 (位置 i - m + 1)。
- 最小周期: n - pi[n-1], 若 n % (n - pi[n-1]) == 0 则 s 由该周期重复构成。

关键点: pi 计算 j = pi[j-1] 回退循环; 匹配时 j==m 后要 j = pi[m-1] 继续 (允许重叠匹配); 下标 0-based; pi[0] = 0。

```
vector<int> prefix_function(const string& s) {
    int n = (int)s.size();
    vector<int> pi(n);
    for (int i = 1; i < n; ++i) {
        int j = pi[i - 1];
        while (j > 0 && s[i] != s[j]) j = pi[j - 1];
        if (s[i] == s[j]) ++j;
        pi[i] = j;
    }
    return pi;
}
vector<int> kmp_match(const string& text, const string& pat) {
    vector<int> pi = prefix_function(pat);
    vector<int> res;
    int j = 0;
    for (int i = 0; i < (int)text.size(); ++i) {
        while (j > 0 && text[i] != pat[j]) j = pi[j - 1];
        if (text[i] == pat[j]) ++j;
        if (j == (int)pat.size()) {
            res.push_back(i - (int)pat.size() + 1); // 匹配起点 (0-based)
            j = pi[j - 1];
        }
    }
    return res;
}
```

05.4 Z 函数

【简介】

z[i] = s 与 s[i..]

的最长公共前缀长度。单模式匹配 (0(n+m))、字符串周期、LCP

查询的另一套工具，常数比 KMP 小。

快速识别：求每个后缀与原串的 LCP；字符串周期；配合 KMP 二选一。

API 接入：

- z_function(s): 返回 z 数组 (z[0] 约定为 0 或 n, 本模板返回 0)。
- 匹配: z[pattern + '#' + text], z[i] == m 处即匹配。
- 周期判定: z[i] + i == n 且 n % i == 0。

关键点：维护 $[l, r]$ 区间复用； $z[0]$ 定义各家不同（本模板 0），使用前确认；分隔符 # 必须不出现在原串。

```
vector<int> z function(const string& s) {
    int n = (int)s.size();
    vector<int> z(n);
    int l = 0, r = 0;
    for (int i = 1; i < n; ++i) {
        if (i <= r) z[i] = min(r - i + 1, z[i - 1]);
        while (i + z[i] < n && s[z[i]] == s[i + z[i]]) ++z[i];
        if (i + z[i] - 1 > r) { l = i; r = i + z[i] - 1; }
    }
    return z;
}
```

05.5 Trie 字典树

【简介】

字符/数字树形前缀结构：插入 $O(\text{len})$ 、查询前缀 $O(\text{len})$ 。用于前缀统计、异或最大、字符串集合、AC 自动机/01 Trie 的基座。

快速识别：“前缀”查询/统计；字符串集合；01 Trie（异或最值）见 05.6。

API 接入：

- insert(s) 插入一个串（计数 +1）；erase(s) 删除一个串（计数 -1，计数归零的节点保留但不可见）；count_exact(s) 返回 s 作为完整串的出现次数。
- count_prefix(s) 返回以 s 为前缀的串数；clear() 清空整棵树。
- 节点池 $\text{nxt}[u][c]$ （c 为字符编号）；0 号 = 根； $\text{end_cnt}[u]$ 是 u 结尾的完整串数， $\text{sub_cnt}[u]$ 是子树串数。
- 字符集小（26/10）用数组，大用 map。

关键点：节点池动态分配（vector 节点）；根 = 0，空节点 = 0；erase 删除的是“计数”而不是物理节点（物理删除会破坏其他串的路径）； sub_cnt 沿插入路径累加、沿删除路径减；erase 不存在的串无操作（可先 count_exact 判）。

```
struct Trie {
    vector<array<int, 26>> nxt{1}; // 节点池, 0 = 根
    vector<int> sub_cnt{0}, end_cnt{0}; // 子树串数 / 结尾串数
    int new_node() { nxt.push_back({}); sub_cnt.push_back(0); end_cnt.push_back(0); return (int)nxt.size() - 1; }
    void insert(const string& s) {
        int u = 0;
        for (char c : s) {
            int id = c - 'a';
            if (!nxt[u][id]) nxt[u][id] = new_node();
            u = nxt[u][id];
            sub_cnt[u]++;
        }
        end_cnt[u]++;
    }
    bool erase(const string& s) { // 删除一个 s; 不存在返回 false
        if (count_exact(s) == 0) return false;
        int u = 0;
        for (char c : s) {
            u = nxt[u][c - 'a'];
            sub_cnt[u]--;
        }
        end_cnt[u]--;
        return true;
    }
    int count_exact(const string& s) const { // s 作为完整串的出现次数
        int u = 0;
        for (char c : s) {
            int id = c - 'a';
            if (!nxt[u][id]) return 0;
            u = nxt[u][id];
        }
        return end_cnt[u];
    }
    int count_prefix(const string& s) const { // 以 s 为前缀的串数
        int u = 0;
        for (char c : s) {
            int id = c - 'a';
            if (!nxt[u][id]) return 0;
            u = nxt[u][id];
        }
        return sub_cnt[u];
    }
}
```

```
void clear() { nxt.assign(1, {}); sub_cnt.assign(1, 0); end_cnt.assign(1, 0); }
};
```

05.6 01 Trie（最大异或对 / 可持久化区间异或）

【简介】

把整数按二进制位建 Trie， $O(\log V)$ 求“与 x 异或最大的数”；可持久化版支持区间 $[l, r]$ 内与 x 异或最大。

快速识别：数组中两数异或最大/最小；区间内与 x 异或最大；异或 $\geq k$ 计数。

API 接入：

- 普通版：insert(x)；max_xor(x) 返回与 x 异或最大值（遍历位贪心）。
- 可持久化版：root[i] 前缀版本；query(l, r, x) 区间内与 x 异或最大值（版本差分）。
- 计数版：维护 sz，统计异或 $< k$ 的数对。

关键点：从高位到低位贪心（30..0，值域到 2^{30} 用 30..0，i64 用 62..0）；每位存在相反分支就走（异或位 = 1）；可持久化 = 每插入一个数 clone 一条链；区间异或查询注意版本差分公式。

```
struct BinaryTrie {
    vector<array<int, 2>> nxt{1}; // 0 = 根
    vector<int> sz{0}; // 子树元素个数（含重复）
    int new_node() { nxt.push_back({0, 0}); sz.push_back(0); return (int)nxt.size() - 1; }
    void insert(int x) { // 插入一个 x（计数 +1）
        int u = 0;
        for (int b = 30; b >= 0; --b) {
            int c = (x >> b) & 1;
            if (!nxt[u][c]) nxt[u][c] = new_node();
            u = nxt[u][c];
            sz[u]++;
        }
    }
    bool erase(int x) { // 删除一个 x; 不存在返回 false
        int u = 0;
        for (int b = 30; b >= 0; --b) {
            int c = (x >> b) & 1;
            if (!nxt[u][c] || sz[nxt[u][c]] == 0) return false;
            u = nxt[u][c];
        }
        u = 0;
        for (int b = 30; b >= 0; --b) { u = nxt[u][(x >> b) & 1]; sz[u]--; }
        return true;
    }
    int count(int x) const { // x 的出现次数
        int u = 0;
        for (int b = 30; b >= 0; --b) {
            int c = (x >> b) & 1;
            if (!nxt[u][c]) return 0;
            u = nxt[u][c];
        }
        return sz[u];
    }
    int max_xor(int x) const { // 与 x 异或的最大值
        int u = 0, res = 0;
        for (int b = 30; b >= 0; --b) {
            int c = (x >> b) & 1;
            if (nxt[u][c ^ 1] && sz[nxt[u][c ^ 1]] > 0) { res |= (1 << b); u = nxt[u][c ^ 1]; }
            else u = nxt[u][c];
        }
        return res;
    }
    int min_xor(int x) const { // 与 x 异或的最小值
        int u = 0, res = 0;
        for (int b = 30; b >= 0; --b) {
            int c = (x >> b) & 1;
            if (nxt[u][c] && sz[nxt[u][c]] > 0) u = nxt[u][c];
            else { res |= (1 << b); u = nxt[u][c ^ 1]; }
        }
        return res;
    }
    // 统计集合中与 x 异或后 < k 的元素个数（k 为 [0, 2^31)）
    int count_xor_less(int x, int k) const {
        int u = 0, res = 0;
        for (int b = 30; b >= 0; --b) {
            if (!u) break;
            int cx = (x >> b) & 1, ck = (k >> b) & 1;
            if (ck) {
```

```

    if (nxt[u][cx]) res += sz[nxt[u][cx]]; // 异或位 =
0 < 1, 全部计入
    u = nxt[u][cx ^ 1]; // 异或位 =
1 继续
    } else {
        u = nxt[u][cx]; // 异或位必
    }
    }
    return res;
}
// 集合中第 k 大 (k 从 0 开始) 的异或结果, 返回对应的集合元素
int kth(int k) const {
    int u = 0, res = 0;
    for (int b = 30; b >= 0; --b) {
        int go0 = nxt[u][0] ? sz[nxt[u][0]] : 0;
        if (k < go0) u = nxt[u][0];
        else { k -= go0; res |= (1 << b); u = nxt[u][1]; }
    }
    return res;
}
};
// 可持久化 01 Trie: root[i] 存前缀版本, 区间查询 = 两版本节点相减 (复用 0
2.22 节点池思想)

```

05.7 AC 自动机

【简介】

Trie + fail 指针：一次扫描文本匹配所有模式串。0(总长 + 文本长)。多模式匹配的标配。

快速识别：多个模式串匹配文本；统计每个模式出现次数；禁串计数 DP (配合 DP)。

API 接入：

- build() 在 insert 完所有模式后调用 (BFS 建 fail)。
- query(text)：扫描返回匹配计数 (可选模式出现次数累计)。
- fail 树：fail 指针反向构成树，子树 = 以某状态为后缀的模式数 (配合 dfn + BIT 或线段树)。
- AC DP: dp[i][u] 走 i 步到状态 u 的合法方案 (避免禁串)。

关键点：insert → build → query 顺序；fail[u] = 最长真后缀的节点；完整 fail 转移 (子不存在时用 fail 的对应子，见代码 transform)；统计出现次数要把 cnt 沿 fail 树累加。

```

struct AhoCorasick {
    vector<array<int, 26>> nxt{1}; // 完整转移表 (含 fail 补
    齐)
    vector<int> fail{0}, cnt{0}; // cnt: 模式结束计数
    int new_node() { nxt.push_back({}); fail.push_back(0); cnt.push_
    back(0); return (int)nxt.size() - 1; }
    void insert(const string& s) {
        int u = 0;
        for (char c : s) {
            int id = c - 'a';
            if (!nxt[u][id]) nxt[u][id] = new node();
            u = nxt[u][id];
        }
        cnt[u]++;
    }
    void build() {
        queue<int> q;
        for (int c = 0; c < 26; ++c) if (nxt[0][c]) q.push(nxt[0][c])
    };
    while (!q.empty()) {
        int u = q.front(); q.pop();
        cnt[u] += cnt[fail[u]]; // 累加 fail
        链的计数
        for (int c = 0; c < 26; ++c) {
            int& v = nxt[u][c];
            if (v) { fail[v] = nxt[fail[u]][c]; q.push(v); }
            else v = nxt[fail[u]][c]; // 补齐转移
        }
    }
    i64 query(const string& t) { // 文本中模式
    串出现总次数
        int u = 0;
        i64 res = 0;
        for (char c : t) {
            u = nxt[u][c - 'a'];
            res += cnt[u];
        }
        return res;
    }
}

```

```
};
```

05.8 Manacher (回文串)

【简介】

0(n) 求所有回文半径：d1[i] 奇回文、d2[i]

偶回文。所有回文子串、最长回文、回文计数。

快速识别：最长回文子串/子序列；回文子串数量；“回文”相关高频题。

API 接入：

- manacher(s)：返回 (d1, d2)；d1[i] = 以 i 为中心奇数回文半径 (含中心)，d2[i] = 以 i-1, i 为中心偶数回文半径。
- 最长奇回文 = 2*d1[i]-1；最长偶回文 = 2*d2[i]。
- 以 i 为中心的回文个数 = d1[i] (奇) + d2[i+1] (偶，按位置)。

关键点：维护当前最右回文 [l, r] 复用；d1 与 d2 公式不同 (d2 的初始值)；下标 0-based；回文串总数 = sum(d1) + sum(d2)。

```

pair<vector<int>, vector<int>> manacher(const string& s) {
    int n = (int)s.size();
    vector<int> d1(n), d2(n);
    for (int i = 0, l = 0, r = -1; i < n; ++i) {
        int k = (i > r) ? 1 : min(d1[l + r - i], r - i + 1);
        while (i - k >= 0 && i + k < n && s[i - k] == s[i + k]) ++k;
        d1[i] = k--;
        if (i + k > r) { l = i - k; r = i + k; }
    }
    for (int i = 0, l = 0, r = -1; i < n; ++i) {
        int k = (i > r) ? 0 : min(d2[l + r - i + 1], r - i + 1);
        while (i - k - 1 >= 0 && i + k < n && s[i - k - 1] == s[i +
        k]) ++k;
        d2[i] = k--;
        if (i + k > r) { l = i - k - 1; r = i + k; }
    }
    return {d1, d2};
}

```

05.9 回文自动机 PAM (回文树)

【简介】

回文串的有穷自动机：在线插入字符，0(n)

建树，支持本质不同回文子串数、每种回文出现次数、回文后缀。

快速识别：需要“所有回文子串”的结构信息 (计数、出现次数)；在线添加；Manacher 不够时。

API 接入：

- PAM: init() (两个根：奇根 0、偶根 1)；extend(pos) 在线加入 s[pos]，返回以 pos 结尾的最长回文长度。
- num[u]：以该节点结尾的回文后缀个数 (fail 链长度)。
- 统计出现次数：按 fail 树拓扑 (从大到小编号) 累加 cnt[fail[u]] += cnt[u]。
- 本质不同回文子串数 = 节点数 - 2。

关键点：get_fail 沿 fail 跳直到 s[pos-len-1] == s[pos]；奇根 len=-1、偶根 len=0；节点 0/1 是哨兵不计数；fail[u] 是最长真回文后缀；插入前先 get_fail(last)。

```

struct PAM {
    struct Node { int next[26] = {}; int fail = 0; int len = 0; int
    num = 0; i64 cnt = 0; };
    vector<Node> tr;
    string s;
    int last = 0;
    PAM() {
        tr.resize(2);
        tr[0].len = -1; // 奇根
        tr[1].len = 0; // 偶根
        tr[0].fail = tr[1].fail = 0;
    }
    int new_node() { tr.push_back(Node{}); return (int)tr.size() - 1
    ; }
    int get_fail(int u, int pos) {
        while (pos - 1 - tr[u].len < 0 || s[pos - 1 - tr[u].len] !=
        s[pos]) u = tr[u].fail;
        return u;
    }
    void extend(int pos) {
        int c = s[pos] - 'a';
    }
}

```

```

int u = get_fail(last, pos);
if (!tr[u].next[c]) {
    int v = new_node();
    tr[v].len = tr[u].len + 2;
    tr[v].fail = tr[get_fail(tr[u].fail, pos)].next[c];
    tr[v].num = tr[tr[v].fail].num + 1;
    tr[u].next[c] = v;
}
last = tr[u].next[c];
tr[last].cnt++;
}
void build(const string& str) {
    s = str;
    for (int i = 0; i < (int)s.size(); ++i) extend(i);
    for (int i = (int)tr.size() - 1; i >= 2; --i) tr[tr[i].fail]
.cnt += tr[i].cnt;
}
};

```

05.10 后缀数组 SA（基数排序）

【简介】

后缀排序：sa[i]（第 i 小的后缀起点）、rnk[i]（后缀 i 的排名）、height[i]（sa[i-1] 与 sa[i] 的 LCP）。O(n log n)。子串比较、LCP、重复子串、不同子串计数。

快速识别：后缀排序类问题：最长重复子串、不同子串数量、子串字典序第 k 小、两子串比较。

API 接入：

- build_sa(s): 返回 (sa, rnk, height); s 0-based, 结尾不加哨兵（或加 '\$', 各实现约定不同, 用前确认）。
- 不同子串数 = n(n+1)/2 - sum(height)。
- 最长公共前缀: LCP(i, j) = min(height[rnk[i]+1 .. rnk[j]]) (RMQ)。
- 子串比较: rnk 直接比较两后缀; 子串与子串用 LCP。

关键点：倍增 + 基数排序; height 定义 height[i] = LCP(sa[i-1], sa[i]), height[0] = 0; LCP 查询用 ST

表 (O(2.13) O(1)); 加哨兵时 sa

会多一个元素, 注意偏移。子串不同计数公式是经典。

```

struct SuffixArray {
    int n;
    vector<int> sa, rnk, height;
    SuffixArray(const string& s) { build(s); }
    void build(const string& s) {
        n = (int)s.size();
        sa.assign(n, 0); rnk.assign(n, 0); height.assign(n, 0);
        vector<int> tmp(n);
        iota(all(sa), 0);
        for (int i = 0; i < n; ++i) rnk[i] = s[i];
        for (int k = 1; k < n; k <= 1) {
            auto cmp = [&](int a, int b) {
                if (rnk[a] != rnk[b]) return rnk[a] < rnk[b];
                int ra = a + k < n ? rnk[a + k] : -1;
                int rb = b + k < n ? rnk[b + k] : -1;
                return ra < rb;
            };
            sort(all(sa), cmp);
            tmp[sa[0]] = 0;
            for (int i = 1; i < n; ++i) tmp[sa[i]] = tmp[sa[i - 1]]
+ cmp(sa[i - 1], sa[i]);
            rnk = tmp;
        }
        int h = 0;
        for (int i = 0; i < n; ++i) {
            if (rnk[i] == 0) { h = 0; continue; }
            int j = sa[rnk[i] - 1];
            while (i + h < n && j + h < n && s[i + h] == s[j + h])
+ h;
            height[rnk[i]] = h;
            if (h) --h;
        }
    }
    // LCP(i, j): i, j 为原串下标
    int lcp(int i, int j) const {
        if (i == j) return n - i;
        int a = rnk[i], b = rnk[j];
        if (a > b) swap(a, b);
        int res = INT_MAX;
        for (int k = a + 1; k <= b; ++k) res = min(res, height[k]);
        return res;
    }
};

```

```

}
};

```

05.11 后缀自动机 SAM

【简介】

O(n) 构建, 压缩了所有子串信息: 本质不同子串数、出现次数、最长公共子串、子串定位。状态 = endpos 等价类。

快速识别: 子串集合统计: 本质不同子串数、某子串出现次数、多串最长公共子串、字典序第 k 小子串。

API 接入:

- SAM: extend(c) 逐字符插入 (构造传 maxlen); count_distinct() 本质不同子串数 = sum(len - len[link])。
- 出现次数: 所有非 clone 状态 occurrence=1, 按 len 从大到小累加到 link; occ(u) 即出现次数。
- 最长公共子串: 对另一个串沿转移走, 失配沿 fail 退。
- 第 k 小本质不同子串: 拓扑序 DP 计数后贪心。

关键点: clone 状态 occurrence 必须置 0; last

指向整个串; 转移用

map (字符集任意) 或数组; 统计出现次数前要先按 len

排序状态; 本质不同子串公式 len[u] - len[link[u]]。

```

struct SuffixAutomaton {
    struct State { int link = -1, len = 0; i64 occ = 0; map<char, int> next; };
    vector<State> st;
    int last = 0;
    SuffixAutomaton(int max len = 0) {
        st.reserve(2 * max len);
        st.push_back(State{});
    }
    void extend(char c) {
        int cur = (int)st.size();
        st.push_back(State{});
        st[cur].len = st[last].len + 1;
        st[cur].occ = 1; // 非 clone: 新结
        int p = last;
        while (p != -1 && !st[p].next[c]) { st[p].next[c] = cur; p = st[p].link; }
        if (p == -1) st[cur].link = 0;
        else {
            int q = st[p].next[c];
            if (st[p].len + 1 == st[q].len) st[cur].link = q;
            else {
                int clone = (int)st.size();
                st.push_back(st[q]);
                st[clone].len = st[p].len + 1;
                st[clone].occ = 0; // clone 不是新结
                while (p != -1 && st[p].next[c] == q) { st[p].next[c] = clone; p = st[p].link; }
                st[q].link = st[cur].link = clone;
            }
            last = cur;
        }
    }
    i64 count_distinct() const {
        i64 ans = 0;
        for (int i = 1; i < (int)st.size(); ++i) ans += st[i].len - st[st[i].link].len;
        return ans;
    }
    // 每个状态的出现次数 (endpos 大小): 构造完所有 extend 后调用一次
    vector<i64> endpos_sizes() {
        int sz = (int)st.size();
        vector<int> order(sz), cnt(sz + 1, 0);
        for (int i = 0; i < sz; ++i) cnt[st[i].len]++; // 计数排序 (按 len)
        for (int i = 1; i <= sz; ++i) cnt[i] += cnt[i - 1];
        for (int i = sz - 1; i >= 0; --i) order[--cnt[st[i].len]] = i;
        for (int i = sz - 1; i > 0; --i) { // 沿 link 树自底向上累加
            int u = order[i];
            st[st[u].link].occ += st[u].occ;
        }
        vector<i64> res(sz);
        for (int i = 0; i < sz; ++i) res[i] = st[i].occ;
        return res;
    }
    // o
    cc[状态] = 该状态子串出现次数
};

```

```

    }
    // 统计子串 t 的出现次数: 沿转移走到对应状态后查 occ; 走不到 = 0
    i64 count_occurrences(const string& t, const vector<i64>& occ) const {
    onst {
        int u = 0;
        for (char c : t) {
            auto it = st[u].next.find(c);
            if (it == st[u].next.end()) return 0;
            u = it->second;
        }
        return occ[u];
    }
};

```

05.12 最小表示法 (Booth) 与循环同构

【简介】

$O(n)$

求循环串的最小字典序表示 (所有循环位移中最小者) 及起始位置。

快速识别: 循环串字典序最小/最大表示; 两个串是否循环同构。

API 接入:

- minimal_rotation(s): 返回最小表示的起始下标 (0-based); 比较时 $s[(i+k)\%n]$ 。
- 循环同构判定: `minimal_rotation(a) == minimal_rotation(b)` (或串 a 在 a+a 中查 b, 用 KMP)。
- 整数数组版: 字符比较换整数比较。

关键点: 双指针 i, j + 扩展 k; 失配时 i/j 跳到

$i+k+1/j+k+1$ (跳过不可能起点); 返回 $\min(i, j)$; 注意 $i=j$ 时 $j++$ 。

```

int minimal_rotation(const string& s) {
    int n = (int)s.size();
    int i = 0, j = 1, k = 0;
    while (i < n && j < n && k < n) {
        char a = s[(i + k) % n], b = s[(j + k) % n];
        if (a == b) ++k;
        else if (a > b) { i = i + k + 1; if (i == j) ++i; k = 0; }
        else { j = j + k + 1; if (i == j) ++j; k = 0; }
    }
    return min(i, j);
}

```

05.13 LCS / 编辑距离 / 通配符匹配

【简介】

最长公共子序列 ($O(nm)$)

DP, 可输出方案; 编辑距离 (插入/删除/替换); 通配符匹配 (? 和 *)。三者是序列 DP 的基础题型。

快速识别: 两个序列求最长公共子序列/子串; 字符串变换最小代价; 模式串含 ?/* 的匹配。

API 接入:

- LCS: $dp[i][j] = a[i]==b[j] ? dp[i-1][j-1]+1 : \max(dp[i-1][j], dp[i][j-1])$; 方案回溯。
- 编辑距离: $dp[i][j] = \min(dp[i-1][j]+1, dp[i][j-1]+1, dp[i-1][j-1] + (a[i]!=b[j]))$ 。
- 通配符: * 匹配任意串 ($dp[i][j] = dp[i-1][j] || dp[i][j-1]$), ? 匹配单字符。
- 大 n 用 Bit-parallel / 最长公共子串用 SAM/后缀数组 (见 05.14)。

关键点: 下标从 1 开始 ($dp[0][*]=0$); LCS 计数要开两个

DP; 编辑距离允许的操作按题意 (可替换则用 min 三项); 通配符 * 的转移有“匹配 0 个或多个”两种来源。

```

int lcs_length(const string& a, const string& b) {
    int n = (int)a.size(), m = (int)b.size();
    vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));
    for (int i = 1; i <= n; ++i)
        for (int j = 1; j <= m; ++j)
            dp[i][j] = a[i] == b[j] ? dp[i-1][j-1] + 1 : max(dp[i-1][j], dp[i][j-1]);
    return dp[n][m];
}

int edit_distance(const string& a, const string& b) {
    int n = (int)a.size(), m = (int)b.size();
    vector<vector<int>> dp(n + 1, vector<int>(m + 1));
}

```

```

for (int i = 0; i <= n; ++i) dp[i][0] = i;
for (int j = 0; j <= m; ++j) dp[0][j] = j;
for (int i = 1; i <= n; ++i)
    for (int j = 1; j <= m; ++j)
        dp[i][j] = min({dp[i-1][j] + 1, dp[i][j-1] + 1,
                        dp[i-1][j-1] + (a[i-1] != b[j-1])});
return dp[n][m];
}

```

05.14 多串最长公共子串 (SAM / 后缀数组)

【简介】

多个字符串的最长公共子串 (连续)。SAM: 对第一个建 SAM, 其余串扫描维护最大匹配长度; 或后缀数组: 拼接后相邻 height 最大且覆盖所有串。

快速识别: “若干串的最长公共连续子串” (注意不是子序列); 多串公共部分。

API 接入:

- SAM 版: 对 s1 建 SAM; 对每个 si 跑 cur_len (沿转移 +1, 失配沿 fail 退到 len[link] 再试), $ans = \max(ans, cur_len)$ 。
- SA 版: 拼接 (不同串用不同分隔符), height 上滑窗: 窗口覆盖所有串时取 $\min(height)$ 最大。
- 依赖: SuffixAutomaton (05.11 节), 抄板时一起抄上。

关键点: SAM 失配回退 while (p && !next.count(c)) p = link[p] 后若仍无转移则 len=0 重新开始; 每跑完一个串不要重置 SAM 状态到 0 (记录 cur 节点与 cur_len); SA 版分隔符必须互不相同且小于字符集。

```

// SAM 版最长公共子串 (对 s1 建 SAM, 跑其余串):
int longest_common_substring(const SuffixAutomaton& sam, const string& t) {
    int v = 0, cur = 0, best = 0;
    for (char c : t) {
        while (v && !sam.st[v].next.count(c)) { v = sam.st[v].link; cur = sam.st[v].len; }
        if (sam.st[v].next.count(c)) { v = sam.st[v].next.at(c); ++cur; }
        best = max(best, cur);
    }
    return best;
}

```

05.15 字符串技巧速查 (border / 周期 / 子序列)

【简介】

字符串高频结论与技巧: 最小周期、border

结构、子序列自动机、字典序第 k 子串、回文子序列计数等速查。

快速识别: 字符串结论题 (周期/border); 子序列相关问题; 第 k 大/小子字符串结构。

API 接入:

- 最小周期: $n - \text{pi}[n-1]$ (KMP) 或 $z[n-k] == k$ 且整除。
- 序列自动机: $\text{nxt}[i][c] = i$ 之后第一个 c 的位置 (倒序构建), 子序列匹配 $O(\text{len})$ 。
- 字典序第 k 小子串: SAM 拓朴计数后贪心 (本质不同版) / 后缀数组 (位置不同版)。
- 最少插入字符成回文: $n - \text{LCS}(s, \text{rev}(s))$ (子序列版); 区间 DP 版见 09 章。

关键点: 周期与 border 关系: s 有周期 p 当且仅当 $n - p$ 是 border; 序列自动机倒序构建 $\text{nxt}[i] = \text{nxt}[i+1]$ 再更新; 第 k 小子串分清“本质不同”与“位置不同”。

```

// 序列自动机 (小写字母版): nxt[i][c] = 位置 i 之后 (含 i) 第一个 c 的下标, 无则 n
vector<array<int, 26>> seq_automaton(const string& s) {
    int n = (int)s.size();
    vector<array<int, 26>> nxt(n + 1);
    nxt[n].fill(n);
    for (int i = n - 1; i >= 0; --i) {
        nxt[i] = nxt[i + 1];
        nxt[i][s[i] - 'a'] = i;
    }
    return nxt;
}

```



```
// 子序列匹配: pos = 0; for c in t: pos = nxt[pos][c-'a']; if (pos == n)
不存在; ++pos;
```

05.16 广义后缀自动机（多串 SAM）

【简介】

对多个字符串构建的

SAM: 所有串的所有子串都能被识别。用法: 插入完一个串后把 last 重置回根再插下一个。可统计多串本质不同子串数、每个串的子串出现次数。

快速识别: 多个模式串的子串统计（本质不同子串总数、某子串在这些串出现）; 多串 LCS 的另一种做法。

API 接入:

- 复用 05.11 的 SuffixAutomaton: sam.extend(c) 逐字符; 每插入完一个串执行 sam.last = 0。
- 多串本质不同子串总数 = sum(len[u] - len[link[u]]) (同单串公式)。
- 每个串的子串出现次数: endpos_sizes (同 05.11)。
- 注意: 同一个串可能被重复插入导致多余状态, 用“已存在转移则直接走过去”优化 (可选)。

关键点: last 重置到 0 是广义 SAM

的唯一区别: 重复串会创建冗余状态但不影响正确性 (统计时 len 差可能为 0); 多串出现次数统计需要记录每个串的终止状态再沿 link 累加。

```
// 广义 SAM 用法 (复用 05.11 SuffixAutomaton):
// SuffixAutomaton sam(total len);
// for (string& s : str) {
//     for (char c : s) sam.extend(c);
//     sam.last = 0; // 关键: 插完一个串回到根
// }
// i64 distinct = sam.count_distinct(); // 多串本质不同子串总数
```

06 数论与组合

06.1 模算术与 ModInt

【简介】

模意义下的整数封装: 自动取模的四则运算 + 幂 + 逆元。让 DP/组合数代码干净且防溢出。

快速识别: 所有模 p 运算; 组合数/DP 取模; 负数取模规范化。

API 接入:

- ModInt<MOD> a(x) 构造 (x 可为负数, 自动规范化); + - * / (除法用逆元); pow(e); inv()。
- a.val() 取底层值; 比较 ==。
- 模数编译期常量 (质数保证逆元存在)。

关键点: 负数构造取模修正 (x % MOD + MOD) % MOD

MOD; 除法要求模数质数 (费马小定理) 或先用 exgcd; 中间乘法用 i64 防溢出 (MOD < 1e9 时乘积 < 1e18 安全)。

```
template <i64 MOD>
struct ModInt {
    i64 v;
    ModInt(i64 x = 0) { v = (x % MOD + MOD) % MOD; }
    ModInt operator+(const ModInt& o) const { return ModInt(v + o.v); }
};
ModInt operator-(const ModInt& o) const { return ModInt(v - o.v); }
ModInt operator*(const ModInt& o) const { return ModInt(v * o.v); }
ModInt pow(i64 e) const {
    ModInt r(1), a(*this);
    while (e) { if (e & 1) r = r * a; a = a * a; e >>= 1; }
    return r;
}
ModInt inv() const { return pow(MOD - 2); } // 需 MOD 为质数
ModInt operator/(const ModInt& o) const { return *this * o.inv(); }
bool operator==(const ModInt& o) const { return v == o.v; }
};
using Mint = ModInt<1000000007>;
```

06.2 扩展欧几里得 exgcd 与线性同余

【简介】

求 $ax + by = \gcd(a, b)$ 的一组整数解; 解线性同余 $ax \equiv b \pmod{m}$; 求任意模数逆元。

快速识别: 一次不定方程整数解; 模意义除法 (模数非质数); CRT 的前置。

API 接入:

- exgcd(a, b, x, y): 返回 $g = \gcd(a, b)$, 且 $a*x + b*y = g$ 。
- inv_mod(a, m): 返回 a 模 m 的逆元 (要求 $\gcd(a, m)=1$; 用 exgcd)。
- solve_congruence(a, b, m): 解 $ax \equiv b \pmod{m}$, 返回最小非负解或 -1 无解。
- 通解: $x = x_0 + t * (m/g)$ 。

关键点: exgcd 递归返回后 $x = y'$, $y = x' - (a/b)*y'$; 通解步长 = m/g ; 解的数量 = g 的因子相关; 负数解规范到 $[0, m)$ 。

```
i64 exgcd(i64 a, i64 b, i64& x, i64& y) {
    if (!b) { x = 1; y = 0; return a; }
    i64 g = exgcd(b, a % b, y, x);
    y -= a / b * x;
    return g;
}
i64 inv_mod(i64 a, i64 m) { // 要求 gcd(a, m) == 1
    i64 x, y;
    exgcd(a, m, x, y);
    return (x % m + m) % m;
}
// 解 ax ≡ b (mod m): 返回最小非负解, 无解返回 -1
i64 solve_congruence(i64 a, i64 b, i64 m) {
    i64 x, y, g = exgcd(a, m, x, y);
    if (b % g) return -1;
    i64 step = m / g;
    x = (i128)x * (b / g) % m;
    return (x % step + step) % step;
}
```

06.3 线性筛 (素数 + 欧拉函数 + mu)

【简介】

$O(n)$ 筛出素数, 同时线性求欧拉函数 phi、莫比乌斯 mu、约数个数等积性函数。

快速识别: $1e6 \sim 1e7$ 内素数表; 需要 phi/mu 数组做反演/前缀和; 积性函数预处理。

API 接入:

- linear_sieve(n): 返回 is_prime[] (或 primes 列表) + phi[] + mu[]。
- phi[x]: 1..x 中与 x 互质的个数; mu[x]: 莫比乌斯函数。
- 扩展积性函数: 在“x % primes[j] == 0”分支里维护幂因子。

关键点: 每个合数只被最小质因子筛一次; phi 递推: $x \neq p$ 时

$\phi[x*p] = \phi[x]*p$, 否则 $\phi[x*p] =$

$\phi[x]*(p-1)$; mu: 平方因子时 mu=0; 数组大小 n+1, is_prime 用 char 省内存。

```
struct LinearSieve {
    int n;
    vector<char> is_comp;
    vector<int> primes, phi, mu;
    LinearSieve(int n) : n(n) {
        is_comp.assign(n + 1, 0);
        phi.assign(n + 1, 0);
        mu.assign(n + 1, 0);
        phi[1] = mu[1] = 1;
        for (int i = 2; i <= n; ++i) {
            if (!is_comp[i]) { primes.push_back(i); phi[i] = i - 1; mu[i] = -1; }
            for (int p : primes) {
                if ((i64)p * i > n) break;
                is_comp[i * p] = 1;
                if (i % p == 0) {
                    phi[i * p] = phi[i] * p;
                    mu[i * p] = 0;
                    break; // 最小质因子已处理
                }
                phi[i * p] = phi[i] * (p - 1);
            }
        }
    }
};
```

```

        mu[i * p] = -mu[i];
    }
}
};

```

06.4 素性测试 Miller-Rabin

【简介】

$O(k \log n)$ 确定性/概率判素。用一组固定底数对 64 位整数确定性判素。

快速识别：大数 ($> 1e12$) 判素；Pollard-Rho 的前置。

API 接入：

- `is_prime(x)`: x 为 i64 (含负数/0/1 返回 false)。
- 内部用 `mul_mod` (i128) 防溢出；底数表覆盖 2^{64} 。
- 原理：费马小定理 + 二次探测 ($x-1 = d \cdot 2^s$ 分解)。

关键点：64 位乘法必须用 `__int128`；底数

{2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37} 覆盖 $<$

$3.3e24$ ；先判小素数加速； $x-1$ 分解成 $d \cdot 2^s$ 后逐次平方探测。

```

i64 mul_mod(i64 a, i64 b, i64 mod) { return (i128)a * b % mod; }
i64 mod_pow(i64 a, i64 e, i64 mod) {
    i64 r = 1 % mod; a %= mod;
    while (e) { if (e & 1) r = mul_mod(r, a, mod); a = mul_mod(a, a, mod); e >>= 1; }
    return r;
}
bool is_prime(i64 n) {
    if (n < 2) return false;
    for (i64 p : {2LL, 3LL, 5LL, 7LL, 11LL, 13LL, 17LL, 19LL, 23LL, 29LL, 31LL, 37LL})
        if (n % p == 0) return n == p;
    i64 d = n - 1, s = 0;
    while (d % 2 == 0) d /= 2, ++s;
    for (i64 a : {2LL, 325LL, 9375LL, 28178LL, 450775LL, 9780504LL, 1795265022LL}) { // 64 位确定性底数
        if (a % n == 0) continue;
        i64 x = mod_pow(a, d, n);
        if (x == 1 || x == n - 1) continue;
        bool ok = false;
        for (int r = 1; r < s; ++r) {
            x = mul_mod(x, x, n);
            if (x == n - 1) { ok = true; break; }
        }
        if (!ok) return false;
    }
    return true;
}

```

06.5 因数分解 Pollard-Rho

【简介】

大数质因数分解（期望

$O(n^{1/4})$ ）。基于生日悖论找非平凡因子，递归分解。

快速识别：i64

级大数分解质因数；求大数因子个数/所有因子；配合 MR 判素。

API 接入：

- `pollard_rho(n)`: 返回 n 的一个非平凡因子 (n 为合数)。
- `factorize(n)`: 返回所有质因子（无序，带重复）。
- `unique_factors(n) / divisors(n)` 基于 `factorize`。
- 配合 `is_prime` 递归终止。
- 依赖：本节的 `mul_mod/mod_pow/is_prime` (06.4 节定义在上方)，整节一起抄。

关键点：随机化（种子随机，被卡时换种子）； $f(x) = (x^2 + c) \% n$ 的 Floyd 判环；gcd 判因子；乘法全程 i128； n 为质数时先 MR 判掉再进 Pollard-Rho；大数求所有因子要先去重再组合。

```

mt19937_64 rng_rho(chrono::steady_clock::now().time_since_epoch().count());
i64 pollard_rho(i64 n) {
    if (n % 2 == 0) return 2;
    while (true) {
        i64 c = rng_rho() % (n - 1) + 1;
        i64 x = rng_rho() % n, y = x, d = 1;
        auto f = [&](i64 v) { return (mul_mod(v, v, n) + c) % n; };
        while (d == 1) {
            x = f(x);

```

```

        y = f(f(y));
        d = std::gcd(abs(x - y), n);
    }
    if (d != n) return d;
}
}
void factorize_rec(i64 n, vector<i64>& res) {
    if (n == 1) return;
    if (is_prime(n)) { res.push_back(n); return; }
    i64 d = pollard_rho(n);
    factorize_rec(d, res);
    factorize_rec(n / d, res);
}
vector<i64> factorize(i64 n) { vector<i64> res; factorize_rec(n, res); return res; }

```

06.6 组合数（阶乘逆元 / Lucas / exLucas）

【简介】

$C(n, k)$ 模 p : p 为质数且 $n < p$ 用阶乘逆元 $O(1)$; n 很大 p 较小用 Lucas; p 为合数用 exLucas。

快速识别：组合数取模； n 大 ($> 1e9$) 模小；模数非质数。

API 接入：

- `init_comb(maxn, MOD)`: 预处理阶乘与阶乘逆元: $C(n, k)$ $O(1)$ (要求 $n < MOD, k \leq n$)。
- `C_lucas(n, k, p)`: n, k 可到 $1e18$, p 质数 ($p \leq 1e5$ 时用)。
- `exLucas(n, k, p)`: 任意模数（分解 p 为质数幂, CRT 合并）。
- `C_small(n, k)`: $n \leq 5000$ 用杨辉三角。
- 依赖: `mod_pow` (01.3 节)，抄板时一起抄上。

关键点: `inv_fact[m] = pow(fact[m], p-2)` 再倒推; Lucas: $C(n, k) \equiv C(n/p, k/p) * C(n \% p, k \% p)$; exLucas 里 p 的幂要单独处理（阶乘去掉 p 因子再乘回）; $k < 0$ 或 $k > n$ 返回 0。

```

struct Comb {
    vector<i64> fact, inv_fact;
    i64 MOD;
    Comb(int maxn, i64 mod) : MOD(mod) {
        fact.resize(maxn + 1); inv_fact.resize(maxn + 1);
        fact[0] = 1;
        for (int i = 1; i <= maxn; ++i) fact[i] = fact[i - 1] * i % MOD;
        inv_fact[maxn] = mod_pow(fact[maxn], MOD - 2, MOD);
        for (int i = maxn; i >= 1; --i) inv_fact[i - 1] = inv_fact[i] * i % MOD;
    }
    i64 C(int n, int k) const {
        if (k < 0 || k > n) return 0;
        return fact[n] * inv_fact[k] % MOD * inv_fact[n - k] % MOD;
    }
};
// Lucas (p 质数, n, k 大):
i64 C_lucas(i64 n, i64 k, i64 p, const Comb& cb) {
    if (k < 0 || k > n) return 0;
    if (n < p) return cb.C((int)n, (int)k);
    return C_lucas(n / p, k / p, p, cb) * cb.C((int)(n % p), (int)(k % p)) % p;
}

```

06.7 特殊组合数: Catalan / Stirling

【简介】

Catalan 数（括号序列、二叉搜索树、凸多边形三角剖分、出栈序列）；Stirling 数（第一类：排列的轮换；第二类：集合划分）。

快速识别：“ n 对括号合法序列数”、“ n 个节点的 BST 数” $\rightarrow$ Catalan; “ n 个元素分到 k 个非空集合” $\rightarrow$ S2; “ n 个元素的排列有 k 个轮换” $\rightarrow$ S1。

API 接入：

- Catalan: $Cat[n] = C(2n, n) - C(2n, n+1)$ 或递推 $Cat[n+1] = Cat[n] * 2(2n+1) / (n+2)$ 。
- S2: $S(n, k) = S(n-1, k-1) + k * S(n-1, k)$, $0(nk)$; 单点公式 $S(n, k) = 1/k! * \sum (-1)^i C(k, i) (k-i)^n$ 。
- S1 (无符号): $s(n, k) = s(n-1, k-1) + (n-1) * s(n-1, k)$ 。
- 快速求一行 S2: NTT/卷积（生成函数）， n 大时用。

关键点: Catalan 递推有除法 (用逆元, mod 质数); S2 递推的 $k \cdot S(n-1, k)$ 方向; S1 的 $(n-1) \cdot$ 系数; 注意 n, k 范围决定用递推 $O(nk)$ 还是公式 $O(n \log n)$ 。

```
// 第二类 Stirling S(n, k) (递推 O(nk))
vector<vector<i64>> S2(int n, int k, i64 MOD) {
    vector<vector<i64>> S(n + 1, vector<i64>(k + 1, 0));
    S[0][0] = 1;
    for (int i = 1; i <= n; ++i)
        for (int j = 1; j <= min(i, k); ++j)
            S[i][j] = (S[i - 1][j - 1] + i * S[i - 1][j]) % MOD;
    return S;
}
// Catalan (mod 质数):
// Cat[0] = 1; Cat[i] = Cat[i-1] * 2 * (2*i-1) % MOD * inv(i+1) % MO
D;
```

06.8 整除分块 (数论分块)

【简介】

$\sum_{i=1}^n \lfloor n/i \rfloor \cdot f(i)$ 类求和: $\lfloor n/i \rfloor$ 取值只有 $O(\sqrt{n})$ 段, 每段等差数列/前缀和快速算。 $O(\sqrt{n})$ 。

快速识别: 求和含 $\lfloor n/i \rfloor$; $\sum$

$\lfloor n/i \rfloor$ 、除数函数求和、莫比乌斯反演的基础。

API 接入:

- for ($l = 1$; $l \leq n$; $l = r + 1$) { $r = n / (n / l)$; 处理段 $[l, r]$; }。
- 每段值 $v = n / l$ 恒定; 贡献 $= v \cdot (f$ 的前缀和 $(r) - f$ 的前缀和 $(l-1))$ 。
- 二维版: n/i 与 m/j 联合分块 (取 $\min$ 端点)。

关键点: $r = n / (n / l)$ 是核心公式; i 从 1 开始、 n 可为 $i64$; 二维分块每次 $r = \min(n/(n/l), m/(m/l))$; 防除零 (n/l 为 0 时结束)。

```
// sum_{i=1}^n floor(n/i) (朴素公式, f(i)=1)
i64 floor_sum(i64 n) {
    i64 ans = 0;
    for (i64 l = 1, r; l <= n; l = r + 1) {
        i64 v = n / l;
        r = n / v;
        ans += v * (r - l + 1);
    }
    return ans;
}
// 通用: ans += v * (pref[r] - pref[l-1]), pref 为 f 的前缀和
```

06.9 莫比乌斯反演

【简介】

$[\gcd(i, j)=1]$ 类计数、约数/倍数关系的转化。核心公式: $f(n) = \sum_{d|n} g(d) \Rightarrow g(n) = \sum_{d|n} \mu(d) f(n/d)$; 以及 $[\gcd(i, j)=1] = \sum_{d|\gcd(i, j)} \mu(d)$ 。

快速识别: $\gcd$ 相关计数 ($\sum_{i \leq n, j \leq m} [\gcd(i, j)=k]$ 的个数、互质对数); 约数条件计数。

API 接入:

- 预处理 μ (06.3 线性筛)。
- 套路 1: $[\gcd(i, j)=1]$ 换成 $\sum_{d|\gcd(i, j)} \mu(d)$, 交换求和顺序。
- 套路 2: $\gcd(i, j)=k$ 个数 = 先算 $\gcd$ 是 k 的倍数 的个数 $F(k)$, 再反演。
- 整除分块加速外层求和。

关键点: 交换求和顺序是核心技巧; 先写“倍数版”再反演; μ 前缀和配合整除分块 $O(\sqrt{n})$; 注意 $i \leq n, j \leq m$ 时 $\min(n, m)$ 为分块上界。

```
// 例: sum_{i=1}^n sum_{j=1}^m [gcd(i, j) == 1] (n, m <= 1e7, 预处理 mu 前缀和)
// ans = sum_{d=1}^{min(n, m)} mu[d] * (n/d) * (m/d) (整除分块加速)
i64 coprime_pairs(i64 n, i64 m, const vector<i64>& mu_pref) {
    i64 ans = 0, lim = min(n, m);
    for (i64 l = 1, r; l <= lim; l = r + 1) {
        i64 vn = n / l, vm = m / l;
        r = min(n / vn, m / vm);
        ans += (mu_pref[r] - mu_pref[l - 1]) * vn * vm;
    }
}
```

```
return ans;
}
```

06.10 杜教筛 (大范围积性函数前缀和)

【简介】

$O(n^{2/3})$ 求大范围 ($1e10 \sim 1e12$) 积性函数前缀和, 如 $\sum \phi(i)$ 、 $\sum \mu(i)$ 。公式: $S(n) = \dots$ 用卷积与整除分块递归。

快速识别: n 到 $1e10+$, 求 ϕ/μ 前缀和; Min_25 的轻量替代 (只支持少数函数)。

API 接入:

- $\text{pref_phi}(n) / \text{pref_mu}(n)$ (记忆化 + 线性筛小范围打底)。
- 公式: $S_{\phi}(n) = n(n+1)/2 - \sum_{d=2}^n S_{\phi}(n/d)$; $S_{\mu}(n) = 1 - \sum_{d=2}^n S_{\mu}(n/d)$ 。
- 用 `unordered_map<i64, i64>` 记忆化; 预处理 $n^{2/3}$ 内前缀和。

关键点: 整除分块递归 (d 从 2 到 $n, n/d$

分块); 记忆化防止指数爆炸; 线性筛打底到 $n^{2/3}$

平衡复杂度; $S(1)=\phi(1)=1$ 边界。

```
// 杜教筛求 phi/mu 前缀和 (n 可到 1e11); 依赖 06.3 线性筛 (抄板时一起抄上)
// 用法: DuSieve du(1e6); du.S phi(n); du.S mu(n);
struct DuSieve {
    static const int BASE = 1000000; // 打底规模 (n^(2/3) 量级)
    vector<i64> phi_pref, mu_pref;
    unordered_map<i64, i64> memo_phi, memo_mu;
    DuSieve() {
        LinearSieve ls(BASE); // 06.3 线性筛
        phi_pref.assign(BASE + 1, 0);
        mu_pref.assign(BASE + 1, 0);
        for (int i = 1; i <= BASE; ++i) {
            phi_pref[i] = phi_pref[i - 1] + ls.phi[i];
            mu_pref[i] = mu_pref[i - 1] + ls.mu[i];
        }
    }
    i64 S_phi(i64 n) {
        if (n <= BASE) return phi_pref[n];
        if (memo_phi.count(n)) return memo_phi[n];
        i64 res = n * (n + 1) / 2; // sum phi = n(n+1) / 2 - sum_{d=2}^n S(n/d)
        for (i64 l = 2, r; l <= n; l = r + 1) {
            i64 v = n / l;
            r = n / v;
            res -= (r - l + 1) * S_phi(v);
        }
        return memo_phi[n] = res;
    }
    i64 S_mu(i64 n) {
        if (n <= BASE) return mu_pref[n];
        if (memo_mu.count(n)) return memo_mu[n];
        i64 res = 1; // sum mu = 1 - sum_{d=2}^n S(n/d)
        for (i64 l = 2, r; l <= n; l = r + 1) {
            i64 v = n / l;
            r = n / v;
            res -= (r - l + 1) * S_mu(v);
        }
        return memo_mu[n] = res;
    }
};
```

06.11 中国剩余定理 CRT / exCRT

【简介】

解同余方程组 $x \equiv a_i \pmod{m_i}$: 模数两两互质用 CRT ($O(k \log m)$); 不互质用 exCRT (合并归纳)。求最小非负解。

快速识别: 一组模不同数的同余条件 x ; “除以 3 余 2, 除以 5 余 3”类。

API 接入:

- `crt(rem, mod)`: rem/mod 等长数组; 返回最小非负解; 无解返回 -1。
- exCRT 逐步合并: $x \equiv x \pmod{M}$ 与 $x \equiv a \pmod{m}$ 合并成新同余。
- 合并公式: $x' = x + M * t, t$ 解 $M * t \equiv a - x \pmod{m}$ 。
- 依赖: `exgcd` (06.2 节), 抄板时一起抄上。

关键点: exCRT 合并时用 exgcd 解 $M \cdot t \equiv (a-x) \pmod m$, 无解条件 $\gcd(M, m) \nmid (a-x)$; 中间量用 i128 防溢出 (M 可能很大); CRT 要求互质否则必须 exCRT; 结果规范到 $[0, lcm)$ 。

```
// exCRT: 合并所有同余, 返回最小非负解或 -1
// 合并公式:  $x' = x + M \cdot (t \cdot u \bmod (m/g))$ ,  $t$  是  $(a-x)/g$  的商,  $u$  是  $(M/g)$  在模  $(m/g)$  下的逆元
i64 exCRT(vector<i64>& rem, vector<i64>& mod) {
    i64 x = rem[0], M = mod[0];
    for (int i = 1; i < (int)rem.size(); ++i) {
        i64 a = rem[i], m = mod[i];
        i64 g = std::gcd(M, m);
        if ((a - x) % g) return -1;
        i64 t = ((a - x) / g) % (m / g);
        i64 u, w;
        exgcd(M / g, m / g, u, w);
        u = (u % (m / g) + m / g) % (m / g);
        i128 newM = (i128)(M / g) * m; // 新模数: 先
        i128 add = (i128)M * (t * u % (m / g)); // 增量: add <
        newM // vv < 2*newM
        i128 vv = (i128)x + add; // vv < 2*newM
        , 条件减法代替取模
        if (vv >= newM) vv -= newM;
        x = (i64)vv;
        M = (i64)newM;
        if (x < 0) x += M;
    }
    return x;
}
```

06.12 BSGS / exBSGS (离散对数)

【简介】

解 $a^x \equiv b \pmod p$: BSGS $O(\sqrt{p})$ (p 质数); exBSGS 处理 $\gcd(a, p) \neq 1$ 。

快速识别: “求最小 x 使 $a^x \equiv b$ ”; 离散对数; 密码学/循环节类问题。

API 接入:

- bsgs(a, b, p): 返回最小非负 x ; 无解 -1 (要求 $\gcd(a, p)=1$)。
- exbsgs(a, b, p): 一般情形。
- 预处理表 $m = \text{ceil}(\sqrt{p})$, $a^{(j \cdot m)} \cdot b$ 查表。
- 依赖: mod_pow (01.3 节), 抄板时一起抄上。

关键点: 大步小步: 存 a^i ($i < m$) 到哈希表, 枚举大步 j 查 $b \cdot a^{-j \cdot m}$; exBSGS 先除公因数直到互质 (b 也要除); 无解判断 ($b=1$ 时 $x=0$); p 为合数时用 exBSGS。

```
i64 bsgs(i64 a, i64 b, i64 p) {
    if (b % p == 1 % p) return 0;
    unordered_map<i64, i64> mp;
    i64 m = (i64)ceil(sqrt((long double)p));
    i64 cur = 1;
    for (i64 j = 0; j < m; ++j) {
        if (!mp.count(cur)) mp[cur] = j; // 存最小指数
        cur = cur * a % p;
    }
    i64 am = mod_pow(a, m, p);
    i64 inv_am = mod_pow(am, p - 2, p); // 要求 gcd(a, p)=1
    cur = b;
    for (i64 i = 0; i <= m; ++i) {
        if (mp.count(cur)) return i * m + mp[cur];
        cur = cur * inv_am % p;
    }
    return -1;
}
```

06.13 原根 / 二次剩余 / 离散对数补充

【简介】

原根 (模 p 的生成元, p 质数时 $\phi(p)=p-1$ 阶); 二次剩余 $x^2 \equiv a \pmod p$ 用 Tonelli-Shanks 解; NTT 原根配合。

快速识别: 求最小原根; 解模意义平方根; NTT 需要原根 (998244353 的原根是 3)。

API 接入:

- primitive_root(p): 返回最小原根 (p 质数)。判定: $g^{\frac{p-1}{q}} \neq 1$ 对所有 $p-1$ 的质因子 q 。

- tonelli_shanks(a, p): 返回一个平方根 x (另一个是 $p-x$); 无解返回 -1 (勒让德符号判)。
- 常用 NTT 模数: 998244353 原根 3; 1004535809 原根 3; 469762049 原根 3。
- 依赖: mod_pow (01.3 节), 抄板时一起抄上。

关键点: 原根存在性: 模数 $2/4/p^k/2p^k$ (p 奇质数); Tonelli-Shanks 要求 p 奇质数; 二次剩余判定 = 勒让德符号 $a^{\frac{p-1}{2}} \equiv 1$; $a=0$ 返回 0。

```
// 最小原根 (p 质数)
i64 primitive_root(i64 p) {
    i64 phi = p - 1;
    vector<i64> fac;
    i64 x = phi;
    for (i64 q = 2; q * q <= x; ++q) if (x % q == 0) {
        fac.push_back(q);
        while (x % q == 0) x /= q;
    }
    if (x > 1) fac.push_back(x);
    for (i64 g = 2; ++g) {
        bool ok = true;
        for (i64 q : fac) if (mod_pow(g, phi / q, p) == 1) { ok = false; break; }
        if (ok) return g;
    }
}
```

06.14 容斥原理与子集枚举

【简介】

多个条件的并集/交集计数: $|A \cup B \cup C| = \sum |A| - \sum |A \cap B| + \dots$; 或“恰好满足”用反演。配合子集枚举 $O(2^k)$ 。

快速识别: 多个约束条件求“至少/恰好满足一个”的数量; 互斥条件计数; 错排/限定位置。

API 接入:

- 模板: for mask in $0..(1 \ll k)-1$, 按 popcount 奇偶加減 $f(\text{mask})$ 。
- “恰好满足 k 个”: $\text{ans} = \sum_{i=k} (-1)^{i-k} C(i, k) \cdot g(i)$ ($g(i)$ = 至少满足 i 个)。
- 错排: $D[n] = (n-1)(D[n-1] + D[n-2])$ 。

关键点: 容斥系数的符号按 popcount 奇偶; $1 \ll k$ 在 $k \geq 31$ 用 1LL; 先想清楚 $f(\text{mask})$ 好不好算 (通常是“强制满足 mask 内条件的数量”); “恰好”用二项式反演。

```
// 例: 求  $[1..N]$  中不被任何质数列表  $p$  整除的数
// ans = N - sum_{非空子集 S}  $(-1)^{|S|+1} \cdot \text{floor}(N / \text{prod}(S))$ 
// 二项式反演: “恰好  $k$  个” =  $\sum_{i=k} C(i, k) \cdot (-1)^{i-k} \cdot \text{at\_least}(i)$ 
```

06.15 生成函数与多项式 (见 07 章)

【简介】

组合计数的高级工具: 普通生成函数 (组合)、指数生成函数 (排列/标号结构)。FFT/NTT 加速卷积见 07 章。

快速识别: 方案数题用生成函数推导闭合式; “每类物品取几个” $\rightarrow$ 普通 GF; “标号集合” $\rightarrow$ EGF。

API 接入:

- 普通 GF: $A(x) = \sum a_n x^n$; 卷积对应组合合并。
- EGF: $A(x) = \sum a_n x^n / n!$; 用于排列、集合划分 (exp)。
- 常见闭合式: $1/(1-x)^k$ (隔板法)、 $(1+x)^k$ 、 e^x (集合)、 $1/(1-x-x^2)$ (Fibonacci)。

关键点: 系数转 EGF 要除以 $n!$; 组合恒等式与 GF 对应: $[x^n] 1/(1-x)^k = C(n+k-1, k-1)$; 乘法规则: 普通 GF 卷积 = 组合, EGF 卷积 = 带组合数的排列合并 (标号结构用 $C(i+j, i)$ 合并)。

```
// 常用结论 (模板速查):
//  $[x^n] 1/(1-x) = 1$ 
//  $[x^n] 1/(1-x)^k = C(n+k-1, k-1)$  (k 类物品不限量取 n 个)
//  $[x^n] 1/(1-x^2) = [n \text{ 为偶数}]$  (只能取偶数个)
//  $[x^n] (1+x)^k = C(k, n)$ 
//  $[x^n] 1/(1-x-x^2) = \text{Fibonacci}(n+1)$ 
//  $[x^n] x/(1-x-x^2) = \text{Fibonacci}(n)$ 
// 隔板法: n 个相同球分 k 组 (可空) =  $C(n+k-1, k-1)$ 
```


06.16 Burnside 引理与 Polya 定理

【简介】

等价类计数：轨道数 = $(1/|G|) * \sum_{g \in G} \text{fix}(g)$, $\text{fix}(g)$ = 在置换 g 下不变的元素数。Polya 用于“用 m 种颜色染 n 个位置”： $\text{fix}(g) = m^{\{\text{cycle}(g)\}}$ 。

快速识别：旋转/翻转对称下的不同染色方案数；等价类计数。

API 接入：

- 旋转对称 (n 个珠子, k 种颜色) : $\text{ans} = 1/n * \sum_{i=0}^{n-1} k^{\{\text{gcd}(i,n)\}}$ 。
- 一般置换群：枚举每个置换，数轮换个数 c ，累加 k^c ，除以群大小。
- 取模时除以群大小用逆元。
- 依赖：mod_pow/inv_mod (01.3 / 06.2 节)，抄板时一起抄上。

关键点：fix(g) 的计算（不动点）是核心；旋转 i 步的轮换数 = $\text{gcd}(i, n)$ ；正方形/立方体的对称群要列出所有置换；群大小 $|G|$ 是分母（旋转 $n +$ 翻转 n 等）。

```
// 项链：n 个位置，m 种颜色，只考虑旋转：ans = 1/n * Σ_{i=0}^{n-1} m^{\gcd(i,n)}
i64 necklace_count(i64 n, i64 m, i64 MOD) {
    i64 ans = 0;
    for (int i = 0; i < n; ++i) ans = (ans + mod_pow(m, std::gcd(i, (int)n), MOD)) % MOD;
    return ans * inv_mod(n % MOD, MOD) % MOD;
}
// 含翻转：n 奇 -> + n*m^{(n+1)/2}; n 偶 -> + n/2*(m^{n/2} + m^{(n/2+1)});
// 总群大小 2n
```

06.17 AtCoder floor_sum 与数论杂项速查

【简介】

$\sum_{i=0}^{n-1} \text{floor}((a*i + b) / m)$ 的 $O(\log m)$ 算法（类欧几里得）；数论常用结论速查（除数函数、平方数、连分数）。

快速识别：等差数列取整求和；类欧算法题。

API 接入：

- floor_sum(n, m, a, b): 返回 $\sum_{i=0}^{n-1} \text{floor}((a*i+b)/m)$ 。
- 变体：sum_floor(($a*i$)% m ...) 等可转化。

关键点：递归式是类欧核心： $a \geq m$ 或 $b \geq m$ 先拆整数部分； $a < m$ && $b < m$ 时翻转（把 i 换成 $\text{floor}((a*i+b)/m)$ 的上界）；实现照抄标准版。

```
// AtCoder floor_sum O(log m)
i64 floor_sum(i64 n, i64 m, i64 a, i64 b) {
    i64 ans = 0;
    if (a >= m) { ans += (n - 1) * n * (a / m) / 2; a %= m; }
    if (b >= m) { ans += n * (b / m); b %= m; }
    i64 y_max = (a * n + b) / m, x_max = y_max * m - b;
    if (y_max == 0) return ans;
    ans += (n - (x_max + a - 1) / a) * y_max;
    ans += floor_sum(y_max, a, m, (a - x_max % a) % a);
    return ans;
}
```

06.18 扩展卢卡斯 exLucas（组合数模任意合数）

【简介】

$C(n, k) \bmod M$, M 为任意合数。做法：分解 $M = \prod p_i^{e_i}$ ，对每个质数幂用“去掉 p 因子 + 卢卡斯”求 $C(n, k) \bmod p^e$ ，最后 CRT 合并。

快速识别：组合数取模且模数非质数； n 大 ($>1e9$)。

API 接入：

- exlucas(n, k, M): 返回 $C(n, k) \bmod M$ ($k < 0$ 或 $k > n$ 返回 0)。
- 依赖：mod_pow (01.3)、exgcd (06.2)。
- n, k 用 i64 (可到 $1e18$ ，内部递推)。

关键点：核心是阶乘去 p 因子：fact_mod_pk(n, p, pk) 返回 $n!$ 去掉所有 p 因子后模 p^e 的值，另数 p 的幂次；CRT 合并用 exgcd 逆元； p^e 可能较大（如 $1e9$ ），乘法用 il28。注意：fact_mod_pk 内部是 $O(pk)$ 循环， pk 超过 $1e6$ 会超时——模数是质数时直接用 06.6 组合数/Lucas，不要用 exLucas；依赖 exCRT (06.11，需

exCRT 修复版)。

```
// exLucas: C(n,k) mod M (M 任意合数)
// 依赖: mod_pow / mul_mod (01.3 节)、exgcd 与 exCRT (06.2 / 06.11 节)，
// 整节一起抄；不要重复定义 mul_mod
i64 fact_mod_pk(i64 n, i64 p, i64 pk) { // n! 去掉 p 因子后 mod pk
    if (n == 0) return 1;
    i64 res = 1;
    // 完整循环: (1..pk 中不被 p 整除的数的积)^{(n/pk)}，再乘剩余部分
    i64 cyc = 1;
    for (i64 i = 1; i <= pk; ++i) if (i % p) cyc = mul_mod(cyc, i, p);
    res = mod_pow(cyc, n / pk, pk);
    for (i64 i = n / pk * pk + 1; i <= n; ++i) if (i % p) res = mul_mod(res, i, pk);
    return mul_mod(res, fact_mod_pk(n / p, p, pk), pk); // 递归去掉 p 因子部分
}
i64 p_power_in_fact(i64 n, i64 p) { // n! 中 p 的指数
    i64 cnt = 0;
    while (n) { n /= p; cnt += n; }
    return cnt;
}
i64 comb_mod_pk(i64 n, i64 k, i64 p, i64 pk) { // C(n,k) mod p^e
    if (k < 0 || k > n) return 0;
    i64 e = p_power_in_fact(n, p) - p_power_in_fact(k, p) - p_power_in_fact(n - k, p);
    i64 a = fact_mod_pk(n, p, pk);
    i64 b = fact_mod_pk(k, p, pk);
    i64 c = fact_mod_pk(n - k, p, pk);
    i64 res = mul_mod(a, mod_pow(b, pk - pk / p - 1, pk), pk); // b 的逆 (欧拉)
    res = mul_mod(res, mod_pow(c, pk - pk / p - 1, pk), pk);
    return mul_mod(res, mod_pow(p, e, pk), pk);
}
i64 exlucas(i64 n, i64 k, i64 M) {
    if (k < 0 || k > n) return 0;
    vector<i64> rem, mods;
    i64 x = M;
    for (i64 q = 2; q * q <= x; ++q) if (x % q == 0) {
        i64 pk = 1;
        while (x % q == 0) { x /= q; pk *= q; }
        rem.push_back(comb_mod_pk(n, k, q, pk));
        mods.push_back(pk);
    }
    if (x > 1) { rem.push_back(comb_mod_pk(n, k, x, x)); mods.push_back(x); }
    return exCRT(rem, mods); // 06.11 CRT 合并
}
```

06.19 Min_25 筛（大范围积性函数前缀和）

【简介】

求积性函数 f 的前缀和 (n 到 $1e11 \sim 1e12$)。核心：把“质数部分贡献”与“合数部分贡献”分离，用整除分块 + DP。适用于 $f(p)$ 是多项式、 $f(p^e)$ 好算的函数。

快速识别： n 巨大 ($1e10+$) 的积性函数前缀和（如 $\sum f(i)$, f 在质数处是多项式）；杜教筛处理不了的非简单函数。

API 接入：

- 模板给：求 $\sum_{i=1}^n f(i)$, f 在质数 p 处 = p^k 之和（改 f_prime_sum）。
- min25_prefix(n): 返回前缀和 (mod 或 i64)。
- 需先筛出 $\sqrt{n}$ 内素数。

关键点： g 数组存“只看质数部分的 f 前缀和”（多项式分拆）；dfs 枚举最小质因子贡献合数部分； $f(1)$ 单独加；改函数只需改质数部分公式与 dfs 中的 $f(p^e)$ 项； $n^{2/3}$ 预处理平衡复杂度。实现长，整段抄，只改 f 定义。

```
// Min 25 筛框架 (求 sum f(i), f(p) = p^1 + p^0 例: 除数函数前缀和和变体)
// 依赖 06.3 线性筛 (sqrt(n) 内素数)
i64 n, sq;
vector<i64> primes;
vector<i64> g; // g[id(x)] = 质数部分 f 前缀和
i64 id1[1000005], id2[1000005]; // 整除分块 id 映射 (大/小)
i64 w[2000005]; // 所有 n/i 的取值
int m = 0;
int get_id(i64 x) { return x <= sq ? id1[x] : id2[n / x]; }
void init_min25(i64 N) {
    n = N; sq = (i64)sqrt(n);
    primes.clear();
    LinearSieve ls((int)sq); // 06.3
    primes = ls.primes;
}
```



```

for (i64 l = 1, r; l <= n; l = r + 1) {
    r = n / (n / l);
    w[++m] = n / l;
    if (w[m] <= sq) id1[w[m]] = m; else id2[n / w[m]] = m;
    g[m] = w[m] - 1; // 例: f(p)=1 (质数个数), 改成多
}
// 项数和
for (int j = 0; j < (int)primes.size(); ++j) {
    i64 p = primes[j];
    if (p * p > n) break;
    for (int i = 1; i <= m && w[i] >= p * p; ++i) {
        g[i] -= g[get id(w[i] / p)] - i;
    }
}
}
i64 dfs(i64 x, int j) { // 枚举最小质因子 >= primes[j]
    if (primes[j] > x) return 0;
    i64 ans = g[get id(x)] - j; // 质数部分贡献
    for (int k = j; k < (int)primes.size(); ++k) {
        i64 p = primes[k];
        if (p * p > x) break;
        i64 pe = p;
        for (int e = 1; pe * p <= x; ++e, pe *= p) {
            ans += (f_pe(pe) * (dfs(x / pe, k + 1) + f_pe(p))); //
        }
        f_pe(pe^e) 按题意
    }
    return ans;
}
i64 f_pe(i64 pe) { return 0; } // 改这里: f(pe^e) 的值 (模板默认 0 占位)
i64 min25_prefix(i64 N) { init_min25(N); return dfs(N, 0) + 1; } //
// +1 是 f(1)

```

06.20 Bell 数 / 错排 / 常用结论速查

【简介】

Bell 数 (n 个元素划分成若干非空集合的方案数 = 第二类 Stirling 之和); 错排 (n 个元素都不在自己位置的排列数); 以及数论常用小结论速查。

快速识别: “ n 个球分到任意个盒子 (盒子非空、无标号)” → Bell; “ n 个人重新排队都不在原位” → 错排; 结论速查用于卡壳时快速翻。

API 接入:

- Bell: $Bell[n] = \sum_k S2(n, k)$ (依赖 06.7 的 $S2$); 模质数可 $O(n^2)$ DP。
- 错排: $D[0]=1, D[1]=0; D[n] = (n-1)*(D[n-1]+D[n-2])$ 。
- 常见结论 (速查):
- 鸽巢原理: $n+1$ 个物品放 n 个盒子, 至少一个盒子有 2 个。
- $\sum_{i=1}^n \lfloor n/i \rfloor \approx n \ln n$ (调和级数)。
- 平方差: $a^2 - b^2 = (a-b)(a+b)$; 奇偶分解。
- gcd 性质: $\gcd(a, b) = \gcd(a-b, b); \gcd(a, \text{lcm}(b, c)) = \text{lcm}(\gcd(a, b), \gcd(a, c))$ 。
- 模性质: $(a+b) \bmod m = ((a \bmod m) + (b \bmod m)) \bmod m$ 。

关键点: Bell 数递推 $B[n+1] = \sum C(n, k)$

$B[k]$ (组合恒等式); 错排公式除递推外有 $D[n] = n! \sum (-1)^k / k!$; 结论速查只给方向, 细节回对应节。

```

// 错排 (递推, mod 质数):
// D[0] = 1, D[1] = 0;
// for (int i = 2; i <= n; ++i) D[i] = (i - 1) * (D[i - 1] + D[i - 2]) % mod;
// Bell 数 ( $O(n^2)$  三角, 依赖 S2 或直接递推):
// B[0] = 1;
// for (int i = 1; i <= n; ++i) { B[i] = 0; for (int k = 0; k < i; ++k) B[i] = (B[i] + C(i-1, k) * B[k]) % mod; }

```

06.21 LTE 引理与连分数 (结论速查)

【简介】

LTE (Lifting The Exponent): 求 $v_p(a^n - b^n)$ (质数幂因子指数) 的精确公式; 连分数: 有理数/二次无理数的连分数表示, 用于 Pell 方程与丢番图逼近。

快速识别: $a^n - b^n$ 能被 p 的多少次幂整除; Pell 方程 $x^2 - Dy^2 = 1$ 的最小解。

API 接入:

- LTE: p 奇质数且 $p \mid a-b$ ($p \nmid a, b$) 时, $v_p(a^n - b^n) = v_p(a-b) + v_p(n)$; n 偶数时 $v_p(a^n + b^n) = v_p(a+b) + v_p(n)$; $p=2$ 有单独公式。
- 连分数: 循环连分数求 Pell 最小解 (暴力迭代到出现循环)。

关键点: LTE 前提 ($p \mid a-b, p \nmid a, b$)

必须满足, 否则公式失效; $p=2$ 的公式不同 ($v_2(a^n - b^n)$ 分奇偶); Pell 最小解 = 连分数循环节末尾的渐近分数。

```

// LTE:  $v_p(a^n - b^n)$  ( $p$  奇质数,  $p \mid a-b, p \nmid a, b$ )
//  $v = v_p(a-b) + v_p(n)$ 
//  $p=2$  特例:
//  $n$  奇:  $v_2(a^n - b^n) = v_2(a-b)$ 
//  $n$  偶:  $v_2(a^n - b^n) = v_2(a-b) + v_2(a+b) + v_2(n) - 1$ 
// 连分数求 Pell  $x^2 - Dy^2 = 1$  最小解: 展开  $\text{sqrt}(D)$  连分数, 取渐近分数试

```

07 线性代数与多项式

07.1 矩阵与矩阵快速幂

【简介】

矩阵乘法 +

快速幂: 把线性递推 (Fibonacci、常系数线性递推、图上走 k 步方案数) 加速到 $O(n^3 \log k)$ 。

快速识别: 线性递推求第 n 项 (n 巨大); 图上“恰好/至多走 k 步”计数; min-plus 矩阵 (最短路版); 状态转移是线性的 DP。

API 接入:

- Mat: $\text{Mat}(n, m)$ 全 0; $\text{unit}(n)$ 单位阵; $\text{operator}*(O(n^3))$, 可换 min-plus)。
- $\text{mat_pow}(A, k)$: A 的 k 次方。
- 递推转矩阵: 状态向量 v , 转移矩阵 T , $v_k = T^k * v_0$ 。
- min-plus: $C[i][j] = \min_k (A[i][k] + B[k][j])$, 用于“恰好 k 条边最短路”。

关键点: 矩阵维数 = 状态数 (通常 $\leq 100, n^3$

才可行); 初始向量 v_0 要与 T

方向一致 (行/列向量约定); min-plus 单位元 =

0 (对角线)、无穷大 (非对角); k 用 i64; 模意义取模。

```

struct Mat {
    int n, m;
    vector<vector<i64>>> a;
    Mat(int n_ = 0, int m_ = 0, i64 v = 0) : n(n_), m(m_), a(n_, vector<i64>(m, v)) {}
    static Mat unit(int sz) {
        Mat I(sz, sz);
        for (int i = 0; i < sz; ++i) I.a[i][i] = 1;
        return I;
    }
    Mat operator*(const Mat& o) const { // 常规乘法 (可换 min-plus)
        Mat r(n, o.m);
        for (int i = 0; i < n; ++i)
            for (int k = 0; k < m; ++k) if (a[i][k])
                for (int j = 0; j < o.m; ++j)
                    r.a[i][j] = (r.a[i][j] + a[i][k] * o.a[k][j]) % mod7;
        return r;
    }
};
Mat mat_pow(Mat A, i64 k) {
    Mat R = Mat::unit(A.n);
    while (k) { if (k & 1) R = R * A; A = A * A; k >>= 1; }
    return R;
}
// Fibonacci T = {{1,1},{1,0}} f_n = (T^n * {f1, f0})[0]

```

07.2 高斯消元 (实数 / 模意义)

【简介】

解线性方程组 $Ax =$

b (实数版消元; 模质数版除以逆元)。判无解/无穷解。

快速识别: n 元一次方程组; 期望/概率方程 (吸收马尔可夫链); 电网电流; 几何求交。

API 接入:

- `gauss_real(aug)`: 增广矩阵 (n 行 n+1 列), 返回解 vector (-1 无解, -2 无穷多解)。
 - `gauss_mod(aug, MOD)`: 模质数版。
 - 稀疏/特殊结构 (带状、三对角) 用专用算法。
- 关键点: 选主元 (最大绝对值/非零) 防精度问题; 模版每行除以主元逆元; 无解判 rank 增广 > rank 系数; 无穷解 = 自由元 > 0; 回代方向从后往前。

```
// 实数高斯消元 (解线性方程组 Ax = b)
// 用法: vector<vector<double>> a(n, vector<double>(n + 1)); 第 i 行存系数 + 常数
// 返回: ans (解); 空 vector = 无解或无穷解 (需要时用 rank 判定)
vector<double> gauss_real(vector<vector<double>> a) {
    int n = (int)a.size();
    for (int col = 0, row = 0; col < n && row < n; ++col) {
        int piv = row;
        for (int i = row + 1; i < n; ++i)
            if (fabs(a[i][col]) > fabs(a[piv][col])) piv = i;
        if (fabs(a[piv][col]) < 1e-12) continue; // 自由元
        swap(a[piv], a[row]);
        for (int i = 0; i < n; ++i) if (i != row) { // 全消 (高斯-约当)
            double f = a[i][col] / a[row][col];
            for (int j = col; j <= n; ++j) a[i][j] -= f * a[row][j];
            ++row;
        }
        // 检查矛盾行: 0 = 非零 -> 无解
        for (int i = 0; i < n; ++i) {
            bool allzero = true;
            for (int j = 0; j < n; ++j) if (fabs(a[i][j]) > 1e-12) { allzero = false; break; }
            if (allzero && fabs(a[i][n]) > 1e-12) return {};
        }
        vector<double> ans(n);
        for (int i = 0; i < n; ++i) { // 每行主元列
            int col = -1;
            for (int j = 0; j < n; ++j) if (fabs(a[i][j]) > 1e-12) { col = j; break; }
            if (col != -1) ans[col] = a[i][n] / a[i][col];
        }
        return ans;
    }
    // 模质数版: 把除法 f = a[i][col] / a[row][col] 换成乘逆元
    // i64 f = a[i][col] * mod_pow(a[row][col], MOD - 2, MOD) % MOD;
```

07.3 行列式 (任意模数)

【简介】

n x n 矩阵行列式取模: 模质数直接高斯消元; 任意模数用“辗转相除消元” (不依赖逆元)。

快速识别: 生成树计数 (Matrix-Tree)、组合问题里的行列式、奇偶性/可逆性判断。

API 接入:

- `det_mod(a, MOD)`: 返回行列式 (MOD 任意, 可为合数)。
 - 模质数版: 普通消元每行乘主元逆元。
 - Matrix-Tree: 拉普拉斯矩阵删一行一列后求行列式。
- 关键点: 任意模数版用辗转相除 (类似 gcd 的消元, 每次交换行要翻转符号); 每交换两行符号取反; 复杂度 $O(n^3 \log \text{MOD})$; n <= 500 可用。

```
// 行列式取模 (任意模数, 辗转相除消元, 不依赖逆元)
// 用法: det_mod(a, MOD): a 为 n x n 矩阵; 返回行列式值
// 模质数时可改用简单高斯消元 (每行乘主元逆元), 速度更快
static i64 det_mod(vector<vector<i64>> a, i64 MOD) {
    int n = (int)a.size();
    i64 ans = 1;
    for (int i = 0; i < n; ++i) {
        // 选主元: 第 i 列找一个非零行换上来 (换行变号)
        for (int j = i + 1; j < n; ++j)
            if (a[i][i] == 0 && a[j][i] != 0) { swap(a[i], a[j]); ans = (MOD - ans) % MOD; }
        if (a[i][i] == 0) return 0;
        ans = ans * (a[i][i] % MOD + MOD) % MOD;
        // 用第 i 行消下面各行: a[j][k] -= a[j][i] * a[i][i] / a[i][i] (模意义乘逆元)
        i64 inv = mod_pow(a[i][i], MOD - 2, MOD); // 要求 MOD 质数; 合数用辗转相除版
        for (int j = i + 1; j < n; ++j) if (a[j][i]) {
            i64 f = a[j][i] * inv % MOD;
            for (int k = i; k < n; ++k)
```

```
        a[j][k] = (a[j][k] - f * a[i][k] % MOD + MOD) % MOD;
    }
    return ans;
}
// Matrix-Tree 定理: 生成树数 = det mod (拉普拉斯矩阵删去任意一行一列, MOD)
// 拉普拉斯 L: L[i][i] = 点 i 度数; L[i][j] = -边数 (i != j)
```

07.4 FFT (任意整数卷积)

【简介】

$O(n \log n)$ 多项式/序列卷积。浮点版: 结果需取整; 大系数/大 n 注意精度。NTT 见 07.5 (模质数首选)。

快速识别: 卷积式 $c[k] = \sum a[i]b[k-i]$; 多项式乘法; 生成函数展开; 大数乘法。

API 接入:

- `vector<i64> convolution_f64(a, b)`: 返回卷积 (已四舍五入为整数); 长度 n+m-1。
 - 优化: 结果取整用 `llround`; 系数大 (> $1e9$) 或 n+m 大 (> $2e5$) 建议 NTT 或拆系数 FFT。
- 关键点: 补零到 2 的幂; 复数用 `complex<double>`; 逆变换除以 n; 精度: 系数乘积总和超过 $1e15$ 会丢精度, 用 NTT; 长度上限与 double 精度有关 ($1e5$ 内安全)。

```
// FFT (蝶形迭代版)
void fft(vector<complex<double>>& a, bool invert) {
    int n = (int)a.size();
    for (int i = 1, j = 0; i < n; ++i) {
        int bit = n >> 1;
        for (; j & bit; bit >>= 1) j ^= bit;
        j ^= bit;
        if (i < j) swap(a[i], a[j]);
    }
    for (int len = 2; len <= n; len <= 1) {
        double ang = 2 * acos(-1.0) / len * (invert ? -1 : 1);
        complex<double> wlen(cos(ang), sin(ang));
        for (int i = 0; i < n; i += len) {
            complex<double> w(1);
            for (int j = 0; j < len / 2; ++j) {
                complex<double> u = a[i + j], v = a[i + j + len / 2];
                w *= wlen;
                a[i + j] = u + v;
                a[i + j + len / 2] = u - v;
            }
        }
    }
    if (invert) for (auto& x : a) x /= n;
}
vector<i64> convolution_f64(const vector<i64>& a, const vector<i64>& b) {
    if (a.empty() || b.empty()) return {};
    int need = (int)a.size() + (int)b.size() - 1, n = 1;
    while (n < need) n <= 1;
    vector<complex<double>> fa(n), fb(n);
    for (int i = 0; i < (int)a.size(); ++i) fa[i] = a[i];
    for (int i = 0; i < (int)b.size(); ++i) fb[i] = b[i];
    fft(fa, false); fft(fb, false);
    for (int i = 0; i < n; ++i) fa[i] *= fb[i];
    fft(fa, true);
    vector<i64> res(need);
    for (int i = 0; i < need; ++i) res[i] = (i64)llround(fa[i].real());
    return res;
}
```

07.5 NTT (模质数卷积)

【简介】

模 998244353 的 FFT (数论变换), 无浮点误差, 竞赛首选。支持长度 $\leq 2^{23}$ 。

快速识别: 模质数卷积; 大系数卷积; 多项式全套操作 (07.6-07.9) 的地基。

API 接入:

- `convolution_ntt(a, b)`: 返回模 998244353 的卷积。
- 模数/原根: 998244353 原根 3; `ntt(a, invert)` 底层。
- 其他模数: 原根不同 (1004535809/469762049 原根 3)。
- 任意模数: 三模 NTT + CRT (或 FFT 拆系数)。

- 依赖: mod_pow (01.3 节), 抄板时一起抄上。

关键点: 长度限制 $n+m-1 \leq 2^{23}$; 系数先规范到 $[0, \text{MOD})$; 空数组返回空; 逆变换后乘 inv_n; 中间乘法 i64。

```
const int NTT_MOD = 998244353, NTT_G = 3;
void ntt(vector<int>& a, bool invert) {
    int n = (int)a.size();
    for (int i = 1, j = 0; i < n; ++i) {
        int bit = n >> 1;
        for (; j & bit; bit >>= 1) j ^= bit;
        if (i < j) swap(a[i], a[j]);
    }
    for (int len = 2; len <= n; len <= 1) {
        int wlen = mod_pow(NTT_G, (NTT_MOD - 1) / len, NTT_MOD);
        if (invert) wlen = mod_pow(wlen, NTT_MOD - 2, NTT_MOD);
        for (int i = 0; i < n; i += len) {
            i64 w = 1;
            for (int j = 0; j < len / 2; ++j) {
                int u = a[i + j];
                int v = (int)(a[i + j + len / 2] * w % NTT_MOD);
                a[i + j] = u + v < NTT_MOD ? u + v : u + v - NTT_MOD;
                a[i + j + len / 2] = u - v >= 0 ? u - v : u - v + NTT_MOD;
                w = w * wlen % NTT_MOD;
            }
        }
    }
    if (invert) {
        int inv_n = mod_pow(n, NTT_MOD - 2, NTT_MOD);
        for (int& x : a) x = (int)(1LL * x * inv_n % NTT_MOD);
    }
}
vector<int> convolution ntt(vector<int> a, vector<int> b) {
    if (a.empty() || b.empty()) return {};
    int need = (int)a.size() + (int)b.size() - 1, n = 1;
    while (n < need) n <= 1;
    a.resize(n); b.resize(n);
    ntt(a, false); ntt(b, false);
    for (int i = 0; i < n; ++i) a[i] = (int)(1LL * a[i] * b[i] % NTT_MOD);
    ntt(a, true);
    a.resize(need);
    return a;
}
```

07.6 FWT (AND / OR / XOR 卷积)

【简介】

位运算卷积: $c[k] = \sum_{i \text{ op } j = k} a[i]b[j]$ (op = AND/OR/XOR), $O(n \log n)$ 。子集卷积的基底。

快速识别: “按位与/或/异或等于

k”的计数卷积; 集合幂级数; 高维前缀和的卷积版。

API 接入:

- fwt_or(a, invert) / fwt_and / fwt_xor (原位变换): convolution_or/and/xor(a, b)。
- 数组长度必须是 2 的幂 (不足补 0)。
- XOR 版逆变换要乘 inv_n。
- 依赖: NTT_MOD/mod_pow (07.5 节定义在上方), 整节一起抄。

关键点: 三种变换的蝶形公式不同 (OR/AND 是前缀和型, XOR 是 Walsh-Hadamard 型); invert 时 OR/AND 直接反符号, XOR 要除 n; 长度 2 的幂。

```
// OR 卷积 (invert=true 为逆变换)
void fwt_or(vector<int>& a, bool invert) {
    int n = (int)a.size();
    for (int len = 2; len <= n; len <= 1)
        for (int i = 0; i < n; i += len)
            for (int j = 0; j < len / 2; ++j)
                a[i + j + len / 2] = (a[i + j + len / 2] + (invert ?
NTT_MOD - a[i + j] : a[i + j])) % NTT_MOD;
}
// AND 卷积: 把上面 += 换成对低位做 (a[i+j] += (invert?-1:1)*a[i+j+len/2])
// XOR 卷积:
void fwt_xor(vector<int>& a, bool invert) {
    int n = (int)a.size();
    for (int len = 2; len <= n; len <= 1)
        for (int i = 0; i < n; i += len)
            for (int j = 0; j < len / 2; ++j) {
                int u = a[i + j], v = a[i + j + len / 2];
```

```
        a[i + j] = (u + v) % NTT_MOD;
        a[i + j + len / 2] = (u - v + NTT_MOD) % NTT_MOD;
    }
    if (invert) {
        int inv_n = mod_pow(n, NTT_MOD - 2, NTT_MOD);
        for (int& x : a) x = (int)(1LL * x * inv_n % NTT_MOD);
    }
}
```

07.7 形式幂级数核心 (inv / ln / exp / sqrt / pow)

【简介】

多项式全套操作: 求逆、ln、exp、开方、幂、除法 (牛顿迭代 + NTT)。生成函数题的核心工具。

快速识别: 需要多项式的逆/exp/ln; 生成函数展开; 常系数线性递推转生成函数。

API 接入:

- 各函数输入 vector<int> (模 998244353), 输出同长度 (截断到 n 项)。
- poly_inv(a, n)、poly_ln(a, n) (要求 $a[0]=1$)、poly_exp(a, n) ($a[0]=0$)、poly_sqrt(a, n)、poly_pow(a, k, n)。
- 组合使用: exp 依赖 inv 与积分/微分; pow = exp($k \cdot \ln$)。

关键点: 牛顿迭代实现 (倍增); 边界条件: ln 需 $a[0]=1$, exp 需 $a[0]=0$, sqrt 需 $a[0]$ 有二次剩余; 长度补 2 的幂; 常数较大, $n > 1e5$ 慎用全套。

```
// 多项式求逆 (倍增, NTT 加速):
// inv[0] = a[0]^{-1}; inv = inv * (2 - a * inv) 每次翻倍
// 多项式 ln: ln(a) = \int a' / a; exp: 牛顿迭代 e = e * (1 - ln(e) + a)
// 多项式 sqrt: s = (s + a/s) / 2
// 多项式 pow: a^k = exp(k * ln(a)) (注意 a[0] 处理)
// 以上均基于 NTT 卷积, 具体实现较长, 抄板时整段复制
```

07.8 多项式多点求值 / 快速插值 (积树)

【简介】

$f(x_i)$ 全部点值 (多点求值 $O(n \log^2 n)$); 从 n 个点插出多项式 (快速插值, $O(n \log^2 n)$)。基于“积树” (分治建乘积树 + 多项式取模)。

快速识别: 给定多项式算多个点值; 拉格朗日插值的高效版; n 大 ($1e5$) 时。

API 接入:

- 多点求值: multipoint_eval(f, xs) 返回 $f(xs[i])$ 。
- 快速插值: interpolate(xs, ys) 返回多项式系数。
- n 小时 ($\leq 2e3$) 用 $O(n^2)$ 朴素版 (见 07.9 Lagrange)。
- 需要多项式取模 (poly_mod)。

关键点: 积树 = 每层节点存 $n(x - x_i)$ 的多项式; 求值 = 从上到下模积树; 插值 = 转置 (对偶) 求值 + 逆元处理; 实现较长, 抄标准版。

```
// 积树 (segment product tree):
// node[l..r] = \prod_{i=l..r} (x - x_i), 底层为 (x - x_i)
// 多点求值: f mod node[l] 后向下递归
// 快速插值: 先求  $g = \prod (x - x_i)$ ,  $g'(x_i)$  作分母, 再做转置求值
```

07.9 Berlekamp-Massey + Kitamasa (线性递推)

【简介】

BM 从序列前几项 ($2k$ 个) 恢复最短线性递推 ($O(k^2)$); Kitamasa 用该递推 $O(k^2 \log n)$ 求第 n 项。数列题“给前几项求第 n 项”的通杀工具。

快速识别: 只给数列前若干项, 求第 n 项 (n 巨大); “观察/猜递推”的替代; 找不到规律时暴力 BM。

API 接入:

- berlekamp_massey(s): s 为 vector<i64> (mod 998244353); 返回递推系数 coef (满足 $s[n] = \sum \text{coef}[i] * s[n-1-i]$)。
- kitamasa(coef, init, n): 返回第 n 项 (0-based 或 1-based 按实现, 用前确认)。

- 要求序列项数 $\geq 2 \times$ 阶数。

关键点：最少需要 $2k$ 项 ($k =$ 递推阶数)；全部运算 mod 998244353; Kitamasa 本质 = 多项式取模 + 快速幂 ($x^n \bmod$ 特征多项式)；结果正确性依赖“递推确实存在”（组合计数题适用，随机数据不适用）。

```
// Berlekamp-Massey: 从序列前若干项恢复最短线性递推 (mod 998244353)
// 用法: auto coef = berlekamp_massey(s); coef.size() = 递推阶数 k
// s[n] = sum_{i=0}^{k-1} coef[i] * s[n-1-i]
const i64 BM_MOD = 998244353;
vector<i64> berlekamp_massey(const vector<i64>& s) {
    vector<i64> C(1, 1), B(1, 1);
    i64 b = 1;
    int L = 0, m = 1;
    for (int n = 0; n < (int)s.size(); ++n) {
        i64 d = 0;
        for (int i = 0; i <= L; ++i) d = (d + C[i] * s[n - i]) % BM_MOD;
        if (d == 0) { ++m; continue; }
        vector<i64> T = C;
        i64 coef = d * mod_pow(b, BM_MOD - 2, BM_MOD) % BM_MOD;
        if ((int)C.size() < (int)B.size() + m) C.resize(B.size() + m);
        for (int i = 0; i < (int)B.size(); ++i)
            C[i + m] = (C[i + m] - coef * B[i] % BM_MOD + BM_MOD) % BM_MOD;
        if (2 * L <= n) {
            L = n + 1 - L;
            B = T;
            b = d;
            m = 1;
        } else ++m;
    }
    // C[0]=1, C[1..L] 是递推系数 (s[n] = -C[1]*s[n-1] - ... )
    vector<i64> coef(L);
    for (int i = 1; i <= L; ++i) coef[i - 1] = (BM_MOD - C[i]) % BM_MOD;
    return coef;
}

// Kitamasa: 用递推 coef 求第 n 项 (n 从 0 开始); init = 前 k 项 (init[i] = 第 i 项)
// 复杂度  $O(k^2 \log n)$ 
i64 kitamasa(const vector<i64>& coef, const vector<i64>& init, i64 n) {
    int k = (int)coef.size();
    if (n < k) return init[n];
    // 多项式乘法 (mod 特征多项式  $x^k - \text{coef}[0]x^{k-1} - \dots - \text{coef}[k-1]$ )
    auto mul = [&](const vector<i64>& a, const vector<i64>& b) {
        vector<i64> c(2 * k - 1, 0);
        for (int i = 0; i < k; ++i) if (a[i])
            for (int j = 0; j < k; ++j) if (b[j])
                c[i + j] = (c[i + j] + a[i] * b[j]) % BM_MOD;
        for (int i = 2 * k - 2; i >= k; --i) if (c[i])
            for (int j = 1; j <= k; ++j)
                c[i - j] = (c[i - j] + c[i] * coef[j - 1]) % BM_MOD;
        // 降幂
        c.resize(k);
        return c;
    };
    vector<i64> res(k, 0), base(k, 0);
    res[0] = 1;
    base[1] = 1;
    while (n) {
        if (n & 1) res = mul(res, base);
        base = mul(base, base);
        n >>= 1;
    }
    i64 ans = 0;
    for (int i = 0; i < k; ++i) ans = (ans + res[i] * init[i]) % BM_MOD;
    return ans;
}

// 用法: 序列 s 至少给  $2k$  项; coef = BM(s); 第 n 项 = kitamasa(coef, s 前 k 项, n)
```

07.10 常系数线性递推的其他工具 (Bostan-Mori / 特征值)

【简介】

Bostan-Mori: $O(k \log k \log n)$ 求有理生成函数第 n 项；比 Kitamasa 快，适合 k 大 n 巨大。特征多项式法：快速幂矩阵的替代。

快速识别：递推阶数 k 较大 ($1e3 \sim 1e4$) + n 巨大；生成函数 $P(x)/Q(x)$ 的第 n 项。

API 接入：

- bostan_mori(P, Q, n): P/Q 为多项式 (mod)，返回 $[x^n] P/Q$ 。
- 常系数递推转生成函数： $Q =$ 特征多项式， P 由初值定。
- 需要 NTT 与多项式求逆。

关键点：核心是偶部/奇部提取： $Q(x)Q(-x)$ 只含偶次项，替换 $x^2 = y$ 后规模减半；实现需要完整多项式工具链； k 小 (< 100) 用 Kitamasa 更简单。

```
// Bostan-Mori 核心:
// while (n > 0):
//     Q neg(x) = Q(-x)
//     S = P * Q neg; T = Q * Q neg
//     if (n 为偶) P = S 的偶次项; else P = S 的奇次项
//     Q = T 的偶次项; n >>= 1
// 最后 return P[0] / Q[0] (乘逆元)
```

07.11 拉格朗日插值 (连续点 / 任意点)

【简介】

从 $n+1$ 个点插出 n 次多项式并求 $f(k)$ 。连续点 $0..n$ 有 $O(n)$ 公式；任意点 $O(n^2)$ 。

快速识别：“ f 是多项式，给 $n+1$ 个点，求 $f(k)$ ”；前缀和/求和的多项式性质（如 $\sum i^k$ 是 $k+1$ 次多项式）； n 大（点连续时）。

API 接入：

- 连续点 $0..n$: interpolate_0n(y, k): $y[i] = f(i)$; 返回 $f(k)$ (模 MOD)。 $O(n)$ 。
- 任意点: interpolate_general(xs, ys, k) $O(n^2)$ 。
- $\sum_{i=1}^n i^k$ 是 $k+1$ 次多项式：取前 $k+2$ 个点插值。
- 依赖：Comb (06.6 节) + mod_pow (01.3 节)，抄板时一起抄上。

关键点：连续点公式需要阶乘与阶乘逆元； $k < 0$ 或 k 在点内直接返回 $y[k]$ ；分母预计算前缀/后缀积；mod 质数。

```
// 连续点 0..n 插值 (y[i] = f(i), 求 f(k), mod 质数)
i64 interpolate_0n(const vector<i64>& y, i64 k, i64 MOD, const Comb& cb) {
    int n = (int)y.size() - 1; // 点数 = n+1
    if (k <= n) return y[k];
    vector<i64> pre(n + 2, 1), suf(n + 2, 1);
    for (int i = 0; i <= n; ++i) pre[i + 1] = pre[i] * ((k - i) % MOD + MOD) % MOD;
    for (int i = n; i >= 0; --i) suf[i] = suf[i + 1] * ((k - i) % MOD + MOD) % MOD;
    i64 ans = 0;
    for (int i = 0; i <= n; ++i) {
        i64 num = pre[i] * suf[i + 1] % MOD;
        i64 den = cb.fact[i] * cb.inv_fact[n - i] % MOD;
        if ((n - i) & 1) den = (MOD - den) % MOD;
        ans = (ans + y[i] * num % MOD * mod_pow(den, MOD - 2, MOD)) % MOD;
    }
    return ans;
}
```

07.12 康托展开与逆康托

【简介】

排列 $\leftrightarrow$ 字典序排名 的双向映射：康托展开 $O(n^2)$ (BIT 优化 $O(n \log n)$) 求排名；逆康托由排名还原排列。

快速识别：排列的字典序编号；“第 k 个排列”。

API 接入：

- cantor(p): p 为 $0..n-1$ 的排列 (0-based)，返回排名 (0-based)。
- inv_cantor(rank, n): 返回第 rank 个排列 (0-based)。
- 公式：rank = $\sum p[i]$ 位置后比 $p[i]$ 小的个数 * $(n-1-i)!$ (0-based 时去掉 +1)。
- 依赖：Fenwick (02.5 节)，抄板时一起抄上。

关键点: 0-based 排名与 1-based 差

1, 用前确认: 小阶乘表预处理; 大 n ($n > 20$) 排名超 $i64$ 时用模意义或高精度。

```
// 康托展开 (0-based 排名, n 小直接用阶乘表):
i64 cantor(const vector<int>& p) {
    int n = (int)p.size();
    vector<i64> fact(n + 1, 1);
    for (int i = 1; i <= n; ++i) fact[i] = fact[i - 1] * i;
    i64 rank = 0;
    Fenwick<int> fw(n); // 标记已用
    for (int i = 0; i < n; ++i) {
        int less = p[i] - fw.sum(p[i]); // 未使用且小于 p[i] 的个数
        rank += (i64)less * fact[n - 1 - i];
        fw.add(p[i] + 1, 1);
    }
    return rank;
}
```

07.13 异或方程组 (GF(2) 高斯消元)

【简介】

A $x = b$ 模 2 的线性方程组 (每个变量 0/1)。用于异或方程组、开关灯问题、图染色奇偶。

快速识别: 异或约束的方程组; 每个开关/灯的状态约束; “翻 0/1 使满足条件”。

API 接入:

- xor_gauss(aug, n, m): 增广矩阵用 vector<bitset<>>; 返回解或自由元数。
- 用 bitset 加速 ($O(n^3/64)$)。
- 解数 = $2^{\text{自由元}}$ 。

关键点: 主元列记录; 每行 bitset 异或消元; 自由元任意取值都是解; 无解判矛盾行; 状态/开关问题先建方程再消元。

```
// GF(2) 高斯消元 (bitset 加速)
int xor_gauss(vector<bitset<512>>& a, int n, int m, vector<int>& free cols) {
    int row = 0;
    for (int col = 0; col < m && row < n; ++col) {
        int piv = row;
        while (piv < n && !a[piv][col]) ++piv;
        if (piv == n) { free cols.push back(col); continue; }
        swap(a[piv], a[row]);
        for (int i = 0; i < n; ++i)
            if (i != row && a[i][col]) a[i] ^= a[row];
        ++row;
    }
    for (int i = row; i < n; ++i) if (a[i][m]) return -1; // 无解
    return m - row; // 自由元个数
}
```

07.14 LGV 引理 (不相交路径计数)

【简介】

有向无环图中, 从起点集 A 到终点集 B 的两两不相交路径数 = 路径计数矩阵的行列式 ($M[i][j]$ = 从 $A[i]$ 到 $B[j]$ 的路径数, $\det(M)$ 即不相交路径数, 带符号)。常用于网格图/棋盘计数。

快速识别: 多起点多终点、要求路径互不相交 (不相交方案数); 网格/树形 DAG 的路径计数。

API 接入:

- 先算路径数矩阵: ways[i][j] = 从起点 i 到终点 j 的路径数 (DP 或组合数)。
- det_mod(ways, MOD) (07.3 节) 求行列式 = 不相交路径数 (模意义)。
- 前提: 图是 DAG, 且任意两条路径若相交必“同向” (LGV 引理成立条件)。

关键点: 行列式的符号对应排列的奇偶性, 通常取绝对值或模意义; 路径数矩阵用组合数 (网格) 或 DP (一般 DAG); n (起点数) 小 (≤ 50) 用 07.3 行列式; LGV 要求起点集与终点集大小相等。

```
// LGV: det(ways) 即不相交路径数。网格例: A[i]=(0, y i) 到 B[j]=(x j, M),
// ways[i][j] = C(x_j + M - y_i, x_j) (组合数, 06.6 节); 然后 det_mod(ways, MOD)。
```

```
// 依赖: Comb (06.6) \ det_mod (07.3)
```

07.15 形式幂级数完整实现 (inv / ln / exp / sqrt / pow)

【简介】

基于 NTT 的多项式全套运算 (截断到 n 项), 是生成函数/多项式题的核心工具。整段抄板, 只改需要的地方。

快速识别: 需要多项式求逆/ln/exp/开方/幂; 生成函数展开; 分治 NTT 的前置。

API 接入:

- poly_inv(a, n): $a[0] \neq 0$ 时的逆; poly_ln(a, n): 需 $a[0] = 1$; poly_exp(a, n): 需 $a[0] = 0$ 。
- poly_sqrt(a, n): 需 $a[0]$ 有二次剩余 (常见直接 = 1); poly_pow(a, k, n): k 为 $i64$ 。
- 全部返回 vector<int> (mod 998244353)。

关键点: 牛顿迭代倍增 (inv/exp/sqrt 每轮翻倍长度); $\ln = \int (a'/a)$; exp 用迭代 $e = e(1 - \ln e +$

$a)$; 前置条件不满足先处理 (如 $a[0] \neq 1$ 时 $\ln$ 需提取首项); 长度补 2 的幂; 常数大, n 超过 $1e5$ 慎用全套。

```
// 依赖: 07.5 NTT 的 convolution_ntt 与 NTT_MOD (整节一起抄)
vector<int> poly_inv(const vector<int>& a, int n) {
    vector<int> res(1, (int)mod pow(a[0], NTT_MOD - 2, NTT_MOD));
    int len = 1;
    while (len < n) {
        len <= 1;
        vector<int> f(min((int)a.size(), len));
        for (int i = 0; i < (int)f.size(); ++i) f[i] = a[i];
        f.resize(len); res.resize(len);
        vector<int> fntt = f, rntt = res;
        ntt(fntt, false); ntt(rntt, false);
        for (int i = 0; i < len; ++i)
            fntt[i] = (int)(1LL * fntt[i] * rntt[i] % NTT_MOD);
        ntt(fntt, true);
        fntt[0] = (2 - fntt[0] + NTT_MOD) % NTT_MOD;
        for (int i = 1; i < len; ++i) fntt[i] = fntt[i] * (NTT_MOD - fntt[i]) % NTT_MOD;
        ntt(fntt, false);
        for (int i = 0; i < len; ++i)
            fntt[i] = (int)(1LL * fntt[i] * rntt[i] % NTT_MOD);
        ntt(fntt, true);
        res = fntt;
    }
    res.resize(n);
    return res;
}

vector<int> poly_deriv(const vector<int>& a) { // 导数
    vector<int> r(max(0, (int)a.size() - 1));
    for (int i = 1; i < (int)a.size(); ++i) r[i - 1] = (int)(1LL * a[i] * i % NTT_MOD);
    return r;
}

vector<int> poly_integ(const vector<int>& a) { // 积分 (需逆元表 inv[i])
    static vector<int> inv{0, 1};
    while ((int)inv.size() <= (int)a.size())
        inv.push_back((int)(NTT_MOD - 1LL * (NTT_MOD / (int)inv.size()) * inv[NTT_MOD % (int)inv.size()] % NTT_MOD));
    vector<int> r(a.size() + 1);
    for (int i = 0; i < (int)a.size(); ++i) r[i + 1] = (int)(1LL * a[i] * inv[i + 1] % NTT_MOD);
    return r;
}

vector<int> poly_ln(const vector<int>& a, int n) { // 要求 a[0] = 1
    vector<int> der = poly_deriv(a), inv = poly_inv(a, n);
    der.resize(n);
    vector<int> mul = convolution_ntt(der, inv);
    mul.resize(n);
    return poly_integ(mul);
}

vector<int> poly_exp(const vector<int>& a, int n) { // 要求 a[0] = 0
    vector<int> res(1, 1);
    int len = 1;
    while (len < n) {
        len <= 1;
        vector<int> t = poly_ln(res, len); // ln(当前 e)
        for (int i = 0; i < len; ++i) {
            i64 val = (i < (int)a.size() ? a[i] : 0) - t[i];
            t[i] = ((val % NTT_MOD) + NTT_MOD) % NTT_MOD;
        }
    }
    res.resize(n);
    return res;
}
```



```
    }
    t[0] = (t[0] + 1) % NTT_MOD; // 1 - ln e
+ a
    res = convolution ntt(res, t);
    res.resize(len);
  }
  res.resize(n);
  return res;
}
// poly_sqrt(a, n): s = (s + a/s)/2 倍增 (依赖 poly_inv); a[0] 需为 1 (或
二次剩余)
// poly_pow(a, k, n): a^k = exp(k * ln a), 需 a[0] != 0 (a[0]=0 时先提公
因子)
```

07.16 Bostan-Mori (有理生成函数第 n 项)

【简介】

$O(k \log k \log n)$ 求 $[x^n] P(x)/Q(x)$ (P, Q 为多项式)。比 Kitamasa 快, 适合 k 大 ($1e3 \sim 1e4$) n 巨大。常系数线性递推转生成函数的标准工具。

快速识别: 递推阶数 k 大 + n 巨大; 生成函数形式的第 n 项。

API 接入:

- `bostan_mori(P, Q, n)`: 返回 $[x^n] P/Q \pmod{998244353}$; P, Q 为系数数组。
- 依赖: `poly` 全套 (07.15) + `NTT` (07.5)。

关键点: 核心 = 偶部/奇部提取: $Q(-x)$ 构造 $Q(x)Q(-x)$ (只含偶次), 替换 $x^2=y$ 规模减半; n 奇偶决定取 $P \cdot Q(-x)$ 的奇/偶部; 最后 $P[0] \cdot \text{inv}(Q[0])$ 。实现依赖完整多项式库, 抄板整段复制。

```
// Bostan-Mori 核心 (依赖 07.5 NTT 与 07.15 poly_inv 等)
// P, Q 为 vector<int> (mod 998244353), P.size() < Q.size() 通常成立
i64 bostan_mori(vector<int> P, vector<int> Q, i64 n) {
  while (n) {
    vector<int> Qn(Q.size());
    for (int i = 0; i < (int)Q.size(); ++i)
      Qn[i] = (i & 1) ? (Q[i] ? NTT_MOD - Q[i] : 0) : Q[i];
    // Q(-x)
    vector<int> S = convolution ntt(P, Qn);
    vector<int> T = convolution ntt(Q, Qn);
    P.clear();
    for (int i = n & 1; i < (int)S.size(); i += 2) P.push_back(S[i]);
    // 奇偶部
    Q.clear();
    for (int i = 0; i < (int)T.size(); i += 2) Q.push_back(T[i]);
    ;
    n >>= 1;
  }
  // 此时 P, Q 都是常数项多项式, 答案 = P[0] * inv(Q[0])
  return P.empty() ? 0 : P[0] * mod_pow(Q[0], NTT_MOD - 2, NTT_MOD) % NTT_MOD;
}
```

07.17 三模 NTT (任意模数卷积)

【简介】

模数不是 NTT 友好素数 (如 $1e9+7$) 时的卷积。用三个 NTT 友好模数分别做 NTT 卷积, 再用 CRT 合并。EC-Final 2024 / 2025 的 D 题都考了, 必抄。

快速识别: $c[k] = \sum a[i]b[k-i]$ 且要求结果对任意模数取模 (尤其 $1e9+7$); FFT 精度不够、目标模数不是 998244353。

API 接入:

- `convolution_any(a, b, MOD)`: a, b 为 `vector<int>` (系数), 返回对 MOD 取模的卷积。
- 结果范围: 单系数和 $<$ 三模乘积 ($\sim 4.7e26$), $i64$ 系数和 ($n \leq 1e5, |a[i]| \leq 1e9$) 安全。
- 依赖: `mod_pow` (01.3)。

关键点: 三个模数 998244353 / 1004535809 / 469762049 原根都是 3, 最大支持长度 $2^{23} / 2^{21} / 2^{26}$; CRT 合并分两步 (先 $m1, m2$ 用 $i128$ 存中间, 再并入 $m3$); 每步取模用 $i128$ 防溢出; 要取任意模数时最后 $\% MOD$ 。

```
// 三模 NTT (任意模数卷积), 依赖 01.3 的 mod_pow
// 三个 NTT 友好模数, 原根均为 3
const i64 M1 = 998244353, M2 = 1004535809, M3 = 469762049;
void ntt_mod(vector<int>& a, bool invert, i64 MOD) {
  int n = (int)a.size();
```

```
  for (int i = 1, j = 0; i < n; ++i) {
    int bit = n >> 1;
    for (; j & bit; bit >>= 1) j ^= bit;
    j ^= bit;
    if (i < j) swap(a[i], a[j]);
  }
  for (int len = 2; len <= n; len <= 1) {
    i64 wlen = mod_pow(3, (MOD - 1) / len, MOD);
    if (invert) wlen = mod_pow(wlen, MOD - 2, MOD);
    for (int i = 0; i < n; i += len) {
      i64 w = 1;
      for (int j = 0; j < len / 2; ++j) {
        int u = a[i + j];
        int v = (int)(a[i + j + len / 2] * w % MOD);
        a[i + j] = (u + v) % MOD;
        a[i + j + len / 2] = (u - v + MOD) % MOD;
        w = w * wlen % MOD;
      }
    }
  }
  if (invert) {
    i64 inv_n = mod_pow(n, MOD - 2, MOD);
    for (int& x : a) x = (int)(x * inv_n % MOD);
  }
}
vector<int> conv_one(const vector<int>& a, const vector<int>& b, i64 MOD) {
  int need = (int)a.size() + (int)b.size() - 1, n = 1;
  while (n < need) n <= 1;
  vector<int> fa(n), fb(n);
  for (int i = 0; i < (int)a.size(); ++i) fa[i] = a[i];
  for (int i = 0; i < (int)b.size(); ++i) fb[i] = b[i];
  ntt_mod(fa, false, MOD); ntt_mod(fb, false, MOD);
  for (int i = 0; i < n; ++i) fa[i] = (int)((i128)fa[i] * fb[i] % MOD);
  ntt_mod(fa, true, MOD);
  fa.resize(need);
  return fa;
}
// 任意模数卷积: a, b 系数 (int), 返回对 MOD 取模的卷积
vector<int> convolution_any(const vector<int>& a, const vector<int>& b, i64 MOD) {
  if (a.empty() || b.empty()) return {};
  vector<int> r1 = conv_one(a, b, M1);
  vector<int> r2 = conv_one(a, b, M2);
  vector<int> r3 = conv_one(a, b, M3);
  // 预计算 CRT 系数 (inv 与模乘积)
  static i64 inv12 = mod_pow(M1 % M2, M2 - 2, M2); // inv (M1 mod M2)
  static i64 m12 = M1 * M2; // < 1e18, i128 保险
  static i64 inv123 = mod_pow(m12 % M3, M3 - 2, M3); // inv (m12 mod M3)
  int n = (int)r1.size();
  vector<int> res(n);
  for (int i = 0; i < n; ++i) {
    // 第一步: 合并 r1, r2 -> x12
    i64 t = (r2[i] - r1[i] % M2 + M2) % M2 * inv12 % M2;
    i128 x12 = (i128)r1[i] + (i128)t * M1; // < m12
    // 第二步: 并入 r3
    i64 x3 = (i64)(x12 % M3);
    i64 t2 = (r3[i] - x3 + M3) % M3 * inv123 % M3;
    i64 x = (i64)(x12 % MOD + (m12 % MOD) * t2 % MOD);
    res[i] = (int)(x % MOD);
  }
  return res;
}
// 用法: auto c = convolution_any(a, b, 1e9 + 7);
```

08 计算几何

08.1 点、向量与线段基础

【简介】

几何运算的原子操作: 叉积 (方向/面积)、点积 (投影/夹角)、线段相交、点到线段距离。一切几何题的底座。

快速识别: 任意几何题先确认会用这些原语: 判断左右/共线、求交点、面积、距离。

API 接入:

- `Point`: x, y (double 或 $i64$); 运算 + - * (数乘)、`dot` (点积)、`cross` (叉积)。

- `cross(o, a, b)`: 向量 $oa \times ob$, >0 逆时针、 <0 顺时针、 $=0$ 共线。
- `seg_intersect(a, b, c, d)`: 线段 ab 与 cd 是否相交 (含端点)。
- `point_to_seg_dist(p, a, b)`: 点到线段距离。
- `sgn(x)`: 符号函数 (浮点容差)。

关键点: 叉积符号决定方向; 浮点比较一律走 `sgn` (eps 容差); 线段相交 = 快速排斥 + 跨立实验; 整数坐标能用 `i64` 就用 `i64` (精确, 无精度问题), 需要除法/开方再转 `double`。

```
const double EPS = 1e-9;
int sgn(double x) { return x < -EPS ? -1 : x > EPS ? 1 : 0; }
struct Point {
    double x, y;
    Point(double x = 0, double y = 0) : x(x), y(y) {}
    Point operator+(const Point& o) const { return {x + o.x, y + o.y}; }
    Point operator-(const Point& o) const { return {x - o.x, y - o.y}; }
    Point operator*(double k) const { return {x * k, y * k}; }
    double dot(const Point& o) const { return x * o.x + y * o.y; }
    double cross(const Point& o) const { return x * o.y - y * o.x; }
};
double cross(const Point& o, const Point& a, const Point& b) { return
    (a - o).cross(b - o); }
double dot(const Point& o, const Point& a, const Point& b) { return
    (a - o).dot(b - o); }
bool on_seg(const Point& p, const Point& a, const Point& b) {
    return sgn(cross(a, b, p)) == 0 && sgn(dot(p, a, b)) <= 0;
}
// 线段相交 (含端点):
bool seg_intersect(const Point& a, const Point& b, const Point& c, const
    Point& d) {
    double d1 = cross(a, b, c), d2 = cross(a, b, d);
    double d3 = cross(c, d, a), d4 = cross(c, d, b);
    if (sgn(d1) * sgn(d2) < 0 && sgn(d3) * sgn(d4) < 0) return true;
    return on_seg(c, a, b) || on_seg(d, a, b) || on_seg(a, c, d) ||
on_seg(b, c, d);
}
// ===== 进阶函数 (08.1b): 交点 / 垂足 / 定圆 / 三角形心 =====
// 两直线交点 (直线 ab 与 cd, 要求不平行):
Point line_intersect(const Point& a, const Point& b, const Point& c,
    const Point& d) {
    Point ab = b - a, cd = d - c;
    double t = (c - a).cross(cd) / ab.cross(cd);
    return a + ab * t;
}
// 点到直线 (ab) 的垂足:
Point foot_of_perp(const Point& p, const Point& a, const Point& b) {
    Point ab = b - a;
    return a + ab * (ab.dot(p - a) / ab.dot(ab));
}
// 点到直线 (ab) 距离:
double point_to_line_dist(const Point& p, const Point& a, const Point&
    b) {
    return fabs(cross(a, b, p)) / sqrt((b - a).dot(b - a));
}
// 点到线段 (ab) 距离:
double point_to_seg_dist(const Point& p, const Point& a, const Point&
    b) {
    Point ab = b - a, ap = p - a;
    double t = ab.dot(ap) / ab.dot(ab);
    t = max(0.0, min(1.0, t));
    Point q = a + ab * t;
    return sqrt((p - q).dot(p - q));
}
// 三点定圆 (外心): 返回 (圆心, 半径); 三点共线时圆心为无穷 (需先判)
pair<Point, double> circumcircle(const Point& a, const Point& b, const
    Point& c) {
    double d = 2 * (a.x * (b.y - c.y) + b.x * (c.y - a.y) + c.x * (a
        .y - b.y));
    double ux = ((a.dot(a)) * (b.y - c.y) + (b.dot(b)) * (c.y - a.y)
        + (c.dot(c)) * (a.y - b.y)) / d;
    double uy = ((a.dot(a)) * (c.x - b.x) + (b.dot(b)) * (a.x - c.x)
        + (c.dot(c)) * (b.x - a.x)) / d;
    Point o(ux, uy);
    return {o, sqrt((a - o).dot(a - o))};
}
// 三角形外心 (同三点定圆)、内心 (角平分线交点):
Point incenter(const Point& a, const Point& b, const Point& c) {
    double la = sqrt((b - c).dot(b - c)), lb = sqrt((c - a).dot(c -
        a)), lc = sqrt((a - b).dot(a - b));
    return Point((la * a.x + lb * b.x + lc * c.x) / (la + lb + lc),
        (la * a.y + lb * b.y + lc * c.y) / (la + lb + lc));
}
```

```
// 三角形重心: 三点坐标平均; 垂心 = a + b + c - 2*外心
Point centroid(const Point& a, const Point& b, const Point& c) { ret
    urn (a + b + c) * (1.0 / 3); }
```

08.2 多边形面积与点包含

【简介】

多边形面积 (鞋带公式)、点是否在多边形内 (射线法/转角法)、多边形周长。

快速识别: 多边形面积/重心; 点在多边形内判定; 多边形的几何性质。

API 接入:

- `polygon_area(poly)`: 返回面积 (可负, 取绝对值); 顶点按顺序 (顺/逆时针均可)。
- `point_in_poly(p, poly)`: 射线法返回 true/false (边界处理可选)。
- `polygon_centroid(poly)`: 重心。
- 依赖: `Point` 基础 (08.1 节), 抄板时一起抄上。

关键点: 鞋带公式 $\sum (x[i]*y[i+1] - x[i+1]*y[i]) / 2$; 射线法注意点在边界上 (先判 `on_seg`); 顶点数 $>= 3$; 自交多边形面积公式不适用。

```
double polygon_area(const vector<Point>& p) {
    double s = 0;
    int n = (int)p.size();
    for (int i = 0; i < n; ++i) s += p[i].cross(p[(i + 1) % n]);
    return fabs(s) / 2;
}
bool point_in_poly(const Point& pt, const vector<Point>& p) {
    int n = (int)p.size();
    bool in = false;
    for (int i = 0, j = n - 1; i < n; j = i++) {
        if (on_seg(pt, p[i], p[j])) return true; // 在边界上
        if (((p[i].y > pt.y) != (p[j].y > pt.y)) &&
            pt.x < (p[j].x - p[i].x) * (pt.y - p[i].y) / (p[j].y - p
                [i].y) + p[i].x)
            in = !in;
    }
    return in;
}
```

08.3 凸包 (Andrew 单调链)

【简介】

点集的最小凸多边形: 按 x 排序后上下链分别维护, $O(n \log n)$ 。返回逆时针凸包顶点。

快速识别: “包围所有点的最小凸多边形”; 凸包上的点/边; 配合旋转卡壳 (08.4)。

API 接入:

- `convex_hull(pts)`: 返回凸包顶点 (逆时针, 不含共线点, 首尾不重复)。
- `convex_hull_with_collinear`: 含边界点的版本 (改 $\leq$ 为 $<$)。
- 凸包面积: 对凸包跑 08.2。
- 依赖: `Point` 基础 (08.1 节), 抄板时一起抄上。

关键点: 排序按 (x, y) ; 叉积 ≤ 0

弹出 (严格凸); 最后一段要回扫; 凸包点集可能退化 (< 3 点); `i64` 坐标可直接用叉积符号不用浮点。

```
vector<Point> convex_hull(vector<Point> p) {
    sort(all(p), [](const Point& a, const Point& b) {
        return a.x != b.x ? a.x < b.x : a.y < b.y;
    });
    p.erase(unique(all(p), [](const Point& a, const Point& b) {
        return a.x == b.x && a.y == b.y;
    }), p.end());
    int n = (int)p.size();
    if (n <= 1) return p;
    vector<Point> h(2 * n);
    int k = 0;
    for (int i = 0; i < n; ++i) {
        while (k >= 2 && sgn(cross(h[k - 2], h[k - 1], p[i])) <= 0)
            --k;
        h[k++] = p[i];
    }
    for (int i = n - 2, t = k + 1; i >= 0; --i) {
```

```

    while (k >= t && sgn(cross(h[k - 2], h[k - 1], p[i])) <= 0)
--k;
    h[k++] = p[i];
}
h.resize(k - 1);
return h;
}

```

08.4 旋转卡壳（凸包直径 / 宽度）

【简介】

凸包上双指针旋转求最远点对距离（直径）、最大三角形、凸多边形最小宽度。 $O(n)$ 。

快速识别：凸包直径（最远点对）；最大面积三角形；多边形宽度。

API 接入：

- `rotating_calipers(hull)`：返回直径平方（`i64/double`）。
- 双指针：对边 i ，旋转 j 直到叉积不再增大。
- 最大三角形：三层嵌套 or 双指针。
- 依赖：Point 基础（08.1 节）+ 凸包（08.3 节），抄板时一起抄上。

关键点：必须先求凸包；双指针 j

单调不回退；用叉积比较（三角形面积 =

$|\text{cross}|/2$ ）；直径端点至少有一对是对踵点。

```

double rotating_calipers_diameter(const vector<Point>& h) {
    int n = (int)h.size();
    if (n == 2) return (h[0] - h[1]).dot(h[0] - h[1]);
    int i = 1;
    double best = 0;
    for (int i = 0; i < n; ++i) {
        while (cross(h[i], h[(i + 1) % n], h[(i + 1) % n]) >
                cross(h[i], h[(i + 1) % n], h[i]))
            i = (i + 1) % n;
        best = max(best, (h[i] - h[i]).dot(h[i] - h[i]));
        best = max(best, (h[(i + 1) % n] - h[j]).dot(h[(i + 1) % n]
- h[j]));
    }
    return best;
}

```

08.5 半平面交

【简介】

n 个半平面（直线一侧）的交集（凸多边形/空/无界）。排序 + 双端队列， $O(n \log n)$ 。

快速识别：“满足所有线性约束的区域”；多边形核（能看到多边形所有边的区域）；可行域。

依赖：Point 与 `line_intersect`（08.1 节），抄板时一起抄上。

API 接入：

- `half_plane_intersection(lines)`：lines 为有向直线（左侧为保留侧）；返回凸包顶点（逆时针）；空集返回空。
- 每条线由两点定义（方向约定：左侧保留）。
- 有界性检查：交集可能无界，题目通常给边界约束。

关键点：按极角排序、去重平行线；队列弹头弹尾（新线与队首/尾交点判断）；结果退化（点/线段）时注意容差；方向约定必须全章统一（左侧保留）。

```

// 半平面交（求 n 个半平面的交集凸多边形；空集返回空）
// 有向直线：点 a 指向点 b，保留**左侧**；交点用 line intersect（08.1b）
struct HalfPlane {
    Point a, b; // 有向直线 a -> b，左侧保留
    double ang; // 极角
};
vector<Point> half_plane_intersection(vector<HalfPlane> hps) {
    for (auto& h : hps) h.ang = atan2(h.b.y - h.a.y, h.b.x - h.a.x);
    sort(all(hps), [](const HalfPlane& x, const HalfPlane& y) { return
x.ang < y.ang; });
    // 去重平行线：同极角保留最靠左的
    vector<HalfPlane> uniq;
    for (auto& h : hps) {
        if (!uniq.empty() && fabs(uniq.back().ang - h.ang) < EPS) {
            // 平行且方向相同：保留更靠左（用叉积判断），这里简单保留
        } else uniq.push_back(h);
    }
    hps = uniq;
    auto on_left = [](const HalfPlane& h, const Point& p) {

```

```

        return sgn(cross(h.a, h.b, p)) >= 0; // p 在直线左侧（含
边界）
    };
    deque<HalfPlane> dq;
    auto inter = [&](const HalfPlane& x, const HalfPlane& y) {
        return line_intersect(x.a, x.b, y.a, y.b);
    };
    for (auto& h : hps) {
        while (dq.size() >= 2 && !on_left(h, inter(dq[dq.size()-1],
dq[dq.size()-2]))) dq.pop_back();
        while (dq.size() >= 2 && !on_left(h, inter(dq[0], dq[1]))) d
q.pop_front();
        // 反向平行（方向相反）-> 交集为空
        if (dq.size() && sgn((h.b - h.a).cross(dq.back().b - dq.back
().a)) == 0 &&
            sgn((h.b - h.a).dot(dq.back().b - dq.back().a)) < 0) ret
urn {};
        dq.push_back(h);
        while (dq.size() >= 3 && !on_left(dq[0], inter(dq.back(), dq[dq.
size()-2]))) dq.pop_back();
        while (dq.size() >= 3 && !on_left(dq.back(), inter(dq[0], dq[1]
))) dq.pop_front();
        if (dq.size() < 3) return {};
        vector<Point> res;
        for (int i = 0; i < (int)dq.size(); ++i)
            res.push_back(inter(dq[i], dq[(i + 1) % dq.size()]));
        return res;
    }
    // 多边形内核（能看到多边形所有边的区域）= 每条边作为半平面求交

```

08.6 圆相关（交点 / 最小圆覆盖）

【简介】

两圆交点、直线与圆交点、点在圆内；最小圆覆盖（随机增量 $O(n)$ 期望）。

快速识别：圆与圆/直线交点；“覆盖所有点的最小圆”。

API 接入：

- `circle_intersect(c1, r1, c2, r2)`：返回交点数组（ $0/1/2$ 个）。
- `line_circle_intersect`：直线与圆。
- `min_enclosing_circle(pts)`：返回（圆心，半径）；随机化顺序防卡。
- 三点定圆：外心 = 中垂线交点。
- 依赖：Point 基础（08.1 节）+ `rng`（01.12 节），抄板时一起抄上。

关键点：最小圆覆盖随机打乱输入（否则最坏 $O(n^3)$ ）；判无交点用距离与半径和差比较（`eps`）；外心公式用叉积/点积，注意三点共线退化。

```

// 直线与圆交点（直线 ab，圆 (c, r)）：返回交点数组（0/1/2 个）
vector<Point> line_circle_intersect(const Point& a, const Point& b,
const Point& c, double r) {
    Point ab = b - a;
    Point f = foot of perp(c, a, b);
    double d2 = (f - c).dot(f - c);
    if (sgn(d2 - r * r) > 0) return {};
    double len = sqrt(max(0.0, r * r - d2));
    double L = sqrt(ab.dot(ab));
    Point dir = ab * (len / L);
    if (sgn(len) == 0) return {};
    return {f - dir, f + dir};
}
// 两圆交点（圆 (c1,r1) 与 (c2,r2)）：返回交点数组（0/1/2 个）
vector<Point> circle_intersect(const Point& c1, double r1, const Poi
nt& c2, double r2) {
    double d = sqrt((c1 - c2).dot(c1 - c2));
    if (sgn(d) == 0) return {}; // 同心圆
    if (sgn(d - r1 - r2) > 0 || sgn(d + fabs(r1 - r2)) < 0) return {
}; // 外离/内含
    double a = (r1 * r1 - r2 * r2 + d * d) / (2 * d);
    Point base = c1 + (c2 - c1) * (a / d);
    double h = sqrt(max(0.0, r1 * r1 - a * a));
    Point dir = Point(-(c2.y - c1.y), c2.x - c1.x) * (h / d);
    if (sgn(h) == 0) return {base};
    return {base - dir, base + dir};
}
// 两圆相交面积：
double circle_intersect_area(const Point& c1, double r1, const Point
& c2, double r2) {
    double d = sqrt((c1 - c2).dot(c1 - c2));
    if (sgn(d - r1 - r2) >= 0) return 0; // 相离/外切

```

```

    if (sgn(d - fabs(r1 - r2)) <= 0) { // 内含/内切:
小圆面积
        double mi = min(r1, r2);
        return acos(-1.0) * mi * mi;
    }
    double a1 = acos((r1 * r1 + d * d - r2 * r2) / (2 * r1 * d));
    double a2 = acos((r2 * r2 + d * d - r1 * r1) / (2 * r2 * d));
    return a1 * r1 * r1 + a2 * r2 * r2 - r1 * d * sin(a1);
}

// 最小圆覆盖 (随机增量)
pair<Point, double> min enclosing circle(vector<Point> p) {
    shuffle(all(p), rng);
    Point c = p[0];
    double r = 0;
    auto dist2 = [&](const Point& a, const Point& b) {
        return (a - b).dot(a - b);
    };
    for (int i = 1; i < (int)p.size(); ++i) {
        if (dist2(p[i], c) <= r * r + EPS) continue;
        c = p[i]; r = 0;
        for (int j = 0; j < i; ++j) {
            if (dist2(p[j], c) <= r * r + EPS) continue;
            c = (p[i] + p[j]) * 0.5;
            r = sqrt(dist2(p[i], p[j])) / 2;
            for (int k = 0; k < j; ++k) {
                if (dist2(p[k], c) <= r * r + EPS) continue;
                // 三点定圆 (外心), 覆盖 p[i], p[j], p[k]
                c = circumcircle(p[i], p[j], p[k]).first; // 定义
                r = sqrt(dist2(p[i], c));
            }
        }
    }
    return {c, r};
}

```

08.7 扫描线 (矩形面积并 / 周长)

【简介】

按 x 排序扫描, y 方向用线段树维护“被覆盖长度”, 矩形面积并 $O((n \log n))$ 。周长并、矩形交类似。

快速识别: 多个矩形的面积并/交; 区间覆盖长度 (一维是差分/线段树, 二维用扫描线)。

API 接入:

- `rectangle_area_union(rects)`: `rects = (x1, y1, x2, y2)` 列表 (闭/开区间约定统一); 返回面积。
- 离散化 y 坐标 + 线段树维护覆盖长度; 边事件 $(x, y1, y2, +1/-1)$ 排序。
- 面积交: 线段树维护覆盖次数 ≥ 2 的长度 (或容斥)。

关键点: y 离散化 (坐标压缩); 线段树节点存“覆盖次数 > 0 的长度”; 事件排序相同 x 先处理; 开区间统一 (半开区间 $[y1, y2)$ 避免端点重复计数); 结果用 $i64$ (坐标可为 int , 面积用 $i64$)。

```

// 矩形面积并 (扫描线 + 线段树, 离散化 y 坐标)
// 用法: rects 为 (x1, y1, x2, y2) (左下-右上); 坐标可到 1e9 (离散化)
// 返回面积 (i64, 坐标 int 时)
struct SegCover { // 线段树: 维护被覆盖长度
    int n;
    vector<int> cnt; // 覆盖次数
    vector<i64> len; // 被覆盖长度 (离散化区间长度)
    vector<i64> ys; // 离散化 y (升序)
    void init(const vector<i64>& ys) {
        ys = ys;
        n = (int)ys.size() - 1; // 区间数 = 点数 - 1
        cnt.assign(4 * n + 4, 0);
        len.assign(4 * n + 4, 0);
    }
    void update(int p, int l, int r, int ql, int qr, int delta) {
        if (qr < l || r < ql) return;
        if (ql <= l && r <= qr) { cnt[p] += delta; }
        else {
            int m = (l + r) / 2;
            update(p * 2, l, m, ql, qr, delta);
            update(p * 2 + 1, m + 1, r, ql, qr, delta);
        }
        if (cnt[p] > 0) len[p] = ys[r + 1] - ys[l]; // 整段覆盖
        else len[p] = (l == r) ? 0 : len[p * 2] + len[p * 2 + 1];
    }
}

```

```

void add(int ql, int qr, int delta) { if (ql <= qr) update(1, 0,
n - 1, ql, qr, delta); }
i64 cover_len() const { return len[1]; }
};
i64 rectangle_area_union(const vector<array<i64, 4>>& rects) {
    struct Ev { i64 x, y1, y2; int type; }; // type: +1 进入, -1 离开
    vector<Ev> evs;
    vector<i64> ys;
    for (auto [x1, y1, x2, y2] : rects) {
        evs.push_back({x1, y1, y2, 1});
        evs.push_back({x2, y1, y2, -1});
        ys.push_back(y1); ys.push_back(y2);
    }
    sort(all(ys)); ys.erase(unique(all(ys)), ys.end());
    sort(all(evs), [](const Ev& a, const Ev& b) { return a.x < b.x; });
    auto id = [&](i64 y) { return int(lower_bound(all(ys), y) - ys.begin()); };
    SegCover seg;
    seg.init(ys);
    i64 ans = 0, last x = evs[0].x;
    for (auto& e : evs) {
        ans += seg.cover_len() * (e.x - last x);
        seg.add(id(e.y1), id(e.y2) - 1, e.type); // 半开区间 [y1, y2)
        last x = e.x;
    }
    return ans;
}
// 面积交: 线段树维护覆盖次数 >= 2 的长度 (节点存 cov1/cov2 两档)

```

08.8 KD-Tree (二维动态加点查询)

【简介】

交替按 x/y 中位数划分的空间树; 支持加点、矩形/圆范围查询 (最近点、矩形权值和)。期望 $O(\sqrt{n})$ 查询。

快速识别: 二维点集动态加点 +

矩形/圆形范围统计; 最近点对动态版; 点不太多 ($1e5$ 内)。

API 接入:

- KDTree: `insert(pt, val)`; `query_rect(x1,y1,x2,y2)` 矩形内权值和; `nearest(p)` 最近点。
- 构建版: `build(pts)` 离线建树。
- 剪枝: 子树边界矩形与查询无交则剪。

关键点: 节点存 `mnx/mny/mxx/mxy` (子树包围盒); 分割维交替 (或按方差选); 插入不旋转会退化——最坏 $O(n)$ 链, 必要时离线 `build`; 范围查询剪枝靠包围盒。

```

// KD-Tree (离线构建版): 二维点权值查询 (矩形和 / 最近点)
// 用法: KDTree kd(pts, vals); kd.query_rect(x1,y1,x2,y2); kd.nearest(p);
struct KDTree {
    struct Node { i64 x, y, v, sum; int lc = 0, rc = 0; i64 mnx, mny, mxx, mxy; };
    vector<Node> tr;
    int n;
    struct P { i64 x, y, v; };
    vector<P> pts;
    KDTree(const vector<P>& p) : pts(p) {
        n = (int)p.size();
        tr.assign(4 * n + 4, Node{});
        for (auto& t : tr) { t.mnx = t.mny = LINF; t.mxx = t.mxy = -LINF; } // 空盒
        build(1, 0, n - 1, 0);
    }
    void pull(int p) {
        tr[p].sum = tr[p].v + tr[tr[p].lc].sum + tr[tr[p].rc].sum;
        tr[p].mnx = min({tr[p].x, tr[tr[p].lc].mnx, tr[tr[p].rc].mnx});
        tr[p].mny = min({tr[p].y, tr[tr[p].lc].mny, tr[tr[p].rc].mny});
        tr[p].mxx = max({tr[p].x, tr[tr[p].lc].mxx, tr[tr[p].rc].mxx});
        tr[p].mxy = max({tr[p].y, tr[tr[p].lc].mxy, tr[tr[p].rc].mxy});
    }
    void build(int p, int l, int r, int dim) {
        if (l > r) return;
        int m = (l + r) / 2;
        // 按当前维中位数划分 (nth element)
        nth_element(pts.begin() + l, pts.begin() + m, pts.begin() + r + 1,
            [dim](const P& a, const P& b) { return dim == 0 ? a.x < b.x : a.y < b.y; });
        tr[p].x = pts[m].x; tr[p].y = pts[m].y; tr[p].v = pts[m].v;
    }
}

```



```

    build(p * 2, l, m - 1, dim ^ 1);
    build(p * 2 + 1, m + 1, r, dim ^ 1);
    // 叶子边界
    if (l == r) tr[p].mnx = tr[p].mxx = tr[p].x, tr[p].mny = tr[p].mxy = tr[p].y;
    pull(p);
}
// 矩形 [x1, x2]x[y1, y2] 内权值和
i64 query_rect(int p, i64 x1, i64 y1, i64 x2, i64 y2) {
    if (!p) return 0;
    if (tr[p].mxx < x1 || tr[p].mnx > x2 || tr[p].mxy < y1 || tr[p].mny > y2)
        return 0; // 与矩形无交
    if (x1 <= tr[p].mnx && tr[p].mxx <= x2 && y1 <= tr[p].mny && tr[p].mxy <= y2)
        return tr[p].sum; // 完全包含
    i64 res = 0;
    if (x1 <= tr[p].x && tr[p].x <= x2 && y1 <= tr[p].y && tr[p].y <= y2)
        res += tr[p].v;
    return res + query_rect(tr[p].lc, x1, y1, x2, y2) + query_rect(tr[p].rc, x1, y1, x2, y2);
}
i64 query_rect(i64 x1, i64 y1, i64 x2, i64 y2) { return query_rect(1, x1, y1, x2, y2); }
// 最近点距离平方 (用包围盒距离剪枝): 空树返回 LINF
i64 nearest(int p, i64 x, i64 y, i64& best) {
    if (!p) return best;
    i64 d = (tr[p].x - x) * (tr[p].x - x) + (tr[p].y - y) * (tr[p].y - y);
    best = min(best, d);
    i64 d1 = box_dist(tr[tr[p].lc], x, y), d2 = box_dist(tr[tr[p].rc], x, y);
    if (d1 < d2) {
        if (d1 < best) nearest(tr[tr[p].lc], x, y, best);
        if (d2 < best) nearest(tr[tr[p].rc], x, y, best);
    } else {
        if (d2 < best) nearest(tr[tr[p].rc], x, y, best);
        if (d1 < best) nearest(tr[tr[p].lc], x, y, best);
    }
    return best;
}
i64 nearest(i64 x, i64 y) { i64 best = LINF; return nearest(1, x, y, best); }
static i64 box_dist(const Node& t, i64 x, i64 y) { // 点到子树包围盒的最短距离平方
    if (t.mxx < t.mnx || t.mxy < t.mny) return LINF; // 空盒 (未 build 的节点)
    i64 dx = 0, dy = 0;
    if (x < t.mnx) dx = t.mnx - x;
    else if (x > t.mxx) dx = x - t.mxx;
    if (y < t.mny) dy = t.mny - y;
    else if (y > t.mxy) dy = y - t.mxy;
    return dx * dx + dy * dy;
}
};
// 动态加点版: 不旋转, 最坏 O(n); 点数 1e5 内仍可用, 必要时定期重建

```

08.9 计算几何杂项速查

【简介】

常用几何结论与技巧: Pick 定理、曼哈顿距离变换、向量旋转、点在凸多边形内、闵可夫斯基和 (凸多边形相撞判定)。

快速识别: 格点多边形内部点数 (Pick); 曼哈顿距离最值 (变换坐标); 凸多边形相交/包含。

API 接入:

- Pick: $S = I + B/2 - 1$ (S 面积、 I 内部整点、 B 边界整点, 均为整数)。
- 曼哈顿距离: $|x|+|y|$ 变换为 $\max(|x+y|, |x-y|)$ 方向——最值 = 4 个方向上的 $\max\text{-}\min$ 。
- 向量旋转 θ : $(x \cos\theta - y \sin\theta, x \sin\theta + y \cos\theta)$ 。
- 闵可夫斯基和: $A+B = \{a+b\}$; 两凸多边形相交当且仅当原点在 A 与 $(-B)$ 的和内。

关键点: Pick 定理只适用于格点多边形; 曼哈顿转切比雪夫: $(x+y, x-y)$; 闵可夫斯基和 = 把两个凸包按极角归并 ($O(n+m)$)。

```

// 曼哈顿距离最值: 对每个点算 4 个方向值, 各取 max-min
// d1 = x + y, d2 = x - y, d3 = -x + y, d4 = -x - y
// 最大曼哈顿距离 = max(max(d1)-min(d1), max(d2)-min(d2), ...)
// 向量旋转 90 度: (-y, x); 180: (-x, -y); 270: (y, -x)

```

08.10 立体几何基础 (3D 点 / 向量 / 平面 / 体积)

【简介】

三维空间基础运算: 点/向量、叉积 (法向量)、点积 (夹角)、点到平面距离、四面体体积、直线与平面交点。WIDA 3D 几何核心子集。

快速识别: 三维几何题 (空间点线面关系、体积、距离); n 维推广的向量技巧。

API 接入:

- Point3: x, y, z ; dot (点积)、cross (叉积 = 法向量)、len()、normalize()。
- 平面: $ax + by + cz + d = 0$ 或 (法向量 n , 过点 p): point_plane_dist(p, n, q)。
- 四面体体积: $|(b-a) \cdot ((c-a) \times (d-a))| / 6$ (混合积)。
- 直线与平面交点: 参数 t 代入。
- 空间两点距离、球相关同理推广。

关键点: 叉积顺序决定法向量方向 (右手系); 混合积 = 平行六面体体积; 四点共面 混合积 = 0; 点到平面距离带符号 (法向量单位化); 浮点 eps 处理同 2D。

依赖: EPS/sgn (08.1 节), 抄板时一起抄上。

```

struct Point3 {
    double x, y, z;
    Point3(double x_ = 0, double y_ = 0, double z_ = 0) : x(x_), y(y_), z(z_) {}
    Point3 operator+(const Point3& o) const { return {x + o.x, y + o.y, z + o.z}; }
    Point3 operator-(const Point3& o) const { return {x - o.x, y - o.y, z - o.z}; }
    Point3 operator*(double k) const { return {x * k, y * k, z * k}; }

    double dot(const Point3& o) const { return x * o.x + y * o.y + z * o.z; }
    Point3 cross(const Point3& o) const { // 叉积 (法向量)
        return {y * o.z - z * o.y, z * o.x - x * o.z, x * o.y - y * o.x};
    }
    double len() const { return sqrt(dot(*this)); }
    Point3 normalize() const { double l = len(); return l > EPS ? *this * (1.0 / l) : *this; }
};
// 四面体体积 (a,b,c,d 四个顶点): |混合积| / 6
double tetra_volume(const Point3& a, const Point3& b, const Point3& c, const Point3& d) {
    return fabs((b - a).dot((c - a).cross(d - a))) / 6.0;
}
// 点到平面距离: 平面过点 q、法向量 n (已归一化时) 或未归一化除 |n|
double point_plane_dist(const Point3& p, const Point3& q, const Point3& n) {
    return fabs((p - q).dot(n)) / n.len();
}
// 直线 (a, 方向 dir) 与平面 (过 q, 法向量 n) 的交点 (平行返回 false)
bool line_plane_intersect(const Point3& a, const Point3& dir, const Point3& q, const Point3& n, Point3& res) {
    double denom = dir.dot(n);
    if (fabs(denom) < EPS) return false; // 平行
    double t = (q - a).dot(n) / denom;
    res = a + dir * t;
    return true;
}
// 空间两点距离: sqrt((a-b).dot(a-b)); 球与球交点 / 球与平面交线类推

```

09 动态规划

09.1 背包家族 (0/1 / 完全 / 多重 / 分组)

【简介】

容量维 DP: 0/1 (每件选或不选)、完全 (不限次)、多重 (限次, 二进制拆分或单调队列)、分组 (每组选一个)。核心差异 = 容量循环方向。

快速识别: “选物品凑容量最大价值/方案数”; 每件可选 0/1 次 $\rightarrow$ 0/1; 不限次 $\rightarrow$ 完全; 每种限 c 件 $\rightarrow$ 多重; 物品分组每组选一 $\rightarrow$

分组。

API 接入：

- 0/1: for i: for (j = V; j >= w[i]; --j) dp[j] = max(dp[j], dp[j - w[i]] + v[i]); (容量倒序)。
- 完全：容量正序 (可重复选)。
- 多重：二进制拆分 (每件拆成 1, 2, 4, ... 个的打包) 后跑 0/1; 或用单调队列 $O(nV)$ 。
- 分组: for 组: for (j = V; j >= 0; --j) for 组内物品: ... (组内正序、组间倒序)。
- 恰好装满: dp 初始化 $-INF$ (只 dp[0]=0); 方案数: dp[j] += dp[j-w]。

关键点: 0/1 倒序、完全正序是最高频错误点; 多重二进制拆分用 i64 防 $1 < k$ 溢出; “恰好容量”与“不超过容量”初始化不同; 背包方案数在取模下做。

```
// 0/1 背包 (恰好容量版: dp 初始 -LINF)
vector<i64> knap01(const vector<i64>& w, const vector<i64>& v, int V) {
    vector<i64> dp(V + 1, -LINF);
    dp[0] = 0;
    for (int i = 0; i < (int)w.size(); ++i)
        for (int j = V; j >= w[i]; --j)
            dp[j] = max(dp[j], dp[j - w[i]] + v[i]);
    return dp;
}
// 多重背包 (二进制拆分):
// 数量 c 拆成 1, 2, 4, ..., rem 个一组, 每组当一件 0/1 物品
```

09.2 LIS / 最长上升子序列

【简介】

$O(n \log n)$ 求最长 (严格) 上升子序列长度; 可输出方案; LIS 计数、带权 LIS。

快速识别: 最长递增/递减子序列; LIS 计数; 二维偏序的最长链 (先排序一维)。

API 接入:

- lis_length(a): 返回严格 LIS 长度 (lower_bound); 非严格用 upper_bound。
- lis_with_path(a): 返回方案 (下标)。
- 带权 LIS: 值域 BIT 维护 dp = max(dp + w)。
- 最长链 (套娃/信封): 按一维排序后另一维 LIS。

关键点: 贪心数组 d 只存长度不存方案; lower_bound (严格) / upper_bound (非严格); 输出方案要记录每长度末尾位置再回溯; 值域大离散化。

```
int lis_length(const vector<i64>& a) {
    vector<i64> d;
    for (i64 x : a) {
        auto it = lower_bound(all(d), x); // 严格递增: 非严格用 upper_bound
        if (it == d.end()) d.push_back(x);
        else *it = x;
    }
    return (int)d.size();
}
```

09.3 区间 DP

【简介】

dp[l][r] 合并/分割区间的最优值, 按长度从小到大枚举。石子合并、矩阵链乘、括号序列、回文分割。

快速识别: “区间合并/拆分的最小/最大代价”; 石子合并; 括号匹配计数; 回文分割。

API 接入:

- 骨架: for len: for l: r = l+len-1; dp[l][r] = min_k(dp[l][k] + dp[k+1][r] + cost(l,r))。
- 石子合并 (环形): 数组复制一份处理环。
- 括号序列计数: dp[l][r] = dp[l+1][r] + dp[l][k]*dp[k+1][r] (匹配位置拆分)。
- 回文最少分割: dp[i] = min(dp[j] + 1) if s[j+1..i] 是回文 (预处理回文矩阵)。

- 优化: Knuth (满足四边形不等式时 $O(n^2)$)、平行四边形。

关键点: len 从小到大 (依赖更短区间); 环形处理 = 长度 $2n$ 后取 max; 初始化 dp[i][i]; cost 函数要能 $O(1)$ (前缀和)。

```
// 石子合并 (线性, 最小代价): 前缀和 pre
// for (len = 2; len <= n; ++len)
//     for (l = 1; l + len - 1 <= n; ++l) {
//         r = l + len - 1;
//         dp[l][r] = min over k: dp[l][k] + dp[k+1][r] + (pre[r]-pre[l-1]);
//     }
// 环形: 复制为 2n 长度, 答案 = min over l: dp[l][l+n-1]
```

09.4 状压 DP (TSP 骨架)

【简介】

状态用二进制表示子集: dp[mask][i] = 已访问 mask、当前在 i 的最优值。TSP、Hamilton 路径、分配问题。

快速识别: n <= 20 的选点/顺序问题; 子集状态; 分配/覆盖。

API 接入:

- TSP: dp[mask][i] = min(dp[mask ^ (1<<i)][j] + w[j][i]); 答案 = min over i dp[full][i] + w[i][0]。
- 枚举子集转移: sub = (mask-1) & mask 遍历。
- 初始化: dp[1<<i][i] = 0。

关键点: 状态从 1 开始 (空集哨兵); $1 < i$ 在 $i \geq 31$ 溢出用 1LL; 复杂度 $O(2^n n^2)$, n <= $18 \sim 20$; 用 __builtin_ctz/popcount 优化枚举。

```
// TSP n <= 18 0-based
// vector<vector<i64>> dp(1 << n, vector<i64>(n, LINF));
// for (int i = 0; i < n; ++i) dp[1 << i][i] = 0;
// for (int mask = 1; mask < (1 << n); ++mask)
//     for (int i = 0; i < n; ++i) if (dp[mask][i] < LINF)
//         for (int j = 0; j < n; ++j) if (!(mask >> j & 1))
//             dp[mask | (1 << j)][j] = min(dp[mask | (1 << j)][j], dp[mask][i] + dist[i][j]);
// ans = min_i dp[(1<<n)-1][i] + dist[i][0];
```

09.5 数位 DP (骨架)

【简介】

按位处理数字上限的计数: dp[pos][state][tight]。统计 [L, R] 内满足数位条件的数 (不含某数字、数位和限制、回文数位等)。

快速识别: “区间内多少个满足数位条件”; 数字 1e18 级; 逐位决策。

API 接入:

- 骨架: dfs(pos, state, tight, lead); 记忆化 dp[pos][state][tight] (lead = 前导零)。
- 答案 = solve(R) - solve(L-1)。
- 状态按题意 (出现次数、数位和、相邻关系)。

关键点: tight 维不可省 (受上限约束时不能直接记忆化); lead (前导零) 影响条件判断 (如 0

是否算出现); 数位数组从高位到低位; 记忆化前确认 tight=0 与 lead=0 的状态才可复用; 范围大用字符串读入 + 数位分解。

```
// 数位 DP 骨架: 统计 [0, x] 内不含数字 4 的个数
i64 dp[20][2];
i64 dfs(const string& s, int pos, bool tight) {
    if (pos == (int)s.size()) return 1;
    if (!tight && dp[pos][tight] != -1) return dp[pos][tight];
    int lim = tight ? s[pos] - '0' : 9;
    i64 res = 0;
    for (int d = 0; d <= lim; ++d) {
        if (d == 4) continue;
        res += dfs(s, pos + 1, tight && d == lim);
    }
    if (!tight) dp[pos][tight] = res;
    return res;
}
i64 count_ok(i64 x) {
    if (x < 0) return 0;
    memset(dp, -1, sizeof(dp));
    return dfs(to_string(x), 0, true);
}
// ans = count_ok(R) - count_ok(L - 1)
```

09.6 概率与期望 DP

【简介】

期望 DP 套路: $E = \sum p_i * (E_{next} + cost)$; 含自环的用移项公式; 树上随机游走用树 DP + 高斯消元 (环多时)。

快速识别: “期望步数/次数”; 随机过程建模; 状态转移图 (DAG 直接 DP, 有环高斯消元)。

API 接入:

- DAG 期望: 拓扑序 DP 或记忆化。
- 自环: $E = p*(E+1) + (1-p)*(E_{next}+1) \rightarrow E = (1 + (1-p)*E_{next})/p$ 。
- 方程组: 多个状态的期望相互依赖 $\rightarrow$ 高斯消元。
- 树上随机游走: 每个点 E 由父/子表示, 两遍 DFS 消元 (经典 $O(n)$ 技巧)。

关键点: **期望 = $\sum p * (\text{步长} + \text{后续期望})$ **；自环必须移项 (不能直接记忆化死循环)；概率题先列转移再定 DP 方向；浮点输出 setprecision; 大期望用 long double 防误差。

```
// 含自环期望: 状态 i 有 p 概率自环、q 概率去 j (p+q=1):
// E[i] = p*(E[i]+1) + q*(E[j]+1)
// => E[i] = (1 + q*(E[j]-E[i])) / p
// 通用做法: E[i] = 1 +  $\sum$  (转移) prob * E[to], 含自环的项移到左边, 联立高斯消元
```

09.7 DP 优化: 单调队列

【简介】

转移 $dp[i] = \max/\min_{j \in [i-k, i-1]} (dp[j] + f(i,j))$, f 可拆分成只依赖 j 和只依赖 i 时用单调队列 $O(n)$ 。

快速识别: 转移窗口是连续区间 (滑窗最值); “长度不超过 k 的段”型 DP。

API 接入:

- 通用: $dp[i] = \text{query_max}(\text{窗口内 } dp[j] + g(j)) + h(i)$; 单调队列存 j。
- 窗口边界: $j \geq i - k$ 。
- 二维转一维: 先枚举行/列。

关键点: 队列里存下标, 按 $dp[j] + g(j)$

单调; 先弹过期再取队首; 与 02.15 滑动窗口同结构。

```
// 例: dp[i] = max {i-k <= j < i} (dp[j] + a[i]) -> a[i] 提出
// deque 维护 j, 按 dp[j] 单调递减:
// for i: while (dq.front() < i - k) pop front;
// dp[i] = (dq 空 ? 0 : dp[dq.front()]) + a[i];
// while (dq 非空 && dp[dq.back()] <= dp[i]) pop back;
// dq.push_back(i);
```

09.8 DP 优化: 斜率优化 / Li Chao

【简介】

转移形如 $dp[i] = \min(k_{ij} * x_i + b_{ij})$ (直线求值): 斜率/查询单调用凸包队列 $O(n)$; 不单调用 Li Chao 树 $O(n \log X)$ 。

快速识别: 转移展开后是“一堆直线取最值”; $dp[j] + \text{交叉项}$ (如 $dp[j] + (x_i - x_j)^2$)。

API 接入:

- 凸包版: add_line(k, b) 要求斜率单调; query(x) 要求 x 单调 (HullMin)。
- Li Chao: LiChao lc(XL, XR) 任意顺序; add_line; query(x)。
- 转化: 把 DP 式展开成 $y = kx + b$ 形式, k、x 分别是谁要想清楚。

关键点: 先展开成标准式再决定用哪个; 凸包版交叉相乘用 i128 防溢出; Li Chao 要知道 x 值域; 维护最小值/最大值各一套比较方向。

```
// 斜率优化凸包版 (HullMin): 斜率单调递增 + 查询 x
单调递增时用,  $O(n)$  均摊 // 用法: HullMin h; h.add_line(k, b); i64 best = h.query(x); struct HullLine { i64 k, b; i64
```

```
eval(i64 x) const { return k * x + b; } }; // 判断 b
是否冗余 (下凸包, 斜率递增): 交叉相乘用 i128 防溢出 bool
hull_bad(const HullLine& a, const HullLine& b, const
HullLine& c) { return (i128)(b.b - a.b) * (b.k - c.k) >=
(i128)(c.b - b.b) * (a.k - b.k); } struct HullMin {
deque<HullLine> q; void add_line(i64 k, i64 b) { // 要求 k
单调不减 HullLine cur{k, b}; while (q.size() >= 2 &&
hull_bad(q[q.size() - 2], q.back(), cur)) q.pop_back();
q.push_back(cur); } i64 query(i64 x) { // 要求 x 单调不减
while (q.size() >= 2 && q[0].eval(x) >= q[1].eval(x))
q.pop_front(); return q.front().eval(x); } }; //
求最大值: hull_bad 换 <=, query 的 >= 换 <= (镜像)
```

```
// Li Chao Tree (最小值, 整数值域 [XL, XR])
struct Line { i64 k, b; i64 eval(i64 x) const { return k * x + b; }
};
struct LiChao {
i64 XL, XR;
struct Node { Line l; int lc = 0, rc = 0; bool has = false; };
vector<Node> tr{Node{}};
LiChao(i64 l, i64 r) : XL(l), XR(r) {}
int new_node() { tr.push_back(Node{}); return (int)tr.size() - 1
};
void insert(int p, i64 l, i64 r, Line nw) {
if (!tr[p].has) { tr[p].l = nw; tr[p].has = true; return; }
i64 m = (l + r) / 2;
Line lo = tr[p].l, hi = nw;
if (lo.eval(m) > hi.eval(m)) swap(lo, hi);
tr[p].l = lo;
if (l == r) return;
if (lo.eval(l) > hi.eval(l)) {
if (!tr[p].lc) tr[p].lc = new_node();
insert(tr[p].lc, l, m, hi);
} else if (lo.eval(r) > hi.eval(r)) {
if (!tr[p].rc) tr[p].rc = new_node();
insert(tr[p].rc, m + 1, r, hi);
}
}
void add_line(i64 k, i64 b) { insert(1, XL, XR, {k, b}); }
i64 query(int p, i64 l, i64 r, i64 x) {
i64 res = tr[p].has ? tr[p].l.eval(x) : LINf;
if (l == r) return res;
i64 m = (l + r) / 2;
if (x <= m && tr[p].lc) res = min(res, query(tr[p].lc, l, m,
x));
if (x > m && tr[p].rc) res = min(res, query(tr[p].rc, m + 1,
r, x));
return res;
}
i64 query(i64 x) { return query(1, XL, XR, x); }
};
```

09.9 DP 优化: 决策单调性 (分治优化 / Knuth)

【简介】

$dp[i] = \min_{j < i} (dp[j] + cost(j+1, i))$ 且最优决策点随 i 单调不减 $\rightarrow$ 分治优化 $O(n \log n)$; 区间 DP 满足四边形不等式 $\rightarrow$ Knuth $O(n^2)$ 。

快速识别: 区间/分段 DP 且决策点单调 (打表验证); cost 满足四边形不等式 (如 $cost = (\text{sum})^2$, $cost = |a-b|$)。

API 接入:

- 分治优化: solve(l, r, optL, optR) 递归, 每次算 mid 的最优决策并缩小范围。
- Knuth: $dp[l][r] = \min(dp[l][k] + dp[k][r] + cost)$ 且 $opt[l][r-1] \leq opt[l][r] \leq opt[l+1][r]$ 。
- 验证单调性: 暴力打小表确认决策点单调再上优化。

关键点: 必须先验证决策单调性 (不是所有 DP 都满足); 分治优化的 cost 计算要支持快速获取; Knuth 的决策范围限定 $[opt[l][r-1], opt[l+1][r]]$; 四边形不等式条件: $cost(a,c) + cost(b,d) \leq cost(a,d) + cost(b,c)$ ($a \leq b \leq c \leq d$)。

```
// 分治优化框架:
// void solve(int l, int r, int optL, int optR) {
// if (l > r) return;
// int mid = (l + r) / 2;
// 遍历 j in [optL, min(optR, mid-1)] 找 dp[mid] 最优决策 opt
// solve(l, mid - 1, optL, opt);
```

```
// solve(mid + 1, r, opt, optr);
// }
```

09.10 DP 优化: WQS 二分 / Slope Trick

【简介】

WQS (带权二分): “恰好选 k 个”的最优化, 把限制改造成每选一个扣 c 的罚项, 二分 c 逼近 k ; Slope Trick: 凸分段线性函数 DP 用堆维护斜率变化点, $O(n \log n)$ 。

快速识别: WQS: 恰好 k 个 + 无 k 限制时好算; Slope Trick: 转移是 $|x-a|$ 类绝对值/取 $\max/\min$ 的凸 DP。

API 接入:

- WQS: $f(c)$ = 最优值 - $c * k'$ (k' = 实际选的个数); 二分 c 使 k' 逼近 k ; 答案 = $f(c) + c * k$ 。要求 $f(c)$ 关于 c 单调且最优选数是 c 的单调函数 (凸性)。
- Slope Trick: 维护堆 (大根堆存左半斜率点、小根堆存右半); $|x-a|$ 平移; 取 $\max$ 用两个堆。

关键点: WQS 要求凸性 (最优选数随 c 单调); 二分的是“惩罚系数”; 计数要同时维护选数 (dp 值 + 选数一起存); Slope Trick 只适用凸函数, 改板时先确认凸性; 堆里存的是斜率变化的 x 坐标。

```
// WQS 二分 (带权二分) 骨架:
// 问题: 恰好选 k 个的最优化 (无 k 限制时好算)
// 做法: 二分惩罚 c (每选一个额外扣 c), 跑普通 DP 并记录实际选数 cnt
// if (cnt >= k) 答案可行, 提高 c (减少选择); else 降低 c
// 最终答案 = dp 最优值 + c * k (注意: 多个 c 对应同一 k 时取首个 cnt >= k 的)
// 前提: 最优选数随 c 单调 (凸性), DP 同时维护 (最优值, 选数)

// Slope Trick: 维护凸分段线性函数 (一阶差分单调), 用两个堆存斜率变化点
// 经典应用: dp[i] = min(dp[i-1], ...) + |x - a| 类;
// 例: 使数组变非降的最小操作数 (每次可 +1/-1)
// 用法: 对每个 a[i] 执行 push(a[i]); 答案 = 大根堆顶 (见下)
struct SlopeTrick {
    priority_queue<i64> L; // 左半斜率变化点 (大根堆)
    priority_queue<i64, vector<i64>, greater<i64>> R; // 右半 (小根堆)

    i64 min f = 0; // 函数最小值
    // 加 |x - a|: 插入 a, 更新最小值
    void add_abs(i64 a) {
        if (!L.empty() && a < L.top()) {
            min f += L.top() - a; // 最小值上升
            R.push(L.top()); L.pop();
            L.push(a);
        } else {
            L.push(a);
        }
        if (!R.empty() && a > R.top()) {
            min f += a - R.top();
            L.push(R.top()); R.pop();
            R.push(a);
        } else {
            R.push(a);
        }
        // 平衡: 保证 L.size() 与 R.size() 差 <= 1 (多数模板只维护 L)
        while ((int)L.size() > (int)R.size() + 1) { R.push(L.top());
            L.pop(); }
        while ((int)R.size() > (int)L.size() + 1) { L.push(R.top());
            R.pop(); }
    }
    i64 get_min() const { return min f; }
};

// 例: 变非降 (可用 L 维护): 对每个 a: L.push(a); if (L.top() > a) { ans
+= L.top() - a; L.pop(); L.push(a); }
// 取 max (非增) 镜像。Slope Trick 只适用凸函数, 用前确认凸性。
```

09.11 轮廓线 DP / 插头 DP (骨牌覆盖)

【简介】

逐格推进的状压 DP: 状态 = 当前轮廓线上的覆盖情况 (如每列是否被横放骨牌占用)。经典: $n \times m$ 铺 $1 \times 2/1 \times 3$ 骨牌方案数。

快速识别: 铺砖/覆盖计数 (小 n 、大 m : 把短边压成状态); 棋盘覆盖方案数。

API 接入:

- 状态: $dp[row][mask]$ 或逐格 $dp[mask]$: $mask$ 表示当前行/轮廓被“从上一行伸下来的”骨牌占用的列。
- 转移: 对每个空格枚举放法 (不放/横放/竖放) 更新 $mask$ 。
- 答案: $dp[n][0]$ (最后没有伸出)。

关键点: 竖放骨牌在状态里表现为“下一行对应位被占用”; $mask$ 位表示该格已被占用; 复杂度 $O(n * 2^m * \text{转移数})$, m 取 $\min(n, m)$; 障碍格要跳过并清对应位。

```
// 铺 1x2 骨牌 (n 行 m 列, m 小): dp[col][mask] 或逐格
// 对当前格 (i, j):
// 若 mask 位 j 为 1: 已被占用, 直接推进
// 否则: 横放 (j+1 位置 1) 或竖放 (下一行同列置位)
// 答案 = dp[结束][0]
```

09.12 DP 技巧速查 (滚动数组 / SOS DP / 子集卷积)

【简介】

DP 常用加速技巧: 滚动数组 (空间降维)、SOS DP (子集/超集前缀和 $O(n * 2^n)$)、子集卷积 (快速子集卷积, 配合 FWT)、计数技巧 (差值转正、map 压缩状态)。

快速识别: 空间不够用滚动; 子集求和类 (每个 $mask$ 的所有子集和); “恰好/至少”类计数。

API 接入:

- SOS: for i: for mask: if $(mask \gg i \& 1)$ $f[mask] += f[mask \wedge (1 \ll i)]$ (子集和)。
- 超集和: 反过来 $f[mask] += f[mask | (1 \ll i)]$ 。
- 差值转正: $a[i] - b[i]$ 加偏移量做下标。
- 状态压缩 map: 状态稀疏时用 $\text{map<int, i64> / unordered_map}$ 。

关键点: SOS DP 的层序遍历顺序 (外层位 i 、内层 $mask$); 滚动数组注意覆盖顺序 (当前层依赖上一层); 子集卷积要拆 popcount 分层 (普通 FWT 会重算)。

```
// SOS DP (子集和): f[mask] = sum of f[sub] for sub mask
// for (int i = 0; i < n; ++i)
//     for (int mask = 0; mask < (1 << n); ++mask)
//         if (mask >> i & 1) f[mask] += f[mask ^ (1 << i)];
// 快速子集卷积: g[k][mask] (按 popcount 分层), FWT 后逐层卷积再 FWT 回来
## 10 博弈与概率
```

10.1 SG 函数 (组合博弈)

【简介】

公平组合游戏 (双方同操作集合): 状态 $SG = \text{mex}(\text{后继 SG})$, 异或和为 0 则先手必败。所有“轮流操作、不能动者输”的题通用。

快速识别: 轮流取/移动、无随机、不能操作者输; 多堆/多子游戏 (异或和)。

API 接入:

- $sg[\text{状态}] = \text{mex}(\{sg[\text{后继}]\})$; 记忆化 DFS。
- 多子游戏: 总 $SG =$ 各子 SG 异或; 非 0 先手胜。
- 打表找规律 (周期/公式) 后 $O(1)$ 。

关键点: $\text{mex} =$ 最小未出现非负整数; SG 只对“公平”游戏适用 (双方同操作); 打表是常态——先暴力小范围找 SG 规律; 必胜步 = 找一个后继使异或和为 0。

```
// 记忆化 SG (例: 一堆石子, 每次取 1 或 2 个, 取完者胜)
int sg[1001];
int get_sg(int x) {
    if (sg[x] != -1) return sg[x];
    set<int> s;
    for (int take : {1, 2}) if (x >= take) s.insert(get_sg(x - take));
    int g = 0;
    while (s.count(g)) ++g;
    return sg[x] = g;
}
// 多堆: ans = xor of sg[每堆]; ans != 0 先手胜
```

10.2 Nim 家族速查

【简介】

经典 Nim（异或和判胜负）及变体：Misere Nim、阶梯 Nim、Fibonacci Nim、Wythoff、Moore’s Nim-k、树上删边（Green Hackenbush）。

快速识别：取石子/取物类博弈题：先识别是哪一族（数量、取法限制、胜负条件），再套结论。

API 接入：

- 标准 Nim: $\text{xor} = a[0]^a[1]^{\dots}$ ；非 0 先手胜；必胜一步：找堆使 $a[i] > (a[i] \wedge \text{xor})$ ，改成 $a[i] \wedge \text{xor}$ 。
- Misere Nim（取最后一个输）：全为 1 时看奇偶；否则同标准 Nim。
- 阶梯 Nim（只能从奇数层取）：只看奇数层异或。
- Fibonacci Nim（首取任意、之后不超过上次两倍）：先手胜当且仅当 n 不是 Fibonacci 数。
- Wythoff（两堆，可取任一堆任意个或两堆同数）： $|a-b| * \text{黄金比} = \min(a,b)$ 则必败。
- Moore’s Nim-k（一次最多动 k 堆）：二进制每位和 $\bmod (k+1)$ 判。

关键点：先确认变体再套结论（Misere 与标准只差全 1 特判）；阶梯 Nim 只算奇数层；Wythoff 用浮点黄金比注意精度（用整数近似或 long double）；树上删边 = 每条链长度异或（Colon 原理）。

```
// 标准 Nim 必胜一步：
// int xr = 0; for (int x : a) xr ^= x;
// if (xr == 0) 先手必败；
// for (int &x : a) if ((x ^ xr) < x) { x = x ^ xr; break; } // 输出必胜操作
// Fibonacci Nim: n 是 Fibonacci 数 => 必败
// Wythoff: a <= b, (b - a) * (1+sqrt(5))/2 == a (浮点容差) => 必败
```

10.3 随机算法（爬山 / 模拟退火 / 随机化）

【简介】

近似/概率算法：爬山（邻域贪心）、模拟退火（接受劣解跳出局部最优）、随机打乱/随机化哈希（防构造数据）。

快速识别：几何最优（费马点、最小圆）、连续最优化（答案精度要求不高）；构造数据针对性卡时用随机化。

API 接入：

- 爬山：for 迭代；for 方向：尝试移动，接受更优。
- 模拟退火： T 从高到低，接受概率 $\exp(-\text{delta}/T)$ ；多次重启取最优。
- 随机化哈希：给每个元素随机权值（splitmix64），集合哈希 = 异或/和。

关键点：模拟退火参数（初始温度、降温率、迭代次数）要调；多次运行取最优；随机化哈希防碰撞要 64 位随机；期望得分而非保证 AC——只有确定算法超时才用。

```
// 模拟退火骨架（求函数 f 最小值，x 为连续/离散点）：
// double T = 1000, cooling = 0.999;
// while (T > 1e-8) {
//     auto nxt = neighbor(cur); // 随机邻域
//     double delta = f(nxt) - f(cur);
//     if (delta < 0 || exp(-delta / T) > (double)rng() / UINT64_MAX) cur = nxt;
//     T *= cooling;
// }
// 多次随机初始点重跑，取全局最优
```

10.4 概率题通用技巧

【简介】

概率计数技巧：条件概率、线性期望（期望线性性）、几何分布、二项分布、概率 DP 的“全概率公式”。竞赛概率题 90% 考期望线性性 + 容斥。

快速识别：“期望次数/期望贡献”；每步独立事件；“至少/恰好”概率。

API 接入：

- 期望线性性： $E(X+Y) = E(X)+E(Y)$ 不管相关性——把答案拆成多个指示变量之和。
- 几何分布：成功概率 p ，期望次数 = $1/p$ 。
- 二项： n 次独立试验成功 k 次 = $C(n,k) p^k (1-p)^{(n-k)}$ 。
- 概率取模： p/q 用 q 的逆元表示。

关键点：能拆指示变量就拆（每步贡献独立算）；“直到出现 xxx” = 几何分布/马尔可夫链吸收；概率转模意义 = 乘逆元；注意事件是否独立再乘。

```
// 期望线性性示例：随机排列中“上升对”期望数 =  $C(n,2) * P(\text{前小后大}) = n(n-1)/4$ 
// 几何分布： $E = 1/p$ ；方差 =  $(1-p)/p^2$ 
// 模意义概率： $P = a * \text{inv}(q) \% \text{MOD}$ 
```

11 交互、通信与杂项

11.1 交互题模式

【简介】

程序与评测器一问一答：固定次数内猜出答案（二分、多数投票、分组查询）。必须刷新输出。

快速识别：题面含“interactive”；查询次数上限；输出后要求 flush。

API 接入：

- 输出查询：`cout << "? " << ... << endl`（endl 自带 flush）或 `cout.flush()`。
- 读答案：`cin >> res`。
- 结束：输出！答案 后立即 return。
- 常用模式：二分（单调判定）、比较排序（查询当比较器）、多数投票（Boyer-Moore 或随机）、分组（信息论）。

关键点：每次查询后必须 flush（endl 或 fflush）；本地测试用脚本对拍；查询次数预算好（信息量下限）；注意特殊约定（0/1-base d、是否含端点、答案类型）。

```
// 二分交互骨架（猜隐藏值 x in [l, n]）：
// int l = 1, r = n;
// while (l < r) {
//     int mid = (l + r) / 2;
//     cout << "? " << mid << endl; // endl = flush
//     int res; cin >> res; // res: 0=小于等于, 1=大于（按题面）
//     if (res == 0) r = mid; else l = mid + 1;
// }
// cout << "! " << l << endl;
```

11.2 通信题与信息编码

【简介】

两个程序间通过有限长度字符串通信（bit packing、纠错码、XOR 技巧、随机摘要）。

快速识别：“communication”题；两个程序（A/B）收发字符串；信息量预算。

API 接入：

- 信息量估算：要传 n 种可能 $\rightarrow$ 至少 $\log_2(n)$ bit。
- bit packing：把多个小整数压缩进一个 64 位整数（位段）。
- XOR 技巧：传一个数恢复缺失信息（两数异或）。
- 纠错：Hamming(7,4) 一位纠错；重复发送多数投票。
- 随机摘要：对集合算哈希摘要比对一致性。

关键点：先算信息量下限（能不能传得下）；编码/解码两端对称；纠错码能纠正 d 位错需要海明距离 $2d+1$ ；多用位运算。

```
// bit packing: 把 k 个  $0..(2^b-1)$  的数压缩进 i64
// 编码:  $x = 0$ ; for i:  $x |= (\text{val}[i] << (i * b))$ ;
// 解码:  $\text{val}[i] = (x >> (i * b)) \& ((1 << b) - 1)$ ;
// 两集合一致性（随机哈希）: A 的哈希 = sum splitmix(a_i); B 传回哈希比较
```

11.3 构造题与 checker 思维

【简介】

构造题：输出任意满足条件的方案。套路：归纳构造、模/周期构造、贪心调整、随机验证。输出前自检。

快速识别：“输出一种合法方案”；无最优性要求；special judge。

API 接入:

- 通用: 先想平凡构造 (全 0、全 1、等差、模 3 循环), 再逐步修正。
- 自检: 写 checker 函数 (本地) 验证输出合法再交。
- 常见: 按奇偶分、按模分类、贪心填。

关键点: 验证器 (本地跑一遍检查条件) 是构造题防 WA 的关键; 注意输出格式 (空格/换行、大小写); 边界 (n=1、空串); 构造优先简单规律 (避免复杂易错)。

```
// 本地自检模式:
// bool check(const 输入&, const 输出&) { /* 逐条件验证 */ }
// 构造出答案后先调用 check 再提交 (本地宏控制)
```

11.4 STL 与常用工具速查

【简介】

竞赛高频 STL 与算法函数速查: 容器选择、算法、陷阱。

快速识别: 写代码时查容器/函数选型; 性能与陷阱。

API 接入:

- 容器: `vector` (默认)、`deque` (两端)、`set/multiset` (有序去重/可重)、`map/unordered_map` (哈希)、`priority_queue` (堆, 默认大根)、`stack/queue`。
- 算法: `sort/stable_sort/nth_element` (部分排序)、`lower_bound/upper_bound` (有序二分)、`unique` (去重需先 `sort`)、`next_permutation`、`gcd/lcm` (C++17)、`accumulate` (注意溢出)。
- 陷阱: `unordered_map` 可被卡 (换 `map` 或用随机哈希); `set` 迭代器失效; `priority_queue` 无删除 (懒删除); `string` 拼接 $O(n^2)$ (用 `string::append` 或 `vector<char>`)。

关键点: `nth_element` 找第 k 大 $O(n)$; `lower_bound` 在 `set` 上用 `s.lower_bound` (成员版 $O(\log n)$); `vector<bool>` 是位压缩 (慢, 用 `vector<char>`); `memset` 只适用 0/-1。

```
// 常用:
// nth_element(v.begin(), v.begin() + k, v.end()); // v[k] = 第 k 小 (0-based), O(n)
// auto it = s.lower_bound(x); // set 成员版
// next_permutation(all(v)) // 字典序下一个排列
// i64 g = std::gcd(a, b); // C++17
// 对 unordered_map 防卡: 自定义 splitmix64 哈希
```

11.5 常见陷阱与调试清单

【简介】

WA/TLE/RE 的系统排查清单, 按概率排序。

快速识别: 任何题卡住时按此清单过一遍。

API 接入:

- WA: 下标起点 (查 00.4 索引表)、 k 起点、i64 溢出、模数、边界 (空/单元素/全同)、排序稳定性、比较器严格弱序、浮点 eps、读入格式。
- TLE: 复杂度表核对 (00.2) 和 `cin` 改快读、递归改迭代、`unordered_map` 卡、 $O(n^2)$ 排查、清空方式 (`vector.assign` vs `memset` 大数组)。
- RE: 数组越界 ($n+1$ vs $n+2$)、栈溢出 (递归深改迭代)、除零、空容器 `top()`。
- UB: 未初始化、悬垂引用、 $1 < 63$ 、`INT_MAX+1`、`lambda` 捕获生命周期。

关键点: 先看数据范围再想算法; 对拍 (暴力 vs 正解随机数据) 是找 WA 的利器; 编译开 `-Wall -Wextra`; `assert` 定位 RE; 改完一处重新想全流程而不是盲目试。

```
// 对拍模板 (本地):
// gen.exe > in.txt; brute.exe < in.txt > b.txt; fast.exe < in.txt > f.txt;
// fc b.txt f.txt -> 一致则循环, 不一致则输出 in.txt 定位
```

11.6 样例构造器 (对拍 / 造数据)

【简介】

本地生成随机测试数据 (随机数组、树、图), 配合暴力程序对拍找 WA。写不出正解也要能造数据验证。

快速识别: 本地调试、对拍、验证边界。

API 接入:

- 随机数: `rng()` (01.12); `rng() % n` 取 $[0, n)$ 。
- 随机排列: `shuffle(all(v), rng)`。
- 随机树: 对每个 $i \geq 1$ 连到随机父节点; 随机图: 枚举边对随机加。
- 生成器输出到 `stdout`, 重定向进 `in.txt`。

关键点: 种子固定可复现 (调 bug 时用固定种子); 造数据要有意识覆盖边界 (n=1、全同、最大、0/负); 树/图生成后自检连通与范围。

```
// —— 用法示例: 抄入你的 main() 中 (本地造数据 gen.exe > in.txt) ——
mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());
// 随机数组:
int n = rng() % 10 + 1;
cout << n << '\n';
for (int i = 0; i < n; ++i) cout << (int)(rng() % 20) - 10 << ' ';
// [-10, 10]
cout << '\n';
// 随机树 (n 点, 1-based): 每个点连到随机父节点
// for (int i = 2; i <= n; ++i) cout << i << ' ' << (int)(rng() % (i - 1) + 1) << '\n';
// 随机图 (n 点 m 边, 无重边):
// set<pair<int,int>> es;
// while ((int)es.size() < m) { int u = rng() % n + 1, v = rng() % n + 1; if (u != v) es.insert({min(u,v), max(u,v)}); }
// for (auto [u, v] : es) cout << u << ' ' << v << '\n';
```

11.7 卡常技巧速查

【简介】

常数优化手段合集: TLE

时按顺序尝试。先确认复杂度没问题再卡常。

快速识别: TLE; 复杂度正确但常数大。

API 接入:

- IO: `ios::sync_with_stdio(false)`; `cin.tie(nullptr)`; 超大输入用 01.2 快读; 输出用 `'\n'` 而非 `endl`。
- 容器: `vector` 预分配 `reserve`; `unordered_map` 被卡换 `map` 或自定义哈希; `vector<bool>` 换 `vector<char>`。
- 算法: 循环外提重复计算; `memset` 大数组慢, 用 `fill`; 递归改迭代 (防爆栈)。
- 编译: `-O2`; `#pragma GCC optimize("O3")` (部分 OJ 可用)。
- 位运算: $\% 2$ 用 `& 1`; $\ast 2$ 用 `<< 1`。

关键点: 先测复杂度; 卡常从 IO

开始 (收益最大); 哈希容器被卡时换树容器; 栈溢出先改迭代。

```
// 编译优化 (部分 OJ 支持):
// #pragma GCC optimize(2)
// #pragma GCC optimize(3)
// 自定义哈希防 unordered_map 被卡:
struct CustomHash {
    static ui64 splitmix64(ui64 x) {
        x += 0x9e3779b97f4a7c15ULL;
        x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9ULL;
        x = (x ^ (x >> 27)) * 0x94d049bb133111ebULL;
        return x ^ (x >> 31);
    }
    size_t operator()(i64 x) const {
        static const ui64 FIXED = chrono::steady_clock::now().time_since_epoch().count();
        return splitmix64(x + FIXED);
    }
};
// 用法: unordered_map<i64, int, CustomHash> mp;
```

11.8 近五年东亚赛区真题 → 模板索引

【简介】

2021-2025 ICPC 东亚赛区 (EC-Final + 主要区域赛) 高频算法与本书模板的对照索引。赛前过一遍, 确认每项都会用; 遇到相似题直接翻对应节。

快速识别: 真题考点复习; 赛时“这题像哪年哪题”的定位。

关键点：标注 ★ 的为连续多年出现的高频考点，优先掌握；技巧类（贪心/构造/模拟）不列具体模板但分布在各章。

年份/赛区	真题考点	本书模板
2025 EC-Final	D 三模 NTT ★	07.17 三模 NTT
2025 EC-Final	A 值域拆段+主席树	02.21 主席树
2025 EC-Final	B 计算几何/位运算	08 章 + 01.13
2025 EC-Final	F SCC 缩点+链查询 ★	04.9 SCC + 03.3 树剖
2025 EC-Final	K 区间覆盖（差分）	01.7 差分
2025 EC-Final	M 数论+ST 表	06 章 + 02.13 ST
2025 网络赛	C 区间二分图匹配（线段树优化建图）★	04.27 线段树优化建图 + 04.14 匹配
2025 网络赛	D Min-Max 树形 DP	03.9 树 DP
2025 上海	K 区间 $\min\times\max$ 信息（Beats 变种）	02.10 Beats
2025 上海	哈希+线段树/分块	05.1 哈希 + 02.18 分块
2025 南京	B 半平面最大重叠 ★	08.5 半平面交
2025 南京	F 按位与路径	04.1 最短路 + 01.13
2025 成都	K 覆盖次数（差分前缀和）	01.7 差分
2024 EC-Final	D FFT/三模 NTT ★	07.17
2024 EC-Final	A 贪心+堆	01 章贪心 + 02.16 对顶堆
2024 EC-Final	B 字符串划分 DP	09.3 区间 DP
2024 EC-Final	I 树上 dfn 染色	03.7 Euler 序
2024 杭州	B 按位贪心+线段树 ★	01.13 按位贪心 + 02.7 线段树
2024 南京	E 字符串左移	05.1 哈希
2023 南京	C 原根 / D 平衡树 / G 背包 / J 后缀结构 / M 接雨水	06.13 / 02.24-35 / 09.1 / 05.10-11 / 02.14
2023 济南	B 图划分 / C 开灯 / L 欧拉回路 / M 凸包	02.1 DSU / 02.8 线段树 / 04.13 欧拉 / 08.3 凸包
2022 EC-Final	多项式 / 网络流建模 / 交互	07.15 FPS / 04.17 最小割 / 11.1 交互